Practically Efficient Protocols for Private Computation using Homomorphic Encryption

by

Rasoul Akhavan Mahdavi

A thesis
presented to the University of Waterloo
in fulfillment of the
thesis requirement for the degree of
Doctor of Philosophy
in
Computer Science

Waterloo, Ontario, Canada, 2026

© Rasoul Akhavan Mahdavi 2026

Examining Committee Membership

The following served on the Examining Committee for this thesis. The decision of the Examining Committee is by majority vote.

External Examiner:	Sebastian Angel Associate Professor Department of Computer and Information Science University of Pennsylvania
Supervisor:	Florian Kerschbaum Associate Professor David R. Cheriton School of Computer Science University of Waterloo
Internal Members:	Ian Goldberg Associate Professor David R. Cheriton School of Computer Science, University of Waterloo
	Mohammad Hajiabadi Assistant Professor David R. Cheriton School of Computer Science, University of Waterloo
Internal-External Member:	Douglas Stebila Associate Professor Department of Combinatorics & Optimization, University of Waterloo

Author's Declaration

This thesis consists of material all of which I authored or co-authored: see Statement of Contributions included in the thesis. This is a true copy of the thesis, including any required final revisions, as accepted by my examiners.

I understand that my thesis may be made electronically available to the public.

Statement of Contributions

This thesis includes first-authored and peer-reviewed work that has appeared in conference proceedings.

The contents of Chapter 3 were previously published in a paper which appeared at ACM CCS 2023 [123]. Some modifications have been incorporated for clarification and correctness. It was published in collaboration with Haoyan Ni, Dimitry Linkov, and Florian Kerschbaum. Florian and I initially discussed the main ideas for the paper. Dimitry Linkov developed an initial version of the RCC protocol and RCC-PDTE under my supervision, which I subsequently improved. Haoyan developed the XXCMP protocol and XXCMP-PDTE, also under my supervision. Haoyan and I performed all experimental evaluation and incorporated all results into the paper. The paper was collaboratively written by Florian and myself.

The contents of Chapter 4 were previously published in a paper which appeared at Usenix 2023 [122]. It was published in collaboration with my colleagues. Faezeh, Thomas, John, Xinda, and I developed an early version of the protocol as part of an industry collaboration, which was used to improve our initial design. Afterwards, I developed an updated implementation with additional optimizations, which was used in our experimental evaluation. Nils and I performed the experimental evaluation. The initial manuscript was written by Nils, Bailey, and myself, and was reviewed by Simon, Ehsan, and Florian for final publication.

The work presented in Chapter 5 was done as part of an internship at Google in collaboration with my colleagues, Joon Young Seo, Kevin Yeo, and Sarvar Patel. A part of this work has been accepted to IEEE S&P 2026 and will be published in May 2026. Joon Young, Kevin, Sarvar, and I were all involved in the initial stage of this work, which involved identifying the problem of existing PIR protocols, and designing our new constructions. I implemented all the constructions in our work with assistance from Joon Young on the details of the protocols and specific optimizations. I performed all experimental evaluations and wrote the evaluation section of the paper. Writing was done collaboratively by all co-authors. Joon Young wrote the description for InspiRING and InsPIRe, and associated proofs. I wrote all descriptions for InsPIRe$^{(2)}$ and all associated proofs, and also wrote some additional proofs for InsPIRe. Kevin, Joon Young, Sarvar, and I collaboratively wrote the introduction and Kevin wrote the related work.

Abstract

Digital services have become an indispensable part of our daily lives, particularly services that interact with our most private and sensitive data. With the abundance of such services, users are left to make the difficult choice: can I safely use digital services and products, or does it necessarily come at the cost of my privacy. Private computation techniques empower service providers to perform computation over private data, without the need to observe the data. This not only provides privacy for clients while the data is being used but reduces the risk of incidents such as data leaks for service providers. One commonly used tool for private computation is Homomorphic Encryption (HE), which is a form of encryption that allows computation on data in encrypted form. While homomorphic encryption in theory permits arbitrary computation over encrypted data, in practice, a naive implementation of a desired functionality rarely yields a practical result. For example, one common obstacle when using homomorphic encryption is the high computation time and the large ciphertexts that incur high network costs. However, communication and computation costs are not the only metrics that need to be considered. In my work, we describe problems that arise when homomorphic encryption is used in applications and address these limitations by proposing new techniques and novel protocols. In these new constructions, we not only improve the performance compared to prior work in terms of communication and computation costs but also address additional problems that arise in the deployment of these protocols. Throughout the process, we draw insights on how to design protocols that can be applicable for developers, practitioners, and future researchers. For example, we enable homomorphic comparison of encrypted numbers with higher precision than previous work, using novel representation of numbers that is more suitable for homomorphic encryption. Using this and other building blocks, we propose efficient protocols for decision tree evaluation and private set intersection. Moreover, through our work on private information retrieval, we identify the challenges of using such a protocol in practice and propose novel protocols that are suited for deployment in real-world applications.

Acknowledgements

Florian, as an amazing supervisor, you have made the Ph.D. experience unforgettable and have been a cornerstone of my success. With the amazing opportunities you provided through the years, you have helped me learn so much in my professional life. It was a privilege to have you as an advisor throughout these years.

Thank you my examining commitee, Professor Angel, Goldberg, Stebila, and Haiabadi, for their valuable comments on this thesis.

Thanks to all my amazing colleagues and co-authors throughout the years who made me grow not only as an academic, but also as a person. Thank you Thomas and Shannon for assembling the dream team and giving me the opportunity to work on probably the least stressful project I ever worked on, knowing what a great team we had. Big thanks to you, Miti, for suggesting to work on Peer2PIR which ended up being one of the most interesting projects I worked on, and also set me up for future projects on PIR. A big thanks to my colleagues at Google, Mike, Kevin, and Sarvar who set me up for success, not only during my internship, but also after. Thank you Mike for amazing me every day with your bursts of brilliance and being a last resort that I could count on, every time I was stuck. Thanks Kevin for your amazing guidance and unparalleled wealth of knowledge in literally every possible topic, and making me feel welcome and appreciated. Little Andy will be so lucky to have his own personal version of Google as a father. And thank you Sarvar for the great discussions and mentorship and your relentless pursuit to provide all the necessary conditions for my success at Google.

I want to thank the members of the CrySP lab, which created an environment ripe for collaboration and friendship throughout the years.

Thanks to the long list of amazing friends who were instrumental to my sanity, by listening to my rants, cheering me in my pursuits, and simply making life enjoyable.

Last, I thank my family, who provide nothing but support and encouragement as I ventured into the unknown. Their support will carry me far as I know I will always be able to count on them.

Dedication

To my family, whose only pursuit is the best, for me.

Table of Contents

Examining Committee Membership ii

Author's Declaration iii

Statement of Contributions iv

Abstract v

Acknowledgements vi

Dedication vii

List of Figures xiii

List of Tables xv

1 Introduction 1

1.1 Homomorphic Encryption 2

1.1.1 Considerations when working with HE 3

1.2 Thesis Statement and Contributions 5

1.2.1 Level Up 5

1.2.2 PEPSI 6

1.2.3 InsPIRe 7

2 Background **9**

2.1 Homomorphic Encryption 9

2.2 Constant-weight Encoding 10

3 Level Up: Private Non-Interactive Decision Tree Evaluation using Leveled Homomorphic Encryption **12**

3.1 Introduction 12

3.2 Background and Related Work 15

3.2.1 Non-interactive Private Comparison 15

3.2.2 Decision Trees and PDTE 17

3.2.3 Tree Traversal 18

3.2.4 Range Covers & Point Encoding 19

3.3 Related Work on PDTE 20

3.3.1 Interactive PDTE 20

3.3.2 Non-Interactive PDTE 22

3.4 Our Comparison Operators 24

3.4.1 XXCMP: Extended XCMP Operator 24

3.4.2 Range-Cover Comparison (RCC) Operator 26

3.5 PDTE using Leveled HE 28

3.5.1 Setup and Security Model 28

3.5.2 XXCMP-PDTE: XXCMP + SumPath 29

3.5.3 RCC-PDTE: RCC + SumPath 29

3.6 Evaluation 30

3.6.1 Benchmarking Private Comparison 31

3.6.2 Benchmarking PDTE 32

3.6.3 Summary of Results 36

3.7 Discussion and Future Work 37

3.8 Conclusion 38

4 PEPSI: Practically Efficient Private Set Intersection in the Unbalanced Setting 40

4.1 Introduction . . . 40
4.2 Related Work . . . 42
4.2.1 PSI with Leakage . . . 43
4.2.2 Unbalanced PSI . . . 43
4.2.3 Labelled PSI . . . 44
4.2.4 Circuit PSI . . . 45
4.3 Background . . . 46
4.3.1 Hashing Optimizations . . . 46
4.4 PEPSI: Practically Efficient PSI . . . 48
4.4.1 Dataset Preprocessing . . . 49
4.4.2 Server Computation . . . 51
4.4.3 Result Extraction . . . 52
4.5 PEPSI Variants . . . 52
4.5.1 Optimization for Large Elements . . . 52
4.5.2 Labelled PSI . . . 53
4.5.3 Circuit PSI . . . 54
4.6 Analysis of PEPSI . . . 55
4.6.1 Communication Complexity . . . 57
4.6.2 Computation Complexity . . . 59
4.6.3 Communication/Computation Tradeoff . . . 61
4.7 Evaluation . . . 62
4.7.1 PSI Evaluation and Comparison . . . 63
4.7.2 Labelled PSI Evaluation and Comparison . . . 66
4.7.3 Circuit PSI Evaluation and Comparison . . . 67
4.7.4 Summary of Evaluation . . . 68
4.8 Conclusion . . . 69

5 InsPIRe: PIR with Server-side preprocessing **70**
5.1 Introduction . . . 70
5.1.1 The Case for Server-side Preprocessing with no Offline Communication 72
5.1.2 Technical Overview and Contributions . . . 74
5.2 Preliminaries . . . 75
5.2.1 Common Reference String (CRS) Model . . . 80
5.2.2 Convention of our Pseudocode Presentation . . . 80
5.3 Ring Packing with Preprocessing . . . 81
5.3.1 Revisiting CDKS Packing . . . 81
5.3.2 Packing with Two Key-switching Matrices . . . 83
5.3.3 Partial Packing with One Key-Switching Matrix . . . 91
5.3.4 $\mathsf{InsPIRe}_0$: PIR from Ring Packing . . . 92
5.4 $\mathsf{InsPIRe}^{(2)}$: PIR from Double Ring Packing . . . 93
5.5 InsPIRe: PIR from Ring Packing and Homomorphic Polynomial Evaluation 96
5.5.1 Homomorphic Polynomial Evaluation . . . 97
5.5.2 Putting It Together . . . 100
5.6 Experimental Evaluation . . . 105
5.6.1 Parameterizing $\mathsf{InsPIRe}^{(2)}$ . . . 105
5.6.2 Parameterizing InsPIRe . . . 108
5.6.3 PIR Evaluation with Various Entry Sizes . . . 108
5.6.4 Benchmarking Ring Packing . . . 111
5.7 Applications of InsPIRe . . . 112
5.7.1 Private Queries in IPFS . . . 112
5.7.2 Privacy-Preserving Device Enrollment . . . 113
5.8 Related Works . . . 114
5.9 Conclusions . . . 116

6 Conclusion **117**

6.1 Response to Thesis Statement 117

6.2 Future Directions 119

References **121**

APPENDICES **143**

A PEPSI: Practically Efficient Private Set Intersection in the Unbalanced Setting **144**

A.1 Data Preparation 144

A.2 PSI-kth-Match 144

B InsPIRe: Communication-Efficient PIR with Server-side Preprocessing **147**

B.1 Supplementary Material for Ring Packing 147

B.1.1 LWE to Intermediate Ciphertexts 147

B.1.2 Conversion to RLWE Ciphertext 148

B.2 Analysis of InsPIRe 150

B.3 Security Analysis of InsPIRe 153

B.3.1 Extension to Multiple Queries 156

B.4 Optimizations and Extensions to InsPIRe 158

B.4.1 Multivariate Homomorphic Polynoimal Evaluation 158

B.4.2 Extension to non-power of two t 159

B.4.3 Approximate Gadget Decomposition 160

B.5 Analysis of InsPIRe$^{(2)}$ 161

B.6 Additional Experiments 164

List of Figures

1.1 Overview of Homomorphic Encryption 2

3.1 Edge labelling in tree traversal methods 19
3.2 Range cover and point encoding example 21
3.3 Packing OURC in FV ciphertexts in batched mode. 28
3.4 Amortized Time for private comparison 32
3.6 Communication cost of PDTE as a function of the number of attributes. . 34
3.7 Runtime of PDTE as a function of the number of nodes in the tree. 36
3.5 Runtime and Communication for Private Decision Tree Evaluation over four datasets. 39

4.1 Stages of PEPSI. 48
4.2 Client Dataset Preprocessing in PEPSI 50
4.3 Code length as a function of the Hamming weight for $\bar{\lambda} \in \{16, 32, 48\}$ for $b \leq 4096$. The minimum occurs for a Hamming weight of 8, 8, and 23, respectively. 59
4.4 Optimal Hamming weight as a function of the effective bitlength 60
4.5 Runtime as a function of the Hamming weight 61
4.6 Visualizing the tradeoff between communication cost and runtime in PEPSI. 62

5.1 Communication Cost vs. Online Server Runtime for a PIR query over a 1 GB database. 74
5.2 Overview of the server's computation in InsPIRe 103

5.3 Communication and computation costs of InsPIRe$^{(2)}$ over a 1 GB database 107
5.4 Breakdown of communication and online time of InsPIRe over 1 GB database 109

List of Tables

1.1 The advantage of each of our constructions, aside from improving the speed and communication. 5

2.1 Different encodings for the FV cryptosystem 10

3.1 Summary of notation used in Chapter 3 15
3.2 Properties of Non-interactive Private Decision Tree Evaluation Protocols. . 22
3.3 Number of parallel comparisons in RCC and Folklore comparison as a function of the bit precision. 31
3.4 Characteristics of UCI datasets used in our evaluation 33
3.5 Approximate runtime for private evaluation of balanced decision tree . . . 35
3.6 Communication cost for private evaluation of balanced tree 36

4.1 Comparing the properties of different PSI protocols 42
4.2 Summary of notation for algorithm description. 49
4.3 Polynomial modulus degree and coefficient modulus as a function of the Hamming weight. 58
4.4 Asymptotic complexity of DiPSI vs PEPSI in the unbalanced case, i.e., $m \ll n$. 64
4.5 Private set intersection over 32-bit elements using DiPSI and PEPSI 65
4.6 Private set intersection and Labelled PSI over large elements using the work of Cong et al. and PEPSI 66
4.7 PSI-Sum using PEPSI and the work of Ion et al. [90] 67

5.1 Lattice parameters for $\mathsf{InsPIRe}_0$, $\mathsf{InsPIRe}^{(2)}$, and InsPIRe. 106
5.2 PIR performance metrics for different database sizes with 1-bit entries. . . 110
5.3 PIR performance metrics for different database sizes with 64 B entries. . . 110
5.4 PIR performance metrics for different database sizes with 32 KB entries. . 111
5.5 Concrete costs of packing 2^{12} LWE ciphertexts into RLWE ciphertexts . . 112
5.6 Costs of PIR queries in the three functionalities of IPFS. 113
5.7 Cost of PIR for Chrome Device Enrollment 114

B.1 Performance metrics for different configurations of $\mathsf{InsPIRe}^{(2)}$ 165
B.2 Performance metrics for InsPIRe using different polynomial interpolation degrees. 166

Chapter 1

Introduction

Take a second and count all the digital services with which you have interacted today. Then, consider how much sensitve and personal information you have had to provide in order to recieve many of these services. As our reliance on digital services for daily activities increases, the resulting digital trace of user information creates a substantial and inherent risk of privacy leaks by the providers who collect and manage this data. Aside from interactions between a user and a centralized server interactions, there are also many instances of collaboration between institutions that involve the exchange of private and sensitive information, for the purpose of deriving insights and delivering value to all parties involved.

In such scenarios where we want to perform computation on data from a variety of data sources, we may not be able to do so because of threats to the privacy of individual users, legitimate legal hurdles which prevent companies from sharing user data, and avoiding sharing data to maintain a competitive edge. In all these scenarios, it seems impossible to draw any shared insights and provide services without relinquishing access to the original data. In other words, there seems to be a tension between privacy and utility. Hence, we need techniques to achieve privacy and utility at the same time, i.e., privacy *in use*. On a practical level, such techniques enable users to enjoy services without compromising their privacy, and give institutions which hold valuable datasets, the ability to collaborate, without exposing themselves to risk of data leakage or losing their competitive advantage.

Private computation is a collection of tools and techniques that address this tradeoff, which permits computation on data, without the need to see the data. Using these techniques, we can perform computation on private data, and only reveal the desired outputs. Tools for private computation generally fall into three categories. First, hardware-based

solutions such as Trusted Execution Environments (TEEs) which provide confidentiality based on hardware assumptions [10, 89]. The second category is secure Multi Party Computation (MPC) techniques [70], which allow multiple parties to perform computation on data, without any single individual seeing the underlying data. The third category, which is the primary focus of this work, is solutions based on homomorphic encryption.

1.1 Homomorphic Encryption

Homomorphic Encryption is a form of encryption that allows computation on encrypted data. In its simplest form, homomorphic encryption enables one party, usually denoted as the server, to perform computation over data encrypted by another party, namely the client. This simple form of using homomorphic encryption is visualized in Figure 1.1. The server can perform this computation without interaction with the client, other than receiving the input ciphertexts from the client and sending the response back. The non-interactive model is very useful in applications that involve asymmetry between the client's and server's resources. For example, the server is hosting a database that the client can not store locally, or the client wants to outsource some computation to the server.

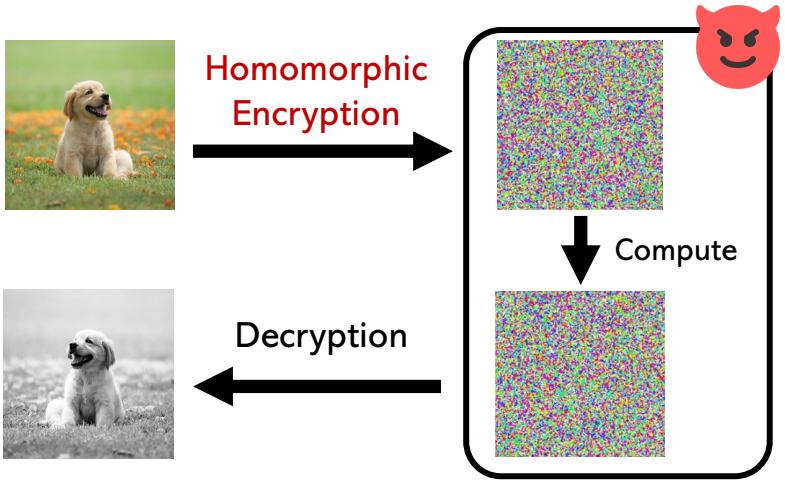


Figure 1.1: Pictorial example usage of homomorphic encryption for applying modifications to an encrypted image.

Evolution of Homomorphic Encryption Schemes. Homomorphic encryption was first proposed by Rivest et al. in 1978 [152], where they posited the existence of *special*

encryption functions that permit computation on encrypted data. From 1978 to 2009, there were constructions for *linear* homomorphic encryption, which only supported one type of encrypted operation (modular addition [137], modular multiplication [69], bit-wise XOR [77], ...), but the existence of a scheme that supported arbitrary encrypted computation remained unsolved for over 30 years. In 2009, in a breakthrough result, Gentry showed how to construct the first *fully* homomorphic scheme, which could perform arbitrary computation over encrypted data [75]. Gentry's initial constructions created the first generation of FHE schemes [75, 166], which proved the existence of fully homomorphic encryption. While this generation of FHE is extremely slow (10^8 seconds for a one-bit operation [76]), it pioneered the use of lattice-based encryption for homomorphic encryption. The two main components of Gentry's construction were a *leveled* homomorphic encryption scheme and the bootstrapping operation. A leveled homomorphic encryption scheme permits computation on encrypted data up to a fixed multiplicative depth, which is determined by the parameters of the encryption scheme. The multiplicative depth of a function is the maximum number of sequential multiplications required in the computation. The bootstrapping operation is a mechanism for resetting the homomorphic noise, which is a property of lattice-based schemes, and accumulates as operations are performed. While leveled schemes were used as a component in constructing a fully homomorphic scheme, they gained independent interest since they could be used without resorting to the bootstrapping operations, albeit for limited functions. This made way for the second generation of schemes, the most notable examples being leveled schemes such as BGV [29] and BFV [29, 71], which offer efficient operations up to a given depth, without the need for bootstrapping. In the third generation, schemes such as FHEW [65] and TFHE [48] were the first to propose an efficient bootstrapping procedure. Lastly, in the fourth generation, CKKS [45] proposed a protocol that operates over rational (and complex) numbers and supports approximate computation. CKKS was further improved to support bootstrapping [23, 39, 44, 81].

1.1.1 Considerations when working with HE

While FHE gives the ability to compute arbitrary functions on encrypted inputs, simply replacing operations over plaintexts with equivalent operations over ciphertexts would result in poor performance. Hence, we require new protocols which take the constraints of FHE into account. Below, we give a non-exhaustive list of considerations when implementing a function over homomorphic ciphertexts.

No branching. Data-dependent operations such as branching on encrypted data are not possible, so we require oblivious algorithms that do not perform any such operations. Oblivious algorithms are necessary for other forms of private computation such as MPC as well, but simply using an oblivious algorithm is not enough.

Cost of operations. Different homomorphic encryption schemes offer different sets of operations. BFV and BGV encode data as polynomials and vectors and offer arithmetic operations (modular addition and multiplication) over these types [157]. TFHE encodes small scalars and permits bitwise operations and small lookup tables [48]. Moreover, different operations are not equally expensive. For example, in BFV and BGV, additions are up to 20 times faster than multiplications, depending on the parameters of the scheme [121, 139]. The cost of each operation depends on the parameters of the cryptosystem, which are chosen such that ciphertexts can accommodate the noise that is accumulated during computation. Since homomorphic (and plaintext) multiplications increase the noise much more than other operations, the main factor that determines the parameters of a homomorphic cryptosystem is the *multiplicative depth*. Hence, in applications implemented in homomorphic encryption, we prefer to reduce the multiplicative depth, albeit at the cost of more overall operations.

Avoid bootstrapping if possible. Given that the bootstrapping operation is still expensive compared to other operations, it is preferable to avoid this operation and use leveled schemes instead, if it fits the desired application. In applications where the function to be evaluated is known beforehand, this is possible. Based on the multiplicative depth of the desired function, we set the parameters of the cryptosystem such that correctness is guaranteed with high probability.

Managing key material. Another consideration is the overhead associated with the maintenance operations such keyswitching and relinearization, on top of the computational cost. FHE operations require evaluation keys, which are needed to perform these maintenance operations on FHE ciphertexts. While in some applications, the existence of these keys is assumed [11, 13, 107], it is not an easy condition to satisfy in other applications. For example, if a client is only making a single request to the server, the keys are a large communication burden [124]. Moreover, these keys must be stored by the server for each potential client, introducing significant server-side overhead and adding complexity during deployment.

1.2 Thesis Statement and Contributions

Given the challenges of real-world deployments of protocols using FHE, we pose the following research question:

How can we have efficient private computation protocols, based on homomorphic encryption, which are suitable for real-world use cases?

In response to this research question, we propose new private computation protocols using homomorphic encryption. Our constructions not only improve performance, in terms of communication and computation costs, but also address hurdles in deployment and real-world use cases. Through these works, we identify shortcomings of existing approaches for private computation using homomorphic encryption and offer solutions to these problems. In the following sections, we briefly describe each of our constructions, the techniques used to improve performance, and the advantages they provide for real-world deployments. Table 1.1 gives a summary of the advantage of each construction, aside from the improvements in communication and computation cost.

Table 1.1: The advantage of each of our constructions, aside from improving the speed and communication.

Work	Protocol	Advantages
Level Up	Private Decision Tree Evaluation	High-precision comparisons
PEPSI	(Circuit) Private Set Intersection	Functions of intersection
InsPIRe	Private Information Retrieval	No offline phase / setup

1.2.1 Level Up

Our first contribution is a protocol for non-interactive private decision tree evaluation (PDTE) that supports arbitrary precision, but is also optimized for leveled homomorphic encryption. In PDTE, the client has an input attribute vector x, and the server holds a decision tree T. The objective is to evaluate the server's decision tree on the client's input, i.e., $T(x)$, such that the server learns nothing about the client's input and the client learns nothing about the server's decision tree, other than $T(x)$.

We focus on non-interactive protocols, where the server performs the computation without the client's involvement, other than providing the input. Non-interactive protocols

are helpful for resource-constrained client devices since the client does not need to remain online or participate in the computation. The primary tool in non-interactive constructions is HE [18, 53, 82, 163].

Decision tree evaluation consists of two primary procedures, comparisons and tree traversal. Hence, to privately evaluate a decision tree, we need a private equivalent of both of these procedures [99].

A private comparison operator can compare an encrypted value with an unencrypted value or two encrypted values. Prior work in PDTE proposed private comparison operators for use in decision tree evaluation, but was limited in the precision they supported. To use existing approaches in the context of decision tree evaluation, we would have to reduce the precision of the comparisons performed in the decision tree, which in turn affects accuracy. Alternatively, we can retrain a model with the given precision requirements, but this results in additional server-side overhead, and is not possible for applications where a specific model is guaranteed (e.g., certification for a specific model). In our work, we propose a new private comparison operator which we denote as a Range Cover Comparison (RCC) which can support comparisons with arbitrary precision. RCC is based on an efficient equality operator that we proposed in previous work [121].

For tree traversal, we use a protocol which we denote as SumPath, which performs the traversal using only homomorphic additions, resulting in a very efficient private tree traversal. While the size of the output of tree traversal is larger using SumPath, the additional overhead we incur is minimal due to our use of homomorphic encryption schemes which operate over vectors.

Using RCC and SumPath, we construct an efficient private decision tree evaluation protocol that can support large trees with over 1000 nodes in less than 2 seconds. Along the way, we make improvements to existing constructions as well, and perform a comprehensive comparison between our constructions and existing work.

1.2.2 PEPSI

As our second contribution, we propose a protocol for private set intersection (PSI) in the unbalanced setting. PSI is a protocol that computes the intersection of two sets while ensuring that nothing beyond the intersection is revealed. A specific form of PSI is unbalanced PSI, in which one party, typically the client, has a substantially smaller set than the second party, the server. Moreover, the client is often a resource-constrained device like a mobile phone or an IoT device with limited network connectivity. Consequently, it benefits the client by minimizing network communication, reducing the number of network round

trips, and offloading computational tasks to the server. In some cases, the goal of the client and server is to compute functions of the intersection, rather than to learn the intersection itself. Circuit PSI is the term used when a function of the intersection is calculated rather than the intersection itself.

We propose PEPSI, an efficient, non-interactive circuit PSI protocol using homomorphic encryption. In PEPSI, the client encrypts its elements using an Homomorphic Encryption (HE) scheme and sends them to the server, which compares the elements homomorphically. The output of each comparison is binary and is used to compute functions. PEPSI does not restrict the function that can be evaluated, adds no communication or computation burden on the client to compute the function, and has communication complexity proportional to the smaller set size.

Our approach also overcomes the impractical runtimes associated with the DiPSI [94] protocol from the literature. We address the slow comparisons by using the efficient constant-weight equality operator [121], permutation-based hashing, and cuckoo hashing. Altogether this reduces the size of the elements that need to be compared such that PEPSI is over four orders of magnitude faster than DiPSI while requiring 90% less communication. For example, PEPSI can compute the intersection of 1024 and one million 32-bit elements in under one second with less than 5 MB of communication.

1.2.3 InsPIRe

In our third work, we address the impractical offline costs of Private Information Retrieval (PIR) using our new construction, InsPIRe. Private information retrieval (PIR) is a protocol that enables users to privately query entries from a public database held by a server. The generic definition of PIR makes it useful in many applications where a client needs private access to a database such as password lookup [111, 160], private blocklist [102], private web search [17, 58, 85], private file systems [124], private streaming [80], private auditing [86], and more.

The performance of PIR has improved significantly in recent years, both in terms of communication and computation costs. However, when deploying PIR in practice, we observe that the limitations of applications are incompatible with the design of PIR protocols. One such example is the use of PIR in IPFS, proposed by Mazmudar et al. [124]. The authors observed that many of the existing PIR protocols relied on an offline phase between the client and server, either to exchange cryptographic keys [2, 4, 9, 11, 33, 125, 132] or to transmit a database-dependent hint [59, 86]. Neither of these approaches is compatible

with the IPFS setup and other applications where an offline setup between the client and server is not possible.

In our work, we propose a new PIR protocol called InsPIRe in the server-side preprocessing model. In this model, there is no communication between the client and server in the offline phase and only the server is required to perform a one-time preprocessing in preparation for the client's query. InsPIRe is fast and has low communication costs, while not requiring any offline communication between the client and server. Our constructions are proposed in the Common Reference String (CRS) model, where all clients and the server have access to a global random string. Under this assumption, we fix the random components of the LWE/RLWE ciphertexts and perform preprocessing on these components. When the client issues a query, we perform a very efficient online phase to respond to the client's query.

Within InsPIRe, we propose two new techniques which are of independent interest. First, we propose a protocol for ring packing with preprocessing which we denote as InspiRING. This protocol packs a vector of LWE ciphertexts into one RLWE ciphertext, with a focus on preprocessing. We show through our evaluation that InspiRING is faster than existing protocols for ring packing, whilst requiring less cryptographic key material. Second, we propose a technique for reducing the size of the PIR response by performing polynomial evaluation. Using a combination of these techniques, we construct InsPIRe and its variants, which are suitable for performing PIR over databases of any size with any entry sizes.

Compared to existing protocols in the same model [110, 126], InsPIRe is both faster and requires less communication per query. For example, for PIR over a 1 GB database, InsPIRe requires less than 480 KB of communication while related work requires over 800 KB of communication.

Chapter 2

Background

2.1 Homomorphic Encryption

Homomorphic Encryption (HE) is a form of encryption that permits computation on the data while in encrypted form. Levelled homomorphic encryption (LHE) schemes permit computation of circuits with a limited multiplicative depth [29, 71]. The parameters of the cryptosystem are chosen based on the multiplicative depth. Hence, we try to design algorithms with a lower multiplicative depth to enhance performance. A fully homomorphic encryption (FHE) scheme permits an unlimited amount of operations with the help of bootstrapping [48, 65]. However, FHE is more computationally expensive and requires large cryptographic keys for setup.

Fan-Vercauteren (FV) Cryptosystem. The Fan-Vercauteren (FV) cryptosystem [71] (also referred to as BFV) is a lattice-based homomorphic cryptosystem. An FV ciphertext is an array of polynomials, each from $R_q = \mathbb{Z}_q[X]/(X^N+1)$, where q is called the *coefficient modulus*. In the simplest case, the ciphertext is only two polynomials. Let $\mathcal{C}$ denote the ciphertext space. N and q determine both the security parameter and how many homomorphic operations can be performed on ciphertexts before decryption is necessary. Inputs in this cryptosystem can be encoded in two formats. Table 2.1 shows the two encoding types and the corresponding operations that can be performed.

Microsoft SEAL [154] implements the FV cryptosystem and supports all the operations mentioned in Table 2.1. We use the SEAL library for our implementations in this work.

Table 2.1: Different encodings for the FV cryptosystem. In all operations, we have $c_1(x), c_2(x) \in \mathcal{C}$ that encrypt $m_1(x), m_2(x)$, respectively. Operations over plaintext polynomials happen in R_p. $\oplus$ and $\otimes$ denote element-wise addition and multiplication modulo p between two vectors. Rotate_k denotes the circular right rotation of a vector by k slots.

Encoding	**Plaintext Space**	**Operation**		
Polynomial	$R_p = \frac{\mathbb{Z}_p[X]}{X^N+1}$	Polynomial Add	$c_1(x), c_2(x) \in \mathcal{C}$	$m_1(x) + m_2(x)$
		Plain Polynomial Mult.	$c_1(x) \in \mathcal{C}$, $m_2(x) \in R_p$	$m_1(x)m_2(x)$
		Polynomial Mult.	$c_1(x), c_2(x) \in \mathcal{C}$	$m_1(x)m_2(x)$
		Oblivious Expansion [11, 38]	$c_1(x) \in \mathcal{C}, k \in \mathbb{N} \cup \{0\}$	Coefficient of x^k in $m_1(x)$
Batched (SIMD)	$\mathbb{Z}_p^{N/2}$	SIMD Add	$c_1(x), c_2(x) \in \mathcal{C}$	$m_1(x) \oplus m_2(x)$
		Plain SIMD Mult.	$c_1(x) \in \mathcal{C}$, $m_2(x) \in \mathbb{Z}_p^{N/2}$	$m_1(x) \otimes m_2(x)$
		SIMD Mult.	$c_1(x), c_2(x) \in \mathcal{C}$	$m_1(x) \otimes m_2(x)$
		Circular (Right) Rotation	$c_1(x) \in \mathcal{C}, k \in \mathbb{N}$	$\text{Rotate}_k(m_1(x))$

2.2 Constant-weight Encoding

Constant-weight equality operators, proposed by Mahdavi and Kerschbaum, are equality operators with a constant multiplicative depth, independent of the bitlength of the operands [121]. To use the constant-weight equality operator, numbers are represented as constant-weight codes with Hamming weight h. Mahdavi and Kerschbaum offer a function for encoding numbers as constant-weight codes. We use this function with the added option of encoding a null value. This is useful in our case since we may need to encode null elements as well. Null elements are encoded as the all-zero string. An equality check between two null values will also return zero (non-match). We include the adjusted algorithm from the work of Mahdavi and Kerschbaum in Algorithm 1. $CW(\ell, h)$ denotes the set of constant-weight codes with length ℓ and Hamming weight h. $[n]$ denotes the set $\{0, 1, \cdots, n-1\}$. To compare constant-weight codewords, we use the arithmetic constant-weight equality operator in this work, which has a multiplicative depth of $1 + \log_2 h$. We use this operator to be able to compare many pairs of numbers simultaneously, and in a SIMD fashion. This operator is shown in Algorithm 2.

Algorithm 1 `CW-Encode` [121]

Input: $x \in [2^n] \cup \{\texttt{Null}\}$, $\ell, h \in \mathbb{N}$ such that $\binom{\ell}{h} \geq 2^n$

if x==`Null` **then**
  return 0^ℓ
$r \leftarrow x$, $h' \leftarrow h$, $y \leftarrow 0^\ell$
for $\ell' = \ell - 1, ..., 1, 0$ **do**
  if $r \geq \binom{\ell'}{h'}$ **then**
    $y[\ell'] = 1$
    $r = r - \binom{\ell'}{h'}$
    $h' = h' - 1$
  if $h = 0$ **then** break
return y

Output: $y \in CW(\ell, h) \cup \{0^\ell\}$

Algorithm 2 Arithmetic Constant-weight Equality Operator [121]

algorithm ARITH-CW-EQ-OP(x, y) // $x, y \in CW(\ell, h) \cup \{0^\ell\}$
  $h' = \sum_{i \in [\ell]} x[i] \cdot y[i]$
  $e = 1/h! \cdot \prod_{i \in [h]} (h' - i)$
return e // $e \in \mathbb{B}$

Chapter 3

Level Up: Private Non-Interactive Decision Tree Evaluation using Leveled Homomorphic Encryption

3.1 Introduction

Decision trees are a well-known ML algorithm which are still used widely in many tasks due to their simplicity, interpretability, and ease of training [83, 149]. Private Decision Tree Evaluation (PDTE) is a protocol for providing a prediction using a private decision tree hosted by a server on a private input provided by a client. At the end of the protocol, the server learns nothing about the client's input (input privacy), and the client learns nothing about the server's decision tree (model privacy) other than the result of the inference and some hyperparameters.

One set of solutions is interactive, where the client and server exchange messages [20, 24, 158, 164] in multiple rounds. These solutions are based on tools such as multi-party computation, secret-sharing and garbled circuits [99]. Another category of solutions is non-interactive approaches, where the client can submit the query and go offline until the response is ready. This is great for the setting where the client lacks computational power or suffers from limited bandwidth. All existing solutions to non-interactive PDTE use leveled or fully homomorphic encryption [5, 53, 117, 163]. Solutions requiring leveled homomorphic encryption select parameters that depend on the comparison's precision. This limits the scalability of the solution significantly [117, 164]. Solutions using fully homomorphic encryption do not suffer the same issue, but individual operations are inefficient due

to the expensive bootstrapping procedure. The efficiency problem is exacerbated because some fully homomorphic schemes do not support SIMD operations, so inferring multiple samples in parallel is not possible.

This work proposes two non-interactive PDTE protocols using leveled homomorphic encryption schemes. We propose a protocol where the multiplicative depth of the entire PDTE protocol does not depend on the tree's structure or the number of client attributes, making it more scalable and practical. Our solution can scale to trees with over 1000 decision nodes and 100 client attributes. At the heart of our proposed protocols are efficient non-interactive comparison operators. First, an extension of XCMP by Lu et al. [117], which we denote as Extended XCMP (XXCMP). This extension supports arbitrary precision and is implemented using automorphisms in the SEAL library. Second, is a comparison operator based on the concept of range covers [97, 155], combined with constant-weight equality operators proposed by Mahdavi and Kerschbaum [121]. We denote this comparison operator as Range Cover Comparison (RCC). Both operators are implemented with FV, an RLWE-based cryptosystem, but in different modes of operation [71]. Comparison operators are the core building block in all non-interactive PDTE protocols [5, 18, 53, 117, 164]. Our proposed comparison operators can efficiently compare numbers with arbitrary precision. In contrast, previous work is limited in precision due to efficiency. XCMP is limited to 13 bits, while the operator used by Cong et al. [53] can only compare 11-bit numbers. The comparison operator of Tueno et al. [163] can compare numbers with arbitrary precision. However, the parameters of the leveled FHE scheme grow with bit precision, limiting the solution's efficiency for high-precision inputs. Our evaluation shows that XXCMP and RCC are up to 100 times faster than the operators proposed by Tueno et al. Since our comparison operators perform comparisons with arbitrary precision, models such as decision trees need not be retrained with low precision to enable private inference.

We use the SumPath algorithm described in Section 3.2.3 to evaluate the paths in the decision tree. SumPath requires no multiplications and hence does not increase the multiplicative depth of the circuit. By combining SumPath with XXCMP and RCC, we propose two new PDTE protocols, XXCMP-PDTE and RCC-PDTE, which use RLWE-based cryptosystems in two different modes of operation. While using RLWE-based cryptosystems with SIMD support has previously been proposed [163], it was only to infer multiple samples in parallel and reduce amortized time. In contrast, our work uses SIMD operations to not only perform multiple inferences but also to speed up even a single inference, which reduces client latency. This approach also allows the client to pack more information into fewer ciphertexts, reducing the overall communication between the client and server when only one inference is performed.

In our evaluation, we train decision trees with different bit precision over commonly

used UCI datasets [64]. Our evaluation shows that XXCMP-PDTE and RCC-PDTE are up to 5 times faster than SortingHats [53], which is a state-of-the-art solution for PDTE. This advantage increases when inferring many samples in parallel. The performance of PDTE, i.e., the communication and computation cost, depends mainly on three factors: precision, the number of decision nodes, and the number of client attributes. We perform an ablation study over these parameters using synthetic decision trees to demonstrate the dependency between these parameters and the performance. Our experiments show that the number of client attributes affects communication and the number of decision nodes affects computation, while bit precision influences both metrics. Moreover, our experiments show that RCC-PDTE is the most scalable solution. It outperforms all other solutions when the number of decision nodes grows, and the number of client attributes increases. Specifically, it can infer decision trees with over 1000 decision nodes and 16-bit precision in under 2 seconds. SortingHats requires more than 10 seconds to evaluate such a decision tree with only 11 bits of precision.

In summary, our contributions are as follows:

- Two non-interactive comparison protocols which we denote as XXCMP and RCC, that can compare numbers with arbitrary precision using leveled homomorphic encryption.
- Two non-interactive PDTE protocols XXCMP-PDTE and RCC-PDTE.
- Evaluation of proposed comparison operators with state-of-the-art protocols.
- Ablation of PDTE over the number of client attributes and the size of the decision tree, which shows RCC-PDTE to be the most scalable solution. Our experiments show that RCC-PDTE can evaluate decision trees with up to 1000 nodes in less than 2 seconds.

In Section 3.2, we review necessary background material such as homomorphic encryption, decision trees and private decision tree evaluation, range covers and tree traversal algorithms. We also compare the properties of related work on non-interactive comparison and non-interactive PDTE in Section 3.3. We describe our constructions, XXCMP and RCC, two non-interactive private comparison protocols in Section 3.4. In Section 3.5, we describe our PDTE protocols, XXCMP-PDTE and RCC-PDTE. In Section 3.6, we compare XXCMP and RCC with other non-interactive comparison protocols and then compare PDTE protocols with our constructions. We conclude in Section 3.7 with a discussion on limitations and future work.

Table 3.1: Summary of notation used in Chapter 3

Symbol	Description
λ	Security parameter
N	Polynomial modulus degree
p	Plaintext modulus
$[[x]]$	Encryption of x
h	Hamming weight
ℓ	Constant-weight code length
n	Bit Precision
$[n]$	$\{0, 1, \cdots, n-1\}$
$\mathbb{B}$	$\{0, 1\}$
$\mathbb{I}$	Indicator function

Symbol	Description
$\mathcal{M}$	$(\mathcal{T}, \mathbf{c}, \mathbf{a}, \mathbf{t}, \mathbf{v})$
$\mathcal{T}$	Decision tree nodes
$\mathcal{D}$	Set of internal decision nodes in $\mathcal{T}$
$\mathcal{L}$	Set of leaf nodes in $\mathcal{T}$
m	Number of decision nodes ($\lvert\mathcal{D}\rvert$)
$\mathbf{x}$	Client attribute vector
$\mathbf{a}$	Node to attribute mapping
$\mathbf{t}$	Threshold value function
$\mathbf{v}$	Leaf value function
d	Depth of the decision tree
k	Number of classification labels

3.2 Background and Related Work

Table 3.1 summarizes the notation used in this chapter. Background on homomorphic encryption and constant-weight equality operators are provided in Chapter 2.

3.2.1 Non-interactive Private Comparison

A comparison operator is a function $f : D \mapsto \{0, 1\}$ such that for $x, y \in D$,

$$f(x, y) = \mathbb{I}[x \leq y] \tag{3.1}$$

A private comparison is a protocol where two inputs, x and y, are provided, such that one or both of them are encrypted. The output of the protocol is $f(x, y)$ in encrypted form.

In this work, we are particularly interested in non-interactive solutions to this problem. This is useful in protocols where the encrypted input is provided by a lightweight client which may go offline after providing the input. Below we describe three non-interactive private comparison protocols from the literature.

Folklore Private Comparison. The folklore comparison algorithm compares two n-bit numbers in binary format. This is identical to how binary numbers are compared in the clear, with some adaptations to make it easier to compute using HE. The multiplicative depth of the algorithm is $1+\log_2(n+1)$, which poses a burden to compute efficiently using HE. Algorithm 3 shows this algorithm which assumes that inputs are binary vectors and all operations are element-wise. We also use the RightShift$_k$ function, which logically shifts the contents of any vector or bitstring by k positions. Additions and multiplications can also be replaced with XOR and AND operations.

Algorithm 3 Folklore Comparison($\mathbb{I}[a < b]$)

Input: $a, b \in \{0,1\}^n$

1: $\theta_{eq} \leftarrow 1 - (a-b)^2$ // Check equality of corresponding bits
2: $\theta_{gt} \leftarrow (1-a) \cdot b$ // Find bit positions where a is less than b
3: $\theta_{\text{PrefixEq}} = \theta_{gt} \cdot \prod_{k=0}^{n-1} \texttt{RightShift}_k(\theta_{eq})$
4: $\theta \leftarrow \sum_i \theta_{\text{PrefixEq}}[i]$

Output: $\theta \in \{0,1\}$ // $\theta = \mathbb{I}[a < b]$

Algorithm 3 works by finding the first index at which a is equal to 0 and b is equal to 1, indicating that a is less than b. θ_{eq} calculates the equality of corresponding bits using one multiplication. θ_{gt} computes the indices i at which $a[i] = 0$ and $b[i] = 1$. Lastly, θ_{PrefixEq} compute θ_{eq} and θ_{gt} such that for each index i, $\theta_{\text{PrefixEq}}[i] = 1$ if all previous are equal and θ_{gt}. At most one of the elements of θ_{PrefixEq} is equal to 1, so a the end we sum the elements of θ_{PrefixEq} to get the final result. This specific implementation of the comparison operator makes it optimal for SIMD implementation.

Variations of folklore comparison have been implemented in many works using leveled homomorphic encryption [82, 163].

XCMP. Lu et al. [117] proposed a comparison operator called XCMP which compares two encrypted numbers using leveled HE. Cong et al. introduced a variation using TFHE where one operand is unencrypted [53]. This relaxation reduces the runtime of the comparison. Algorithm 4 shows this variant of XCMP which compares an encrypted input, a, provided by the client with an unencrypted b provided by the server. The output is $\mathbb{I}[a \leq b]$. Inputs are encoded as RLWE ciphertexts, i.e., Polynomial encoding from Table 2.1.

Algorithm 4 XCMP($\mathbb{I}[a \leq b]$) [117]

algorithm XCMP(A, b) // $A = X^a, a, b \in [N]$
  $T \leftarrow \frac{1}{2} X^{-b} \cdot (1 + X + \cdots + X^{N-1})$
  $R \xleftarrow{\$} R_p$ and $R[0] = 1/2 \mod p$
  $C = A \cdot T + R$
  return C

The result of the comparison is in the constant term of C.

Iliashenko and Zucca [88]. The comparison function of $x, y \in \mathbb{Z}_p$ can be represented as either a bivariate polynomial of the two inputs or a univariate function of the difference. Iliashenko and Zucca showed how to exploit the structure of these polynomials to efficiently evaluate the comparison function. Their main observation was that these polynomials have many zero coefficients which can be ignored. Based on this observation, they showed that comparison using FHE schemes that operate over arithmetic circuits can be efficient.

3.2.2 Decision Trees and PDTE

Decision Trees. A decision tree is a classification algorithm which classifies input data by sequentially checking a series of criteria. The simplest form of a decision tree is represented by a binary tree where each internal node compares an attribute with a threshold. Each leaf is assigned a classification value (or simply a class). To classify a data point, we start at the root of the tree, perform a comparison and move to the right or left child, depending on the result of the comparison. We continue until we reach a leaf and output the class of that leaf.

More formally, a decision tree consists of a set of nodes $\mathcal{T} = \mathcal{D} \cup \mathcal{L}$, where $\mathcal{D}$ and $\mathcal{L}$ are the set of decision nodes and leaf nodes, respectively. There also exists an attribute vector, $\mathbf{x}$, and four functions:

- $\mathbf{c} : \mathcal{D} \mapsto \mathcal{T}^2$ maps each decision node to its left and right children.
- $\mathbf{a} : \mathcal{D} \mapsto [|\mathbf{x}|]$ maps decision nodes to attribute indices.
- $\mathbf{t} : \mathcal{D} \mapsto \mathbb{Z}$ maps internal nodes to threshold values.
- $\mathbf{v} : \mathcal{L} \mapsto \mathbb{Z}$ maps leaf nodes to classification values.

We denote the tuple $\mathcal{M} = (\mathcal{T}, \mathbf{c}, \mathbf{a}, \mathbf{t}, \mathbf{v})$ as the decision tree model.

Private Decision Tree Evaluation (PDTE). Private Decision Tree Evaluation is a protocol between a server and a client where the server holds the model, $\mathcal{M}$, and the client holds the attribute vector, $\mathbf{x}$. The goal is to infer the tree on the client's attribute vector such that the server does not learn anything about the client's input. Moreover, the client should not learn anything about the server's private decision tree other than the classification result and some hyperparameters.

The client may try to steal the model with black-box access to an API through carefully crafted queries to the server [93, 96, 140, 156, 161]. Such an attack is outside the scope of this work, but defences against such attacks is an active area of research [93, 161].

3.2.3 Tree Traversal

One of the main steps in all private decision tree evaluation protocols is traversing the tree from the root to the leaves, as discussed by Kiss et al. [99]. We denote the leaf holding the result of the prediction as the *result leaf* and the value of that leaf as the *result value*. If all comparisons in the decision nodes are computed as a binary output, there are two methods to aggregate the results and compute the result leaf and value. We denote these two methods as *Path Conjugation* and *SumPath*. For each method, the edges in a tree are annotated as shown in Figure 3.1.

In Path Conjugation, for each leaf in the tree, all values on the edges between the root and that leaf are conjugated. Consequently, the result leaf will have a value of one, and all other leaves will have a value of zero. This process can be computationally optimized by reusing computations in inner nodes [163]. Some works, such as those by SortingHats and Tueno, utilize this approach with fully homomorphic encryption and the CMux gate in TFHE [53, 164]. However, this approach is not practical when using leveled homomorphic encryption, given that the multiplicative depth of the circuit will depend on the depth of the tree.

On the other hand, the SumPath method has been proposed previously for use with additive encryption [99, 158] or leveled homomorphic encryption in an arithmetic field [164]. Edges are annotated as indicated in Figure 3.1. For each leaf in the tree, the sum of all the edges in the tree from the root to that leaf is assigned to that leaf. Only the result leaf will have a sum of zero, and all others will be non-zero. By returning the sum on each leaf, plus the masked value on each leaf, the client can infer the correct result value. The advantage of this approach is that no multiplications or conjugations are required, which makes it computationally inexpensive and does not increase the multiplicative depth. This,

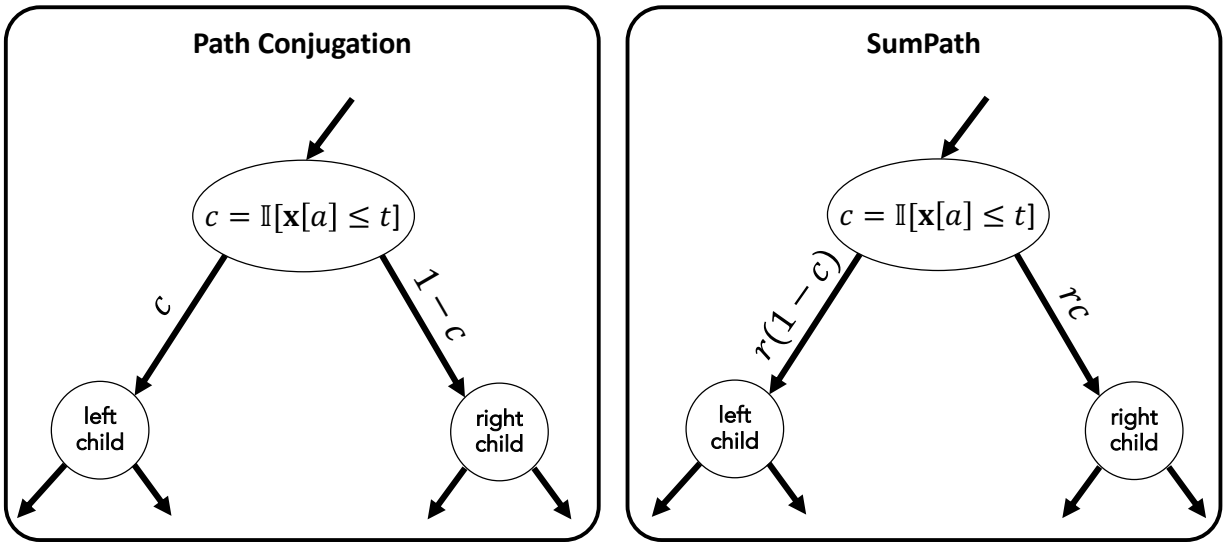


Figure 3.1: Labelling of edges in two tree traversal methods, Path Conjugation and SumPath. $\mathbf{x}$ is the client attribute vector. a, and t are the attribute index and threshold for the parent node, respectively. In SumPath, r is either a random number or 1.

however, comes at the cost of having download cost proportional to the number of leaves in the tree.

3.2.4 Range Covers & Point Encoding

Range covers are a method to represent intervals in a manner that is concise and easy to use in secure computation [97, 155]. Let T be a binary tree of internal nodes of prefixes numbers in $[2^n]$ and leaf nodes of the elements in $[2^n]$. The children of a prefix p are $p|0$ and $p|1$, and the root is the empty prefix. A *range cover* $RC_n(a, b)$ is a set of nodes in T, such that its set of children at the leaf level are all elements in the range $[a, b]$, where $a, b \in [2^n]$. The *best range cover* contains at most $2 \log n$ nodes with at most 2 nodes at each level of T (excluding the root level). A uniform range cover is a modification of the best range cover such that each level of the tree contains exactly 2 nodes [97, 155]. This is achieved by padding the best range cover with dummy nodes at all levels if fewer than two nodes are chosen. The advantage of a uniform range cover is that the size is independent of the interval, which is a requirement in private protocols. For every range $[a, b]$ there exists a best range cover and uniform range cover.

A *point encoding* $PE_n(c)$ of an element $c \in [2^n]$ is the set of nodes from the leaf c to

the root (except the root itself). A point encoding consists of n nodes with one node at each level of T (except the root level). We denote as $RC_n(a,b)[i]$ and $PE_n(c)[i]$ the i-th element of a range cover or encoding, respectively.

We can test the relationship $c \in [a,b]$ given $RC_n(a,b)$ and $PE_n(c)$ by checking to see if there are any common prefix nodes between the range cover and point encoding. We give the full algorithm on how to do this check in Algorithm 5.

In this notation, $RC_n(a,b)$ contains $2n$ prefix nodes and we assume the prefix nodes from level i of the prefix tree are in $RC_n(a,b)[2i]$ and $RC_n(a,b)[2i+1]$ (and some levels may be empty). $PE_n(c)$ contains n prefix nodes and the prefix node from level i of the prefix tree is in $PE_n(c)[i]$. The inclusion algorithm requires $2n$ equality check at most.

Algorithm 5 RC/PE Inclusion

Input: $RC_n(a,b), PE(c,n)$

1: $\theta_{included} = 0$
2: **for** $i \in [n]$ **do**
3: $\quad \theta_{included} = \theta_{included} + \mathbb{I}\left[RC_n(a,b)[2i] == PE_n(c)[i]\right]$
4: $\quad \theta_{included} = \theta_{included} + \mathbb{I}\left[RC_n(a,b)[2i+1] == PE_n(c)[i]\right]$

Output: $\mathbb{I}\left[c \in [a,b]\right]$

Figure 3.2 shows an example: the prefix tree for $[0, 2^3-1]$ with the range cover for $[0,4]$, indicated by the shaded boxes, and point encodings for 2 and 6. We can see that $RC_3(0,4)$ and $PE_3(2)$ have the prefix node corresponding to 0 in common whereas $RC_3(0,4)$ and $PE_3(6)$ have no prefix node in common.

3.3 Related Work on PDTE

3.3.1 Interactive PDTE

Decision trees can be privately evaluated using two-party computation (2PC) or using a combination of 2PC and homomorphic encryption. There are also solutions which use tools such as secret sharing, garbled circuits, and oblivious transfer [20, 31, 99, 116]. Unlike homomorphic encryption, 2PC is usually communication-bound, i.e., reducing the communication complexity or rounds is necessary to increase efficiency. However, since this work

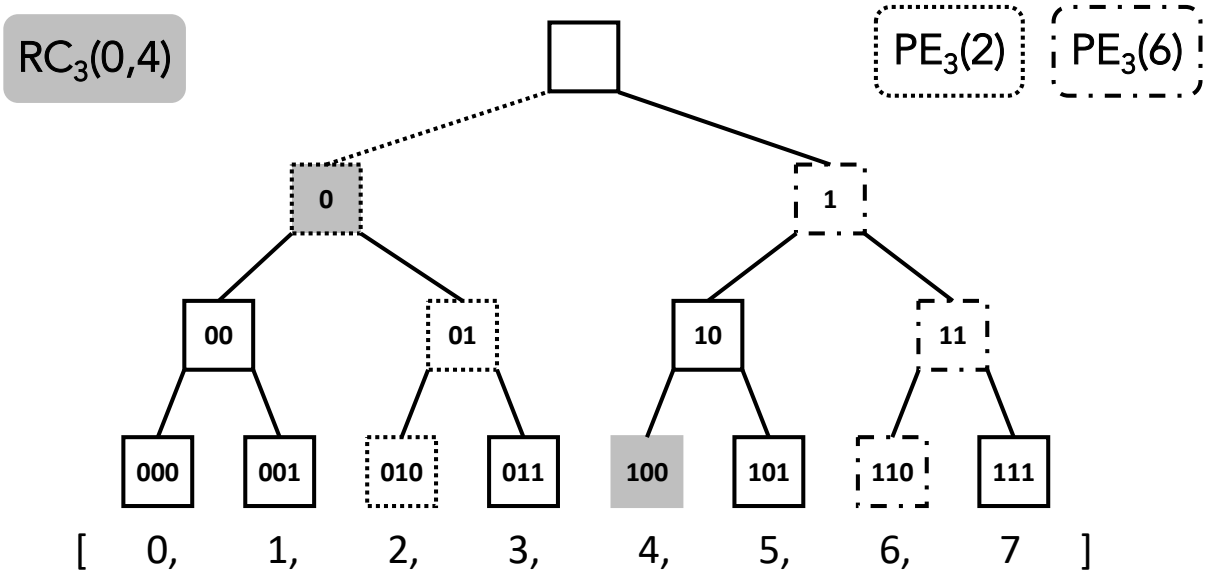


Figure 3.2: Range cover for $[0, 4]$ (shaded boxes) and point encoding of 2 and 6 in domain $[0, 2^3 - 1]$. In this example, $RC_3(0, 4)$ and $PE_3(2)$ have a prefix node in common, given that $2 \in [0, 4]$ but $RC_3(0, 4)$ and $PE_3(6)$ have no prefix node in common since $6 \notin [0, 4]$.

focuses on non-interactive decision tree evaluation, we provide only a few selected examples of the work in private decision trees using 2PC.

2PC, e.g., using Yao's garbled circuits, implements a constant-round protocol. Brickell et al. [31] present an improved constant-round protocol using 2PC and homomorphic encryption. Kiss et al. [99] later surveyed this and a number of other protocols and identified several design options and new combinations of these protocols systematizing the protocols.

However, 2PC usually leads to communication complexity which is exponential in the tree depth since all branches need to be evaluated. Using different techniques, one can also reduce the communication complexity. Tueno et al. [164] propose to use oblivious RAM (ORAM). Bai et al. [20] propose to use additive homomorphic encryption or pseudo-random functions.

Table 3.2: Properties of Non-interactive Private Decision Tree Evaluation Protocols. a is the number of client attributes, d is the depth of the tree, $\mathcal{D}$ is the set of internal decision nodes, and $\mathbf{x}$ is the client attribute vector. * The precision of PROBONITE depends on the choice of parameters for the LWE scheme but is typically less than 8 bits. ** XCMP-PDTE uses a trusted third-party to perform the homomorphic computation.

	PROBONITE [18]	XCMP-PDTE**	PDT-Bin [163]	PDT-Int [163]	SortingHats [53]	XXCMP-PDTE	RCC-PDTE
Supports Unbalanced	×	✓	✓	✓	✓	✓	✓
Attribute Selection	PIR	Clear	Clear	Clear	Clear	Clear	Clear
Comparison	PBS (CT-CT)	XCMP	Folklore	Lin-Tzeng [114]	XCMP-CT-PT	**XXCMP**	**RCC**
Path Evaluation	CMux	[158]	AND	SumPath	CMux	SumPath	SumPath
Batchable	×	×	✓	✓	×	✓	✓
Bit Precision	$< 8^*$	$N = 12, 13$	n	n	$N = 11$	n	n
Levelled or FHE	FHE	LHE	LHE or FHE	LHE(BGV)	FHE(TFHE)	LHE(FV)	LHE(FV)
# of Comparisons	$O(d)$	$\lvert\mathcal{D}\rvert$	$\lvert\mathcal{D}\rvert$	$\lvert\mathcal{D}\rvert$	$\lvert\mathcal{D}\rvert$	$\lvert\mathcal{D}\rvert$	$\lvert\mathcal{D}\rvert$
Query Complexity	$O(\lvert\mathbf{x}\rvert)$	$O(N\lvert\mathbf{x}\rvert)$	$O(n\lvert\mathbf{x}\rvert)$	$O(n\lvert\mathbf{x}\rvert)$	$O(N\lvert\mathbf{x}\rvert)$	$O(N\lvert\mathbf{x}\rvert)$	$O(\ell n\lvert\mathbf{x}\rvert)$
Mult. Depth	N/A	1	$\log_2 n$	$\log_2 n$	N/A	$\lceil\log_2(n/\log_2 N)\rceil$	$1+\log_2 h$

3.3.2 Non-Interactive PDTE

Interactive protocols are great for computational efficiency but usually require high network activity. Moreover, they require several rounds of interaction between the client and server. This is not a good solution if the client is not strong or has a limited network connection. The client may wish to go offline while waiting for the result in many use cases. Non-interactive approaches to private decision tree evaluation primarily use homomorphic encryption. Some works used leveled homomorphic encryption [5, 117, 164] while others require a fully homomorphic scheme [18, 53, 164]. A decision tree is comprised of many comparisons between client attributes and thresholds, so at the heart of all protocols is an efficient comparison operation. The primary approach in many works is to reduce the precision of the comparison to make it efficient [53, 117]. For this to be possible, the decision tree evaluation has to be fine-tuned for low precision. Quantization-aware training is one approach to this. If a model already exists with high precision, it can not be used as it is and has to be retrained. Table 3.2 summarizes the properties of related work. We also provide a short description of how each protocol works.

XCMP PDTE [117]. Lu et al. proposed a non-interactive PDTE protocol based on XCMP, described in Section 3.2.1. All comparisons are performed using XCMP, and all paths are evaluated using the same method as the work of Tai et al. [158]. This protocol's main limitation is that the precision of the comparison which they use is limited to 13 bits, i.e., $\log_2 N$. The protocol's design was such that the model holder could also encrypt the

thresholds of the model and delegate the evaluation to a third-party server. In this case, the third party would learn the tree's structure but not the thresholds.

PDT-Bin & PDT-Int [163]. Tueno, Boev, and Kerschbaum proposed two PDTE protocols using a binary and arithmetic circuit. In their first construction, denoted as PDT-Bin, numbers are represented using binary encoding and compared using a Folklore comparison. Traversal of the decision tree in PDT-Bin is performed using homomorphic AND operations, ultimately yielding a single result that represents the outcome of the classification.

In addition to PDT-Bin, Tueno, Boev, and Kerschbaum introduced another protocol, PDT-Int, which is based on leveled homomorphic encryption, specifically employing the BGV scheme. For the comparison operator in PDT-Int, they adopted a variation of the Lin-Tzeng [114] protocol, which outputs zero in the case of a match and a random number otherwise. To traverse paths in the decision tree, the SumPath technique is employed, and one value corresponding to each leaf node is returned.

PROBONITE [18]. Azogagh et al. proposed a PDTE protocol named PROBONITE which evaluates only one path of the tree [18]. In contrast to other non-interactive protocols, which perform one comparison for each node in the decision tree and evaluate all paths, PROBONITE only evaluates one path. The protocol starts at the root and traverses one path down the tree. This is done by using two subprocedures: 1) *Blind Array Access*, which is used to select the next attribute with which to compare 2) *Blind Node Selection*, which selects the next node to traverse to based on the result of the comparison. The entire protocol is performed using HE, and the comparison is also performed using the functional bootstrapping capability of TFHE. Given that only one comparison is performed at each level, this greatly improves performance by not performing unnecessary comparisons. The drawback is that since the server does not know which threshold to compare with, it must blindly find the correct threshold with the Blind Array Access subprocedure, which can be computationally expensive. Moreover, the comparison happens between two ciphertexts, as opposed to other approaches which compare encrypted attributes with cleartext thresholds [53, 164]. For privacy, imbalanced trees are padded with redundant nodes to a full, balanced tree so that all queries are equally expensive. The protocol only leaks the depth of the tree and not the number of leaves or any other properties of the tree. There is no public implementation of this work available, so we only resort to the theoretical comparison provided in the Table 3.2.

SortingHats [53]. Cong et al. expanded on the idea of XCMP-style comparisons [117]. XCMP focused on comparing two encrypted numbers, but in PDTE, one operand is usually in the clear, making the comparison operation simpler. Cong et al. proposed a faster comparison operation based on XCMP where only one operand is encrypted and the other is in the clear [53]. We describe this protocol in Algorithm 4. The operation is done with only two polynomial multiplications, which is much cheaper than comparing two encrypted numbers which requires 16 polynomial multiplications [117]. Using their proposed operand, they designed a non-interactive private decision tree evaluation protocol called SortingHats. The authors used TFHE-based FHE to implement their protocol. They also proposed using transciphering to reduce communication between the client and server, a method orthogonal to this work to reduce communication costs. The main limitation of SortingHats is the bit precision, which is currently capped at 11 bits, which is equal to the degree of the polynomial ring in which it operates.

3.4 Our Comparison Operators

In this section, we propose two operators for non-interactive private comparison. First is an extension of the XCMP protocol mentioned in Section 3.2 for larger bit precision. Second, is a novel protocol based on range covers in combination with constant-weight codes.

3.4.1 XXCMP: Extended XCMP Operator

XCMP only supports the comparison of numbers smaller than N. We propose an extension based on XCMP which can support numbers of arbitrary size. We denote this as *XXCMP*. The idea is to represent large numbers in base N. High-order digits are compared first and if they are equal, the next digits are compared. For example, let $a = a_1N + a_0$ and $b = b_1N + b_0$ where $a_i, b_i \in [N]$. Then we use the following identity:

$$\mathbb{I}[a > b] = \mathbb{I}[a_1 > b_1] + \mathbb{I}[a_1 = b_1] \cdot \mathbb{I}[a_0 > b_0]. \tag{3.2}$$

For this, we need an equality operator over numbers encoded as XCMP ciphertexts. Within that algorithm, we use another sub-protocol which checks the equality of two numbers in the XCMP format. This protocol uses the oblivious expansion technique proposed by Angel et al. [11]. We use a modification of this procedure which we denote as

OBLIVIOUS-EXPANSION$(P(X), k)$, for ciphertext $P(X)$ and index k, such that the output is a ciphertext $Q(X) = p_k$ where p_k is coefficient of X^k in $P(X)$.

Algorithm 7 shows the extension of XCMP for numbers smaller than N^2. This algorithm requires one homomorphic multiplication.

XXCMP can compare numbers of arbitrary size. Algorithm 6 shows the general XXCMP algorithm which compares two number $a, b \in [N^k]$, for some known parameter k. The output of the comparison is in the constant coefficient of the plaintext. In the general case, to compare numbers smaller than $n = N^k$, the multiplicative depth of the circuit is $\lceil \log_2 k \rceil$ and requires $k(k-1)/2$ homomorphic multiplications.

Algorithm 6 Computing $\mathbb{I}[a > b]$ using Extended XCMP (XXCMP) for $a, b \in [N^k]$ such that $a = (a_{k-1}, \cdots, a_1, a_0)_N$ and $b = (b_{k-1}, \cdots, b_1, b_0)_N$ where $a_i, b_i \in [N]$

algorithm XXCMP($[[A]], b$) // $A \in R_p^2, b \in [N^2]$
  $([[X^{a_{k-1}}]], \cdots, [[X^{a_1}]], [[X^{a_0}]]) \leftarrow [[A]]$
  for $i \in [k]$ **do**
    $[[gt_i]] \leftarrow \text{XCMP}_0([[X^{a_i}]], b_i)$
  for $i \in [k]$ **do**
    $[[eq_i]] \leftarrow$ Oblivious-Expansion($[[X^{a_i}]], b_i$)
  $[[C]] = \sum_{i=0}^{k-1} [[gt_i]] \cdot \prod_{j=i+1}^{k-1} [[eq_j]]$
  return $[[C]]$

Algorithm 7 Computing $\mathbb{I}[a > b]$ using Extended XCMP (XXCMP) for $a, b \in [N^2]$ such that $a = a_1 N + a_0$ and $b = b_1 N + b_0$ where $a_i, b_i \in [N]$. Polynomial arithmetic is modulo $X^N + 1$.

algorithm $\text{XCMP}_0([[X^a]], b)$ // $a, b \in [N]$
  $T \leftarrow -(1 + X + \cdots + X^{N-b-1})$
  $R \xleftarrow{\$} R_p$ and $R[0] = 0 \mod p$
  $[[C_0]] = [[X^a]] \cdot T + R$
  return C_0

algorithm $\text{XXCMP}_2([[A]], b)$ // $A \in R_p^2, b \in [N^2]$
  $[[X^{a_1}]], [[X^{a_0}]] \leftarrow [[A]]$
  $[[gt_0]] \leftarrow \text{XCMP}_0([[X^{a_0}]], b_0)$
  $[[gt_1]] \leftarrow \text{XCMP}_0([[X^{a_1}]], b_1)$
  $[[eq_1]] \leftarrow$ OBLIVIOUS-EXPANSION($[[X^{a_1}]], b_1$)
  $[[C]] = [[gt_1]] + [[eq_1]] \cdot [[gt_0]]$
  return $[[C]]$

3.4.2 Range-Cover Comparison (RCC) Operator

Assume we have $a, b \in [2^n]$ and we want to compute $\mathbb{I}[a \leq b]$. At a high level, the idea is to use the following statement:

$$a \leq b \iff b \in [a, 2^n - 1] \tag{3.3}$$

which is similar to the RC/PE inclusion problem. However, one end of the interval is always the maximum value. Using this constraint, we define a restricted version of a range cover called the *One-sided Uniform Range Cover (OURC)*. The main difference between a typical range cover and OURC is that an OURC consists of only one prefix node in each level of the prefix tree (including the root level). Hence, the inclusion check requires only $n + 1$ equality checks instead of $2n$. This results in a major performance improvement when running the circuit using HE. Algorithm 8 shows the procedure for computing the OURC. The modified OURC/PE inclusion procedure is shown in Algorithm 8. We include the procedure for calculating the point encoding as well.

Algorithm 8 Calculating OURC and PE

algorithm OURC(x, n) // $x \in [2^n]$
  $\theta_{OURC} \leftarrow [\texttt{Null}] * (n + 1)$
  if $x = 0$ **then**
    $\theta_{OURC}[n] = 0$
    return θ_{OURC}
  $s = 2^n - 1$
  $K \leftarrow \{\}$
  while $s \geq x$ **do**
    Find j such that $x - 1 + 2^j \leq s < x - 1 + 2^{j+1}$
    $\theta_{OURC}[j] = 2^{n-j} - 1 - \sum_{k \in K} 2^{k-j}$
    $s \leftarrow s - 2^j$
    Add j to K
  return θ_{OURC}

algorithm PE(y, n) // $y \in [2^n]$
  return $[y, \lfloor y/2 \rfloor, \lfloor y/2^2 \rfloor, \cdots, \lfloor y/2^n \rfloor]$

algorithm PE-RC-Inclusion$(\theta_{OURC}, \theta_{PE})$
  $\theta_{in} = 0$
  for $i \in [n + 1]$ **do**
    $\theta_{in} = \theta_{in} + \mathbb{I}[\text{OURC}(a, n)[i] == \text{PE}(b, n)[i]]$
  return θ_{in}

As shown in Algorithm 8, $n + 1$ equality checks are required for one RCC comparison. We replace the equality checks in each iteration of the for loop with a constant-weight equality operator of Hamming weight h. All other operations are additions so the total

multiplicative depth of the inequality operator only depends on h. Note that the multiplicative depth does not depend on n, the bit precision of the numbers.

OURC Inclusion using Homomorphic Encryption. In our private comparison protocol, the client, which holds a, sends an encryption of $\text{OURC}(a, n)$ to the server. The input is encoded and encrypted such that the comparison is performed efficiently. More specifically, if $\text{OURC}(a, n) = \{a_0, a_1, \cdots, a_n\}$, then constant-weight encoding of a_i is spread across ℓ ciphertexts. Figure 3.3 shows a visualization of how the packing happens. Each blue box contains all the bits of information about $\text{OURC}(a, n)$. Using this encoding, each prefix node, a_i occupies exactly $n + 1$ slots in each ciphertext. The number of occupied slots does not depend on the parameters of the constant-weight code, i.e., the Hamming weight. Multiple pairs of numbers from the client and server can be compared in parallel.

Algorithm 9 details the procedure to encode $\text{OURC}(a, n)$ across multiple FV ciphertexts in batched mode. We provide the packing method for the point encoding as well, which is used in the comparison protocol. We note that this is not the only method to encode values in FV ciphertexts. We elaborate on other packing methods and why we chose this method in Section 3.7.

Algorithm 9 OURC and PE Encoding

algorithm OURC-ENCODE(a, h, ℓ, n) // $a \in [2^n]$
  $[a_0, a_1, \cdots, a_n] \leftarrow \text{OURC}(a, n)$
  for $i \in [n + 1]$ **do**
    $a'_i = \text{CWENCODE}(a_i, h, \ell)$ // $a'_i \in \mathbb{B}^\ell$
  for $i \in [\ell]$ **do**
    $pt_{OURC}[i] = [a'_0[i], a'_1[i], ..., a'_n[i]]$
  return pt_{OURC}

algorithm PE-ENCODE(b, h, ℓ, n) // $b \in [2^n]$
  $[b_0, b_1, \cdots, b_n] \leftarrow \text{PE}(b, n)$
  for $i \in [n + 1]$ **do**
    $b'_i = \text{CWENCODE}(b_i, h, \ell)$ // $b'_i \in \mathbb{B}^\ell$
  for $i \in [\ell]$ **do**
    $pt[i] = [b'_0[i], b'_1[i], ..., b'_n[i]]$
  return pt_{PE}

Given the encryption of $\text{OURC}(a, n)$, we can now outline the procedure for comparison. Algorithm 10 shows this procedure. The red symbols show the values that are encrypted when the procedure is performed between a client and server.

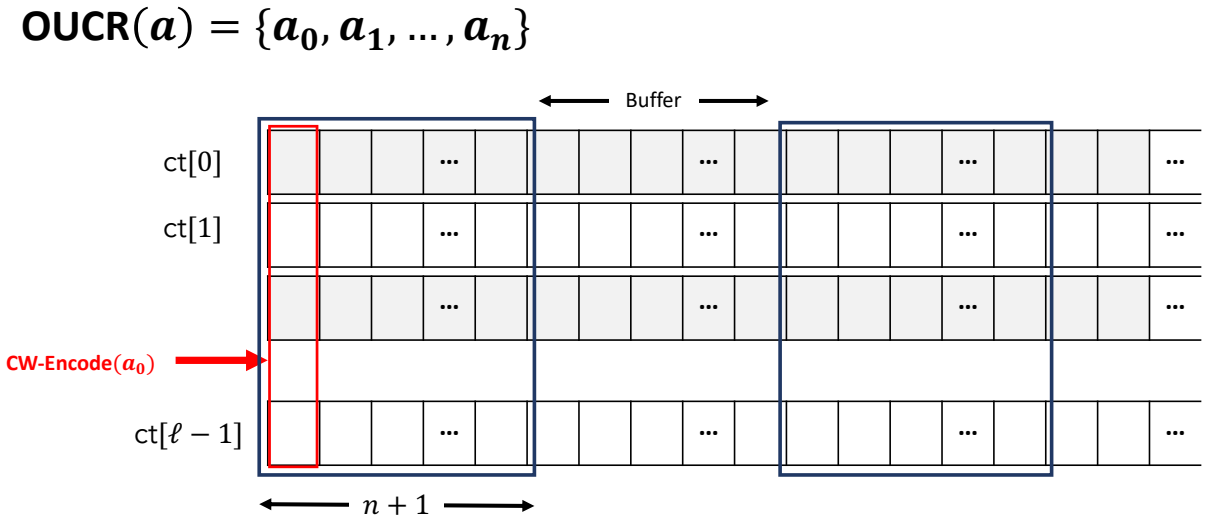


Figure 3.3: Packing OURC in FV ciphertexts in batched mode.

Algorithm 10 RCC Comparison

algorithm RCC-Compare($[[a]], b$)
  $[[a_{ct}]] \leftarrow$ OURC-Encode($[[a]], h, \ell, n$) //Done by client
  $b_{pt} \leftarrow$ PE-Encode(b, h, ℓ, n)
  $[[\theta]] =$ Arith-CW-Eq-Op($[[a_{ct}]], b_{pt}$)
  $[[\theta_{\text{sum}}]] \leftarrow \sum_{i=0}^{n}$ Rotate$_i([[\theta]])$
  $M \leftarrow 0^N$, $M[0] = 1$ //Mask
  $[[\theta_{\text{cmp}}]] \leftarrow [[\theta_{\text{sum}}]] \otimes M$
  return $[[\theta_{\text{cmp}}]]$

3.5 PDTE using Leveled HE

This section describes our two proposed protocols, XXCMP-PDTE and RCC-PDTE. We describe the setup, security model, and details about the two protocols.

3.5.1 Setup and Security Model

Our PDTE protocols work in the client/server model. The server holds a decision tree model, and the client holds a vector of attributes. The goal is for the client to learn nothing about the server's model other than the inference result. The server should learn nothing from the protocol. Our protocol is non-interactive, i.e., the client uploads its query to the server and waits for a response. The client need not be online while the server

processes the query.

Similar to prior work, we work in the semi-honest model, where both the client and server follow the protocol but may try to infer extra information.

Both protocols use homomorphic encryption and assume cryptographic keys such as public keys, evaluation keys, and relinearization keys have been exchanged between the client and server, similar to previous works.

3.5.2 XXCMP-PDTE: XXCMP + SumPath

In XXCMP-PDTE, we combine the XXCMP comparison with the SumPath algorithm. This protocol is implemented using FV ciphertexts in polynomial mode. The protocol has two main parts 1) For each node, compare the correct client attribute to the corresponding node threshold. 2) Run SumPath and get one encrypted result per leaf. Algorithm 11 depicts this algorithm. d.left and d.right denote the left and right exiting edge from d, respectively. The final result can be compressing the leaves into the same ciphertext. Each leaf can occupy one of the coefficients of the final ciphertext.

Variables that are coloured red are encrypted when running the procedure between a server and client.

Algorithm 11 XXCMP-PDTE: PDTE using XXCMP

algorithm XXCMP-PDTE($[[\mathbf{x}]], \mathcal{M}$)
  $(\mathcal{T}, \mathbf{a}, \mathbf{t}, \mathbf{v}) \leftarrow \mathcal{M}$
  for $d \in \mathcal{D}$ **do**
    $[[c]] \leftarrow \text{XXCMP}([[\mathbf{x}[]]\mathbf{a}[d][[]]], \mathbf{t}[d])$
    $d.[[\text{left}]] \leftarrow [[c]]$
    $d.[[\text{right}]] \leftarrow 1 - [[c]]$
  for $\ell \in \mathcal{L}$ **do**
    $[[s(\ell)]] =$ Sum of [[edges]] from root to ℓ
    $r_x, r_y \xleftarrow{\$} \mathbb{Z}_p$
    $[[x(\ell)]] \leftarrow r_x \cdot [[s(\ell)]]$
    $[[y(\ell)]] \leftarrow r_y \cdot [[s(\ell)]] + v(\ell)$
  return $\{([[x(\ell)]], [[y(\ell)]])\}_{\ell \in \mathcal{L}}$

3.5.3 RCC-PDTE: RCC + SumPath

In RCC-PDTE, comparisons are performed using RCC and the tree is traversed using SumPath. One important difference between this protocol and XXCMP-PDTE is that all

comparisons happen in parallel using the batched mode of FV. The results of the comparisons will occupy different slots of the ciphertext. Algorithm 12 outlines this procedure. In the implementation, we also batched the final result into one ciphertext to reduce communication costs.

Algorithm 12 RCC-PDTE: PDTE using RCC

algorithm RCC-PDTE($[[\mathbf{x}]], \mathcal{M}$)
  $\mathcal{T}, \mathbf{a}, \mathbf{t}, \mathbf{v} \leftarrow \mathcal{M}$
  $\mathbf{t}' \leftarrow$ array of thresholds, aligned with client attributes
  $[[c]] \leftarrow$ RCC-Compare($[[\mathbf{x}]], \mathbf{t}'$)
  for $d \in \mathcal{D}$ **do**
    $[[c']] \leftarrow$ Rotate $[[c]]$ to get d to first slot
    $d.[[\text{left}]] \leftarrow 1 - [[c']]$
    $d.[[\text{right}]] \leftarrow [[c']]$
  for $\ell \in \mathcal{L}$ **do**
    $[[s(\ell)]] =$ Sum of [[edges]] from root to ℓ
    $r_x, r_y \xleftarrow{\$} \mathbb{Z}_p$
    $[[x(\ell)]] \leftarrow r_x \cdot [[s(\ell)]]$
    $[[y(\ell)]] \leftarrow r_y \cdot [[s(\ell)]] + v(\ell)$
  return $\{([[x(\ell)]], [[y(\ell)]])\}_{\ell \in \mathcal{L}}$

Choosing the Hamming weight. The Hamming weight used for the constant-weight code directly affects the code length. More specifically, the code length is the smallest ℓ such that

$$\binom{\ell}{h} \geq 2^n \tag{3.4}$$

For $1 \leq h \leq n/2$, the code length decreases as the Hamming weight increases. But for $h > n/2$, the code length increases as the Hamming weight increases. So we do not choose the Hamming weight to be larger than half the bit precision. Given that analysis, the choice of the Hamming weight requires consideration of the trade-off between runtime and communication costs. In our evaluation, we plot several Hamming weights to show the effect.

3.6 Evaluation

In this section, we present our evaluation in two parts. In Section 3.6.1 we benchmark the runtime of our proposed non-interactive private comparison operators in comparison

with existing operators. In Section 3.6.2, we evaluate PDTE algorithms over decision trees trained over UCI datasets. We perform ablation studies to measure the performance of different algorithms with respect to precision, the number of client attributes and the size of the decision tree.

3.6.1 Benchmarking Private Comparison

In this experiment scenario, we assume a client wants to compare its input values with that of the server using a private comparison operator. We measure the computation time for the operators proposed in Section 3.4. Specifically, we benchmark 1) RCC 2) Folklore Comparison 3) XXCMP 4) SortingHats Comparison and 5) the work of Iliashenko et al. [88]. The first three are implemented using Microsoft SEAL [154]. In Figure 3.4 we plot the amortized runtime and communication as a function of the bitlength of the values. RCC is parameterized by the Hamming weight h which has a significant effect on the performance. We plot RCC for multiple Hamming weights to show the effect of the parameter.

All experiments are performed 10 times and the average results are reported. The shaded areas show one standard deviation of error.

XCMP and SortingHats perform one comparison at a time, whereas Folklore and RCC are batched and perform the comparison in a SIMD fashion. The input is encoded in the same format as that described in Figure 3.3. In this encoding, the number of parallel comparisons is a function of the precision and the parameters of the encryption scheme. Table 3.3 shows the number of comparisons for different precisions. The number of parallel comparisons decreases as the precision increases. Due to our encoding method, the number of parallel comparisons does not depend on the Hamming weight, in the case of RCC. The work of Iliashenko et al. also performs comparison in a batched manner. We use the univariate variant of their protocol which, per their experiments, produces the best results.

Table 3.3: Number of parallel comparisons in RCC and Folklore comparison as a function of the bit precision.

Bitlength	8	12	16	20	24	28	32	36
# of Comps	963	655	496	399	334	287	252	224

Figure 3.4 shows that RCC has the smallest amortized runtime for a private comparison

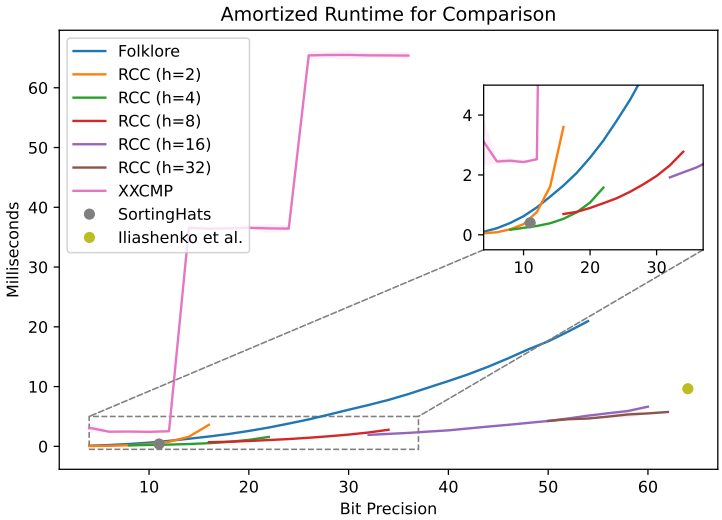


Figure 3.4: Amortized Time for private comparison. The shaded areas indicate one standard deviation of error. Time for a SortingHats comparison is measured by using the benchmarks in their repository. For RCC, at each point, we use the Hamming weight which has the smallest runtime.

for any bit precision. For a bit precision of 11, SortingHats and RCC have an approximately similar runtime, but SortingHats can not extend to higher bit precision.

3.6.2 Benchmarking PDTE

In this subsection, we compare the proposed private decision tree evaluation protocols in terms of server computation time and communication between the client and server. We first benchmark the performance whilst inferring decision trees trained on common UCI datasets and measure the communication and computation overhead. Our experiments over datasets show that the performance of these algorithms is tied to the number of client attributes and the size of the decision tree. Hence, we ablate with respect to these two parameters to understand the effect.

Implementation & Experimental Details

XXCMP-PDTE and RCC-PDTE are implemented as described in Section 3.5. We parallelized some steps in both protocols to enhance performance. Particularly, in RCC-PDTE,

the constant-weight equality operator was shown to be highly parallelizable by Mahdavi and Kerschbaum [121]. We also implement a baseline solution which we denote as *Folklore-PDTE*. Folklore-PDTE is implemented similarly to RCC-PDTE, with only the comparison replaced with the Folklore comparison from Section 3.2.1. All three algorithms are implemented using Microsoft SEAL version 4.0[1], which implements the FV cryptosystem in polynomial and batching mode [154]. Our implementation is publicly available on Github.[2]

For SortingHats, we use the implementation provided by the authors[3]. We activate the parallelization flag and use artificial input. Experiments are conducted on an Intel(R) Xeon(R) Platinum 8368 CPU @ 2.40GHz server running Ubuntu 22.02 with 32 cores. All experiments are repeated 10 times and the average is reported. The shaded areas indicate the standard deviation of the measurements.

Evaluation over Datasets.

We train decision trees over four datasets from the UCI repository [64], Heart, Breast, Spam, and Steel, which are also used in related work [53, 164]. We train decision trees with the desired precision using the Concrete-ML framework [127]. Table 3.4 shows the properties of the datasets. The structure of the decision tree changes as the precision increases. In general, a decision tree with higher precision has fewer nodes.

Table 3.4: Characteristics of UCI datasets used in our evaluation

Name	ID	# of Classes	# of Attributes
Breast	1510	2	30
Steel	1504	2	33
Heart	1565	5	13
Spam	44	2	57

In Figure 3.5, communication and computation are plotted as a function of the precision for Folklore-PDTE, XXCMP-PDTE, and RCC-PDTE. SortingHats does not permit arbitrary precision, so we plot it as one point in the graphs.

[1] https://github.com/microsoft/SEAL
[2] https://github.com/RasoulAM/private-decision-tree-evaluation
[3] https://github.com/KULeuven-COSIC/SortingHat

Figure 3.5 shows the results for four datasets. Folklore-PDTE is consistently the slowest of all solutions. None of the benchmarked approaches is consistently better, but in all cases, it is either XXCMP-PDTE or RCC-PDTE. In communication, Folklore-PDTE is dominant given that it has the most compact representation for a number, but given its impractical runtime, we can dismiss that. Hence, if we disregard that, SortingHats has the least communication overhead compared to XXCMP-PDTE and RCC-PDTE.

These experiments show that there is not a dominant solution that wins in all cases for all metrics. Communication and computation are a function of many factors, such as the number of client attributes and the number of decision nodes. We perform ablations to better understand the effect of each of these parameters.

Ablation over Number of Attributes.

In this experiment, we benchmark PDTE over a synthetic decision tree with a varying number of client attributes. Specifically, we generate a synthetic balanced tree of depth 6 (with 31 decision nodes) with three different bit precisions, $n = 8, 16, 26$. Note that the same results hold for trees of other sizes and shapes (balanced or unbalanced). We plot the communication complexity of RCC-PDTE and XXCMP-PDTE as we vary the number of client attributes from 5 to 100.

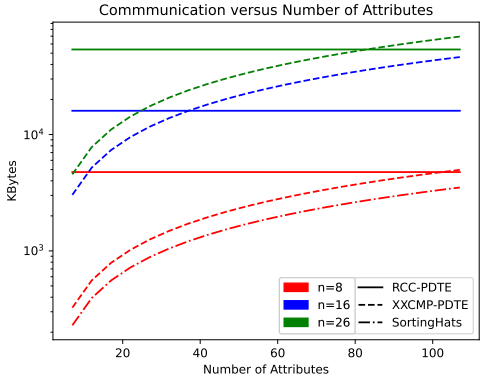


Figure 3.6: Communication cost of PDTE as a function of the number of attributes.

SortingHats and XXCMP-PDTE encrypt each client attribute in a separate RLWE ciphertext. Hence, the communication complexity scales linearly with the number of client attributes (notice the logarithmic vertical axis). For a given precision, RCC-PDTE has constant communication cost due to the use of batch encoding. When the number of

client attributes is small, RCC-PDTE has a higher communication cost compared to other solutions. SortingHats has the smallest communication overhead when precision is less than 11 bits. For precision higher than 11 bits, XXCMP-PDTE is the best solution in terms of communication. While RCC-PDTE has the highest communication cost when the number of attributes is small, this high overhead shrinks as the number of attributes grows and in some cases, it becomes the dominant solution. For example, at 16-bit precision, RCC-PDTE has the least communication overhead once the number of client attributes exceeds 40.

Note that the runtime of both algorithms does not have a noticeable change as the number of attributes changes. Hence, we only report the approximate runtimes of each approach in Table 3.5. Similar to the results shown in Figure 3.5, XXCMP-PDTE is the fastest for a low bit precision, but RCC-PDTE overtakes it for higher bit precision. SortingHats is only applicable for low precision and is slower in that case.

Table 3.5: Approximate runtime for private evaluation of balanced decision tree of depth 6 (with 31 decision nodes). The number of attributes varies from 7-100.

Precision (bits)	Runtime (ms)		
	SortingHats	XXCMP-PDTE	RCC-PDTE
8	648 - 662	133 - 155	149 - 170
16	-	673 - 753	187 - 234
26	-	752 - 845	747 - 966

Ablation over Number of Nodes.

In this experiment, we benchmark PDTE over synthetic trees as we vary the number of decision nodes. We fix the number of attributes to 32, but the same results hold for a different number of attributes. We generate balanced trees with depths up to 10, but the results do not depend on the shape of the tree, and the same results hold if the tree is unbalanced. This is because the tree traversal algorithm, SumPath, only takes up at most 10% of the total runtime. Hence, a change in the shape of the tree does not significantly impact the runtime. Figure 3.7 shows the runtime as a function of the number of nodes for $n = 8, 16, 26$. Note that the horizontal axis is logarithmic.

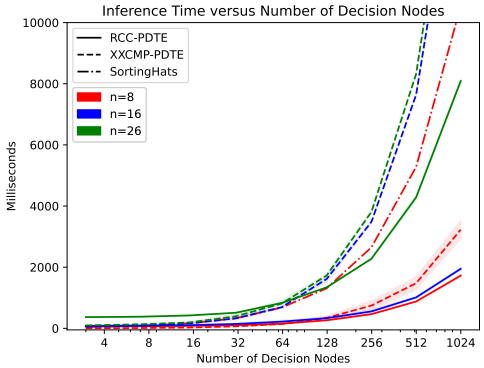


Figure 3.7: Runtime of PDTE as a function of the number of nodes in the tree.

As expected, the runtime of XXCMP-PDTE increases linearly with the number of decision nodes, given that one comparison is performed for each decision node in the tree. Due to batched computations, the runtime of RCC-PDTE only increases if more ciphertexts are required for the comparisons. For low precision, XXCMP-PDTE has better runtime compared to RCC-PDTE. However, for larger decision trees, RCC-PDTE is faster.

In all the reported protocols, the communication cost is a function of the precision and number of client attributes (and the Hamming weight, in the case of RCC-PDTE). Upload cost does not depend on the size of the decision tree. Download costs depend on the size of the tree, but in those cases, the entire output can fit within one ciphertext. Hence, we only report the communication cost of each protocol in Table 3.6.

Table 3.6: Communication cost for private evaluation of balanced tree over an input with 32 attributes. The depth of the tree to evaluate is up to 7.

Precision (bits)	Communication (KB)		
	SortingHats	XXCMP-PDTE	RCC-PDTE
8	2342	1486	4757
16	-	13847	16001
26	-	20770	53795

3.6.3 Summary of Results

For non-interactive private comparison, XXCMP and RCC can be utilized for arbitrary precision, while SortingHats has limited precision. If a single comparison with arbitrary

precision is required, XXCMP is a good option; however, RCC offers a significantly better amortized time for comparing numerous pairs.

In the context of PDTE, for low precision, either SortingHats, XXCMP-PDTE, or RCC-PDTE may be faster. The optimal solution is contingent on a combination of factors, including bit precision, the number of decision nodes, and the number of client attributes. For low precision, SortingHats is superior in terms of communication, but RCC-PDTE and XXCMP-PDTE are generally faster.

When dealing with bit precision higher than 11, XXCMP-PDTE and RCC-PDTE are the only practical available options, with RCC-PDTE proving to be faster, particularly as the number of decision nodes increases.

3.7 Discussion and Future Work

High-precision Applications. While decision trees may be achievable with smaller precision and quantization-aware training, there are other cases where the precision can not be sacrificed. For example, in the case of intrusion detection, servers may want to check to see if an IP, which is a 32-bit number, is in a specific range or not.

Extension of SortingHats. Other protocols such as SortingHats can also be modified to accommodate larger precision. This can be done using a similar algorithm to XXCMP but with a fully homomorphic scheme such as TFHE. Specifically, the equality check that is performed in line 9 of Algorithm 7 can be performed with functional bootstrapping instead.

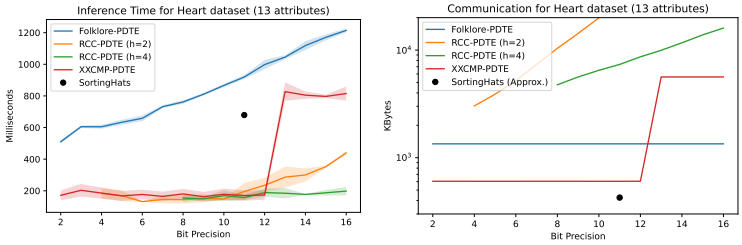
Other Packing Methods. Homomorphic encryption in the batched setting offers a lot of flexibility to choose the packing method. Other packing methods can improve communication, particularly when trying to reduce the time for one inference. Tools such as HELayers [3] can help find the other packing strategies.

Comparison with PROBONITE. There is currently no public implementation for PROBONITE available. We conducted a theoretical comparison of PROBONOTE with other PDTE protocols in Table 3.2, but a practical comparison would also be interesting as part of future work.

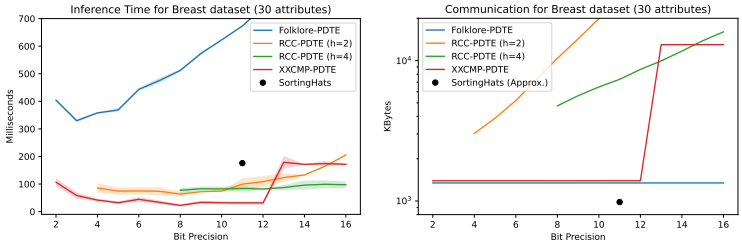
3.8 Conclusion

In this chapter, we propose two protocols for non-interactive private decision tree evaluation leveraging leveled homomorphic encryption, XXCMP-PDTE and RCC-PDTE. These protocols are based on XXCMP and RCC, two non-interactive comparison protocols which can efficiently compare numbers of arbitrary precision with a constant multiplicative depth.

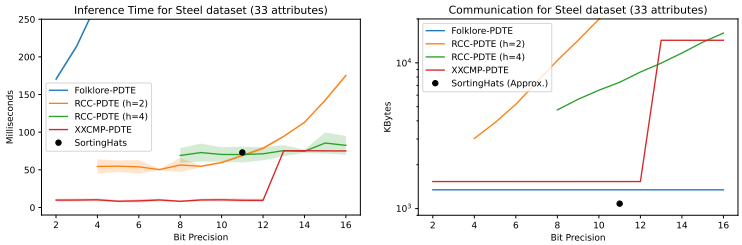
Our experimental analysis demonstrates that several protocols can be used when the client's input features a small number of attributes, the decision tree remains small, and lower precision is acceptable. However, when faced with many client attributes, large decision trees or the necessity for high precision, XXCMP-PDTE and RCC-PDTE emerge as better options compared to SortingHats. In some cases, these two protocols are up to 5 times faster than SortingHats. In very large decision trees, RCC-PDTE is the best solution and can infer a decision tree with over 1000 nodes and 16 bits of precision in under 2 seconds.



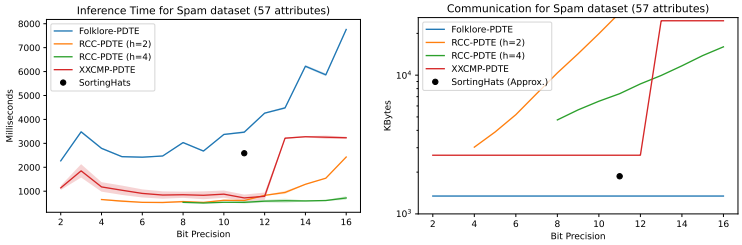
(a) Heart Dataset



(b) Breast Dataset



(c) Steel Dataset



(d) Spam Dataset

Figure 3.5: Runtime and Communication for Private Decision Tree Evaluation over four datasets. For each dataset, the left graph plots the runtime, and the right graph plots the communication. The shaded area shows one standard deviation of error.

Chapter 4

PEPSI: Practically Efficient Private Set Intersection in the Unbalanced Setting

4.1 Introduction

Private Set Intersection (PSI) is designed to compute the intersection of two sets while ensuring that nothing beyond the intersection is revealed. A specific form of PSI is *unbalanced* PSI, in which one party, typically the client, has a substantially smaller set than the second party, the server. Moreover, the client is often a resource-constrained device like a mobile phone or an IoT device with limited network connectivity. Consequently, it benefits the client by minimizing network communication, reducing the number of network round trips, and offloading computational tasks to the server.

In some cases, the goal of the client and server is to compute functions of the intersection, rather than to learn the intersection itself. For instance, in the case of COVID-19 exposure applications, a user simply wants to know if they share any randomly generated codes with those stored on the server, indicating potential exposure [67, 150, 159, 162]; with whom those matches are is irrelevant. Similarly, when gauging the effectiveness of ad campaigns by cross-referencing online advertisements with offline credit card transactions [90], the advertiser is interested in knowing the total sales corresponding to a specific set of credit cards used at certain vendors. Only the total sales amount is of interest, not the individual transactions. *Circuit PSI* is the term used when a function of the intersection is calculated rather than the intersection itself.

In the circuit PSI setting, the work of Kacsmar et al. [94] (referred to as DiPSI) is best suited for the unbalanced non-interactive setting. However, DiPSI still suffers from inefficiency issues. In contrast to DiPSI, the numerous fast state-of-the-art solutions for PSI [36, 54, 147] are limited in terms of being extendable to the circuit PSI setting. Specifically, each state-of-the-art solution is limited in one of the following ways. The first limitation is for solutions that are restricted as to the functions they can compute [90, 109]. They are unable to execute functions beyond a specific predefined set. Despite the fact that protocols designed for specific functions allow targeted optimizations, our evaluation shows that we outperform the state-of-the-art for many specific functions, particularly for large server set sizes. Furthermore, targeted optimizations become obsolete when the function needs to be changed. Second, adapting some solutions to circuit PSI levies a computational and communication burden on the client to compute arbitrary functions, usually in the form of two-party computation [36, 146]. This communication burden is usually proportional to the larger set [36, 90, 146] or depends on the complexity of the task. Such a burden is not acceptable when designing a non-interactive protocol.

Summary of Contributions. In this chapter, we propose PEPSI, an efficient, non-interactive circuit PSI protocol using homomorphic encryption. Homomorphic Encryption (HE) is a form of encryption that permits computation on the data while in encrypted form. In PEPSI, the client encrypts its elements using an HE scheme and sends them to the server, which compares the elements homomorphically. The output of each comparison is binary and is used to compute functions. Our approach removes the limitations that are present in existing work. PEPSI does not restrict the function that can be evaluated, there is no communication or computation burden on the client to compute the function, and it only has communication complexity proportional to the smaller client set size.

Our approach also overcomes the impractical runtimes associated with the DiPSI protocol from the literature [94]. We address the slow comparisons by comparing items using the efficient constant-weight equality operator [121], permutation-based hashing, and cuckoo hashing. Altogether this reduces the size of the elements that need to be compared and the total number of comparions, such that PEPSI is over four orders of magnitude faster than DiPSI and requires 90% less communication. For example, PEPSI can compute the intersection of 1024 and one million 32-bit elements in under one second with less than 5 MB of communication. In this setting, DiPSI could require 600 minutes of runtime and over 200 MB of communication.

PEPSI has a runtime and communication cost comparable to other PSI protocols based on HE [42, 54], which cannot extend to circuit PSI. Moreover, the client is required to

do much less computational work in PEPSI, making it compatible with use cases where the client does not have much computational power, i.e., the unbalanced setting. We empirically compare PEPSI with state-of-the-art protocols in our experiments, showing that PEPSI is even competitive with PSI protocols which cannot extend to circuit PSI. Our work is the first to show that non-interactive circuit PSI can be efficient up to the point it is practical for use in applications.

4.2 Related Work

The term *private set intersection* (PSI) was coined by Freedman et al. [74]. In a PSI protocol, two parties compute the intersection of their respective private sets such that no information is revealed except the intersection itself. If the intersection is only revealed to one of the parties, this is known as one-sided PSI. However, one-sided PSI can be extended to two-sided PSI by running the protocol a second time and switching the roles of the client and server.

Below, we provide a summary of different categories of PSI and different PSI protocols. Table 4.1 also summarizes the properties of different protocols.

Table 4.1: Comparing the properties of different PSI protocols. m and n are the client and server set sizes, respectively. We assume $m \ll n$ in the analysis of works which support the unbalanced setting. DH: Diffie-Hellman. 2PC: two-party computation. PDTE: private decision tree evaluation. *PSI using [90, 120, 173] is done in one round and an extra round is required to compute the sum. Moreover, it can only perform a limited set of functions in only one extra round. **PSI is realized by extending private set membership.

Work	Tools	Unbalanced	Rounds	Comm.	Comp.	PSI	Labelled PSI	Circuit PSI
[148]	B-OPPRF + 2PC	×	$1+\log_2 \lambda$	$O(n)$	$O(n \log n)$	✓	✓	2PC
[36]	RB-OPPRF + 2PC	×	$1+\log_2 \lambda$	$O(n)$	$O(n)$	✓	✓	2PC
[90, 173]	DH	×	1+1 *	$O(n)$	$O(n)$	✓	×	✓*
[51, 120]	OT + PDTE	×	1+1	$O(n)$	$O(n)$	✓	✓**	✓**
[43]	HE	✓	1	$O(m \log n)$	$O(n \log n)$	✓	✓	×
[42, 54]	OPRF + HE	✓	2	$O(m)$	$O(n \log n)$	✓	✓	×
DiPSI [94]	HE	✓	1	$O(m)$	$O(n)$	✓	✓	✓
PEPSI	HE	✓	1	$O(m)$	$O(n)$	✓	✓	✓

4.2.1 PSI with Leakage

In some applications and real-world use cases, such as checking for compromised credentials, the overhead of PSI is high if the client element is compared to all elements. For example, comparing the client credentials to billions of leaked credentials is expensive. To alleviate the overhead, some works propose to only compare the client elements to a subset of the server elements [112, 160]. For example, elements are distributed into buckets by their prefix. To query for a specific element, the client only queries the bucket corresponding to that element instead of the entire set. The server will know that the client's query falls into that specific bucket so such an approach partially leaks information about the client at the cost of better performance. This leakage degrades the security guarantees but may be acceptable in some applications. Thomas et al. [160] predict that the prefix of a user's credentials is common with about 350k-470k other users, assuming each client has one credential in the database.

Our work does not follow this approach and instead offers PSI without leakage. While such an approach requires more computational effort, it offers better security. We note that our protocol could also be combined with bucketization, offering a tunable trade-off between security and performance from the leakage of current protocols [112, 160] to no leakage.

4.2.2 Unbalanced PSI

Unbalanced PSI is a special case of PSI where one party, which we call the client, has a much smaller set compared to the other party, i.e., the server [6, 14, 42, 43, 54, 67, 95, 112]. This is a common assumption in many applications of PSI. For example, in compromised credentials checking (C3) [95, 112], the client wishes to access a large database containing credentials that are leaked on the web. While the set of client credentials is small, e.g., a few hundred credentials, the database of leaked credentials is in the billions and growing [95, 112]. Moreover, the client is usually bandwidth-constrained and has limited computational power. Hence, protocols with limited network round trips and little client-side computation are preferred. A narrower variant of unbalanced PSI exists, dubbed laconic PSI, with the additional constraint that only the server learns the output of the protocol [6, 14].

While there are many efficient state-of-the-art PSI protocols [36, 120, 145, 147, 148], the communication complexity of these solutions scales with the larger set, making them unsuitable for unbalanced PSI. For example, Pinkas et al. and Chandran et al. propose PSI protocols that compute an oblivious PRF for each element of the server set [36, 148].

Ma and Chow propose to construct a private decision tree from the server set, which is encrypted and sent to the client. The client then obliviously traverses this decision tree using OT [120]. Such solutions are useful in high-bandwidth, low-latency networks but fall short over high-latency networks.

Amongst solutions that have no leakage, the current most efficient non-interactive PSI protocols in the unbalanced case are based on homomorphic encryption [42, 43, 54]. Homomorphic encryption (HE) is a type of encryption that allows computations on data in its encrypted form. Chen et al. [43] were among the first to propose a PSI protocol specifically designed for the unbalanced setting using homomorphic encryption. If $\mathbb{Y}$ represents the server set, the key idea is to construct the polynomial $P(x) = r\prod_{y_i \in \mathbb{Y}}(x - y_i)$, for some random value r. The client sends an encryption of its elements to the server, which then homomorphically evaluates $P(x)$ on the client input. For a certain client element x_0, if $x_0 \in \mathbb{Y}$, then $P(x_0) = 0$, otherwise $P(x_0)$ is a random number. The idea of representing the set as a polynomial had been proposed before [100] and Chen et al. included optimizations to reduce the multiplicative depth of the function that is homomorphically evaluated, thereby enhancing efficiency [43]. Chen et al. [42] and Cong et al. [54] built upon the work of Chen et al. [43], employing a combination of Oblivious Pseudorandom Functions (OPRF) and homomorphic encryption. This improved performance extended the security to the malicious model and also allowed the protocol to extend to elements of arbitrary bitlength.

4.2.3 Labelled PSI

A specific variant of PSI is *labelled PSI*, where the server has a private label associated with each element in its private set. The server stores pairs in the form of (y_i, ℓ_i) where y_i is the identifier of the element and ℓ_i is the label associated with that element. The two parties compute the intersection of their sets and output all the pairs (y_i, ℓ_i) where y_i is in the intersection of the two sets. Nothing beyond the elements in the intersection and their corresponding labels are revealed to either one of the parties. When the client has only one element, this problem is equivalent to private information retrieval by keywords, first proposed by Chor et al. [49]. In the context of private contact discovery, the client may wish to retrieve the public keys of users in their contact list [63, 95].

Many PSI protocols can be extended to labelled PSI with additional computation and communication costs. For example, Chen et al. [42] and Cong et al. [54] extend their protocol to labelled PSI by evaluating another polynomial which evaluates to the ℓ_i when $x = y_i$. For the protocols of Pinkas et al. [146] and Chandran et al. [36] the extension to

labelled PSI can be done without changing the asymptotic complexity, but it does result in doubling the concrete communication and computation cost.

4.2.4 Circuit PSI

Another variant of PSI is *circuit PSI*, where the objective is to compute a function of the intersection, rather than the intersection itself. One example is PSI-Cardinality, where the parties compute the size of the intersection [94]. Another example is PSI-Sum [90], where the sum of values associated with elements in the intersection is revealed.

One application for PSI-Sum, known as ad-conversion, is assessing the effectiveness of ad campaigns [90]. A company purchasing an ad through an ad service company, such as Google, would like to know the total purchases made by users who have seen their particular ad. If Google can show a client who paid to have an ad displayed through their system that the ad leads to sales, then it is easier to convince the client to purchase more ads in the future. Such ad-conversion computations can be done by linking those who have seen an ad with their credit card transactions. However, credit card vendors are unwilling to disclose their clients' transactions. PSI-Sum is a solution in this scenario.

In the existing approaches [42, 43, 54], every comparison between a client and server element results in an arithmetic output. An arithmetic output means that if the elements are equal, the output is zero, and the output is a random non-zero number in $\mathbb{Z}_p$, for some p. While using this approach results in fast PSI protocols, computing a function of the intersection is not feasible. To compute a function, the result of each comparison must be converted to a binary output, i.e., one for a match and zero otherwise.

One approach to computing such functions is to use secure two-party computation (2PC) to convert the arithmetic output into a secret-shared binary output. 2PC can then be used to compute any arbitrary function over the binary output. While there is flexibility regarding the functions that can be computed in this approach, the communication complexity of MPC protocols is high. More specifically, the communication complexity depends on the size of both sets and the complexity of the desired function [36, 146].

Another approach is to compare elements from the client and server homomorphically via a homomorphic equality circuit. The output of the comparisons is binary in this case. This is the only non-interactive approach in the literature and is more suitable for the unbalanced setting since it does not require any client interaction. The work of Kacsmar et al. [94], which they call DiPSI, uses this approach. DiPSI achieves asymptotically optimal communication, given that only the client dataset is communicated over the network. The authors use this protocol to compute a differentially private PSI-Sum and other functions.

The main problem with DiPSI is the extremely high runtime. PSI-Sum between 1024 client elements and 1 million server elements can take over 600 minutes. This can mainly be attributed to the homomorphic comparison function that they employ. We propose a solution that compares elements homomorphically, similar to DiPSI but is over four orders of magnitude faster.

Some works take a different approach to circuit PSI by designing a protocol that can compute a very specific function very efficiently [90, 109]. This is in contrast to the other approaches where an arbitrary function can be derived. A specialized protocol outperforms generalized protocols in terms of performance but restricts the functions that can be derived and may leak additional information in the process [90].

Ion et al. [90] propose a protocol for PSI-Sum-with-Cardinality. This protocol outputs, to the server, the sum of values associated with elements in the intersection, whilst also revealing the intersection cardinality to the client. Lepoint et al. [109] proposed Private Join and Compute (PJC) which provides PSI-sum, which is the sum of values associated with elements in the intersection, without leakage of the intersection cardinality. However, their protocol cannot compute any function other than the sum and has an expensive offline phase. PSI-Stats follows a similar approach and supports more functions including arithmetic and geometric mean, standard deviation, and approximate composition [173]. However, this protocol is still limited to the functionalities proposed by the authors. A common feature of these works is they require three rounds of interaction and the final results are revealed to the server.

4.3 Background

We provide relevant background on homomorphic encryption and constant-weight equality operators in Chapter 2. Additional background on hashing optimizations is given below.

4.3.1 Hashing Optimizations

We use a collection of hashing techniques to reduce the total number of comparisons and the cost of each comparison.

Hashing-to-Bins. Hashing-to-bins reduces the number of comparisons in a PSI protocol [73, 94, 145, 148]. Elements are distributed into a predetermined number of bins and comparisons are only performed between client and server elements in the same bin; since

the bin in which an element is placed is determined using hashes of that element. When using this strategy, note that it is necessary to pad each bin with dummy elements up to a predetermined size to prevent the load of the bins from leaking information about the dataset.

There is a possibility that the number of elements in a bin exceeds the predetermined maximum, which would constitute a failure in the protocol. Thus, it is necessary to employ a strategy that manages elements that fall into the same bin.

Cuckoo hashing [136, 171] is a strategy for hashing elements into bins such that elements can be ejected from their current bin and placed in another bin after their initial placement. In Cuckoo hashing [136], there are m elements which are placed in $b = \epsilon m$ bins, for some constant ϵ. Each bin holds at most one element and the placement of each element is determined using k hash functions, denoted as $H_i : \mathcal{X} \mapsto [b]$ for $i = 1, 2, \cdots, k$.

For each element x, we place x in bin $H_1(x)$. If bin $H_1(x)$ was already occupied by x', such that $H_1(x) = H_i(x')$, we evict x' from the bin. The new placement for x' is bin $H_{i+1 \mod k}(x')$. If that bin is empty, we are done; otherwise, we repeat this process, evicting the current resident of the bin. To ensure the protocol terminates, it is necessary to define an upper bound on the number of evictions permitted. If an empty bin is not found with the permitted number of evictions, the element is placed in a stash with a fixed size. Thus, to locate an element x, it is sufficient to look in bin $H_1(x)$, $H_2(x)$, $\cdots$, $H_k(x)$ or in the stash. When selecting parameters ϵ, k, and the stash size, please refer to past work analyzing the failure rates of Cuckoo hashing [15, 98, 136, 147].

Permutation-based Hashing Permutation-based hashing (PBH) is a technique to reduce the length of elements [15, 105, 145, 148]. We describe PBH in the following paragraph using the assumption that the number of bins is $b = 2^c$, for some $c \in \mathbb{N}$.

For a λ-bit element x, assume $x = x_H || x_L$ where $||$ is the concatenation operation and $|x_H| = c$ and $|x_L| = \lambda - c$. In the binning procedure, instead of placing x in bin $H(x)$, we place x_L in bin $b_x = x_H \oplus H(x_L)$. Now, if two elements x and y are placed in the same bin and match, this means that $x_L = y_L$ and $b_x = b_y$. From that, we have

$$b_x = b_y \Rightarrow x_H \oplus H(x_L) = y_H \oplus H(y_L) \Rightarrow x_H = y_H$$

Combining $x_L = y_L$ and $x_H = y_H$, we can deduce that $x = y$. Using this technique, we compare $\bar{\lambda}$-bit elements where $\bar{\lambda} = \lambda - c$. Since we can identify matches using only these shorter elements, we will only store elements of length $\bar{\lambda}$ in our bins. Thus, we term $\bar{\lambda}$ the *effective bitlength* and use it as the bitlength parameter throughout our descriptions and analysis of our protocol.

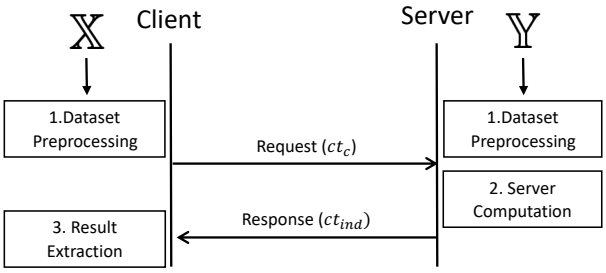


Figure 4.1: Stages of PEPSI.

4.4 PEPSI: Practically Efficient PSI

In this section, we describe our protocol PEPSI in detail; specifically, we describe the necessary steps to compute the intersection between a client set of size m and a server set of size n, where each set contains λ-bit elements. PEPSI is a PSI protocol that operates in one round and consists of three stages: Dataset Preprocessing, Server Computation, and Result Extraction. The client sends a *request message* to the server and the server responds with a *response message*. The request and response message constitute the total communication complexity of the protocol. At the end of the protocol, the client learns the intersection and the server learns nothing about the client set.

Figure 4.1 visualizes the steps of the protocol and the interaction between the client and server. In the base case (no variants), the Server Computation stage simply computes the intersection. This can be extended to other functionalities such as labelled PSI, computing a function on top of the intersection, and laconic PSI. See Section 4.5 for such variants. Table 4.2 summarizes all the notations used in the protocol description.

Table 4.2: Summary of notation for algorithm description.

Input Parameters	
$\mathbb{X}$	Client set
$\mathbb{Y}$	Server set
m	Client set size
n	Server set size
λ	Bitlength of elements
α	Maximum Error rate
Cryptographic Parameters	
N	Polynomial modulus degree
q	Coefficient modulus
t	Plaintext modulus
Binning Parameters	
b	Number of bins in T_c and T_s
γ	Client maximum bin load
μ	Server maximum bin load
$\bar{\lambda}$	Effective bitlength (used in PBH)

Constant-weight Code Parameters	
h	Hamming weight of constant-weight code
ℓ	Constant-weight code length
Auxiliary Notation	
T_c	Client table ($T_c[b][\gamma]$)
T_s	Server table ($T_s[b][\mu]$)
T'_c	T_c with **encoded** elements
T'_s	T_s with **encoded** elements
pt_c	Plaintexts of clients elements
ct_c	Ciphertexts of clients elements
val_c	Ciphertexts of clients values
pt_s	Plaintexts of servers elements
val_s	Plaintexts of server values

4.4.1 Dataset Preprocessing

In the first stage, the client and server must asynchronously preprocess their datasets to prepare them for the next stage. Preprocessing the dataset reduces the protocol latency when a client queries a server. This preprocessing is also helpful for the server, which has a large dataset and can reuse the preprocessed dataset for many queries by various clients. Both parties, client and server, preprocess their data in three steps: hashing optimization, encoding elements, and finally, batching (and encryption). Figure 4.2 visualizes the steps of the preprocessing. Note the actions in each step by a party are dependent on whether they are in the role of client or server. We provide a high-level overview here with relevant pseudocode in Appendix A.1.

Hashing Optimization. Our hashing optimization uses a hashing-to-bins strategy in conjunction with permutation-based hashing. Hashing-to-bins reduces the number of elements we need to compare, and permutation-based hashing reduces the size of elements that are inserted in the table (recall $\bar{\lambda} = \lambda - c$ as the effective bitlength from Section 4.3). The

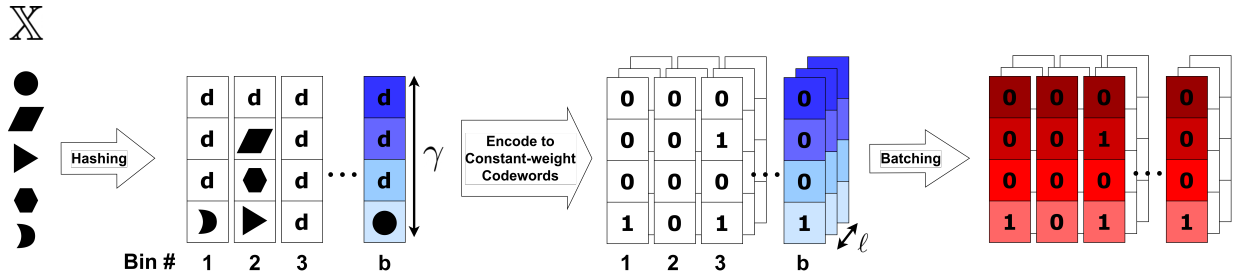


Figure 4.2: Client Dataset Preprocessing in PEPSI. d indicates dummy elements, b is the number of bins, and γ is the clients maximum bin load. From left to right: symbols represent the real-valued payload that is hashed into bins with a maximum bin load γ. The payload is encoded into bits using our constant-weight codewords, whereby the same color indicates the same payload, and finally encrypted into a ciphertext by batching across bins. The preprocessing outputs $\ell \cdot \gamma$ ciphertexts.

selected hashing-to-bins strategy in PEPSI must insert elements into b bins and does not require a stash. Elements that fall into the stash must be compared with all other elements, which severely diminishes the performance of our protocol. Moreover, permutation-based hashing cannot be used in conjunction with a stash. Hence, for the binning portion of the hashing optimization, we can use known hashing strategies that do not have a stash, such as Cuckoo hashing without a stash [146, 148] or Dual hashing [94].

The client and server have predetermined maximum bin loads, γ and μ, respectively, which are a function of their set sizes and the number of bins. After inserting elements into the bins, each party pad bins with dummy elements up to the maximum load of the bin. If more elements are inserted into a bin than the maximum load, the protocol fails. The size of the bins are chosen such that this happens with negligible probability. We denote the client's binning table with b bins and a maximum load of γ as T_c. Similarly, we denote the server's binning table with b bins and a maximum load of μ as T_s.

Encode to Constant-weight Codewords. The client encodes each $\bar{\lambda}$-bit element in T_c as a constant-weight codeword of length ℓ and Hamming weight h. The Hamming weight can be chosen freely but affects the communication-computation tradeoff (see Section 4.6). We set ℓ as a function of $\bar{\lambda}$ and h such that there is a unique representation for every $\bar{\lambda}$-bit element in T_c. So,

$$\ell = \ell(\bar{\lambda}, h) = \min\left\{\ell \in \mathbb{N} \mid \binom{\ell}{h} \geq 2^{\bar{\lambda}}\right\}. \tag{4.1}$$

We denote T_c' as the new table which holds the encoded elements. Dummy elements

are encoded to the all-zero string of length ℓ. The server constructs T_s' from T_s using the same procedure as the client uses to construct T_c' from T_c.

Batching (and Encryption). In the last step, the client restructures T_c' to obtain a table of plaintexts. Recall that plaintexts are vectors of length N. Each position in $pt_c[i][j]$ corresponds to elements from one of the b bins. More concretely, $pt_c[i][*]$ contains the bits of the i-th encoded element in each of the b bins of T_c. We call this the client's i-th batch. Moreover, this process, which we call *batching*, allows the protocol to perform comparisons in different bins simultaneously. Each shade of red in Figure 4.2 shows the contents of one plaintext. The client additionally encrypts each plaintext with its secret key. The server constructs plaintexts pt_s in a similar fashion from the contents of T_s' and the server's i'-th batch is defined similarly. The server does not need to encrypt its plaintexts.

We assume $b = N$ in this example to simplify the explanation, but we show how to relax this assumption in relevant steps of the protocol.

4.4.2 Server Computation

The client sends the ciphertexts it has produced to the server for the next stage. In this stage, the server receives the ciphertexts of the client's elements and performs the set intersection (see Algorithm 13). At a high level, for each $i \in [\gamma]$ and $i' \in [\mu]$, the server compares batch i of the client's dataset with batch i' of the server's dataset. The output of this step is a table of ciphertexts of size $\gamma \times \mu$, which we denote by ct_{eq}. For $i \in [\gamma]$ and $i' \in [\mu]$, $ct_{eq}[i][i']$ denotes the result of comparing elements from batch i from the client and batch i' from the server.

Recall that the arithmetic constant-weight equality operator does not depend on the operands of the comparison. Hence, we can use it as a SIMD operator in line 4 of Algorithm 13.

Algorithm 13 Compute intersection of client and server set

```
1: algorithm INTERSECT(ct_c, pt_s)
2:     for each client batch i ∈ [γ] do
3:         for each server batch i' ∈ [μ] do
4:             ct_eq[i][i'] ← ARITH-CW-EQ-OP(ct_c[i], pt_s[i'])
5:         ct_ind[i] ← Σ_{i'∈[μ]} ct_eq[i][i']
6:     return ct_ind
```

If $b \neq N$, line 4 of Algorithm 13 is repeated $\lceil b/N \rceil$ times.

4.4.3 Result Extraction

The server sends the ciphertexts from the previous stage to the client for the final stage. In this final stage, the client decrypts the ciphertexts received from the server to obtain the indicator vector. The indicator vector specifies whether there was a match in the server set, for each element in the client set. From that, the client extracts the intersection. Algorithm 14 shows the procedure for this stage.

Algorithm 14 Extract intersection from server output

```
1: algorithm ExtractIntersection
2:     for each client batch i ∈ [γ] do
3:         ind[i] ← Dec(ct_ind[i], sk_c)
4:         for each bin k ∈ [b] do
5:             if ind[i][k] = 1 then
6:                 Add T_c[k][i] to intersection
7:     return intersection
```

4.5 PEPSI Variants

Section 4.4 describes PEPSI for computing the intersection of two sets of λ-bit elements. However, the algorithm can be adapted for several other scenarios as well. Each adaptation adds extra steps to the protocol to achieve extra functionality or enhance performance. We describe these adaptations in detail in this section.

4.5.1 Optimization for Large Elements

In some applications, the client and server elements are very large, e.g., file names or 256-bit strings. In these cases, it is inefficient to compare large elements homomorphically. Instead, we map each element to a smaller λ-bit element using a hash function. However, if λ is too small, there may be colliding elements, which results in an incorrect result. Hence, we derive the failure rate of PEPSI if we map m client elements and n server elements to λ-bit elements. Conversely, we choose λ such that the failure rate of PEPSI is less than a given parameter α.

Not all collisions result in failure of the protocol, i.e., an incorrect result. Failure occurs when two unequal elements, one from the server and one from the client, have an identical mapping and are compared to each other by being placed in the same bin. Such an event produces an incorrect result since two unequal elements match. The upper bound on the

failure rate is given in Lemma 1. Note that collision between two client elements (similarly, server elements) does not result in an incorrect result since client elements are not directly compared with each other.

Lemma 1. *When hashing m client elements and n server elements to λ-bit strings, the probability of failure in the protocol due to collisions is upper bounded by*

$$\frac{b\gamma\mu}{2^{\lambda}}, \tag{4.2}$$

where b, γ, and μ are the number of bins, maximum client bin size, and maximum server bin size, respectively.

Proof. Let $\mathbb{P}[B]$ denote the probability of failure due to a collision. Let $\mathbb{P}[B_i]$ denote the probability of there existing a collision between any two of the elements of bin i, for $i \in \{1, 2, \cdots, b\}$. So we have

$$\mathbb{P}[B] \leq \sum_{i \in [b]} \mathbb{P}[B_i]. \tag{4.3}$$

We know that, in a given bin, failure only occurs when two unequal elements, one from the server and the other from the client, have an equal constant-weight mapping. We know that the probability of a collision between two distinct elements from the domain is $2^{-\lambda}$, where the probability is in expectation over all perfect hash functions.

Hence, combining the facts stated above, we have that

$$\mathbb{P}[B] \leq \sum_{i \in [b]} \mathbb{P}[B_i] = \sum_{i \in [b]} \gamma\mu 2^{-\lambda} = b\gamma\mu 2^{-\lambda}. \tag{4.4}$$

□

Using Lemma 1, we can see that, to bound the failure rate by $2^{-\alpha}$, we must choose λ, such that $b\gamma\mu 2^{-\lambda} \leq 2^{-\alpha}$. Hence, we set $\lambda = \alpha + \lceil \log_2(b\gamma\mu) \rceil$.

4.5.2 Labelled PSI

PEPSI can be adapted to labelled PSI by altering the server computation and the result extraction. Algorithm 15 shows the adapted algorithm. In this algorithm, we denote the

server labels as val_s, which is a vector of μ plaintexts. Moreover, the algorithm assumes that the label are non-zero and fit within one plaintext slot, but this can be extended to labels of arbitrary size. Algorithm 15 can be extended to the case where server labels are larger than one plaintext slot. The expensive step of the algorithm, which is matching client and server elements, does not need to be repeated. Instead, we simply repeat line 4 of Algorithm 15 for as many times that is required, depending on the size of the label.

Algorithm 15 Server Computation and Result Extraction for Labelled PSI

1: **algorithm** LABELLEDPSI($ct_c[i]$,$pt_s[i']$)
2: **for each** client batch $i \in [\gamma]$ **do**
3: **for each** server batch $i' \in [\mu]$ **do**
4: $ct_{eq}[i][i'] \leftarrow$ ARITH-CW-EQ-OP($ct_c[i], pt_s[i']$)
5: $ct_{res}[i] \leftarrow \sum_{i'} val_s[i'] \cdot ct_{eq}[i][i']$
6: Send ct_{res} to the client
7: **algorithm** EXTRACTLABELS(ct_{res})
8: **for each** client batch $i \in [\gamma]$ **do**
9: $val[i] \leftarrow \mathsf{Dec}(ct_{res}[i], \mathtt{sk}_c)$
10: **for each** bin $k \in [b]$ **do**
11: **if** $val[i][k] \neq 0$ **then**
12: Add $val[i][k]$ to intersection labels
13: **return** intersection labels

4.5.3 Circuit PSI

PEPSI can additionally be extended to compute functions over the intersection. Such functions include PSI-Cardinality, PSI-Sum [94], PSI-Inner-Product [90, 109], and beyond. We introduce additional functionalities in the appendix.

PSI-Sum and PSI-Cardinality. In PSI-sum, the sum of server values associated with the elements in the intersection are output to the client. Algorithm 16 shows the modified server computation and result extraction for PSI-sum, respectively. Kacsmar et al. use the same algorithm for securely computing PSI-sum and give proof of correctness and security [94]. PSI-Cardinality, which computes the size of the intersection, is a special case where the server values are equal to one.

PSI-Inner-Product. PSI-Inner-Product compute the inner product between values held by the client and values held by the server, for values that have an identifier that falls in the intersection of the two sets. PSI-sum can be generalized to PSI-Inner-Product [109]

Algorithm 16 Server computation and result extraction for PSI-Sum

1: **algorithm** ComputePSISum(ct_c, pt_s)
2: **for each** client batch $i \in [\gamma]$ **do**
3: **for each** server batch $i' \in [\mu]$ **do**
4: $ct_{eq}[i][i'] \leftarrow$ Arith-CW-Eq-Op($ct_c[i], pt_s[i']$)
5: $ct_{sum} \leftarrow \sum_{i' \in [\mu]} val_s[i'] \cdot \sum_i ct_{eq}[i][i']$
6: Sample vector $r \leftarrow [r_0, r_1, \cdots, r_{N-1}]$ s.t. $\sum r_i = 0$
7: $ct_{res} \leftarrow ct_{sum} + r$
8: **return** ct_{res}

9: **algorithm** ExtractPSISum(ct_{res})
10: $res \leftarrow \mathsf{Dec}(ct_{res}, \mathsf{sk}_c)$
11: $ct_{sum} \leftarrow \sum_{k \in [N]} res[i]$
12: **return** ct_{sum}

as well. Assuming that the encrypted client values are denoted as val_c, we simply replace line 5 of Algorithm 16 with

$$ct_{sum} \leftarrow \sum_{i' \in [\mu]} val_s[i'] \cdot \sum_i val_c[i] \cdot ct_{eq}[i][i'].$$

4.6 Analysis of PEPSI

In this section, we provide an overview of the security analysis and details on communication and computation complexities. The communication and computation analysis is necessary to choose optimal parameters and enables PEPSI to achieve competitive performance compared to related work.

Security Analysis. The PEPSI protocol operates in the semi-honest model, i.e., the client and server follow the protocol but may try to infer extra information. The client input privacy is guaranteed due to the use of homomorphic encryption. The noise level in the homomorphic ciphertexts can reveal extra information about the server's dataset. Hence, we need a technique to make the noise level of the output ciphertexts indistinguishable from fresh ciphertexts. This is referred to as *circuit privacy* in the literature [26]. Techniques such as noise flooding [16, 40] achieve this property and can also be used in this work. To use noise flooding, we may require to use larger parameters for correctness and security, or to reduce the size of the plaintext space to allow room to add additional noise. Reducing the size of the plaintext space is possible in our protocol, since inputs are encoded as binary string which only requires a binary plaintext space. Noise flooding is not implemented in the available homomorphic encryption libraries, so we do not perform that step in our

implementation. Aside from the use of larger parameters, noise flooding has negligible effect on runtime, i.e., at the cost of one extra homomorphic addition, so our experimental results are unaffected, if we can use the same parameters. We leave the specific choice of noise flooding technique for future work.

We choose all parameters such that we have 128-bit security.

Extending to the Malicious Model. The malicious security model removes the assumption that adversaries follow the protocol and considers active adversaries behaving arbitrarily. The malicious model for PSI has two limitations: 1) A malicious party may simply substitute its input and learn (parts of) the other party's set once the intersection is revealed, which cannot be prevented in the malicious model. 2) Assuming the use of verifiable computation, in a one-round protocol, arbitrary behaviour is only possible in the input-carrying first message. Hence, additional information can only be computed from the message (output) received in response to this first message. Using techniques such as input verification [34, 62], verifiable homomorphic encryption [101], and zero-knowledge proofs, we can augment our protocol to account for malicious clients and servers. This would significantly impact the performance since all actions performed by the client, including all encryption and public key generation would need to proven. We leave a details description of such a protocol for future work.

Notes on Performance Analysis. The communication and computation costs of PEPSI depend on many parameters, the most apparent of which are the set sizes and the bitlength of elements. Two parameters that can be tuned to optimize performance are the binning strategy and the Hamming weight.

The choice of binning strategy and associated parameters has been extensively discussed in previous works [136, 148], and using existing analysis, we can derive the relevant binning parameters b, γ, and μ, given the set sizes. Hence, in this work, we compute the communication and computation cost as a function of γ, μ, and b instead of the set sizes. As a result, our analysis is agnostic to the binning strategy.

The other parameter we can optimize over is the Hamming weight. The Hamming weight h is a parameter of the constant-weight equality operator. Cryptographic parameters depend solely on h as it determines the multiplicative depth. The code length is a function of the bitlength (and effective bitlength) and the Hamming weight, so it does not need to be optimized. In this section, we discuss how the communication and computation of PEPSI are affected by the Hamming weight and the tradeoff between these two metrics.

Analysis for Parameter Selection. Using the analysis from this section, we define strategies for optimally selecting parameters for PEPSI. There are two obstacles to optimizing the parameters in PEPSI. Firstly, there are circular dependencies between the parameters. For example, the cryptographic parameters, N and q dictate the number of bins, which influences the error rate which in turn influences the parameters of the constant-weight code. However, the Hamming weight of the constant-weight code determines the multiplicative depth, which puts a lower bound on the cryptographic parameters. The second obstacle is that there is no precise closed-form formula for the runtime so it requires experimental evaluation. Moreover, in many cases, there is no definitive choice for the parameters that optimize both communication and computation. In such cases, we visualize the communication computation tradeoff to assist in parameter selection. Hence, optimizing the parameters amounts to a non-trivial task that requires experimental evaluation and careful and systemic selection of the parameters.

4.6.1 Communication Complexity

The total communication complexity of PEPSI consists of the request and response messages. The response message depends on the function that is being evaluated. For example, if the set intersection is returned, the response is as big as the request message. In contrast, if we want the intersection cardinality, the response is only one ciphertext. In all the examples examined in this work, the response is smaller than the request size. Hence, to encompass all of these applications, we analyze and optimize the request size.

Asymptotic Communication Complexity. Theorem 1 derives the asymptotic complexity of the request in PEPSI as a function of the code length, ℓ.

Theorem 1. *The asymptotic complexity of the request message in PEPSI is* $O(b \cdot \gamma \cdot \ell)$*, where* $\ell = \ell(\bar{\lambda}, h)$ *as defined in Equation* (4.1)*.*

Proof. As shown in Section 4.4, in the request, the encrypted plaintexts containing T'_c are sent to the server. T'_c is a table with dimension $b \times \gamma \times \ell$, with one bit in each cell of this table. Hence, the total communication is $O(b \cdot \gamma \cdot \ell)$. □

Corollary 1.1. *If we use Cuckoo hashing in PEPSI, then the communication complexity can be simplified to* $O(m \cdot \ell)$ *where* m *is the client set size and* $\ell = \ell(\bar{\lambda}, h)$ *as defined in Equation* (4.1)*.*

Concrete Communication Cost. Theorem 1 only states the asymptotic complexity of the request size, but in practice, the size of the ciphertext must also be considered. Encoded elements in the client's set are put into $\lceil b/N \rceil \cdot \gamma \cdot \ell$ ciphertexts and each ciphertext in the request is approximately $N \cdot \log_2 q$ bits, where $\log_2 q$ is the FV ciphertext modulus which is used. So overall, the size of the request in PEPSI is $\lceil b/N \rceil \cdot \gamma \cdot \ell \cdot N \cdot \log_2 q$ bits (or simply $b \cdot \gamma \cdot \ell \cdot \log_2 q$ when b is a multiple of N).

Note that the ciphertext modulus depends on the multiplicative depth, which depends only on the Hamming weight. N and q must also satisfy requirements for security which have been outlined in the literature [37]. For each Hamming weight, we select the smallest parameter set, which ensures the correctness of the result while also providing at least 128-bit security. Table 4.3 shows the polynomial modulus degree and bitlength of the coefficient modulus we use for each h.

Table 4.3: Polynomial modulus degree and coefficient modulus as a function of the Hamming weight.

h	1	2	3-4	5-8	9-16	17-32	33-64
$\log_2 N$	12	13	13	13	14	14	14
$\log_2 q$	72	144	168	204	240	276	312

Optimizing Communication Cost. The communication cost is derived as a function of the code length. However, the code length itself depends on the Hamming weight and effective bitlength, as shown in Equation (4.1). Hence, to minimize the communication complexity, we look for the Hamming weight which minimizes $\lceil b/N \rceil \cdot \gamma \cdot \ell \cdot N \cdot \log_2 q$.

We know that γ does not depend on the Hamming weight so it acts as a constant in our optimization and can be removed. So we can minimize $\lceil b/N \rceil \cdot \ell \cdot N \cdot \log_2 q$, where $\ell = \ell(\bar{\lambda}, h)$. The effect of b is more convoluted and must be taken into account in the optimization. For simplicity, we assume $b \leq 4096$, which is less than the smallest possible value for N, so $\lceil b/N \rceil = 1$. We show results for other values of b in the appendix.

Figure 4.3 plots $\lceil b/N \rceil \cdot \ell \cdot N \cdot \log_2 q$ with $b \leq 4096$ as a function of the Hamming weight for $\bar{\lambda} = 16, 32, 48$ to demonstrate that a minimum exists. The minimum communication occurs for $h = 8, 8, 23$, respectively.

We plot the optimal Hamming weight for communication for each effective bitlength in Figure 4.4 assuming $b \leq 4096$. In the case of a tie between two Hamming weights, we choose the smaller Hamming weight since it requires less computation. Generally, as

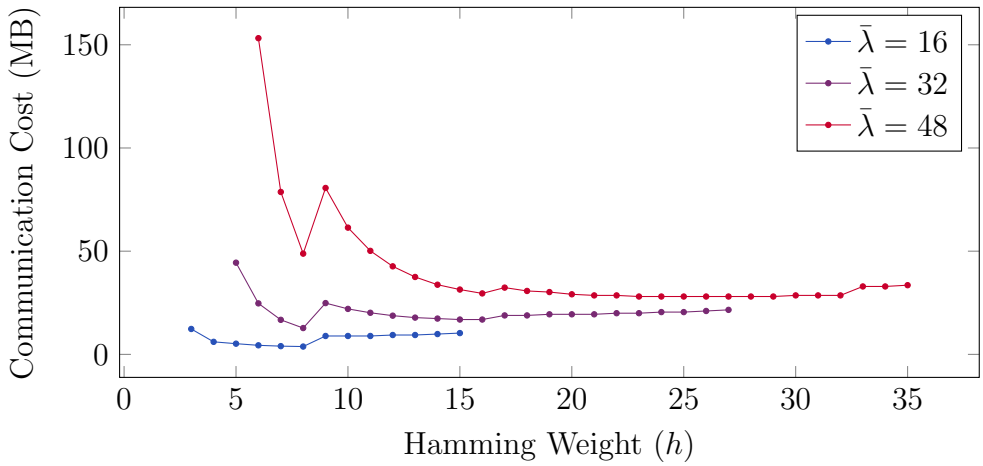


Figure 4.3: Code length as a function of the Hamming weight for $\bar{\lambda} \in \{16, 32, 48\}$ for $b \leq 4096$. The minimum occurs for a Hamming weight of 8, 8, and 23, respectively.

the effective bitlength grows, a larger Hamming weight is required to achieve the smallest communication cost.

4.6.2 Computation Complexity

Amongst the stages of the protocol, the Result Extraction has a small computation overhead and also depends on the specific function that we derive. The data preparation can be done offline. Most of the online computation time is dedicated to the Server Computation stage. Within that stage, the majority of the runtime is dedicated to computing the encrypted indicator vector (Algorithm 13) and is required for any subsequent computation. The server computation algorithms described in Section 4.5.3 may perform additional operations after the indicator vector is derived, but in all cases, these operations are insignificant compared to calculating the indicator vector. Hence, we base our analysis and parameter selection on Algorithm 13 and calculate and optimize the runtime of this algorithm.

Approximate Server Computation Complexity. We approximate the computational complexity of the Server Computation stage by counting the number of expensive homomorphic operations, i.e., plaintext and homomorphic multiplications.

Theorem 2. *The number of homomorphic operations in Algorithm 13 is* $(\ell \cdot \texttt{PM} + h \cdot \texttt{M}) \cdot$

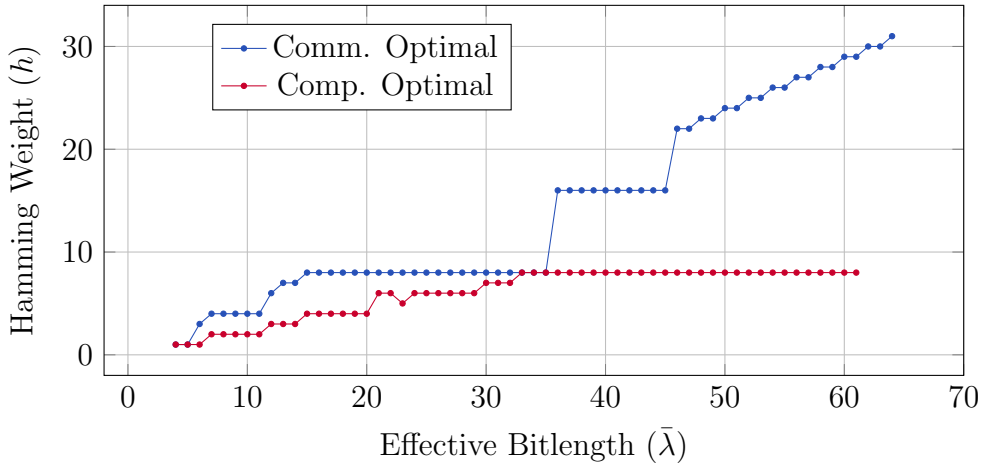


Figure 4.4: Hamming weight which optimizes communication and computation as a function of the effective bitlength ($\bar{\lambda}$) in blue and red, respectively. We assume $b \leq 4096$ in these graphs.

$\lceil b/N \rceil \cdot \gamma \cdot \mu$ *where* `PM` *and* `M` *denote plaintext and homomorphic multiplication, respectively, and* $\ell = \ell(\bar{\lambda}, h)$.

Proof. The constant-weight equality operator in line 3 of Algorithm 13 consists of ℓ homomorphic multiplications and h plaintext multiplications [121]. In the general case when the number of bins is larger than N, line 4 of Algorithm 13 is performed $\lceil b/N \rceil$ times. Moreover, this line is repeated in two loops of length γ and μ, hence the total number of operations is $(\ell \cdot \texttt{PM} + h \cdot \texttt{M}) \cdot \lceil b/N \rceil \cdot \gamma \cdot \mu$. □

Optimizing Computation Cost. Given that we do not have a closed-form formula for the computational complexity of the homomorphic operators, optimizing the computation complexity is not possible. However, the concrete computation cost of PEPSI can be optimized empirically. Optimization is performed over the Hamming weight since we assumed that the bitlength, b, γ, and μ are given.

The client and server set sizes determine γ and μ, based on the binning strategy. The choice of h does not affect any of these parameters and only influences ℓ. Hence, γ and μ can be treated as constants in the optimization and it suffices to minimize $\left\lceil \frac{b}{N} \right\rceil \cdot (\ell \cdot \texttt{PM} + h \cdot \texttt{M})$. Theoretically, optimizing this term is difficult since there are no closed-form formulas for the runtime of homomorphic operations. Instead, we empirically optimize it by trying different Hamming weights. We set $\gamma = \mu = 1$ to eliminate the variance introduced by

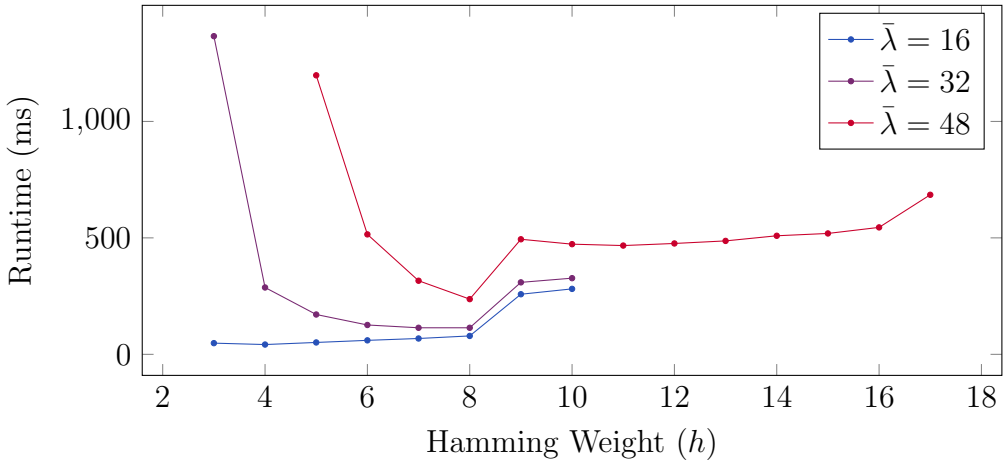


Figure 4.5: Runtime as a function of the Hamming weight for $\bar{\lambda} \in \{16, 32, 48\}$ for $b \leq 4096$. The minimum occurs for a Hamming weight of 4, 8, and 8, respectively.

those parameters and run PEPSI using only one thread. Figure 4.5 shows the reuslt of this comparison.

Figure 4.4 shows, in red, the Hamming weights which optimize computation time for each effective bitlength assuming that $b \leq 4096$. Similar to the communication, the optimal Hamming weight for runtime increases as $\bar{\lambda}$ increases.

4.6.3 Communication/Computation Tradeoff

Note that choosing parameters that optimize the computation would increase communication costs and vice versa. In practice, it may be reasonable to choose parameters that fall somewhere between the two extremes. Figure 4.6 visualizes the communication/computation tradeoff for the intersection as the Hamming weight varies. We plot the communication/computation tradeoff for $\bar{\lambda} = 16, 24, 32$ assuming that $b \leq 4096$.

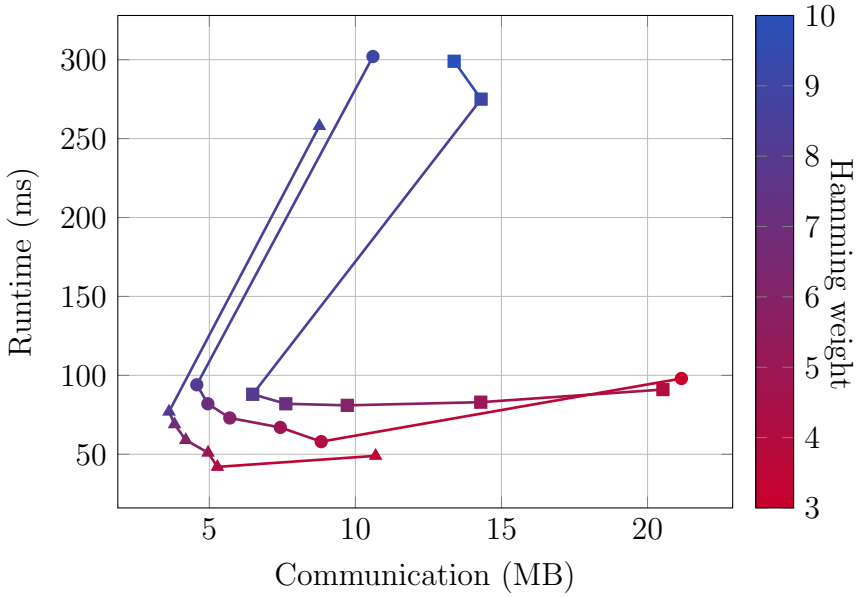


Figure 4.6: Visualizing the tradeoff between communication cost and runtime in PEPSI for $\bar{\lambda} = 16$ (triangles), $\bar{\lambda} = 24$ (squares), and $\bar{\lambda} = 32$ (circles) with $b \leq 4096$. Each point indicated running PEPSI with a specific Hamming weight which varies from 3 (on the lower right side) to 9 (on the upper left). We set $\gamma = \mu = 1$ in this example to isolate the effect of other parameters.

Notice that in some cases, the Hamming weight which is optimal for communication may incur a high burden on the computation for very little gain, and vice versa. For example, in Figure 4.3 for $\bar{\lambda} = 48$, the communication cost decreases very slightly (less than 6%) as we increase h from 16 to 23. However, in Figure 4.5, the computation cost is much lower for $h = 16$ compared to $h = 23$ (over 40%).

4.7 Evaluation

Experimentation Details. We implement PEPSI in C++ using the Microsoft SEAL[1] library and measure runtime and communication. The SEAL library implements a variant of the FV cryptosystem and supports the operations we use (recall Section 2.1). Our implementation of PEPSI and the code used in our evaluation is available on Github[2].

[1] https://github.com/microsoft/SEAL
[2] https://github.com/RasoulAM/pepsi

We run experiments using two scenarios: 1) 32-bit elements and 2) large elements. 32-bit elements can be used to represent phone numbers in private contact discovery [94, 95]. Large elements are useful for applications such as checking for compromised credentials, where credentials could be strings of arbitrary length [112]. Parameters for these two scenarios are selected as follows:

Parameters for 32-bit Elements. Based on Figure 4.4, the best Hamming weight we can choose for the communication cost is $h = 8$, which results in $N = 2^{13}$ and $\log_2 q = 192$. As explained in Section 4.4, we round the number of bins up to a multiple of N for efficiency. Consequently, in our experiments, we have $b = N = 2^{13}$, $\bar{\lambda} = 19$, and $\ell = 24$. For our binning strategy, we use Cuckoo hashing with three hash functions without a stash, so $\gamma = 1$. We choose the parameters of Cuckoo hashing such that the failure rate is less than 2^{-40}. Based on the analysis in the literature [146, 148], the number of bins must be at least $1.27m$. The server bin size depends on the server set size and the number of bins. We use upper bounds for the server bin load found in the literature [136] and confirm the numbers experimentally using a simulation of the binning strategy.

Parameters for Large Elements. We choose the parameters such that the failure rate due to colliding elements is less than $\alpha = 2^{-40}$, so λ changes as the server set size increases. The binning strategy is Cuckoo hashing with three hash functions and no stash, so $\gamma = 1$. As a result, $b = N = 16384, \lambda = 40 + \lceil \log_2(b \cdot \gamma \cdot \mu) \rceil = 40 + \lceil \log_2(b \cdot \mu) \rceil, \bar{\lambda} = 40 + \lceil \log_2(\mu) \rceil$.

4.7.1 PSI Evaluation and Comparison

Comparison with DiPSI [94]. DiPSI [94] is the most similar work in the literature and operates in one round, similar to PEPSI. While PEPSI uses a homomorphic equality operator, similar to DiPSI, it improves on DiPSI in several aspects. The two main differences are the use of a more efficient equality operator and permutation-based hashing. These differences result in significant improvements in concrete runtime and communication costs. Moreover, the advantage is increased by careful optimization of the parameters of PEPSI, as was mentioned in Section 4.6. Such optimizations are not required in DiPSI given that there is no variable parameter in the equality operator.

Table 4.1 shows that the asymptotic complexity of DiPSI and PEPSI is identical, but if we include parameters regarding the bitlength of elements, there is a difference. Table 4.4 displays the asymptotic complexity of the two protocols. The table also shows that the

multiplicative depth of PEPSI does not depend on the bitlength, which results from our choice of equality operator.

Table 4.4: Asymptotic complexity of DiPSI vs PEPSI in the unbalanced case, i.e., $m \ll n$.

Protocol	Communication	Computation	Mult. Depth
DiPSI	$O(m\lambda)$	$O(n \cdot \lambda)$	$\log_2 \lambda$
PEPSI	$O\left(m \cdot \ell(\bar{\lambda}, h)\right)$	$O\left(n \cdot \ell(\bar{\lambda}, h)\right)$	$1 + \log_2 h$

All our experiments follow the unbalanced setting where the server has more elements than the client. DiPSI performs the intersection over 32-bit elements, so we do the same for a fair comparison. Cong et al. [54] allows to use elements of arbitrary length so we compare with the variant of PEPSI which can do the same. We measure all messages exchanged as the total communication and for computation, we measure the server's total runtime. We repeat experiments three times and report the average. However, the runtime of DiPSI is prohibitively high, so we take numbers from their paper [94].

Table 4.5 shows runtime of DiPSI and PEPSI for intersection of sets with 32-bit elements. This table shows that DiPSI is much slower than our work. PEPSI is at least four orders of magnitude faster than DiPSI. Most of this speedup can be attributed to the faster comparison operator, which we use in PEPSI. The communication of PEPSI is much smaller as well, which stems from 1) better parameters for the cryptosystem, 2) permutation-based hashing in conjunction with Cuckoo hashing, and 3) optimal Hamming weight, which minimizes communication. PEPSI requires 90% less communication compared to DiPSI. DiPSI can extend to labelled PSI and circuit PSI as well, which increases the total runtime and communication. We omit DiPSI from the rest of the evaluation due to its low performance.

To summarize, PEPSI is a strict improvement over DiPSI. It is non-interactive, similar to DiPSI, requires strictly less communication, is over three orders of magnitude faster, and is capable of all functionalities that DiPSI offers.

Comparison with Cong et al. [54] Cong et al. [54] is the state-of-the-art amongst PSI protocols that only require two rounds of interaction. The work of Cong et al. has strictly improved over the work of Chen et al. [42, 43]. Cong et al. denote the time required to compute the OPRF of server elements as *offline* time. We report the offline and online time separately. We use the publicly available implementation of the work of Cong et

Table 4.5: Private set intersection over 32-bit elements using DiPSI and PEPSI. The bin size in PEPSI is $b = 8192$ in all cases. DNF indicates experiments which did not finish in less than an hour. * We copied the runtimes and communicating cost for DiPSI from their paper [94]. Note that the runtime of DiPSI is in minutes.

n	m	DiPSI [94] Time **(mins)**	DiPSI [94] Comm. (MB)	μ	PEPSI Time (s) Offline	PEPSI Time (s) Online	PEPSI Comm. (MB) Req.	PEPSI Comm. (MB) Resp.
2^{20}	1024	600*	200*	526	0.11	0.83	4.1	0.11
	2048	350*	200*			0.81	4.1	0.11
	4096	190*	200*			1.5	8.1	0.22
2^{24}	1024	DNF	DNF	6710	1.5	8.9	4.1	0.11
	2048	DNF	DNF			9.0	4.1	0.11
	4096	DNF	DNF			17.5	8.1	0.22
2^{28}	1024	DNF	DNF	100565	19.9	133	4.1	0.11
	2048	DNF	DNF			133	4.1	0.22
	4096	DNF	DNF			260	8.1	0.22

al. [54], which is published on Github[3] and is called APSI. We use parameters provided with their code for each client and server set size pair. Table 4.6 compares PEPSI and the work of Cong et al. [54] for intersection of sets with large elements.

The communication cost of PEPSI is more than the work of Cong et al. in PSI and labelled PSI, but the gap narrows as the size of the labels increases. Regarding runtime, most of the server runtime (over 95%) in the work of Cong et al. [54] is spent computing a pseudo-random function for each server element in the offline phase. They state that this step could be computed offline and stored on the server. This is a valid assumption for some applications but does not apply to scenarios in which the server database frequently changes or when we require the overall runtime to be small. In the case of PSI (without labels), for smaller server set sizes, the protocol of Cong et al. has a better total runtime. However, for server sets with over 2^{24} elements, the total runtime of PEPSI is less than the work of Cong et al., with PEPSI requiring less time in the offline phase, but more time in the online phase. Hence, in the case of PSI, PEPSI is preferable for large server sets that frequently change.

[3] https://github.com/microsoft/APSI

Table 4.6: Private set intersection and Labelled PSI over large elements using the work of Cong et al. and PEPSI. The number of bins is set to $b = 16384$ in PEPSI for all cases.

Label Size (Bytes)	n	m	Cong et al. [54] Time (s) Offline	Cong et al. [54] Time (s) Online	Cong et al. [54] Comm. (MB) Request	Cong et al. [54] Comm. (MB) Response	μ	PEPSI Time (s) Offline	PEPSI Time (s) Online	PEPSI Comm. (MB) Request	PEPSI Comm. (MB) Response
No Label	2^{20}	1024	5.6	0.44	1.3	1.2	296	0.11	7.8	30.0	0.22
		2048	5.7	0.82	2.7	1.5			7.1	30.0	0.22
		4096	6.1	0.94	3.5	2.2			7.0	30.0	0.22
	2^{24}	1024	99	1.2	1.8	2.1	3487	1.5	75.4	33.7	0.22
		2048	97	1.5	3.0	2.3			75.6	33.7	0.22
		4096	97	1.8	5.2	2.6			75.9	33.7	0.22
	2^{28}	1024	1770	7.3	3.5	9.5	50812	19.9	1144	39.2	0.22
		2048	1720	7.4	6.0	9.5			1154	39.2	0.22
		4096	1790	7.7	8.5	9.8			1141	39.2	0.22
32	2^{20}	256	92	2.4	4.2	3.8	296	0.11	7.2	30.7	2.9
		4096	95	2.4	4.4	3.5			7.0	30.7	2.9
	2^{22}	256	535	2.9	4.2	6.7	976	0.45	21.5	32.1	2.9
		4096	530	3.3	4.4	6.9			21.3	32.1	2.9
	2^{24}	256	DNF	DNF	DNF	DNF	3487	1.5	76.0	34.5	2.9
		4096	DNF	DNF	DNF	DNF			75.0	34.5	2.9
288	2^{20}	256	567	2.0	2.7	9.1	296	0.11	7.1	30.7	26.0
		4096	578	1.7	2.7	9.1			7.2	30.7	26.0
	2^{22}	256	3501	13.6	4.2	37.4	976	0.45	21.5	32.1	26.0
		4096	3388	14.2	4.4	35.3			21.5	32.1	26.0
	2^{24}	256	DNF	DNF	DNF	DNF	3487	1.5	76.5	34.5	26.0
		4096	DNF	DNF	DNF	DNF			76.1	34.5	26.0

4.7.2 Labelled PSI Evaluation and Comparison

We compare with the work of Cong et al., which is the state-of-the-art in non-interactive labelled PSI. They use the same parameters for the experiment but vary the label size. We choose 32-byte and 288-byte labels in our experiments. The results of this comparison are summarized in Table 4.6.

However, the work of Cong et al. has a high overhead when extending to labelled PSI instead of PSI. The computation cost of Cong et al. scales with the size of the label. In contrast, our work requires negligible additional server time to compute labelled PSI. The total runtime of PEPSI is consistently less than Cong et al. and the gap widens as the server set size increases, but PEPSI still requires more time in the online phase. So, in summary,

Table 4.7: PSI-Sum using PEPSI and the work of Ion et al. [90]. Times are reported in seconds, and communication is in MegaBytes. The number of bins is set to $b = 16384$ in PEPSI for all cases.

n	m	Ion et al. [90] Time (s)	Ion et al. [90] Comm. (MB)	μ	PEPSI Time (s) Offline	PEPSI Time (s) Online	PEPSI Comm. (MB)
2^{18}	1024	52.9	20.0	100	0.01	3.1	27.6
	2048	71.9	20.7			2.7	27.6
	4096	64.8	21.9			2.7	27.6
	8192	74.7	24.4			2.8	27.6
2^{20}	1024	199	78.2	296	0.11	7.0	30.0
	2048	254	78.9			7.1	30.0
	4096	214	80.1			7.0	30.0
	8192	221	82.6			6.8	30.0
2^{22}	1024	950	311	976	0.44	21.4	31.4
	2048	797	312			21.4	31.4
	4096	791	313			21.4	31.4
	8192	808	315			21.5	31.4
2^{24}	1024	3153	1240	3487	1.5	75.8	33.7
	2048	3380	1240			76.3	33.7
	4096	3120	1250			75.8	33.7
	8192	3140	1250			75.6	33.6

in the case of labelled PSI, PEPSI is consistently faster than Cong et al., particularly for large, dynamic server sets and large labels but requires more communication.

4.7.3 Circuit PSI Evaluation and Comparison

Our evaluation of circuit PSI is focused on non-interactive solutions, which are compatible with the unbalanced setting. Hence, we omit solutions based on 2PC [146, 148], which require interaction between the client and server to compute the function. DiPSI can extend to circuit PSI and evaluate arbitrary functions but has a prohibitively high runtime, as we saw in Table 4.5. Recall that the work of Chen et al. [42, 43] and Cong et al. [54] cannot extend to circuit PSI. Instead, we compare with the work of Ion et al. [90], which is also a circuit PSI protocol but is limited to only a few functions: PSI-Cardinality and PSI-Sum-with-Cardinality. We use the public implementation[4] which is provided by the authors. PSI-Stats [173] performs the same operations as Ion et al. in the first two rounds of

[4] https://github.com/google/private-join-and-compute

interaction and requires more operations when computing more complex functions. In the case of computing the PSI-Sum, PSI-Stats is identical to the work of Ion et al. Given that there is no publicly available implementation of PSI-Stats, we resort to the measurements from the work of Ion et al.

In the experiments that we perform in this section, we set the client set size to be $m \in \{1024, 2048, 4096, 8192\}$ and vary the server set size to observe the effect on communication and computation. The entire communication across all rounds is recorded and the runtime denotes the server runtime. Ion et al. [90] map elements to 256-bit strings using hash functions, so we use the variant of PEPSI with large elements to have a fair comparison. Table 4.7 summarizes the results of the experiments in this section.

Note that the cardinality is leaked whilst computing the sum in the work of Ion et al. [90]. In contrast, PEPSI does not have such leakage and is advantageous in this regard. Moreover, Ion et al. [90] is very limited in the functions that it can compute. Another observation is that the communication of the work of Ion et al. scales linearly with the server set size [90], whereas the communication cost of PEPSI increases at a much lower rate. For this reason, PEPSI is advantageous in the unbalanced setting and can scale to much larger server set sizes. The runtime of both protocols increases as the server size increases, but PEPSI is over 100 times faster than the work of Ion et al. [90]. One reason that PEPSI is faster than the work of Ion et al. [90] is the low per-element computation cost due to the use of batched homomorphic encryption. In contrast, the work of Ion et al. uses expensive modular exponentiations. The client must also perform expensive modular exponentiations, whereas in PEPSI, the client requires only a small amount of computation.

4.7.4 Summary of Evaluation

In our evaluation of PSI, we found that PEPSI demonstrates runtime that is comparable with other approaches [42, 54] but requires more communication. However, PEPSI excels particularly when the server size is very large and dynamic. In the context of labelled PSI, PEPSI is consistently faster than existing methods but requires more communication. The advantage of PEPSI increases as the label size increases. For circuit PSI, and specifically for PSI-Sum, PEPSI is faster than existing non-interactive approaches [90, 94], especially when dealing with larger server sets. Additionally, PEPSI has a significant advantage in communication cost, as it only depends on the client set size, making it highly efficient for large server sets. Furthermore, PEPSI offers the flexibility to easily extend to other functions, a capability that is not feasible with related work.

4.8 Conclusion

We propose PEPSI, a practical non-interactive circuit PSI protocol using homomorphic encryption. Some state-of-the-art PSI protocols cannot extend to circuit PSI. Those extending to circuit PSI require an interactive step with the client to compute the function or have the communication cost proportional to the server set size. These limitations are undesirable in the unbalanced setting. DiPSI, the only solution which does not have these limitations, has impractical runtimes. PEPSI addresses all these problems and proposes an efficient circuit PSI protocol with low communication overhead.

We use multiple techniques, such as constant-weight equality operators and permutation-based hashing that greatly improve the runtime and communication cost of PEPSI and result in a protocol that is competitive with existing work. We achieve competitive runtime and communication through careful optimization of parameters such as the Hamming weight.

PEPSI can compute the intersection of 1024 client elements with one million server elements in less than one second with less than 5 MB of communication. Functions such as sum and cardinality can be computed with negligible additional runtime. PEPSI is over four orders of magnitude faster than DiPSI, and 20 times faster than the work of Ion et al. [90].

Chapter 5

InsPIRe: PIR with Server-side preprocessing

5.1 Introduction

Private information retrieval (PIR) is a powerful cryptographic protocol enabling users to privately query entries from a public database held by a server. In this protocol, the server holds a database with N entries and the client wishes to retrieve the i-th entry. PIR ensures the client's query index i remains hidden from the server. The strong privacy guarantees of PIR have been critically leveraged to design privacy-preserving systems for many applications such as advertising [79], blocklists [102], certificate transparency [86], contact discovery [22, 63, 95], databases [169], file systems [124], media consumption [80], metadata-hiding communication [4, 11, 12, 104, 131], password leak checks [9, 112, 160] and web search [85]. PIR has also seen adoption for real-world deployments in industry by Apple [13], Google [1] and Microsoft [107].

PIR has been studied in both the single-server [103, 135, 138] and multiple, non-colluding server settings [27, 50, 68]. While multi-server PIR schemes are efficient, their privacy relies on stronger, non-collusion trust assumptions that may be difficult to materialize in practice. In our work, we focus exclusively on single-server PIR to avoid the these challenges.

When evaluating the efficiency of PIR protocols, there are two core cost metrics that must be considered. First, the amount of computational cost necessary to serve PIR requests. The computational cost can be measured using throughput, which is the ratio of

the database size and the server time (equivalently, the amount of time needed per database entry). The second important metric is the communication (bandwidth) needed for each PIR query. Over the past decade, many works have studied the problem of designing efficient PIR protocols that we survey below.

Server-Stored Client-Specific Keys. Starting from XPIR [2], there has been a long line of work designing PIR protocols based on the RLWE hardness assumption [4, 9, 11, 33, 125, 132]. These works obtain small online communication, but have two main drawbacks. First, they require substantially larger computational costs during queries compared to other PIR schemes (see below). Even worse, these works requires the server to store client-specific keys of several megabytes in size that must be used during PIR queries, which is very limiting in practice. These client-specific keys must be uploaded to the server (for example, either with the first query or in an offline preprocessing step). So, the total communication cost is substantially large, especially if users only perform a small number of PIR queries. Also, server storage grows with the number of users limiting applicability for systems with very large user bases.

Client-Stored Database Hints. Another line of work has considered the case of PIR with offline preprocessing where the client must retrieve and store hints about the database. Several recent works including SimplePIR and DoublePIR [86] as well as FrodoPIR [59] build LWE-based PIR schemes in this class. They obtain higher throughput compared to the prior RLWE-based PIR schemes while maintaining slightly larger, but comparable query communication. However, the database hint, that must be downloaded by each user before issuing any PIR queries, is substantially large. Furthermore, hints must be re-downloaded by all users each time the database changes.

Taking this paradigm to an even more extreme, recent work [55] has shown that client-stored database hints can obtain PIR schemes with online query time sublinear in the database size obtaining even higher throughput. Unfortunately, practical instantiations [151, 175] require every user to stream the entire database during offline preprocessing. Furthermore, the database must be streamed every $\tilde{O}(\sqrt{n})$ PIR queries unlike the prior LWE-based schemes [59, 86]. The work of ThorPIR [72] avoid streaming the entire database by, instead, using heavy cryptographic primitives requiring up to multiple hours of server computation for just one client to retrieve a hint.

Server-side Preprocessing with No Offline Communication. The main drawback of the prior two classes of PIR schemes usually come from the communication required in the offline preprocessing steps. To solve this, recent works including Tiptoe [85], HintlessPIR [110], YPIR [126], RLWEPIR [124], WhisPIR [61], and KSPIR [119] construct PIR protocols in the Common Reference String (CRS) model where there is no offline

communication. This solves many of the problems from the prior schemes that require offline communication. Unfortunately, these works end up sacrificing online communication during PIR queries to eliminate offline communication. For example, YPIR [126] requires up to megabytes of online communication more compared to PIR with client-stored hints.

In a recent breakthrough work, Lin et al. [115] showed the first PIR scheme with silent preprocessing and sublinear query time. Unfortunately, the concrete cost remains large to date rendering the construction impractical [134].

Our Question. Given the current state of affairs in PIR, we are left to navigate one of two obstacles. If we insist on both high-throughput and low query communication, then we must solve the challenges and costs associated with the offline communication for either server-stored client-specific keys or client-downloaded database hints. In contrast, if we aim to avoid offline communication and require PIR schemes with silent preprocessing, we must stomach substantial additional communication for each PIR query. This leads to the following question:

Can we construct a PIR without offline communication obtaining high-throughput and low query communication?

In this work, we answer in the affirmative with a new class of PIR protocols that obtain both high-throughput and low query communication without offline communication.

5.1.1 The Case for Server-side Preprocessing with no Offline Communication

PIR with server-side preprocessing is very useful in many applications of PIR. We list four circumstances under which server-side preprocessing is beneficial, compared to other approaches in the literature. This is not an exhaustive list and we leave more applications for PIR with server-side preprocessing to future work.

Singular Query. A common assumption in the PIR literature is that the user makes many queries such that any setup cost may be amortized. Under this assumption, PIR protocols with large initial offline communication costs are justified as the per-query costs become small over time. For example, protocols that require server-stored cryptographic keys [11, 125, 132] assume that these keys are used for many queries. Protocols that use database-dependent client-stored hints assume that the downloaded hint is used across multiple queries over some longer period of time [59, 86]. Mazmudar et al. [124] argued

that in many applications (e.g., IPFS), the client may only make one query to a server. In these settings, protocols with large initial costs make the usage of PIR infeasible. It is also not hard to see the PIR applicability challenges extending to even more use cases where clients only perform a few number of PIR queries infrequently over a long period of time. In such applications, PIR with server-side preprocessing that avoid large setup costs are critical.

Cold Start. In some applications, the client may not have enough time to perform offline communication. An example is the use of PIR for device enrollment in Chrome devices [1]. Given that the PIR query is performed upon startup, there has not been any opportunity to exchange information in the offline phase. Within these usages of PIR, the initial setup must be performed online at the same time as the PIR query. As a result, protocols with expensive preprocessing (such as streaming the database or uploading large cryptographic keys) would incur substantial costs to use PIR. In contrast, PIR protocols with server-side preprocessing may be used immediately even if the client has no time to perform offline communication.

User Anonymity. There has been work that suggests using PIR in the context of anonymity networks such as Tor [130]. In this context, the usage of PIR requiring per-client storage on the server, also known as a per-client state, is contradictory to the goal of anonymity. If a per-client state is used, the server can correlate queries made by the client, which reveals metadata. Henzinger et al. [86, Appendix B] also demonstrated the risk of client state recovery attacks by an active adversary when the client reuses a secret key. Once again, PIR with server-side preprocessing has none of these drawbacks and may be used in anonymity networks without these privacy concerns.

Large User Bases. We can also consider the setting where PIR is used to serve information to large user bases. For example, one can the consider password leak checks application that is served to all browser users [9, 107]. Once again, we could consider PIR protocols with large preprocessing costs. For protocols with server-stored keys, the PIR service must store a large per-client state for every browser user. The storage costs grow linearly in the user base size, which would be infeasible in this application. Similarly, PIR protocols with client-stored hints would require executing an expensive offline phase with each browser user that is once again challenging to scale. In either of the above types of PIR protocols, if the client loses its storage for any reason, the entire offline phase must be executed again. PIR with server-side preprocessing does not suffer from any of these scalability issues.

Database Changes. We can also consider the natural setting where the database may update on a frequent basis. For PIR with client-held hints, any application with a con-

stantly changing database will require modifying the offline database-dependent hints that have been given to clients. In many cases, this requires either re-executing the offline phase or running some specialized protocol for updating client hints [87, 175]. Nevertheless, the necessity of executing an additional protocol with every client for each database update adds substantial complexity to the system. In contrast, PIR with server-side preprocessing only requires the server to locally compute and update its internal database representation to handle database changes.

5.1.2 Technical Overview and Contributions

Efficient PIR with Server-side Preprocessing. As our main contribution, we present a new PIR protocol, InsPIRe, that simultaneously obtains high-throughput and low query communication using only server-side preprocessing in the CRS model, without any offline communication. Specifically, we obtain higher throughput and lower communication than all state-of-the-art PIR protocols in the CRS and silent preprocessing model such as YPIR [126] and HintlessPIR [110]. We observe this advantage across various database sizes with different payload sizes. For example, for retrieving a 32 KB payload from a 32 GB database, InsPIRe has 10% higher throughput and 67-90% lower communication costs compared to HintlessPIR [110] and YPIR [126]. Moreover, we can parameterize InsPIRe to optimize for other metrics, e.g., lower communication. InsPIRe can achieve 78-93% reduction in communication whilst still having better throughput than HintlessPIR and YPIR. Figure 5.1 shows a full comparison with related work.

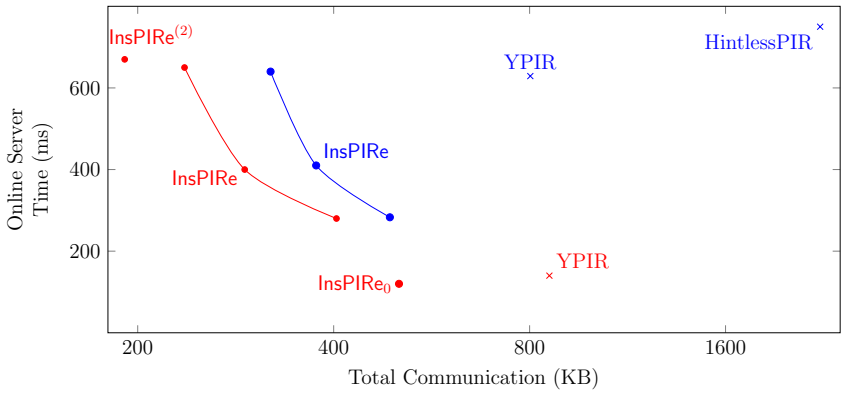


Figure 5.1: Communication Cost vs. Online Server Runtime for a PIR query over a 1 GB database. Red points retrieve 1-bit entry and blue points retrieve a 32KB entry.

Ring Packing with Preprocessing. At the core of our techniques, we develop a novel

ring packing algorithm InspiRING that translates LWE ciphertexts into a RLWE ciphertext (Section 5.3) using three steps. In the first step, LWE ciphertexts are transformed to an intermediate representation with the message encoded as a constant term of the plaintext polynomial. This is achieved by carefully applying the trace function, expressed in terms of the two generators of the Galois group. Next, ciphertexts are aggregated to obtain one ciphertext in the intermediate representation that encodes all the original messages. Finally, the result is converted back to the standard RLWE ciphertext format using only two cryptographic keys. In the context of PIR, InspiRING requires substantially less cryptographic material to compress PIR responses while simultaneously obtaining better throughput.

Homomorphic Polynomial Evaluation. We also present a novel technique to further reduce the size of the PIR response by representing database entries using polynomials. We show how to reduce the PIR response size by evaluating polynomials homomorphically in an efficient manner (Section 5.5).

Applications. We show that our new protocol, InsPIRe, can be applied to improve efficiency in several real-world deployments. To showcase the improvements across a wide-range of realistic database sizes and shapes, we show that InsPIRe can improve the efficiency of two applications that heavily rely upon PIR with silent preprocessing. First, we show that InsPIRe is a better alternative for issuing private queries in the InterPlanetary File System (IPFS), where PIR queries are issued over databases of various sizes with varying payload sizes. In this context, our work can improve communication by up to 51% and computation by up to 86% over prior work [124]. Secondly, we consider the private device enrollment that is deployed by Google [1] where PIR is used to privately check membership of identifiers in a database. We show InsPIRe requires less than 300 KB communication to respond to a request in about 800 ms, for a database of 40M identifiers. In summary, InsPIRe enables improvements across a wide set of applications and databases.

5.2 Preliminaries

Basic Notation. We use lowercase bold letters to denote vectors and uppercase bold letters to denote matrices. We use $[a_1, \ldots, a_n]$ to denote an $1 \times n$ row matrix and $[a_1, \ldots, a_n]^\top$ to denote an $n \times 1$ column matrix. For a set R, we use $R^n = R^{n\times 1}$ to denote the set of $n \times 1$ column vectors over R while $R^{1\times n}$ to denote the set of $1 \times n$ row vectors over R. We will use R and $R^{1\times 1}$ interchangeably. For a vector $\mathbf{a}$, we use $\mathbf{a}[j]$ to index the j-th element and $\mathbf{a}[i : j]$ to denote a slice in interval $[i, j)$. For a polynomial p, we denote $p[k_1 : k_2]$ as

the subset of coefficients of p, with degree in $[k_1, k_2)$. If it is clear from the context, we will treat a row vector as a tuple and vice versa and interchange them freely without explicit casting.

For a discrete probability distribution D defined over the set S, we use $x \leftarrow D(S)$ to denote sampling an element from S according to the distribution D. We use $U(S)$ to denote a uniform distribution of a set S.

We use λ to denote the security parameter.

Gaussian and Subgaussian Variables. A random variable X is subgaussian with parameter σ if $\Pr[|X| > t] \leq 2\exp(-\pi t^2/\sigma^2)$ for all $t \geq 0$. A discrete Gaussian variable with width σ is subgaussian with parameter σ. If X is subgaussian with σ, cX is subgaussian with parameter $|c|\sigma$. The sum $X_1 + \ldots + X_k$ of independent subgaussian variables (with parameters $\sigma_1, \ldots, \sigma_k$) is subgaussian with parameter $\sqrt{\sum_i \sigma_i^2}$ [143].

Cyclotomic Rings. We use cyclotomic rings $R = \mathbb{Z}[X]/(X^d + 1)$ and $R_q = \mathbb{Z}_q[X]/(X^d + 1)$, where d is a power of two. Ring elements $p(X) \in R$ may be denoted as p if clear from context.

LWE, RLWE, and MLWE. In our work, we will use both LWE- and RLWE-based private key encryption. We use $\chi(\mathbb{Z})$ to denote an error distribution over $\mathbb{Z}$ and $\chi(R)$ to denote an error distribution over the coefficients of R. We naturally extend to $\chi(\mathbb{Z}^n)$ for an error distribution over $\mathbb{Z}^n$ and $\chi(R^n)$ for an error distribution over R^n.

An LWE encryption of a message $m \in \mathbb{Z}_p$ is the pair $(\mathbf{a}, b) \in \mathbb{Z}_q^d \times \mathbb{Z}_q$ such that $b = -\langle \mathbf{a}, \mathbf{s} \rangle + e + \Delta \cdot m$ where $\mathbf{s} \leftarrow \chi(\mathbb{Z}^d)$ is a secret key, $e \leftarrow \chi(\mathbb{Z})$ is a small error value, and Δ is a scaling factor that is typically $\Delta = \lfloor q/p \rfloor$. Note, the secret key $\mathbf{s}$ enables decryption message m by computing $b + \langle \mathbf{a}, \mathbf{s} \rangle = e + \Delta \cdot m \in \mathbb{Z}_q$. When e is small, this allows retrieving $m \in \mathbb{Z}_p$ by rounding.

An RLWE [71] encryption of a message $m \in R_p$ consists of the pair $(a, b) \in R_q \times R_q$ such that $b = -as + e + \Delta \cdot m$ where $s \leftarrow \chi(R)$ is a secret key, $e \leftarrow \chi(R)$ is an error polynomial and Δ is a scaling factor that is typically $\Delta = \lfloor q/p \rfloor$. Given the secret key s, it is possible to decrypt the message m by computing $b + as = e + \Delta \cdot m \in R_q$ such that m can be retrieved by rounding as long as e is small.

While we do not directly use it in our work, we draw some useful analogies to MLWE encryption schemes [29, 106] in our packing algorithm in Section 5.3 [29, Section 2.3].

Throughout this chapter, we use d to denote both the dimension of LWE samples and the degree of RLWE samples. In both encryption schemes, we will use the version where

the public random components of the ciphertexts are fixed, which remain secure as long as the secret keys are re-sampled [144].

Gadget Matrices. [128] Given a modulus $q \in \mathbb{N}$ and a decomposition base $z \in \mathbb{N}$, we use $\mathbf{g_z} = [1, z, \ldots, z^{\ell-1}]^\top \in \mathbb{Z}_q^\ell$ where $\ell = \lceil \log q / \log z \rceil$ to denote a one-dimensional gadget matrix. We use $\mathbf{g_z^{-1}} : \mathbb{Z}_q \to \mathbb{Z}^{1\times\ell}$ to denote the base-z digit decomposition operator on an integer in $\mathbb{Z}_q$, where each component of the output vector is an integer in $[-z/2, z/2)$. We extend $\mathbf{g_z^{-1}} : \mathbb{Z}_q \to \mathbb{Z}^{1\times\ell}$ to $\mathbf{g_z^{-1}} : R_q \to R^{1\times\ell}$ to operate over the ring R_q where the digit decomposition is applied to each coefficient of the element in R_q independently.

RLWE-RGSW External Product [48]. For $\mathbf{g_z} \in \mathbb{Z}_q^\ell$ and identity matrix $\mathbf{I}_2$, let $\mathbf{G}_{2,z} = \mathbf{I}_2 \otimes \mathbf{g}_z \in R_q^{2\ell\times 2}$. An RGSW encryption of message $m \in R_p$ is $[\mathbf{a}', \mathbf{b}'] \in R_q^{2\ell\times 2}$:

$$[\mathbf{a}', \mathbf{b}'] := [\mathbf{a}, -s\mathbf{a} + \mathbf{e}] + m \cdot \mathbf{G}_{2,z}$$

where $\mathbf{a} \leftarrow U(R_q^{2\ell})$ is the random component vector and $\mathbf{e} \leftarrow \chi(R_q^{2\ell})$ is the noise vector. For a RLWE ciphertext $\mathsf{RLWE}(m_0) = (a, b)$ and a RGSW ciphertext $\mathsf{RGSW}(m_1) = [\mathbf{a}', \mathbf{b}']$, the external product is defined as:

$$\begin{aligned} \boxdot : &\mathsf{RLWE}(m_0) \times \mathsf{RGSW}(m_1) \to \mathsf{RLWE}(m_0 m_1) \\ &(a'', b'') := [\mathbf{g_z^{-1}}(a), \mathbf{g_z^{-1}}(b)] \cdot [\mathbf{a}', \mathbf{b}']. \end{aligned}$$

The result is a RLWE ciphertext of the product $m_0 m_1$.

RLWE Key-Switching. Given a RLWE ciphertext $(a, b) = (a, -as + m + e)$ encrypted under a secret key s, the goal of the key-switching procedure is to transform the ciphertext (a, b) to be encrypted under a different key s'. Let z and ℓ be the gadget parameters. The key-switching procedure consists of a pair of algorithms $\mathsf{KS} = (\mathsf{Setup}, \mathsf{Switch})$ defined as:

- $\mathsf{KS.Setup}(s, s')$: On input source secret $s \in R_q$, target secret $s' \in R_q$
  1. Sample $\mathbf{w} \leftarrow U(R_q^\ell)$ and $\mathbf{e} \leftarrow \chi(R^\ell)$.
  2. Compute $\mathbf{y} \leftarrow -s' \cdot \mathbf{w} + s \cdot \mathbf{g}_z + \mathbf{e}$
  3. Output $\mathbf{K} = [\mathbf{w}, \mathbf{y}] \in R^{\ell\times 2}$ as the key-switching matrix.
- $\mathsf{KS.Switch}((a, b), \mathbf{K})$: On input a RLWE ciphertext $(a, b) = (a, -as + m + e)$, a key-switching matrix $\mathbf{K} \in R^{\ell\times 2}$ that switches from s to s'
  1. Output $(a', b') \leftarrow (0, b) + \mathbf{g_z^{-1}}(a) \cdot \mathbf{K}$.

We refer to the paper of Chen et al. [41, Section 2.4] for more details.

Galois Group. For the cyclotomic ring R that we consider in this work, the Galois group is a group of ring automorphisms, with each element:

$$\tau_g : R \to R$$
$$\tau_g(p) = p(X^g)$$

for $g \in \{1, 3, \ldots, 2d-1\}$. The Galois group is isomorphic to the additive group $\mathbb{Z}_{d/2} \times \mathbb{Z}_2$ and can be generated using two generators [66].

For a non-negative integer i, we use τ_g^i (resp. τ_g^{-i}) to denote the Galois group element corresponding to i applications of τ_g (resp. τ_g^{-1}).

For a matrix $\mathbf{M} \in R^{n \times m}$, we extend the use of τ_g and denote $\tau_g(\mathbf{M})$ to mean that τ_g is applied to each entry of $\mathbf{M}$. We extend this notion naturally to τ_g^i as well.

Independence Heuristic. Following prior works [11, 110, 125, 132], we assume the independence heuristic where the error terms of all intermediate computations are independent. As a result, we analyze the variance of the noise vector as opposed to the worst case magnitude. In particular, we leverage the fact that variance is additive for independent subgaussian variables to analyze noise.

PIR. We consider single-server PIR schemes with single roundtrip queries consisting of the tuple of four algorithms (Setup, Query, $\mathsf{Respond}$, $\mathsf{Extract}$):

- $\mathsf{Setup}(1^\lambda, \mathbf{D}) \to (\mathsf{pp}, \mathbf{D}')$: Executed by the server, this algorithm receives security parameter 1^λ and database $\mathbf{D}$, and outputs public parameters pp and encoded database $\mathbf{D}'$. Output pp is shared with the client.
- $\mathsf{Query}(\mathsf{pp}, \mathsf{idx}) \to (\mathsf{st}, \mathsf{qry})$: Executed by the client, this algorithm receives public parameters pp and the DB index idx, and outputs client state st and query qry.
- $\mathsf{Respond}(\mathsf{pp}, \mathbf{D}', \mathsf{qry}) \to \mathsf{resp}$: Executed by the server, this algorithm receives public parameters pp, encoded database $\mathbf{D}'$, and query qry, and outputs response resp.
- $\mathsf{Extract}(\mathsf{pp}, \mathsf{st}, \mathsf{resp}) \to \mathsf{entry}$: Executed by the client, this algorithm receives public parameters pp, client state st, and server's response resp and outputs the entry.

Since we operate in the CRS model, we also implicitly pass a common random string to all algorithms. In PIR with server-side preprocessing, the public parameters pp do not

depend on the database (expect possibly the size of the database). This can be formalized by seperating the Setup algorithm into two separate algorithms that output pp and $\mathbf{D}'$, respectively. However, this does not impact security so we maintain the current notation for simplicity in the proof.

Our work follows standard security and correctness definitions from prior works [86, 126]. Below, we provide the security and correctness definitions of PIR used in our proofs.

Security. We define single query privacy for a PIR scheme using a standard indistinguishability game.

1. **Setup Phase:**
   (a) On input security parameter 1^λ, $\mathcal{A}$ outputs a database $\mathbf{D}$.
   (b) The challenger computes $(\mathsf{pp}, \mathbf{D}') \leftarrow \mathsf{Setup}(1^\lambda, \mathbf{D})$ and gives it to $\mathcal{A}$.
2. **Challenge Phase:**
   (a) $\mathcal{A}$ outputs a pair of challenge indices $(\mathsf{idx}^{(0)}, \mathsf{idx}^{(1)})$.
   (b) The challenger chooses a bit $b \in \{0, 1\}$ uniformly at random.
   (c) The challenger computes the challenge query $(\mathsf{st}^{(b)}, \mathsf{qry}^{(b)}) \leftarrow \mathsf{Query}(\mathsf{pp}, \mathsf{idx}^{(b)})$. The challenger sends $\mathsf{qry}^{(b)}$ to $\mathcal{A}$.
3. **Guess Phase:** $\mathcal{A}$ outputs a bit b'.

The advantage of the adversary $\mathcal{A}$ in this game is:

$$\mathsf{Adv}^{\mathsf{QP}}_{\mathcal{A}}(\lambda) = |\Pr[b' = b] - (1/2)|$$

We say that a PIR scheme achieves single query privacy if for any PPT adversary $\mathcal{A}$, its advantage $\mathsf{Adv}^{\mathsf{QP}}_{\mathcal{A}}(\lambda)$ is a negligible function in λ. As in SimplePIR [86], we may extend the notion of single query privacy to a sequence of Q queries.

Correctness. For all $\lambda \in \mathbb{N}$, all databases $\mathbf{D}$, and all indices idx, let $(\mathsf{pp}, \mathbf{D}') \leftarrow \mathsf{Setup}(1^\lambda, \mathbf{D})$, $(\mathsf{st}, \mathsf{qry}) \leftarrow \mathsf{Query}(\mathsf{pp}, \mathsf{idx})$ and $\mathsf{resp} \leftarrow \mathsf{Respond}(\mathsf{pp}, \mathbf{D}', \mathsf{qry})$. For a correctness parameter δ, we say that a PIR protocol is $(1 - \delta)$-correct if

$$\Pr[\mathsf{Extract}(\mathsf{pp}, \mathsf{st}, \mathsf{resp}) = \mathbf{D}[\mathsf{idx}]] \geq 1 - \delta.$$

5.2.1 Common Reference String (CRS) Model

Our construction works in the common reference string model that assumes that all parties have access to a global random string, generated by a trusted entity. In this work, this reference string is generated randomly, without any structure, which is a weaker assumtion than the common reference sting model. Under this assumption, we fix the random components of the LWE/RLWE ciphertexts, and the scheme remains secure as long as fresh secret keys are generated for each query (with multiplicative factor security loss proportional to the number of queries). Prior work such as SimplePIR [86], HintlessPIR [110], and YPIR [126] also work in this model.

In a setting such as PIR, the commmon reference string can be generated using MPC techniques involving parties other than the one that is responding to the queries to ensure it has been generated randomly and without any bias.

5.2.2 Convention of our Pseudocode Presentation

In the CRS model, the message-independent components of the ciphertexts are fixed before the online phase, enabling precomputation of protocol parts dependent solely on these message-independent components. Typically, the offline and the online phases of a protocol are described using separate sets of pseudocode. Instead, we combine both phases into a single, integrated pseudocode description.

To clearly distinguish these phases within a unified structure, operations performable during offline preprocessiong are highlighted. This convention requires a two-pass interpretation of our protocol:

1. **Offline Preprocessing Pass:** Only highlighted portions are executed and generated data is cached. Non-highlighted parts are disregarded.

2. **Online Execution Pass:** Non-highlighted portions are executed using cached values from highlighted portions.

We believe this integrated style offers a more intuitive grasp of the interplay between precomputed and online operations without having to jump back and forth between them.

5.3 Ring Packing with Preprocessing

In this section, we present InspiRING, our ring packing construction transforming LWEs into a RLWE ciphertext. InspiRING is specifically designed for the CRS model where the random components of ciphertexts are fixed and can be preprocessed. InspiRING will be the core building block of our low query communication and high-throughput PIR protocol. Prior LWE-based PIR schemes [85, 110] essentially used existing packing/bootstrapping algorithms in a blackbox manner, which mainly target a more general setting where the random parts of the ciphertexts are not necessarily known ahead and can be preprocessed.[1] Our observation is that, if we design algorithms where offline preprocessing utilize fixed random components of the ciphertexts, then we can obtain a more efficient ring packing algorithm. In comparison to the CDKS ring packing [41] used in YPIR [126], our construction is practically more efficient and has better analytical and concrete noise growth while only requiring *two key-switching matrices*. In contrast, CDKS [41] requires $\lg d$ key-switching matrices, increasing PIR communication.

Notational Convention. For the presentation in this section, we adopt a notational convention that omits the message scaling factor Δ and the error term e and assume they are part of the plaintext message. For example, the LWE pseudorandom component $b = -\langle \mathbf{a}, \mathbf{s} \rangle + e + \Delta m$ will be written as $b = -\langle \mathbf{a}, \mathbf{s} \rangle + m'$ where $m' = e + \Delta m$. For this section, we emphasize that this notational convenience does not impact the correctness or security of the described algorithm. On the other hand, the noise introduced during the packing will be written and analyzed explicitly.

5.3.1 Revisiting CDKS Packing

Before we describe our construction, we will first re-examine the CDKS packing algorithm [41] which is also used in YPIR [126]. While certain insights from the CDKS algorithm will be directly incorporated into our construction, our primary focus here is to understand why CDKS requires many key-switching matrices. In this discussion, d represents both the dimension of the initial LWE ciphertexts and the degree of the target ring.

The CDKS algorithm begins by interpreting the input LWE ciphertexts denoted as $(\mathbf{a}, b = -\langle \mathbf{a}, \mathbf{s} \rangle + m) \in \mathbb{Z}_q^d \times \mathbb{Z}_q$ as elements of $R_q \times R_q$ where $R_q = \mathbb{Z}_q[X]/(X^d + 1)$. This

[1] For PIR, these schemes did take advantage of preprocessing the random components of the ciphertexts to improve the online computation, but only by "simulating" the underlying algorithm in the offline phase.

embedding process treats the LWE ciphertext as an RLWE ciphertext, $(\tilde{a}, \tilde{b}) \in R_q \times R_q$. In this interpretation, $\tilde{b}$ corresponds to the original b (treated as a constant term polynomial), and $\tilde{a}$ is $\sum_{i=0}^{d-1} \mathbf{a}[i] \cdot X^{-i}$. Consequently, $\tilde{b}$ can be formulated as:

$$\tilde{b} = -\tilde{a}\tilde{s} + \tilde{m} \pmod{q}. \tag{5.1}$$

In this equation, $\tilde{s} = \sum_{i=0}^{d-1} \mathbf{s}[i] \cdot X^i$ represents the LWE secret $\mathbf{s}$ as a polynomial. A key outcome of this transformation is that the constant term of $\tilde{m}$ holds the original message m. However, the other coefficients of $\tilde{m}$ are filled with arbitrary values due to the embedding. If these other coefficients were zero, packing multiple messages would be simple: multiplying the RLWE ciphertext by X^i cyclically shifts its coefficients by i positions, and thus, by applying suitiable X^i multiplications and summing the resulting ciphertexts, a single RLWE ciphertext could embed all the LWE plaintexts.

For ring packing, it is necessary to zero out specific coefficient slots. This creates space for the plaintext messages from other ciphertexts to be merged into a single, consolidated RLWE ciphertext.

CDKS addresses this challenge by incrementally merging the input ciphertexts through a recursive process. This packing procedure can be viewed as a complete binary tree with depth of $\lg(d)$. The leaves of this tree are the RLWE-embedded LWE ciphertexts (Eq. 5.1). At each internal node, two RLWE ciphertexts are combined into a single, valid RLWE ciphertext. This merging step employs an automorphism to zero out the necessary coefficient slots of the plaintext message. Specifically, these automorphisms preserve certain coefficients (for already packed messages) while negating others. When a ciphertext is added to its automorphic image, the negated coefficients cancel each other, therby freeing up slots for the merging process. A subesequent key-switching operation is then performed to eliminate the extraneous term from the automorphism.

A critical aspect of this method is that the specific automorphism applied depends on the number of messages already packed. Consequently, each level in the recursion tree utilizes a distinct automorphism. This necessitates a unique key-switching matrix at each level to remove the extraneous term generated by the automorphism. As a result, this packing strategy inherently requires $\lg(d)$ key-switching matrices, which results in large cryptographic material.

It is important to note that the CDKS algorithm was not initially developed with the CRS model in mind. In the CRS model, the random components of ciphertexts are known and can be preprocessed. This raises a crucial question: can we leverage the CRS model to devise a more efficient ring packing algorithm to reduce the substantial cryptographic overhead of the existing methods such as CDKS?

5.3.2 Packing with Two Key-switching Matrices

We affirm the above question by presenting our packing algorithm that can efficiently translate d input LWE ciphertexts into a single RLWE ciphertext using only *two key-switching matrices*. Here, d is the dimension of the input LWE ciphertexts and also the degree of the target ring. Our construction is divided into three stages as follows:

1. *LWE to Intermediate Ciphertexts:* The goal of this initial stage is to reinterpret each LWE ciphertext as a *larger* intermediate representation (reminiscent of a Module-LWE ciphertext [29, 106]). Crucially, this intermediate ciphertext encrypts the original LWE message as a *constant term polynomial* and retains necessary homomorphic properties for packing. This transformation is important because a) the constant term message enables straightforward aggregation of multiple such ciphertexts (Stage 2) and b) a carefully designed "correlated structure" in these intermediate ciphertexts enables our construction to ultimately rely on just two key-switching matrices (Stage 3).

2. *Aggregation of Intermediate Ciphertexts:* Stage 2 leverages homomorphic properties of the transformed ciphertexts to combine multiple such ciphertexts into one consolidated ciphertext. Since each transformed ciphertext embeds the original LWE message as a constant term polynomial, the aggregation becomes straightforward. The aggregated ciphertext's plaintext polynomial embeds the input LWE messages into distinct coefficients.

3. *Conversion to RLWE Ciphertext:* The final stage converts the intermediate ciphertext (which now holds all packed messages) into a standard, compact RLWE ciphertext. This is achieved via an iterative key-switching procedure that "collapses" the components, efficiently utilizing the correlated structure and Galois automorphic images of two elementary key-switching matrices.

At first, the transformation of LWE ciphertexts into significantly larger intermediate ciphertext in the initial stage might appear counterintuitive to achieving efficiency. However, the key to this approach lies in the nature of these expanded structures. The bulk of the data within these intermediate forms is determined solely by the fixed random components of the original LWE ciphertexts and the key-switching matrices. In the CRS model where these random elements are fixed and known beforehand, the computationally intensive operations involved in constructing and processing these larger structures can be mainly moved to an offline preprocessing phase. This idea of investing more in offline computation is a recurring principle across all the three stages of our packing algorithm. It allows a

substantial reduction in online computation time and minimizes overhead associated with cryptographic materials during the online packing process.

Stage 1: LWE to Intermediate Ciphertext. The first stage transforms each LWE input ciphertext into a specially structured ciphertext that resembles a Module-LWE (MLWE) ciphertext [29, 106]. Specifically, we will construct an intermediate ciphertext where the random component and the secret key are *modules* over the ring $R_q = \mathbb{Z}_q[X]/(X^d + 1)$. The pseudorandom component is an element in R_q and will embed the original LWE message as a constant term polynomial, masked by an inner product between the random component and the secret key. However, the crucial difference from a typical MLWE ciphertext is the carefully designed correlations among the secret key components. Looking ahead, this correlated structure of the secret key components is essential for enabling the use of two key-switching matrices in Stage 3.

The core idea begins with embedding the input LWE ciphertext, $(\mathbf{a}, b = -\langle \mathbf{a}, \mathbf{s} \rangle + m)$, as an RLWE ciphertext $(\tilde{a}, \tilde{b} = -\tilde{a}\tilde{s} + \tilde{m})$ as in CDKS [41] (see Eq. 5.1). However, as discussed previously, the non-zero coefficients of the embedded plaintext message need to be zeroed out to enable packing. CDKS handles this by incrementally merging the ciphertexts in a recursive fashion, only freeing up necessary coefficient slots required for that stage of merging.

Instead, our insight is to transform the RLWE interpretation $(\tilde{a}, \tilde{b})$ into a carefully designed intermediate representation *up front* that is more amenable to packing. Specifically, we aim to construct an intermediate ciphertext that embeds the original LWE message as a constant term polynomial, e.g., message of the form $\hat{m}(X) = m$. Looking ahead, this "sparsity" of the message allows multiple such ciphertexts to be aggregated without interference between messages.

To achieve this, we leverage the following formulation of the trace function (defined as the sum of Galois conjugates [66]) expressed in terms of two generators of the Galois group. We recall that for the cyclotomic ring R in this work, the Galois group is isomorphic to the additive group $\mathbb{Z}_{d/2} \times \mathbb{Z}_2$ [66]. τ_g and τ_h in Lemma 2 correspond to the two generators $(1, 0)$ and $(0, 1)$ in $\mathbb{Z}_{d/2} \times \mathbb{Z}_2$.

Lemma 2. *Let $p(X) \in \mathbb{Z}[X]/(X^d + 1)$ such that $p(X) = \sum_{i=0}^{d-1} c_i X^i$ where d is a power of two. Let $g = 5$ and $h = 2d - 1$, and define $Tr : R \to R$ as*

$$Tr(p) := \sum_{j=0}^{d/2-1} \tau_g^j(p) + \tau_h \circ \tau_g^j(p).$$

Then $Tr(p) = d \cdot c_0$.

First, we state a useful number theoretic fact from [66].

Lemma 3. *[66] Let d be a power of two and $\gamma < d$ also a power of two. Let $g = 2d/\gamma + 1 \in \mathbb{Z}_{2d}^*$. Then $\mathsf{ord}(g) = \gamma$.*

Lemma 4. *Let d be a power of two and $\gamma < d$ also a power of two. Let $g = 2d/\gamma + 1 \in \mathbb{Z}_{2d}^*$. Then the map $g^i \pmod{2d} \mapsto g^i + g - 1 \pmod{2d}$ is a bijection.*

Proof. First, we see that $g^i \equiv 1 \pmod{g-1}$, which can be proved by induction. By Lemma 3, $\mathsf{ord}(g) = \gamma$. Pigeon hole principle implies that the elements of $\{g^i \pmod{2d} \mid 0 \leq i < \gamma\}$ are precisely of the form $g + j(2d/\gamma)$ for $0 \leq j < \gamma$. Therefore, the map $g^i \pmod{2d} \mapsto g^i + g - 1 \pmod{2d}$ is a bijection. □

The next lemma is central to the proof of Lemma 2.

Lemma 5. *Let $p(X) = \sum_{j=0}^{d-1} c_j X^j$. Let $\gamma < d$ be a power of two integer and $g = 2d/\gamma + 1$. Then,*

$$\pi_\gamma(p) := \sum_{i=0}^{\gamma-1} \tau_g^i(p) \implies \pi_\gamma(p) = \gamma \sum_{\gamma | i} c_i X^i.$$

Proof. We have $\tau_g^i(p) = \sum_{j=0}^{d-1} c_j X^{jg^i}$ and so

$$\pi_\gamma(p) = \sum_{i=0}^{\gamma-1} \sum_{j=0}^{d-1} c_j X^{jg^i} = \sum_{j=0}^{d-1} c_j \Big(\sum_{i=0}^{\gamma-1} X^{jg^i} \Big).$$

Thus, it suffices to show that $\sum_{i=0}^{\gamma-1} X^{jg^i} = \gamma \cdot X^j$ if $\gamma \mid j$ and 0 otherwise.

Suppose that $\gamma \mid j$. It suffices to show that $X^{jg} = X^j$, which would imply that $X^{jg^i} = X^j$ for all $0 \leq i < \gamma$. Now, since $g = 2d/\gamma + 1$, $jg = 2dj/\gamma + j$. But since $\gamma \mid j$, it follows that $2dj/\gamma$ is a multiple of $2d$, which implies that $X^{jg} = X^{2dj/\gamma + j} = X^j$. Therefore, we have $\sum_{i=0}^{\gamma-1} X^{jg^i} = \gamma \cdot X^j$. On the other hand, suppose that $\gamma \nmid j$. We want to show that $\sum_{i=0}^{\gamma-1} X^{jg^i} = 0$. Let $\omega = X^{j(g-1)}$. Then,

$$\omega \sum_{i=0}^{\gamma-1} X^{jg^i} = \sum_{i=0}^{\gamma-1} X^{j(g^i + g - 1)}.$$

By Lemma 4, the map $g^i \pmod{2d} \mapsto g^i + g - 1 \pmod{2d}$ is a bijection, which implies that

$$\sum_{i=0}^{\gamma-1} X^{j(g^i+g-1)} = \sum_{i=0}^{\gamma-1} X^{jg^i}.$$

This shows that $(\omega - 1)\sum_{i=0}^{\gamma-1} X^{jg^i} = 0$. It remains to show that $\omega \neq 1$, which would prove that $\sum_{i=0}^{\gamma-1} X^{jg^i} = 0$ because $\mathbb{Z}[X]/(X^d + 1)$ is an integral domain. But $\omega = X^{j(g-1)} = X^{2dj/\gamma} = 1$ if and only if $2dj/\gamma$ is a multiple of $2d$ if and only if $\gamma \mid j$. But since $\gamma \nmid j$, it follows that $\omega \neq 1$, completing the proof. □

Finally, we prove the main lemma using the above.

Proof of Lemma 2. Applying Lemma 5 with $\gamma = d/2$ and $g = 5$ yields

$$\pi_{d/2}(p) = (d/2)\left(c_0 + c_{d/2}X^{d/2}\right)$$

. Now

$$\tau_h \circ \pi_{d/2}(p) = \frac{d}{2}\left(c_0 + c_{d/2} \cdot \tau_h(X^{d/2})\right) = \frac{d}{2}\left(c_0 - c_{d/2}X^{d/2}\right)$$

But $\mathrm{Tr} = \pi_{d/2} + \tau_h \circ \pi_{d/2}$ implying $\mathrm{Tr}(p) = d \cdot c_0$. □

The key insight to forming the intermediate ciphertext lies in how its components are constructed from the initial RLWE-like interpretation $(\tilde{a}, \tilde{b})$. The first (random) component of the new intermediate ciphertext is a vector of d polynomials $\hat{\mathbf{a}} \in R_q^d$. The first half of $\hat{\mathbf{a}}$ is defined as $\hat{\mathbf{a}}[j] = d^{-1} \cdot \tau_g^j(\tilde{a})$ for $j < d/2$. The second half is the automorphic image of the first half by τ_h, e.g. $\hat{\mathbf{a}}[d/2 : d] = \tau_h(\hat{\mathbf{a}}[0 : d/2])$.

The original $\tilde{b}$ is re-interpreted to serve as the pseudorandom component of this new intermediate ciphertext. Specifically, suppose we define the corresponding intermediate secret key vector (module) as $\hat{\mathbf{s}} \in R_q^d$ where the first half is $\hat{\mathbf{s}}[j] = \tau_g^j(\tilde{s})$ and the second half is the automorphic image of the first half by τ_h, e.g. $\hat{\mathbf{s}}[d/2 : d] = \tau_h(\hat{\mathbf{s}}[0 : d/2])$. Then, the component $\tilde{b}$ satisfies:

$$\tilde{b} = -\langle \hat{\mathbf{a}}, \hat{\mathbf{s}} \rangle + \hat{m} \pmod{q} \tag{5.2}$$

We note that this equation is derived from the application of the operator Tr from Lemma 2 appropriately on Eq. 5.1 (detailed in Appendix B.1.1). In particular, the pair $(\hat{\mathbf{a}}, \tilde{b})$ forms an MLWE-like ciphertext encrypting message $\hat{m}(X) = m$.

Importantly, the structure $(\hat{\mathbf{a}}, \tilde{b})$ maintains necessary homomorphic properties (akin to MLWE) for packing. It supports plaintext absorption (multiplication to each component) and homomorphic addition (component-wise addition), which is crucial for Stage 2.

In summary, Stage 1 converts an LWE ciphertext $(\mathbf{a}, b)$ (encrypting m) into an intermediate ciphertext encrypting $\hat{m}(X) = m$, which we denote as $\mathsf{IRCtx}(\hat{m})$ (stands for Intermediate Representation CipherTeXt).

We present the formal pseudocode of this transformation in Algorithm 17's subroutine TRANSFORM. See Appendix B.1.1 for further detailed mathematical derivation of $\mathsf{IRCtx}(\hat{m})$.

Stage 2: Aggregation of Intermediate Ciphertexts. Stage 2 combines d IRCtx ciphertexts (from Stage 1) into a single IRCtx, packing their messages as distinct coefficients.

Consider d input LWE ciphertexts: $(\mathbf{a}_0, b_0)$, $\ldots$, $(\mathbf{a}_{d-1}, b_{d-1})$, where the k-th ciphertext encrypts m_k. Applying Stage 1's TRANSFORM to each $(\mathbf{a}_k, b_k)$ yields: $\mathsf{IRCtx}(\hat{m}_k) \leftarrow \text{TRANSFORM}(\mathbf{a}_k, b_k)$, where $\hat{m}_k(X) = m_k$ is a constant term polynomial.

We then multiply each $\mathsf{IRCtx}(\hat{m}_k)$ by X^k and sum the results. Leveraging the homomorphic properties of IRCtx, we obtain the following:

$$\sum_{k=0}^{d-1} \mathsf{IRCtx}(\hat{m}_k) \cdot X^k = \mathsf{IRCtx}(\sum_{k=0}^{d-1} \hat{m}_k X^k).$$

We denote this aggregated plaintext message as $\hat{m}_{agg} := \sum_{k=0}^{d-1} \hat{m}_k X^k$ and the corresponding ciphertext encrypting it $\mathsf{IRCtx}(\hat{m}_{agg}) = (\hat{\mathbf{a}}_{agg}, \tilde{b}_{agg})$. The above operation positions the original plaintext messages in the LWE ciphertexts, m_k, into the coefficients of $\hat{m}_{agg}$. However, the intermediate ciphertext produced at this stage, $\mathsf{IRCtx}(\hat{m}_{agg})$, is considerably large, comprising $d+1$ elements from the ring R_q. The formal pseudocode of this stage is provided in Algorithm 17's PACK procedure, Line 4.

Stage 3: Conversion to RLWE Ciphertext. The aggregated intermediate ciphertext $\mathsf{IRCtx}(\hat{m}_{agg}) = (\hat{\mathbf{a}}_{\mathbf{agg}}, \tilde{b}_{agg}) \in R_q^d \times R_q$ is currently represented by $d+1$ ring elements in R_q and is effectively encrypted under a vector of secret key shares $\hat{\mathbf{s}}$ with $\hat{\mathbf{s}}[j] = \tau_g^j(\tilde{s})$ and $\hat{\mathbf{s}}[j + d/2] = \tau_h \circ \tau_g^j(\tilde{s})$ for $j < d/2$.

As a reminder, $\tilde{b}_{agg}$ can be expressed as (from Eq. 5.2):

$$\begin{aligned}
\tilde{b}_{agg} &= -\langle \hat{\mathbf{a}}_{agg}, \hat{\mathbf{s}} \rangle + \hat{m}_{agg} \pmod q \\
&= -\left(\sum_{j=0}^{d/2-1} \hat{\mathbf{a}}_{agg}[j] \cdot \tau_g^j(\tilde{s}) \right) \pmod q \\
&\quad - \left(\sum_{j=0}^{d/2-1} \hat{\mathbf{a}}_{agg}[j + d/2] \cdot \left(\tau_h \circ \tau_g^j(\tilde{s})\right) \right) + \hat{m}_{agg} \pmod q.
\end{aligned}$$

To obtain a standard two-component RLWE ciphertext encrypted under a single base secret key $\tilde{s}$, we must reduce the number of these components by re-expressing the masking part $\langle \hat{\mathbf{a}}_{\mathbf{agg}}, \hat{\mathbf{s}} \rangle$ with respect to a single base secret key $\tilde{s}$.

At a high level, this reduction is achieved through an iterative process that systematically "collapses" the components. In each step, a RLWE key-switching procedure (see Section 5.2 for full description) is applied, which re-expresses the pseudorandom component originally masked by one secret key share $\tau_g^k(\tilde{s})$ (resp. $\tau_h \circ \tau_g^k(\tilde{s})$) so that its masking now relies on a different share $\tau_g^{k-1}(\tilde{s})$ (resp. $\tau_h \circ \tau_g^{k-1}(\tilde{s})$), effectively reducing the number of distinct key shares in the overall ciphertext masking. This iterative process eventually consolidates all parts under two key shares $\tilde{s}$ and $\tau_h(\tilde{s})$.

This process relies on a set of key-switching matrices, but all necessary key-switching matrices are automorphic images of a single elementary key-switching matrix which we denote as $\mathbf{K}_g = \mathsf{KS.Setup}(\tau_g(\tilde{s}), \tilde{s})$. This is because if $\mathbf{K}_g$ transforms a ciphertext component from being encrypted under $\tau_g(\tilde{s})$ to $\tilde{s}$, then its automorphic image $\tau_g^k(\mathbf{K}_g)$ (resp. $\tau_h \circ \tau_g^k(\mathbf{K}_g)$) will transform a component from $\tau_g^{k+1}(\tilde{s})$ to $\tau_g^k(\tilde{s})$ (resp. $\tau_h \circ \tau_g^{k+1}(\tilde{s})$ to $\tau_h \circ \tau_g^k(\tilde{s})$).

This creates a "telescoping" effect where each step in the reduction uses a key related to $\mathbf{K}_g$ by an appropriate Galois automorphism, allowing the component reduction to be performed efficiently using variants of just one base key-switching matrix. After $d-2$ such iterations, the number of random components of $\mathsf{IRCtx}(\hat{m}_{agg})$ is reduced to two, with the resultant ciphertext encrypted under two secret key shares $\tilde{s}$ and $\tau_h(\tilde{s})$. To eliminate the key share $\tau_h(\tilde{s})$, we perform the final key-switching using $\mathbf{K}_h = \mathsf{KS.Setup}(\tau_h(\tilde{s}), \tilde{s})$. The final result is a standard two-component RLWE ciphertext (a_{fin}, b_{fin}), where $b_{fin} = -a_{fin} \cdot \tilde{s} + \hat{m}_{agg} + e_{tot}$ and e_{tot} is the total accumulated noise from the key-switchings.

The formal pseudocode of this stage is presented in Algorithm 17's subroutine Collapse. The full details of this iterative component reduction algorithm, including the specific key-switching steps, are provided in Appendix B.1.2.

Putting It Together. The discussions in the preceding sections can be summarized as the unified algorithm InspiRING.Pack. The pseudocode is in Algorithm 17. We remind the readers that the highlighted parts of the algorithm correspond to the preprocessable components in the CRS model, as mentioned in Section 5.2.2. We denote $\tilde{s}$ as the natural polynomial interpretation of the LWE secret $\mathbf{s}$.

Preprocessing in the CRS Model. A key strength of the InspiRING algorithm is its inherent suitability for preprocessing in the CRS model where the random components of the ciphertexts are fixed. As detailed in the preceding stages, a substantial portion of the

Algorithm 17 InspiRING algorithm

algorithm PACK($[\mathbf{A}, \mathbf{b}] \in \mathbb{Z}_q^{d \times (d+1)}, \mathbf{K}_g = [\mathbf{w}_g, \mathbf{y}_g], \mathbf{K}_h = [\mathbf{w}_h, \mathbf{y}_h] \in R_q^{\ell \times 2}$)
  for $k = 0 \dots d-1$ **do**
    $(\hat{\mathbf{a}}_k, \tilde{b}_k) \leftarrow$ TRANSFORM$(\mathbf{A}[k], \mathbf{b}[k])$
  $(\hat{\mathbf{a}}_{\mathbf{agg}}, \tilde{b}_{agg}) \leftarrow \sum_{k=0}^{d-1} (\hat{\mathbf{a}}_k, \tilde{b}_k) \cdot X^k$
  $(a_{fin}, b_{fin}) \leftarrow$ COLLAPSE$((\hat{\mathbf{a}}_{agg}, \tilde{b}_{agg}), [\mathbf{w}_g, \mathbf{y}_g], [\mathbf{w}_h, \mathbf{y}_h])$
  return $(a_{fin}, b_{fin}) \in R_q \times R_q$

— Subroutines —

algorithm TRANSFORM$((\mathbf{a}, b) \in \mathbb{Z}_q^d \times \mathbb{Z}_q)$
  $\tilde{a} \leftarrow d^{-1} \sum_{i=0}^{d-1} \mathbf{a}[i] X^{-i}$ // $d \cdot d^{-1} = 1 \mod q$
  $\tilde{b} \leftarrow b$
  Initialize $\hat{\mathbf{a}} \in R_q^d$
  for $j = 0 \dots d/2 - 1$ **do**
    $\hat{\mathbf{a}}[j] \leftarrow \tau_g^j(\tilde{a})$
    $\hat{\mathbf{a}}[j + d/2] \leftarrow \tau_h \circ \tau_g^j(\tilde{a})$
  return $(\hat{\mathbf{a}}, \tilde{b}) \in R_q^d \times R_q$

algorithm COLLAPSE$((\hat{\mathbf{a}}_{agg}, \tilde{b}_{agg}) \in R_q^d \times R_q, \mathbf{K}_g = [\mathbf{w}_g, \mathbf{y}_g] \in R_q^{\ell \times 2}, \mathbf{K}_h = [\mathbf{w}_h, \mathbf{y}_h] \in R_q^{\ell \times 2}$)
  $\hat{\mathbf{a}}_1 \leftarrow \hat{\mathbf{a}}_{agg}[0 : d/2]$
  $\hat{\mathbf{a}}_2 \leftarrow \hat{\mathbf{a}}_{agg}[d/2 : d]$
  $(a_1, b_1) \leftarrow$ COLLAPSEHALF$((\hat{\mathbf{a}}_1, \tilde{b}_{agg}, [\mathbf{w}_g, \mathbf{y}_g], I))$
  $(a_2, b_2) \leftarrow$ COLLAPSEHALF$((\hat{\mathbf{a}}_2, b_1, [\mathbf{w}_g, \mathbf{y}_g], \tau_h))$
  $(a, b) \leftarrow$ COLLAPSEONE$((a_1, a_2], b_2), [\mathbf{w}_h, \mathbf{y}_h])$
  return $(a, b) \in R_q \times R_q$

algorithm COLLAPSEHALF$((\hat{\mathbf{a}}_{half}, \tilde{b}_{half}) \in R_q^{d/2} \times R_q, \mathbf{K}_g = [\mathbf{w}_g, \mathbf{y}_g] \in R_q^{\ell \times 2}, \rho \in \{I, \tau_h\}$)
  Rename $(\hat{\mathbf{a}}_{half}, \tilde{b}_{half})$ as $(\mathbf{a}^{(d/2-1)}, b^{(d/2-1)})$
  for $k = d/2 - 1 \dots 0$ **do**
    $\mathbf{K}_g^{k-1} \leftarrow \rho \circ \tau_g^{k-1}(\mathbf{K}_g)$
    $(\mathbf{a}^{(k-1)}, b^{(k-1)}) \leftarrow$ COLLAPSEONE$((\mathbf{a}^{(k)}, b^{(k)}), \mathbf{K}_g^{k-1})$
  return $(\mathbf{a}^{(0)}, b^{(0)}) \in R_q \times R_q$

algorithm COLLAPSEONE$((\mathbf{a}, b) \in R_q^k \times R_q, \mathbf{K} = [\mathbf{w}, \mathbf{y}] \in R_q^{\ell \times 2}$)
  $(a', b') \leftarrow$ KS.Switch$((\mathbf{a}[k-1], b), [\mathbf{w}, \mathbf{y}])$
  $\mathbf{a}' \leftarrow [\mathbf{a}[0], \dots, \mathbf{a}[k-3], \mathbf{a}[k-2] + a']^\top \in R_q^{k-1}$
  return $(\mathbf{a}', b') \in R_q^{k-1} \times R_q$

data manipulated during the packing process depends solely on the random components of the initial LWE ciphertexts and the key-switching matrices $\mathbf{K}_g$ and $\mathbf{K}_h$. This characteristic allows for significant portions of the computation to be performed offline, making the online

packing phase very efficient.

The benefits of this approach are evident in the first two stages. In Stage 1, the random components of the resultant $\mathsf{IRCtx}(\hat{m}_k)$ ciphertexts are entirely determined by the random vectors of the input LWE ciphertexts. Similarly, in Stage 2, the bulk of operations involves combining these preprocessable random components. The online aspects of these stages are minimal: the re-interpretation of the LWE pseudorandom components b_k into their ring element counterparts $\tilde{b}_k$ requires no actual computation, and their aggregation into $\mathsf{IRCtx}(\hat{m}_{agg})$ is a straightforward process of assigning the constant $\tilde{b}_k$ values as polynomial coefficients.

The advantages extend to Stage 3, though perhaps less immediately obvious. This stage involves an iterative reduction of the intermediate ciphertext's components using key-switchings. After Stage 2, the $\mathsf{IRCtx}(\hat{m}_{agg})$ consists of d random components (which, as established, can be preprocessed offline) and a single pseudorandom polynomial component $\tilde{b}_{agg}$ (which must be processed online). The iterative key-switching procedure is designed to maintain an important invariant: throughout the reduction, the evolving random components remain dependent only on the initial, preprocessable random components of the input LWE ciphertexts and the key-switching matrices $\mathbf{K}_g$ and $\mathbf{K}_h$. This crucial property ensures that most data necessary to update the pseudorandom component during each step of the reduction can also be precomputed. Consequently, the online operations in Stage 3 are significantly efficient, contributing to the overall efficiency of the online packing algorithm.

Theorem 3. InspiRING *in the CRS model can pack d LWE ciphertexts in $O(d^3 + \ell d^2 \lg d)$ offline time and $O(\ell \cdot d^2)$ online time where ℓ is the dimension of the key-switching matrix.*

Proof. We provide the proof for the offline and online time separately below:

- **Offline time:** First, we require d iterations of the Transform function. Each iteration of Transform performs $d/2$ iterations of the for loop which performs automorphisms on $\tilde{a}$, which happens in $O(d)$. Overall, this gives the $O(d^3)$ term. After that, we perform $d+1$ iterations of CollapseOne, each of which performs one keyswitch. As part of each keyswitch, we perform ℓ polynomial multiplications, which require conversion to and from NTT format. So each keyswitch is $O(\ell d \lg d)$, resulting in $O(\ell d^2 \lg d)$ runtime for this part. Overall, the complexity is $O(d^3 + \ell d^2 \lg d)$

- **Online time:** In the CollapseHalf subroutine, we perform $d+1$ iterations of CollapseOne. Each iteration of CollapseOne performs ℓ polynomial multiplications as

part of KS.Switch. Since keys and ciphertexts are in NTT format, we require d multiplications to perform each polynomial multiplication. Overall, this results in complexity of $O(\ell d^2)$.

□

Furthermore, our algorithm also results in smaller noise growth (both asymptotically and practically).

Theorem 4. *Let the error distribution χ be subgaussian with parameter σ_χ. Let ℓ be the dimension of the key-switching matrix and z be the decomposition base. Under the independence heuristic,* InspiRING *incurs an additive noise $e_{pack} \in R_q$, which has subgaussian coefficients with parameter σ_{pack} and $\sigma_{pack}^2 \leq \ell d^2 z^2 \sigma_\chi^2/4$.*

Our packing scheme has the same asymptotic running time, but much faster concrete running time as well as smaller noise growth compared to CDKS [41]. We corroborate our analysis with experimental evaluation in Section 5.6.4.

Security. Beyond RLWE hardness, our packing scheme relies on the standard *circular security* assumption, as key-switching matrices encrypt (scaled) automorphic images of the secret key. This assumption is common in lattice based FHE literature [19, 28–30, 41, 75, 92] and PIR [11, 33, 110, 125, 126, 132].

5.3.3 Partial Packing with One Key-Switching Matrix

InspiRING shows how to pack d LWE ciphertexts into one RLWE ciphertext. This algorithm can be simplified to pack $\gamma \leq d/2$ LWE ciphertexts as well. There are two advantages to partial packing compared to full packing. First, we show how to achieve this using only one key switching matrix instead of two, which reduces the required key material by half. The second advantage is that since only γ LWEs are packed within each RLWE ciphertext, we keep the first γ coefficients of b_{fin} and discard the remaining coefficients. This is useful as means to reduce communication when transmitting ciphertexts over the network and is also used in one of our PIR constructions. Algorithm 18 shows the algorithm for partial packing which we denote as PartialInspiRING.

Theorem 5. PartialInspiRING *in the CRS model can pack $\gamma \leq d/2$ LWE ciphertexts in $O(\gamma^2 d + \ell\gamma d \lg d)$ offline time and $O(\ell\gamma d)$ online time where ℓ is the dimension of the key-switching matrix.*

Algorithm 18 PartialInspiRING algorithm for packing γ LWE ciphertexts

algorithm PartialPack($[\mathbf{A}, \mathbf{b}] \in \mathbb{Z}_q^{\gamma \times (d+1)}, \mathbf{K}_g = [\mathbf{w}_g, \mathbf{y}_g] \in R_q^{\ell \times 2}$)
  for $k = 0 \dots \gamma - 1$ **do**
    $(\hat{\mathbf{a}}_k, \tilde{b}_k) \leftarrow$ TransformPartial($\mathbf{A}[k], \mathbf{b}[k]$)
  $(\hat{\mathbf{a}}_{\mathbf{agg}}, \tilde{b}_{agg}) \leftarrow \sum_{k=0}^{\gamma-1} (\hat{\mathbf{a}}_k, \tilde{b}_k) \cdot X^k$
  $(a_{fin}, b_{fin}) \leftarrow$ CollapsePartial($\gamma, (\hat{\mathbf{a}}_{agg}, \tilde{b}_{agg}), [\mathbf{w}_g, \mathbf{y}_g]$)
  return $(a_{fin}, b_{fin}) \in R_q \times R_q$

— Subroutines —

algorithm TransformPartial(γ, $(\mathbf{a}, b) \in \mathbb{Z}_q^d \times \mathbb{Z}_q$)
  $\tilde{a} \leftarrow \gamma^{-1} \sum_{i=0}^{d-1} \mathbf{a}[i] X^{-i}$ // $\gamma \cdot \gamma^{-1} = 1 \mod q$
  $\tilde{b} \leftarrow b$
  Initialize $\hat{\mathbf{a}} \in R_q^\gamma$
  for $j = 0 \dots \gamma - 1$ **do**
    $\hat{\mathbf{a}}[j] \leftarrow \tau_g^j(\tilde{a})$
  return $(\hat{\mathbf{a}}, \tilde{b}) \in R_q^\gamma \times R_q$

algorithm CollapsePartial(γ, $(\hat{\mathbf{a}}_{partial}, \tilde{b}_{partial}) \in R_q^\gamma \times R_q, \mathbf{K}_g = [\mathbf{w}_g, \mathbf{y}_g] \in R_q^{\ell \times 2}$)
  Rename $(\hat{\mathbf{a}}_{partial}, \tilde{b}_{partial})$ as $(\mathbf{a}^{(\gamma-1)}, b^{(\gamma-1)})$
  for $k = \gamma - 1 \dots 0$ **do**
    $\mathbf{K}_g^{k-1} \leftarrow \tau_g^{k-1}(\mathbf{K}_g)$
    $(\mathbf{a}^{(k-1)}, b^{(k-1)}) \leftarrow$ CollapseOne($(\mathbf{a}^{(k)}, b^{(k)}), \mathbf{K}_g^{k-1}$)
  return $(\mathbf{a}^{(0)}, b^{(0)}) \in R_q \times R_q$

Theorem 6. *Let the error distribution χ be subgaussian with parameter σ_χ. Let ℓ be the dimension of the key-switching matrix and z be the decomposition base. Under the independence heuristic,* PartialInspiRING *incurs an additive noise $e_{pack} \in R_q$, such that the first γ coefficients of e_{pack} are subgaussian with parameter σ_{pack} and $\sigma_{pack}^2 \leq \ell \gamma d z^2 \sigma_\chi^2 / 4$.*

5.3.4 InsPIRe$_0$: PIR from Ring Packing

A direct application of InspiRING (and PartialInspiRING) to PIR results in our first PIR construction InsPIRe$_0$. InsPIRe$_0$ is instantiated on top of DoublePIR [86] by using InspiRING or PartialInspiRING to pack the result of the DoublePIR responses. This protocol is useful when the entry size is small (e.g., 1 bit) and when optimizing for specific metrics such as runtime, as we show in the evaluation.

5.4 InsPIRe$^{(2)}$: PIR from Double Ring Packing

Our next construction based on our new ring packing algorithm is denoted as InsPIRe$^{(2)}$. This protocol takes advantage of two layers of ring packing, specifically partial ring packing. The algorithm is designed such that most of the work is done in an offline phase. At a high level, InsPIRe$^{(2)}$ consists of two levels of packing as follows:

- We perform a first level of PIR using LWE and perform partial packing on the resulting LWE ciphertexts using γ_0 as the packing parameter. The packed RLWE ciphertexts are modulus switched and decomposed into plaintexts, which are used as the database in the second layer of PIR.
- In the second level, we start again with PIR using LWE ciphertexts and pack the resulting LWE ciphertexts again. However, the packing in this level is done in two parts, using two different packing parameters, γ_1 and γ_2.

We present our detailed PIR protocol InsPIRe$^{(2)}$ = (Setup, Query, Respond, Extract) using PartialInspiRING in Algorithms 19, 20, 21, 22. The database consists of N records, each of which is an element of $\mathbb{Z}_p^{\gamma_0}$. The database is, however, restructured into a matrix $\mathbf{D} \in \mathbb{Z}_p^{t\gamma_0 \times N/t}$ For convenience, we assume that the public parameters pp are globally available to the subroutines of the PIR algorithms and do not explicitly specify them as inputs.

Algorithm 19 InsPIRe$^{(2)}$.Setup

algorithm SETUP($1^\lambda, \mathbf{D} \in \mathbb{Z}_p^{t\gamma_0 \times N/t}$)
  $\mathsf{rp} \leftarrow (d(\lambda), q(\lambda), \tilde{q}, \chi(\lambda), p)$ as the RLWE params
  $\mathsf{gp}_{ks} \leftarrow (z_{ks}, \ell_{ks})$ as the gadget params for the KS matrices
  $\mathsf{qry}_{\mathsf{off}} \leftarrow$ GENFIXEDQUERYPARTS()
  $\tau \leftarrow \lceil \tilde{q}/p \rceil$
  $\mathsf{pp} \leftarrow (N, t, \gamma_0, \gamma_1, \gamma_2, \mathsf{rp}, \tilde{q}, \tau, \mathsf{gp}_{ks}, \mathsf{qry}_{\mathsf{off}})$
  return $(\mathsf{pp}, \mathbf{D})$

algorithm GENFIXEDQUERYPARTS()
  $\mathbf{A_0} \leftarrow U(\mathbb{Z}_q^{N/t \times d})$
  $\mathbf{A_1} \leftarrow U(\mathbb{Z}_q^{t \times d})$
  $\{\mathbf{w}_g^{(\gamma_i)}\}_{i\in[3]} \leftarrow U(R_q^{\ell_{ks}})$
  return $\mathsf{qry}_{\mathsf{off}} = (\mathbf{A}_0, \mathbf{A}_1, \{\mathbf{w}_g^{(\gamma_i)}\}_{i\in[3]})$

Theorem 7. *Let ℓ_{ks} be the gadget parameter of the key-switching matrices. In the CRS model,* InsPIRe$^{(2)}$ *runs in offline time*

$$O(N\gamma_0 d + t(\gamma_0^2 d + \gamma_0 \ell_{ks} d \lg d) + \tau t d^2 + \tau d(\gamma_1 d + \ell_{ks} d \lg d))$$

Algorithm 20 $\mathsf{InsPIRe}^{(2)}$.Query

algorithm QUERY($\mathsf{pp}, \mathsf{idx}$)
  Parse pp as $(N, t, \gamma_0, \gamma_1, \gamma_2, \mathsf{rp}, \tilde{q}, \tau, \mathsf{gp}_{ks}, \mathsf{qry}_{\mathsf{off}})$
  Parse $\mathsf{qry}_{\mathsf{off}}$ as $(\mathbf{A_0}, \mathbf{A_1}, \{\mathbf{w}_g^{(\gamma_i)}\}_{i\in[3]})$
  Parse idx as (i_0, i_1)
  $\mathbf{s} \leftarrow \chi(\mathbb{Z}^d)$
  $\tilde{s} \leftarrow \sum_{k=0}^{d-1} \mathbf{s}[k] \cdot X^k$
  $\mathbf{b_0} \leftarrow$ GENQUERY$(\mathbf{A_0}, \mathbf{s}, N/t, i_0)$
  $\mathbf{b_1} \leftarrow$ GENQUERY$(\mathbf{A_1}, \mathbf{s}, t, i_1)$
  for $i \in \{0, 1, 2\}$ **do**
    $\mathbf{y}_g^{(\gamma_i)} \leftarrow$ GENKSMATY$(\tilde{s}, \mathbf{w}_g^{(\gamma_i)}, \tau_g)$
  $\mathsf{qry} \leftarrow (\mathbf{b_0}, \mathbf{b_1}, \{\mathbf{y}_g^{(\gamma_i)}\}_{i\in[3]})$
  return $(\mathsf{st} = \tilde{s}, \mathsf{qry})$

algorithm GENQUERY($\mathbf{A}, \mathbf{s}, n, i$)
  Initialize $\mathbf{b} \in \mathbb{Z}_q^n$
  for $k \in [n]$ **do**
    $m_k \leftarrow \mathbb{I}[k = i]$
    $e_k \leftarrow \chi(\mathbb{Z})$
    $\mathbf{b}[k] \leftarrow -\langle \mathbf{A}[k], \mathbf{s}\rangle + e_k + \Delta m_k$ // $\Delta = \lfloor q/p \rceil$
  return b

algorithm GENKSMATY($\tilde{s}, \mathbf{w}, \rho \in \{\tau_g\}$)
  $\mathbf{e} \leftarrow \chi(R_q^{\ell_{ks}})$
  $\mathbf{y} \leftarrow \tilde{s}\mathbf{w} + \mathbf{e} + \rho(\tilde{s}) \cdot \mathbf{g}_{z_{ks}}$
  return y

and online time

$$O(N\gamma_0 + t\gamma_0 d\ell_{ks} + t\tau\gamma_0 + \tau d^2\ell_{ks} + \tau\gamma_0 d(\gamma_2 + \ell_{ks}\lg d)).$$

Theorem 8. *Let $\alpha = |\{\gamma_0, \gamma_1, \gamma_2\}|$, which denotes the number of distinct values for the packing parameter. The total communication cost of* $\mathsf{InsPIRe}^{(2)}$ *is equal to*

$$\alpha \cdot d\ell_{ks}\log_2 q + (N/t)\log_2 q + t\log_2 q + (\tau d/\gamma_1)(d+\gamma_1)\log_2\tilde{q} + (\tau\gamma_0/\gamma_2)(d+\gamma_2)\log_2\tilde{q}$$

Proof. The total communication consists of three components, the packing keys, the query, and the response. We require one packing key for each distinct value of γ_i, each of which is size $d\ell_{ks}\log_2 q$. The two indicator vectors are of length N/t and t, for the first and second layer, respectively, which amounts to size $(N/t)\log_2 q + t\log_2 q$. Lastly, the response consists of $\mathsf{resp_1}$ and $\mathsf{resp_2}$, which are of size $(\tau d/\gamma_1)(d+\gamma_1)\log_2\tilde{q}$ and $(\tau\gamma_0/\gamma_2)(d+\gamma_2)\log_2\tilde{q}$, respectively. □

Theorem 9. *Let the error distribution χ be subgaussian with parameter σ_χ. For $i \in \{0, 1, 2\}$, let e_i denote the error term of an RLWE ciphertext in* $\mathsf{resp_i}$*, defined as in Algorithm 21. Moreover, let $e[:\gamma]$ denote a polynomial which only consists of the first γ coefficients of e. Then for $i \in \{0, 1, 2\}$, under the independence heuristic, $\tilde{e}_i[:\gamma_i] = \tilde{e}_i^{(1)} + \tilde{e}_i^{(2)}$*

Algorithm 21 $\mathsf{InsPIRe}^{(2)}$.Respond

algorithm RESPOND(pp, $\mathbf{D} \in \mathbb{Z}_p^{t\gamma_0 \times N/t}$, qry)
  Parse pp as $(N, \gamma_0, \gamma_1, \gamma_2, \mathsf{rp}, \tilde{q}, \tau, \mathsf{gp}_{ks}, \mathsf{gp}_{gsw}, \mathsf{qry}_{\mathsf{off}})$
  Parse $\mathsf{qry}_{\mathsf{off}}$ as $(\mathbf{A_0}, \mathbf{A_1}, \{\mathbf{w}_g^{(\gamma_i)}\}_{i\in[3]})$
  Parse qry as $(\mathbf{b_0}, \mathbf{b_1}, \{\mathbf{y}_g^{(\gamma_i)}\}_{i\in[3]}))$
  for $i \in \{0, 1, 2\}$ **do**
    $\mathbf{K}_g^{(\gamma_i)} \leftarrow [\, \mathbf{w}_g^{(\gamma_i)}, \mathbf{y}_g^{(\gamma_i)}]$
  $[\, \mathbf{H_0}, \mathbf{b}'_\mathbf{0}] \leftarrow \mathbf{D} \cdot [\, \mathbf{A_0}, \mathbf{b_0}]$
  $\mathsf{resp}_0 \leftarrow \mathsf{InspiRING}.$PARTIALPACKBATCH$(t, \gamma_0, [\, \mathbf{H_0}, \mathbf{b}'_\mathbf{0}], \mathbf{K}_g^{(\gamma_0)})$ // $\mathsf{resp}_0 \in \mathbb{Z}_{\tilde{q}}{}^{t\times(d+\gamma_0)}$
  $\mathbf{D_1} = [\, \overline{\mathbf{D}}_\mathbf{1} \mid \underline{\mathbf{D}}_\mathbf{1}\,] \leftarrow$ DECOMPOSE$(\mathsf{resp}_0, \tilde{q}, p)$ // $\mathbf{D_1} \in \mathbb{Z}_p{}^{t\times\tau(d+\gamma_0)}$
  $\mathbf{H_1} = \begin{bmatrix} \overline{\mathbf{H}}_\mathbf{1} \,|\overline{\mathbf{b}}'_\mathbf{1} \\ \underline{\mathbf{H}}_\mathbf{1} \,|\underline{\mathbf{b}}'_\mathbf{1} \end{bmatrix} \leftarrow [\, \overline{\mathbf{D}}_\mathbf{1} \mid \underline{\mathbf{D}}_\mathbf{1}\,]^T \cdot [\, \mathbf{A_1} \,|\mathbf{b_1}]$ // $\mathbf{H_1} \in \mathbb{Z}_q^{\tau(d+\gamma_0)\times(d+1)}$
  $\mathsf{resp}_1 \leftarrow \mathsf{InspiRING}.$PARTIALPACKBATCH$(\lceil \frac{\tau d}{\gamma_1} \rceil, \gamma_1, [\, \overline{\mathbf{H}}_\mathbf{1}, \overline{\mathbf{b}}'_\mathbf{1}], \mathbf{K}_g^{(\gamma_1)})$
  $\mathsf{resp}_2 \leftarrow \mathsf{InspiRING}.$PARTIALPACKBATCH$(\lceil \frac{\tau\gamma_0}{\gamma_2} \rceil, \gamma_2, [\, \underline{\mathbf{H}}_\mathbf{1}, \underline{\mathbf{b}}'_\mathbf{1}], \mathbf{K}_g^{(\gamma_2)})$
  return $\mathsf{resp} = \{\mathsf{resp}_1, \mathsf{resp}_2\}$

algorithm INSPIRING.PARTIALPACKBATCH$(\ell, \gamma, [\mathbf{H}, \mathbf{b}'], \mathbf{K}_g)$
  for $k \in [\ell]$ **do**
    $\mathbf{H_k} \leftarrow \mathbf{H}[k\gamma : k\gamma + \gamma][:]$
    $\mathbf{b}'_\mathbf{k} \leftarrow \mathbf{b}'[k\gamma : k\gamma + \gamma]$
    $(a_k, b_k) \leftarrow \mathsf{InspiRING}.$PARTIALPACK$(\gamma, [\mathbf{H_k}, \mathbf{b}'_\mathbf{k}], \mathbf{K}_g)$
    $(\tilde{a}_k, \tilde{b}_k) \leftarrow (\lfloor \tilde{q}a_k/q \rceil, \lfloor \tilde{q}b_k/q \rceil)$
  return $\{(\tilde{a}_k[:], \tilde{b}_k[:\gamma])\}_{k\in[\ell]}$

algorithm DECOMPOSE$(\mathbf{C}, \tilde{q}, p, \tau)$
  for $i \in [\tau]$ **do**
    $\mathbf{C}_i = \lfloor p^i \mathbf{C}/\tilde{q} \rceil \mod p$
  $\mathbf{D} = [\mathbf{C}_0|\mathbf{C}_1|\cdots|\mathbf{C}_{\tau-1}]$
  return $\mathbf{D}$

such that

$$||e_i^{(1)}||_\infty \leq f(q, \tilde{q}, p) = \frac{1}{2}(2 + (\tilde{q} \mod p) + \frac{\tilde{q}}{q}(q \mod p))$$

and $\tilde{e}_i^{(2)}$ *has subgaussian coefficients with parameter* $\tilde{\sigma}_i$ *where*

$$\tilde{\sigma}_0^2 \leq \Sigma_0 = d\sigma_\chi^2/4 + (\tilde{q}/q)^2((N/t)p^2\sigma_\chi^2 + \ell_{ks}\gamma_0 d z_{ks}^2 \sigma_\chi^2/4) \tag{5.3}$$

$$\tilde{\sigma}_1^2 \leq \Sigma_1 = d\sigma_\chi^2/4 + (\tilde{q}/q)^2(tp^2\sigma_\chi^2 + \ell_{ks}\gamma_1 d z_{ks}^2 \sigma_\chi^2/4) \tag{5.4}$$

$$\tilde{\sigma}_2^2 \leq \Sigma_2 = d\sigma_\chi^2/4 + (\tilde{q}/q)^2(tp^2\sigma_\chi^2 + \ell_{ks}\gamma_2 d z_{ks}^2 \sigma_\chi^2/4) \tag{5.5}$$

Algorithm 22 $\mathsf{InsPIRe}^{(2)}$.Extract

algorithm $\textsc{Extract}(\mathsf{pp}, \mathsf{st}, \mathsf{resp})$
  Extract $\tilde{s}$ from st
  Parse resp as $\{\mathsf{resp}_1, \mathsf{resp}_2\}$
  $a \leftarrow \textsc{BatchPartialDecrypt}(\tilde{s}, \mathsf{resp}_1, \gamma_1)$ // $a \in \mathbb{Z}_q^{1 \times \tau d}$
  $b \leftarrow \textsc{BatchPartialDecrypt}(\tilde{s}, \mathsf{resp}_2, \gamma_2)$ // $b \in \mathbb{Z}_q^{1 \times \tau \gamma_0}$
  $\mathsf{ct} \leftarrow \textsc{Compose}([a|b], \tilde{q}, p, \tau)$
  $\mathsf{res} \leftarrow \textsc{BatchPartialDecrypt}(\tilde{s}, \mathsf{ct}, \gamma_0)$
  return res

algorithm $\textsc{Compose}(\mathbf{D}, \tilde{q}, p, \tau)$
  Parse $\mathbf{D}$ as $[\mathbf{C}_0|\mathbf{C}_1|\cdots|\mathbf{C}_{\tau-1}]$
  $\mathbf{C} = \sum_{i \in [\tau]} p^{\tau-1-i}\mathbf{C}_i$
  return $\mathbf{C}$

algorithm $\textsc{BatchPartialDecrypt}(\tilde{s}, \mathsf{resp}, \gamma)$
  Parse resp as $\{ct_i\}_{i \in [k]}$
  for $i \in [k]$ **do**
    $p_i \leftarrow \mathsf{RLWE.Dec}(\tilde{s}, ct_i)$
    $pt_i \leftarrow p_i[:\gamma]$
  return $pt = [pt_0|pt_1|\cdots|pt_{k-1}]$ // $pt \in \mathbb{Z}_q^{1 \times k\gamma}$

Theorem 10. *For $f(q, \tilde{q}, p)$ and Σ_i defined as in Theorem 9, then if*

$$\delta_0 \geq 2t\gamma_0 \exp\left(-\pi(\tilde{q}/2p - f(q, \tilde{q}, p))^2/\Sigma_0^2\right)$$
$$\delta_1 \geq 2\tau d \exp\left(-\pi(\tilde{q}/2p - f(q, \tilde{q}, p))^2/\Sigma_1^2\right)$$
$$\delta_2 \geq 2\tau\gamma_0 \exp\left(-\pi(\tilde{q}/2p - f(q, \tilde{q}, p))^2/\Sigma_2^2\right)$$

then $\mathsf{InsPIRe}^{(2)}$ is $(1 - \delta_0 - \delta_1 - \delta_2)$-correct.

The full analysis of $\mathsf{InsPIRe}^{(2)}$ is given in Appendix B.5.

5.5 InsPIRe: PIR from Ring Packing and Homomorphic Polynomial Evaluation

In this section, we present InsPIRe, our low query communication and high-throughput PIR protocol that uses our InspiRING ring packing with preprocessing algorithm as a building block. One can follow the prior PIR frameworks such as YPIR [126] and replace their ring packing algorithms with InspiRING. This immediately results in an improved PIR scheme. However, we will present a new PIR protocol that both leverages InspiRING as well as polynomial evaluation to obtain further communication improvements.

We start by presenting an overview of the prior framework using ring packing and highlight some limitations. Initially, suppose the database consists of N entries, each as an

element in $\mathbb{Z}_p^d$. Note, the choice of using d here is for simplicity and we will generalize later. We introduce an additional configurable parameter t and model the plaintext database as a matrix $\mathbf{D} \in \mathbb{Z}_p^{td \times N/t}$. Each column of the matrix corresponds to t database entries of $\mathbb{Z}_p^d$.

During the query phase, the client queries for the column i that contains the desired entry. To achieve this, the client generates LWE encryptions of a one-hot selection vector of length N/t. All ciphertexts encrypt zero except the i-th ciphertext which encrypts one. The N/t LWE ciphertexts can be represented as the rows of the matrix $[\mathbf{A}, \mathbf{b}] \in \mathbb{Z}_q^{N/t \times (d+1)}$ which are sent to the server. Next, the server performs matrix-matrix multiplication to obtain $\mathbf{D} \cdot [\mathbf{A}, \mathbf{b}]$, which yields td LWE encryptions of the i-th column of $\mathbf{D}$. At this point, each consecutive chunk of d LWE ciphertexts corresponds to a database entry over $\mathbb{Z}_p^d$. Finally, the server uses ring packing (such as InspiRING) to translate each chunk of d LWE ciphertexts into a single RLWE ciphertext before returning the resulting t RLWE ciphertexts back to the client.

In this scheme, the server returns encryptions corresponding to the entire column of t entries. To reduce the response size, we ideally want the server to only return the desired entry instead. Previous approaches, such as OnionPIR [132] and Spiral [125], employed encryptions of one-hot selection vectors for this step. However, these methods typically require $\Theta(\lg(t))$ ciphertexts or large evaluation keys, adding significant communication.

5.5.1 Homomorphic Polynomial Evaluation

We present a new PIR protocol that introduces a new way of encoding the plaintext database. Instead of explicitly representing each column as a concatenation of t database entries, our key idea is to implicitly represent it as coefficients of a polynomial that *evaluates* to the entries in that column for some publicly fixed evaluation points.

With this new database encoding, the initial homomorphic column selection will obtain encrypted polynomial coefficients. Afterwards, we use homomorphic polynomial evaluation to extract the desired entry. By carefully constructing the polynomial, we can perform the evaluation using a *single* RGSW ciphertext. Furthermore, our design utilizes RLWE-RGSW external products (see Section 5.2) to minimize noise growth during homomorphic evaluation. By choosing the evaluation points appropriately, our homomorphic evaluation will only incur *additive* noise growth for each multiplication.

Polynomials for Database Encoding. We proceed more formally to define the polynomial representations for each database column. For convenience, we will assume that N and t are powers of two, but we will generalize later. Recall that our plaintext database

is modeled as $\mathbf{D} \in \mathbb{Z}_p^{td \times N/t}$ where each column contains t entries from $\mathbb{Z}_p^d$. We denote $\mathbf{y}_j^{(i)} \in \mathbb{Z}_p^d$ as the j-th entry in the i-th column.

We will use polynomials to represent each column of the matrix $\mathbf{D}$. We first begin by re-interpreting each $\mathbf{y}_j^{(i)} \in \mathbb{Z}_p^d$ as an element in $R_p = \mathbb{Z}_p[X]/(X^d + 1)$ by mapping the d elements in $\mathbb{Z}_p$ as the polynomial coefficients. We will denote these re-interpretations as $y_j^{(i)} \in R_p$.

Let $R_p[Z]$ be the polynomial ring over R_p. Imagine that we have t distinct points $z_0, \ldots, z_{t-1} \in R_p$ that are fixed publicly beforehand. We will pick these t points later. For each column i, suppose there exists a polynomial $h^{(i)}(Z) = \sum_{j=0}^{t-1} c_j^{(i)} Z^j \in R_p[Z]$ such that the following holds:

$$h^{(i)}(z_j) = y_j^{(i)}, \forall j \in \{0, 1, \ldots, t-1\}.$$

Suppose the server can compute the coefficients $c_0^{(i)}, \ldots, c_{t-1}^{(i)} \in R_p$ of the polynomial $h^{(i)}(Z)$ for each column i. We construct encoded database $\mathbf{D}' \in \mathbb{Z}_p^{td \times N/t}$, where each column i consists of the corresponding polynomial coefficients $c_j^{(i)}$ interpreted as elements in $\mathbb{Z}_p^d$.

Homomorphic Polynomial Evaluation. Suppose the client wants to retrieve the k-th entry from the i-th column, $y_k^{(i)}$. Consider the point after the server has performed initial homomorphic selection of column i *on the encoded database* $\mathbf{D}'$. At this point, the server holds t RLWE encryptions of the coefficients $c_0^{(i)}, \ldots, c_{t-1}^{(i)}$ of the polynomial $h^{(i)}(Z)$. Since the evaluation points z_j are publicly fixed and the polynomials $h^{(i)}(Z)$ are constructed such that $h^{(i)}(z_j) = y_j^{(i)}$ for each $0 \leq j < t$, the client can send a *single* RGSW ciphertext encrypting the evaluation point z_k to the server. The server can then homomorphically evaluate the polynomial $h^{(i)}(Z)$ using the RLWE encrypted coefficients $c_j^{(i)}$ and a single RGSW encrypted evaluation point z_k. This would obtain the desired entry $h^{(i)}(z_k) = y_k^{(i)}$. To perform evaluation, we will use the Horner-style method (see EvalPoly in Algorithm 25), that only involves RLWE-RGSW external products and RLWE additions.

So far, we have not discussed how the evaluation points z_j are chosen. For our PIR protocol to be efficient, we want the evaluation points z_j to satisfy the following properties:

1. *Noise Management during Evaluation:* Homomorphic polynomial evaluation involves large-depth multiplications (e.g., in Horner's method), each increasing the noise in the resulting ciphertext. We want to strategically choose the evaluation points so that the noise growth from multiplications is minimal (ideally, additive).

2. *Interpolation in Rings:* The evaluation points must guarantee a polynomial interpolation within the ring R_p. Since R_p is not necessarily a field, the standard conditions

for interpolation (requiring distinct points) are not sufficient. We need to ensure the existence of the interpolating polynomial for any set of database entries.

Unit Monomials as Evaluation Points. To address the challenges of noise management and interpolation, we propose a specific choice for the evaluation points. Our key insight is to select evaluation points $z_0, \ldots, z_{t-1}$ that are both *unit monomials* and *roots of unity*. Here, we will define unit monomials as elements in R_p of the form $\pm X^k$ where $0 \leq k < d$. If the RGSW ciphertext encrypting the evaluation point encrypts a unit monomial, the external product operation incurs only *additive noise* growth, rather than multiplicative.

Lemma 6. *[Theorem 2.19 [125], adapted] Given* $\mathsf{RLWE}(m_0)$*, let* $\mathsf{RGSW}(m_1)$ *be a fresh ciphertext encrypting a unit monomial, e.g.* $m_1 = \pm X^k$ *for some* $0 \leq k < d$*. Let the error distribution* χ *be subgaussian with parameter* σ_χ*, and let* ℓ *and* z *be the gadget parameters of the RGSW ciphertext. Under the independence heuristic, the external product* $\mathsf{RLWE}(m_0) \boxdot \mathsf{RGSW}(m_1)$ *incurs additive noise* $e_{ep} \in R_q$ *whose coefficients are subgaussian with parameter* σ_{ep} *and* $\sigma_{ep}^2 \leq \ell d z^2 \sigma_\chi^2 / 2$*.*

By ensuring that all $z_0, \ldots, z_{t-1}$ are unit monomials, the entire homomorphic evaluation process (involving $t-1$ multiplications and additions) incurs only additive noise.

Next, we discuss how to guarantee polynomial interpolation within R_p using these unit monomial evaluation points. To guarantee interpolation, our key insight is to use powers of a *primitive t-th root of unity* ω as the evaluation points. Specifically, for $t \leq 2d$ a power of two, we will choose a primitive t-th root $\omega = X^{2d/t}$. With this choice, we define the evaluation points as powers of ω: $z_k := \omega^k$ for $k = 0 \ldots t-1$. Crucially, we see that all the evaluation points z_k are unit monomials. These points are distinct and the Vandermonde matrix (e.g., DFT matrix) formed by them is invertible over R_p, assuming p is odd [32]. Essentially, the polynomial interpolation then amounts to a matrix-vector product between the corresponding IDFT matrix and the vector of $y_j^{(i)}$s, which can be done efficiently using Cooley-Tukey style FFT algorithm.

Constraints and Generalizations. This specific construction imposes two main constraints:

- $t \leq 2d$: This is required for the evaluation points to be unit monomials. While our experimental evaluations indicate that $t \ll 2d$ often suffices for effective PIR, scenarios requiring larger folding factor ($t > 2d$) are possible. We discuss extensions using multivariate polynomial interpolation in Appendix B.4.1.

- t **is a power of two:** This is required for the Cooley-Tukey style FFT algorithm used for the interpolation during database encoding. If t is not a power of two, a computationally less efficient Lagrange interpolation can be used instead. We present this Lagrange interpolation based method in Appendix B.4.2.

5.5.2 Putting It Together

We present our PIR protocol $\mathsf{InsPIRe} = (\mathsf{Setup}, \mathsf{Query}, \mathsf{Respond}, \mathsf{Extract})$ using InspiRING and homomorphic polynomial evaluation in Algorithms 23, 24, 25, 26. For convenience, we assume that the public parameters pp are globally available to the subroutines of the PIR algorithms and do not explicitly specify them as inputs. For instance, we assume InspiRING uses the RLWE parameters rp and the gadget parameters gp_{ks}. Note that GenKsMatY() in Algorithm 24 is essentially the same as KS.Setup except it only generates the pseudo-random component of the key-switching matrix.

Algorithm 23 InsPIRe.Setup

algorithm SETUP($1^\lambda, \mathbf{D} \in \mathbb{Z}_p^{td \times N/t}$)
  $\mathsf{rp} \leftarrow (d(\lambda), q(\lambda), \chi(\lambda), p)$ as the RLWE params
  $\mathsf{gp}_{ks} \leftarrow (z_{ks}, \ell_{ks})$ as the gadget params for the KS matrices
  $\mathsf{gp}_{gsw} \leftarrow (z_{gsw}, \ell_{gsw})$ as the gadget params for the RGSW ciphertext
  $\mathsf{qry}_{\mathsf{off}} \leftarrow$ GENFIXEDQUERYPARTS()
  $\mathsf{pp} \leftarrow (N, t, \mathsf{rp}, \mathsf{gp}_{ks}, \mathsf{gp}_{gsw}, \mathsf{qry}_{\mathsf{off}})$
  $\mathbf{D}' \leftarrow$ ENCODEDB($\mathbf{D}$)
  return $(\mathsf{pp}, \mathbf{D}')$

algorithm ENCODEDB($\mathbf{D} \in \mathbb{Z}_p^{td \times N/t}$)
  Initialize $\mathbf{D}' \in \mathbb{Z}_p^{td \times N/t}$
  for $i = 0 \dots N/t - 1$ **do**
    $y_j^{(i)} \leftarrow j$-th element in column i interpreted as R_p for $0 \le j < t$
    $[c_0^{(i)}, \dots, c_{t-1}^{(i)}]^\top \leftarrow$ INTERPOLATE($[y_0^{(i)}, \dots, y_{t-1}^{(i)}]^\top$)
    Cast $[c_0^{(i)}, \dots, c_{t-1}^{(i)}]^\top$ as a vector over $\mathbb{Z}_p^{td}$, and embed into $\mathbf{D}'[:][i]$
  return $\mathbf{D}'$

algorithm INTERPOLATE($[y_0, \dots, y_{t-1}]^\top \in R_p^t$)
  $\mathbf{c} \leftarrow$ COOLEYTUKEY($[y_0, \dots, y_{t-1}]^\top$)
  return $t^{-1} \cdot \mathbf{c}$

algorithm COOLEYTUKEY($[y_0, \dots, y_{m-1}]^\top \in R_p^m$)
  if $m = 1$ **then**
    return $[y_0]^\top$
  $\mathbf{c}_{even} \leftarrow$ COOLEYTUKEY($[y_0, y_2, \dots, y_{m-2}]^\top$)
  $\mathbf{c}_{odd} \leftarrow$ COOLEYTUKEY($[y_1, y_3, \dots, y_{m-1}]^\top$)
  Initialize $\mathbf{c} \in R_p^m$
  for $i = 0 \dots m/2 - 1$ **do**
    $\omega \leftarrow X^{-2d/m \cdot i}$
    $\mathbf{c}[i] \leftarrow \mathbf{c}_{even}[i] + \omega \cdot \mathbf{c}_{odd}[i]$
    $\mathbf{c}[i + m/2] \leftarrow \mathbf{c}_{even}[i] - \omega \cdot \mathbf{c}_{odd}[i]$
  return $\mathbf{c}$

algorithm GENFIXEDQUERYPARTS()
  $\mathbf{A} \leftarrow U(\mathbb{Z}_q^{N/t \times d})$
  $\mathbf{w}_g \leftarrow U(R_q^{\ell_{ks}})$
  $\mathbf{w}_h \leftarrow U(R_q^{\ell_{ks}})$
  return $\mathsf{qry}_{\mathsf{off}} = (\mathbf{A}, \mathbf{w}_g, \mathbf{w}_h)$

Algorithm 24 InsPIRe.Query

algorithm QUERY($\mathsf{pp}, \mathsf{idx}$)
  Parse pp as $(N, t, \mathsf{rp}, \mathsf{gp}_{ks}, \mathsf{gp}_{gsw}, \mathsf{qry}_{\mathsf{off}})$
  Parse $\mathsf{qry}_{\mathsf{off}}$ as $(\mathbf{A}, \mathbf{w}_g, \mathbf{w}_h)$
  Parse idx as (i, j)
  $\mathbf{s} \leftarrow \chi(\mathbb{Z}^d)$
  $\tilde{s} \leftarrow \sum_{k=0}^{d-1} \mathbf{s}[k] \cdot X^k$
  Initialize $\mathbf{b} \in \mathbb{Z}_q^{N/t}$
  for $k = 0 \ldots N/t - 1$ **do**
    $m_k \leftarrow k = i$
    $e_k \leftarrow \chi(\mathbb{Z})$
    $\mathbf{b}[k] \leftarrow -\langle \mathbf{A}[k], \mathbf{s} \rangle + e_k + \Delta m_k$ // $\Delta = \lfloor q/p \rceil$
  $\omega \leftarrow X^{2d/t}$
  $\mathsf{RGSW}(\omega^j) \leftarrow \mathsf{RGSW.Enc}(\tilde{s}, \omega^j)$
  $\mathbf{y}_g \leftarrow$ GENKSMATY$(\tilde{s}, \mathbf{w}_g, \tau_g)$
  $\mathbf{y}_h \leftarrow$ GENKSMATY$(\tilde{s}, \mathbf{w}_h, \tau_h)$
  $\mathsf{qry} \leftarrow (\mathbf{b}, \mathsf{RGSW}(\omega^j), \mathbf{y}_g, \mathbf{y}_h)$
  return $(\mathsf{st} = \tilde{s}, \mathsf{qry})$

algorithm GENKSMATY($\tilde{s}, \mathbf{w}, \rho \in \{\tau_g, \tau_h\}$)
  $\mathbf{e} \leftarrow \chi(R_q^{\ell_{ks}})$
  $\mathbf{y} \leftarrow \tilde{s}\mathbf{w} + \mathbf{e} + \rho(\tilde{s}) \cdot \mathbf{g}_{z_{ks}}$
  return $\mathbf{y}$

Algorithm 25 InsPIRe.Respond

algorithm RESPOND(pp, $\mathbf{D}' \in \mathbb{Z}_q^{td \times N/t}$, qry)
  Parse pp as $(N, t, \mathsf{rp}, \mathsf{gp}_{ks}, \mathsf{gp}_{gsw}, \mathsf{qry}_{\mathsf{off}})$
  Parse $\mathsf{qry}_{\mathsf{off}}$ as $(\mathbf{A}, \mathbf{w}_g, \mathbf{w}_h)$
  Parse qry as $(\mathbf{b}, \mathsf{RGSW}(\omega^j), \mathbf{y}_g, \mathbf{y}_h)$
  $\mathbf{K}_g \leftarrow [\mathbf{w}_g, \mathbf{y}_g]$
  $\mathbf{K}_h \leftarrow [\mathbf{w}_h, \mathbf{y}_h]$
  $[\mathbf{H}, \mathbf{b}'] \leftarrow \mathbf{D}' \cdot [\mathbf{A}, \mathbf{b}]$
  for $k = 0 \ldots t - 1$ **do**
    $\mathbf{H}_k \leftarrow \mathbf{H}[kd : kd + d)][:]$
    $\mathbf{b}'_k \leftarrow \mathbf{b}'[kd : kd + d]$
    $(a_k, b_k) \leftarrow \mathsf{InspiRING.PACK}([\mathbf{H}_k, \mathbf{b}'_k], \mathbf{K}_g, \mathbf{K}_h)$
    Rename (a_k, b_k) to $\mathsf{RLWE}(c_k)$
  $\mathsf{resp} \leftarrow$ EVALPOLY$(\mathsf{RLWE}(c_0), \ldots, \mathsf{RLWE}(c_{t-1}), \mathsf{RGSW}(\omega^j))$
  return resp

algorithm EVALPOLY($\mathsf{RLWE}(c_0), \ldots, \mathsf{RLWE}(c_{t-1}), \mathsf{RGSW}(\omega^j)$)
  $\mathsf{ct} \leftarrow \mathsf{RLWE}(c_{t-1})$
  for $k = t - 2 \ldots 0$ **do**
    $\mathsf{ct} \leftarrow \mathsf{ct} \boxdot \mathsf{RGSW}(\omega^j) + \mathsf{RLWE}(c_k)$
  return ct

Algorithm 26 InsPIRe.Extract

1: **algorithm** EXTRACT(pp, st, resp)
2: Extract $\tilde{s}$ from st
3: $y_j^{(i)} \leftarrow \mathsf{RLWE.Dec}(\tilde{s}, \mathsf{resp})$
4: **return** $y_j^{(i)}$

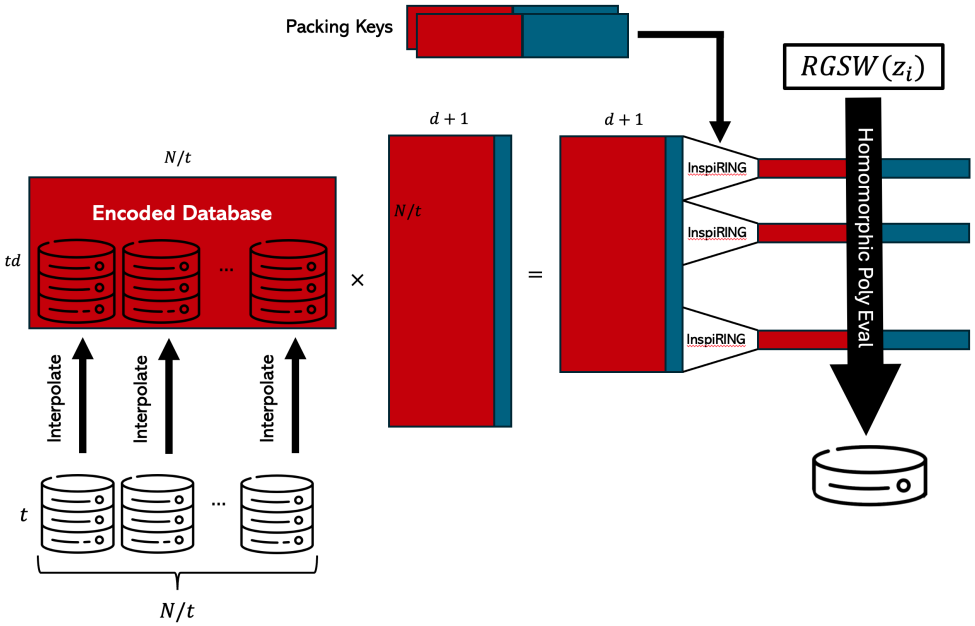


Figure 5.2: Overview of the server's computation in InsPIRe. Red parts indicate options that known offline and can be preprocessed. Blue parts are generated online by the client.

Preprocessing in the CRS Model. We remind the readers that the highlighted parts of InsPIRe.Respond correspond to the preprocessable parts of the algorithm prior to serving queries (see Section 5.2.2). These correspond to parts that only depend on the outputs from InsPIRe.Setup. We note that this extends to InspiRING.Pack as well (Algorithm 17).

Generalizing to Arbitrary Entry Size. The pseudocode assumes that each entry is encoded as $\mathbb{Z}_p^d$ of length d. To support arbitrary length m, we use standard reductions:

- $m > d$**:** Divide each entry into m/d chunks. Construct m/d databases $\in \mathbb{Z}_q^{td \times N/t}$ where the i-th database contains the i-th chunk. In the Respond algorithm, the server processes the client's query on all databases.

- $m < d$: Bundle d/m entries as a single entry over $\mathbb{Z}_p^d$. The client queries for the bundle with the desired entry.

Theorem 11. *Let the error distribution χ be subgaussian with parameter σ_χ. Let ℓ_{ks} and z_{ks} be the gadget parameters of the key-switching matrices. Let ℓ_{gsw} and z_{gsw} be the gadget parameters of the RGSW ciphertext. The error polynomial of the RLWE ciphertext* ct *has the form $e = e_{main} + e_{overflow} \in R_q$. Under the independence heuristics, e_{main} has subgaussian coefficients with parameter $\sigma_{main}^2 \leq Np^2\sigma_\chi^2 + t\ell_{ks}d^2z_{ks}^2\sigma_\chi^2/4 + t\ell_{gsw} \cdot dz_{gsw}^2 \cdot \sigma_\chi^2/2$ and $||e_{overflow}||_\infty \leq tp/2$.*

The $e_{overflow}$ term is introduced from the homomorphic polynomial evaluation because the sum of the embedded plaintexts typically overflows the R_p plaintext space. However, Theorem 11 shows that this additional $e_{overflow}$ is negligible in practice. The full proof is given in Appendix B.2.

Theorem 12. *For subgaussian χ with parameter σ_χ, define $\bar{\sigma}_{main}^2 = Np^2\sigma_\chi^2 + t\ell_{ks}d^2z_{ks}^2\sigma_\chi^2/4 + t\ell_{gsw} \cdot dz_{gsw}^2 \cdot \sigma_\chi^2/2$. Then, for $\delta > 2d\exp\left(-\pi(\Delta/2 - tp/2)^2/\bar{\sigma}_{main}^2\right)$,* InsPIRe *is $(1-\delta)$-correct.*

Theorem 13. *Let ℓ_{ks} and ℓ_{gsw} be the gadget parameters of the key-switching matrices and the query RGSW ciphertext respectively. In the CRS model,* InsPIRe *runs in offline time $O(Nd^2 + t(d^3 + \ell_{ks}d^2 \lg d))$ and online time $O(Nd + t\ell_{ks}d^2 + t\ell_{gsw}d\lg(d))$.*

Theorem 14. *The total communication cost of* InsPIRe *is*

$$d\ell_{ks}\log_2 q + (N/t)\log_2 q + 4\ell_{gsw}d\log_2 q + 2d\log_2 q$$

Security. The security of our protocol relies on the RLWE hardness assumption and the circular security assumption for the key-switching matrices used in packing (see Section 5.3). The full proof of security is given in Appendix B.3.

Comparison with InsPIRe$_0$. Because InsPIRe$_0$ is built on top of DoublePIR, it can only support small entry size such as 1 bit. On the other hand, InsPIRe is much more flexible and can support arbitrary entry size with very low communication overhead. As we show in Section 5.6, InsPIRe can even obtain lower communication than InsPIRe$_0$, but the downside is the higher server runtime from packing during the initial homomorphic column selection. In contrast, InsPIRe$_0$ packs the DoublePIR response consisting of a smaller number of LWE ciphertexts. In general, InsPIRe is the recommended choice unless the entry size is very small and server runtime is the primary metric to optimize.

Further optimizations to InsPIRe using approximate gadget decomposition is discussed in Appendix B.4.3.

5.6 Experimental Evaluation

In this section, we compare our constructions $\mathsf{InsPIRe}_0$, $\mathsf{InsPIRe}^{(2)}$, and $\mathsf{InsPIRe}$ with existing efficient PIR protocols with server-side preprocessing. In particular, we do not compare with protocols that rely on client-specific cryptographic keys [11, 125], database-dependent hints [86] that need to be sent to the client beforehand, or streaming the database in an offline phase [175]. In our experimental evaluation, we are particularly interested in two metrics: the server's online runtime and the total communication required for a single PIR query. We compare our proposed protocols with state-of-the-art PIR protocols, for small (1 bit), medium (64 B), and large (32 KB) entries.

Implementation and Evaluation Details. We implemented $\mathsf{InspiRING}$ with about 3,000 lines of Rust using building blocks for RLWE operations from spiral-rs.[2] Using $\mathsf{InspiRING}$ and building blocks from the YPIR implementation,[3] we implemented $\mathsf{InsPIRe}^{(2)}$ in an additional 3,000 lines of Rust and $\mathsf{InsPIRe}$ in an additional 2,000 lines of Rust. Our code is publicly available on Github.[4] We perform all experimental evaluations on an Intel Xeon CPU @ 2.6 GHz with support for AVX512, where we run all experiments in single-threaded mode for fair comparison with related work. We are primarily interested in two metrics, the total communication and the online server runtime. We provide breakdowns of the communication and runtime as well. In all experiments, offline runtimes are measured once and online runtimes are averaged over 5 runs with a standard deviation of less than 5%. Table 5.1 summarized our parameter choices. For $\mathsf{InsPIRe}_0$, we use parameters identical to those used in YPIR to isolate the benefits of our new protocol. $\mathsf{InsPIRe}_0$ uses two different sets of parameters for the first and second layer, which are listed in the table. For $\mathsf{InsPIRe}^{(2)}$ and $\mathsf{InsPIRe}$, we use lattice parameters which provide 128-bit security based on the lattice-estimator [7] and correctness parameter $\delta = 2^{-40}$.

5.6.1 Parameterizing $\mathsf{InsPIRe}^{(2)}$

In this section, we describe how to choose parameters of $\mathsf{InsPIRe}^{(2)}$, specifically $\{\gamma_i\}_{i\in[3]}$ and t to optimize performance and balance the communication/computation tradeoff. For communication costs, we measure the upload and download costs, and break down the upload costs into the key material (the packing keys) and the query (the encrypted LWE indicator vector and RGSW ciphertext). We also break down the server's online time into five parts:

[2] https://github.com/menonsamir/spiral-rs/
[3] https://github.com/menonsamir/ypir/
[4] https://github.com/google/private-membership/tree/main/research/InsPIRe

Table 5.1: Lattice parameters for $\mathsf{InsPIRe}_0$, $\mathsf{InsPIRe}^{(2)}$, and $\mathsf{InsPIRe}$.

	d	$\log_2 q$	σ_χ	p	ℓ_{KS}	ℓ_{GSW}	z_{KS}	z_{GSW}
$\mathsf{InsPIRe}_0$	1024	32	6.4	256	-	-	-	-
	2048	56	6.4	8192	3	-	2^{19}	-
$\mathsf{InsPIRe}^{(2)}$	2048	53	6.4	65536	3	-	2^{19}	-
$\mathsf{InsPIRe}$	2048	56	6.4	65535	3	3	2^{19}	2^{19}

the first matrix multiplication, the first packing, the second matrix multiplication, and the second and third packing.

Note that γ_0 determines the size of the return entry, i.e., an entry of size $\gamma_0 \log_2 p$ bits is returned. In choosing the values of $\{\gamma_i\}_{i\in[3]}$, similar values allows reuse of the key material, hence reducing overall communication. In contrast, different parts of the protocol benefit from different values of γ. For example, the second packing (which uses γ_1) packs τd ciphertexts with offline preprocessing, but the overall runtime of the second packing does not depend on the value of γ_1 (Theorem 7), so it is best to set γ_1 to be largest value possible, i.e., $\gamma_1 = d/2$. In contrast, large values of γ_0 and γ_2 increase the online runtime, so we choose them to be small and set $\gamma_0 = \gamma_2$ to reuse the key material. While other combinations of $\{\gamma_i\}_{i\in[3]}$ are possible, we see that no other combination of parameters achieve such small total communication cost, while also maintaining competitive runtime.

The choice of t offers a communication-computation trade-off. The total communication is minimized at $t = \sqrt{N}$, for fixed $\{\gamma_i\}_{i\in[3]}$, based on Theorem 8, but smaller values of t reduce the computation cost, based on Theorem 7. So we vary the value of t as part of our experiments to observe the effect. Figure 5.3 shows the results for a 1 GB database. Results for other database sizes are given in Appendix B.6.

Figure 5.3 shows that as we reduce t (increase N/t), the query size increases, but the query always makes up the majority of the total communication. The smallest total communication for retrieving an entry size of 16B, 32B, and 64B is 211 KB, 185 KB, and 172 KB, respectively. The communication cost can be increased to achieve better runtime. The majority of the runtime is used for the first packing, which is consistent with the analysis of Theorem 7.

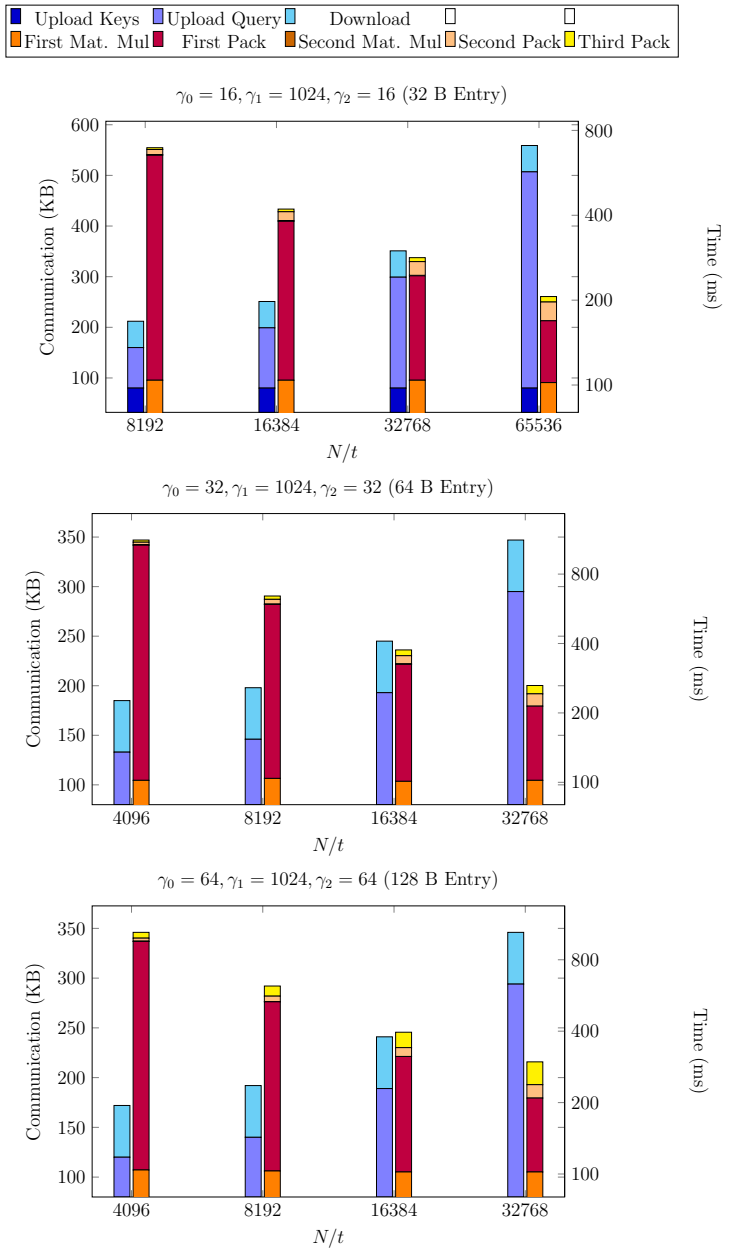


Figure 5.3: Comparison of communication and computation costs of $\mathsf{InsPIRe}^{(2)}$ over a 1 GB database for different parameter sets and entry sizes. From top to bottom: 32 B, 64 B, and 128 B entry sizes.

5.6.2 Parameterizing InsPIRe

InsPIRe introduces the interpolation degree (t) as a new parameter of the PIR protocol which can not be chosen in a simple optimization. To guide in choosing this parameters, we first show the effect of the interpolation degree on the performance of InsPIRe. In summary, we show that the choice of the interpolation degree results in a tradeoff between communication and computation.

Figure 5.4 shows the performance metrics of InsPIRe as we vary the interpolation degree, for a 1 GB database. More results for other database sizes are given in Appendix B.6. For communication costs, we measure the upload and download costs, and break down the upload costs into the key material (the packing keys) and the query (the encrypted LWE indicator vector and RGSW ciphertext). We also break down the server's online time into three parts: the first matrix multiplication, the packing, and the polynomial evaluation.

For the server online runtime, Figure 5.4 shows that, for a fixed database size, a larger interpolation degree results in a higher runtime. For large enough interpolation degree, the most costly step in the protocol is the packing, which is compatible with the analysis in Theorem 13.

For the communication costs, the key size and the response size are fixed, regardless of the interpolation degree. The query consists of two parts, the encrypted indicator vector and the RGSW encrypted evaluation point. The size of the RGSW ciphertext is fixed (84 KB), regardless of the interpolation degree. The length of the encrypted indicator vector depends on the first dimension of the database, which inversely depends on the size of each row, which is linear in the interpolation degree. Hence, for larger interpolation degree, the query size reduces, which in turn reduces the overall communication cost.

5.6.3 PIR Evaluation with Various Entry Sizes

In the final section of our evaluation, we compare all our construction with existing work across various database sizes with different entry sizes. Tables 5.2, 5.3, and 5.4 show the comparison between our constructions and related work for 1-bit, 64B, and 32KB entries, respectively. In each case, we compare with the most competitive protocols that support the specified entry size.

Our evaluation shows that for all configurations, our proposed constructions outperform existing work. For 1-bit entries, $\mathsf{InsPIRe}_0$ is strictly better than related work, both in terms of communication cost and server runtime. Table 5.2 shows at least one data point for InsPIRe that outperforms both in communication and server runtime. However, InsPIRe

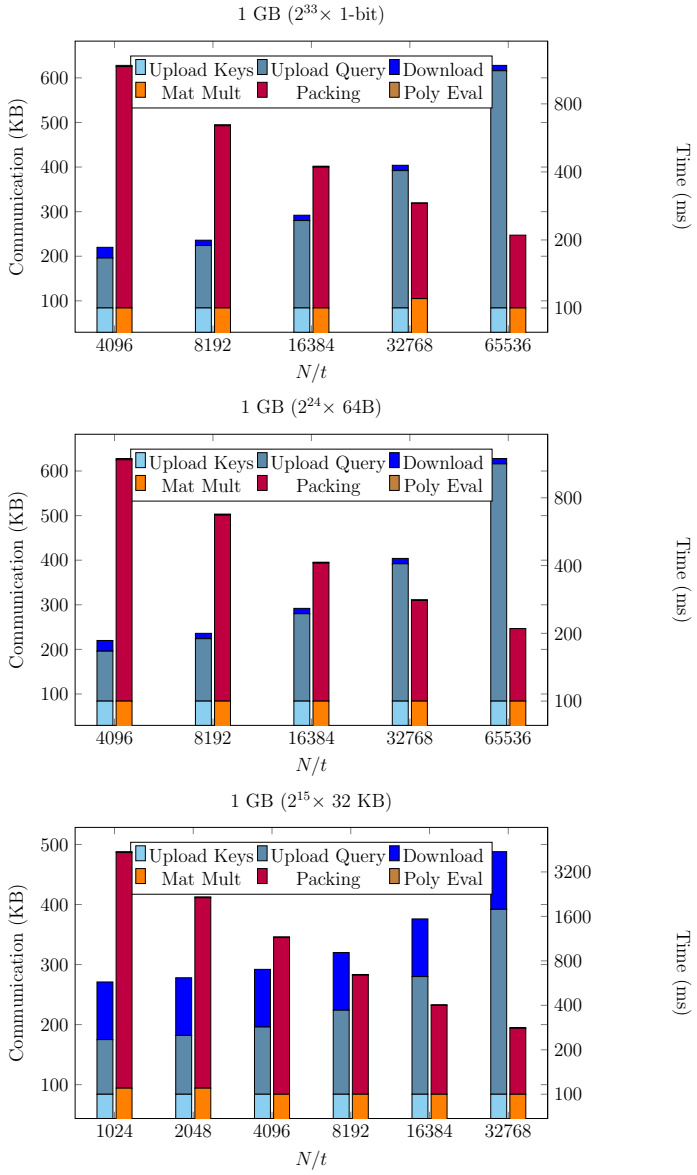


Figure 5.4: Breakdown of Communication and Online Time of InsPIRe over 1 GB database for various entry sizes as a function of the first dimension (N/t). In each case, we use the largest polynomial degree that satisfies correctness.

can achieve even lower communication costs at the cost of higher runtime as shown in Table B.2.

Table 5.2: PIR performance metrics for different database sizes with 1-bit entries.

Metric	YPIR	SimpleYPIR	KSPIR	HintlessPIR	InsPIRe$_0$	InsPIRe$^{(2)}$						InsPIRe		
1 GB ($2^{33}\times$ 1-bit)														
Offline	32 s	119 s	14 s	213 s	43 s	103 s	66 s	47 s	42 s	37 s	35 s	87 s	77 s	99 s
Upload (Keys)	462 KB	462 KB	2352 KB	360 KB	84 KB	80 KB	80 KB	80 KB	80 KB	80 KB	80 KB	84 KB	84 KB	84 KB
Upload (Query)	384 KB	112 KB	14 KB	128 KB	384 KB	40 KB	60 KB	109 KB	113 KB	214 KB	215 KB	140 KB	196 KB	308 KB
Download	12 KB	228 KB	224 KB	1748 KB	36 KB	52 KB	52 KB	52 KB	52 KB	52 KB	52 KB	12 KB	12 KB	12 KB
Total Comm.	858 KB	802 KB	2590 KB	2236 KB	504 KB	172 KB	191 KB	241 KB	244 KB	345 KB	347 KB	236 KB	292 KB	404 KB
Server Time	140 ms	600 ms	780 ms	750 ms	120 ms	1100 ms	670 ms	470 ms	440 ms	360 ms	320 ms	650 ms	400 ms	280 ms
Throughput	7420 MB/s	1720 MB/s	1310 MB/s	1370 MB/s	8750 MB/s	930 MB/s	1530 MB/s	2200 MB/s	2310 MB/s	2880 MB/s	3230 MB/s	1580 MB/s	2530 MB/s	3670 MB/s
8 GB ($2^{36}\times$ 1-bit)														
Offline	187 s	283 s	118 s	1500 s	200 s	402 s	235 s	157 s	118 s	94 s		581 s	597 s	579 s
Upload (Keys)	462 KB	462 KB	2688 KB	360 KB	84 KB	80 KB	80 KB	80 KB	80 KB	80 KB		84 KB	84 KB	84 KB
Upload (Query)	1024 KB	448 KB	14 KB	512 KB	1024 KB	106 KB	132 KB	225 KB	431 KB	855 KB		532 KB	532 KB	980 KB
Download	12 KB	444 KB	224 KB	3316 KB	36 KB	52 KB	52 KB	52 KB	52 KB	52 KB		12 KB	96 KB	12 KB
Total Comm.	1498 KB	1354 KB	2926 KB	4188 KB	1144 KB	238 KB	264 KB	357 KB	562 KB	986 KB		628 KB	712 KB	1076 KB
Server Time	830 ms	1850 ms	5910 ms	2030 ms	810 ms	4680 ms	2670 ms	1850 ms	1390 ms	1150 ms		1360 ms	1340 ms	1120 ms
Throughput	9830 MB/s	4420 MB/s	1390 MB/s	4040 MB/s	10060 MB/s	1750 MB/s	3070 MB/s	4440 MB/s	5880 MB/s	7150 MB/s		6010 MB/s	6110 MB/s	7320 MB/s
32 GB ($2^{38}\times$ 1-bit)														
Offline	724 s	706 s	445 s	5700 s	731 s	589 s	421 s	379 s	339 s	318 s	291 s	2306 s	2128 s	2340 s
Upload (Keys)	462 KB	462 KB	2912 KB	360 KB	84 KB	80 KB	80 KB	80 KB	80 KB	80 KB	80 KB	84 KB	84 KB	84 KB
Upload (Query)	2048 KB	896 KB	14 KB	1024 KB	2048 KB	265 KB	450 KB	477 KB	861 KB	874 KB	1709 KB	980 KB	1876 KB	1876 KB
Download	12 KB	888 KB	224 KB	6452 KB	36 KB	52 KB	52 KB	52 KB	52 KB	52 KB	52 KB	96 KB	12 KB	96 KB
Total Comm.	2522 KB	2246 KB	3150 KB	7836 KB	2168 KB	397 KB	582 KB	609 KB	993 KB	1006 KB	1841 KB	1160 KB	1972 KB	2056 KB
Server Time	3390 ms	5610 ms	50570 ms	5910 ms	3390 ms	7390 ms	5280 ms	5260 ms	4430 ms	4410 ms	4010 ms	4320 ms	3880 ms	3820 ms
Throughput	9680 MB/s	5840 MB/s	650 MB/s	5550 MB/s	9670 MB/s	4430 MB/s	6210 MB/s	6230 MB/s	7400 MB/s	7430 MB/s	8180 MB/s	7580 MB/s	8450 MB/s	8570 MB/s

Table 5.3: PIR performance metrics for different database sizes with 64 B entries.

Metric	SimpleYPIR	KSPIR	HintlessPIR	InsPIRe$^{(2)}$						InsPIRe				
1 GB ($2^{24}\times$ 64B)														
Offline Time	119 s	14 s	213 s	103 s	66 s	47 s	42 s	37 s	35 s	87 s	77 s	98 s	99 s	
Upload (Keys)	462 KB	2352 KB	360 KB	80 KB	80 KB	80 KB	80 KB	80 KB	80 KB	84 KB	84 KB	84 KB	84 KB	
Upload (Query)	112 KB	14 KB	128 KB	40 KB	60 KB	109 KB	113 KB	214 KB	215 KB	140 KB	196 KB	308 KB	532 KB	
Download	228 KB	224 KB	1748 KB	52 KB	52 KB	52 KB	52 KB	52 KB	52 KB	12 KB	12 KB	12 KB	12 KB	
Total Comm.	802 KB	2590 KB	2236 KB	172 KB	191 KB	241 KB	244 KB	345 KB	347 KB	236 KB	292 KB	404 KB	628 KB	
Server Time	600 ms	780 ms	750 ms	1100 ms	670 ms	470 ms	440 ms	360 ms	320 ms	650 ms	400 ms	280 ms	210 ms	
Throughput	1720 MB/s	1310 MB/s	1370 MB/s	930 MB/s	1530 MB/s	2200 MB/s	2310 MB/s	2880 MB/s	3230 MB/s	1580 MB/s	2530 MB/s	3670 MB/s	4850 MB/s	
8 GB ($2^{27}\times$ 64B)														
Offline Time	283 s	118 s	1500 s	402 s	235 s	157 s	118 s	94 s		581 s	597 s	579 s	467 s	
Upload (Keys)	462 KB	2688 KB	360 KB	80 KB	80 KB	80 KB	80 KB	80 KB		84 KB	84 KB	84 KB	84 KB	
Upload (Query)	448 KB	14 KB	512 KB	106 KB	132 KB	225 KB	431 KB	855 KB		532 KB	532 KB	980 KB	1876 KB	
Download	444 KB	224 KB	3316 KB	52 KB	52 KB	52 KB	52 KB	52 KB		12 KB	96 KB	12 KB	96 KB	
Total Comm.	1354 KB	2926 KB	4188 KB	238 KB	264 KB	357 KB	562 KB	986 KB		628 KB	712 KB	1076 KB	2056 KB	
Server Time	1850 ms	5910 ms	2030 ms	4680 ms	2670 ms	1850 ms	1390 ms	1150 ms		1360 ms	1340 ms	1120 ms	1020 ms	
Throughput	4420 MB/s	1390 MB/s	4040 MB/s	1750 MB/s	3070 MB/s	4440 MB/s	5880 MB/s	7150 MB/s		6010 MB/s	6110 MB/s	7320 MB/s	8050 MB/s	
32 GB ($2^{29}\times$ 64B)														
Offline Time	706 s	446 s	5700 s	589 s	421 s	379 s	339 s	318 s	291 s	2306 s	2128 s	2340 s	2114 s	2111 s
Upload (Keys)	462 KB	2912 KB	360 KB	80 KB	80 KB	80 KB	80 KB	80 KB	80 KB	84 KB	84 KB	84 KB	84 KB	84 KB
Upload (Query)	896 KB	14 KB	1024 KB	265 KB	450 KB	477 KB	861 KB	874 KB	1709 KB	980 KB	1876 KB	1876 KB	3668 KB	7252 KB
Download	888 KB	224 KB	6452 KB	52 KB	52 KB	52 KB	52 KB	52 KB	52 KB	96 KB	12 KB	96 KB	12 KB	12 KB
Total Comm.	2246 KB	3150 KB	7836 KB	397 KB	582 KB	609 KB	993 KB	1006 KB	1841 KB	1160 KB	1972 KB	2056 KB	3764 KB	7348 KB
Server Time	5610 ms	50570 ms	5910 ms	7390 ms	5280 ms	5260 ms	4430 ms	4410 ms	4010 ms	4320 ms	3880 ms	3820 ms	3630 ms	3510 ms
Throughput	5840 MB/s	650 MB/s	5550 MB/s	4430 MB/s	6210 MB/s	6230 MB/s	7400 MB/s	7430 MB/s	8180 MB/s	7580 MB/s	8450 MB/s	8570 MB/s	9030 MB/s	9330 MB/s

The breakdown of the communication cost shows that the cryptographic key material of InsPIRe$_0$ and InsPIRe is over 4 times smaller than all other protocols. This is due to the smaller keys used in InspiRING. Similarly, the query and the response size of InsPIRe are smaller than that of all other protocols. This is mainly due to our homomorphic polynomial evaluation technique that allows us to efficiently reduce the PIR response using small additional query overhead.

In summary, InsPIRe strictly improves over existing PIR schemes with server-side preprocessing, and may also be parameterized to optimize specific metrics such as communi-

Table 5.4: PIR performance metrics for different database sizes with 32 KB entries.

Metric	SimplePIR	KSPIR	HintlessPIR	InsPIRe					
1 GB ($2^{15}\times$ 32 KB)									
Offline Time	119 s	14 s	213 s	307 s	177 s	106 s	83 s	72 s	86 s
Upload (Keys)	462 KB	2352 KB	360 KB	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB
Upload (Query)	112 KB	14 KB	128 KB	91 KB	98 KB	112 KB	140 KB	196 KB	308 KB
Download	228 KB	224 KB	1748 KB	96 KB	96 KB	96 KB	96 KB	96 KB	96 KB
Total Comm.	802 KB	2590 KB	2236 KB	271 KB	278 KB	292 KB	320 KB	376 KB	488 KB
Server Time	600 ms	780 ms	750 ms	4270 ms	2210 ms	1170 ms	640 ms	410 ms	280 ms
Throughput	1720 MB/s	1310 MB/s	1370 MB/s	240 MB/s	460 MB/s	880 MB/s	1600 MB/s	2500 MB/s	3620 MB/s
8 GB ($2^{18}\times$ 32 KB)									
Offline Time	283 s	118 s	1500 s	547 s	470 s	598 s	597 s	538 s	467 s
Upload (Keys)	462 KB	2688 KB	360 KB	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB
Upload (Query)	448 KB	14 KB	512 KB	140 KB	196 KB	308 KB	532 KB	980 KB	1876 KB
Download	444 KB	224 KB	3316 KB	96 KB	96 KB	96 KB	96 KB	96 KB	96 KB
Total Comm.	1354 KB	2926 KB	4188 KB	320 KB	376 KB	488 KB	712 KB	1160 KB	2056 KB
Server Time	1850 ms	5910 ms	2030 ms	5220 ms	3010 ms	1920 ms	1340 ms	1120 ms	1020 ms
Throughput	4420 MB/s	1390 MB/s	4040 MB/s	1570 MB/s	2720 MB/s	4260 MB/s	6110 MB/s	7280 MB/s	8050 MB/s
32 GB ($2^{20}\times$ 32 KB)									
Offline Time	706 s	446 s	5700 s	2052 s	2581 s	2569 s	2306 s	2340 s	2053 s	1945 s
Upload (Keys)	462 KB	2912 KB	360 KB	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB
Upload (Query)	896 KB	14 KB	1024 KB	196 KB	308 KB	532 KB	980 KB	1876 KB	3668 KB	7252 KB
Download	888 KB	224 KB	6452 KB	192 KB	96 KB	96 KB	96 KB	96 KB	96 KB	96 KB
Total Comm.	2246 KB	3150 KB	7836 KB	472 KB	488 KB	712 KB	1160 KB	2056 KB	3848 KB	7432 KB
Server Time	5610 ms	50570 ms	5910 ms	15470 ms	7640 ms	5480 ms	4320 ms	3820 ms	3650 ms	3500 ms
Throughput	5840 MB/s	650 MB/s	5550 MB/s	2120 MB/s	4290 MB/s	5980 MB/s	7580 MB/s	8570 MB/s	8980 MB/s	9360 MB/s

cation.

5.6.4 Benchmarking Ring Packing

In this section, we provide a benchmark of InspiRING compared to existing ring packing algorithms such as CDKS [41] and the packing algorithms used in HintlessPIR [110]. In this benchmark, we pack 2^{12} LWE ciphertexts and measure the offline and online runtime of this procedure. Additionally, we report the size of additional material the client must provide for packing, i.e., the packing keys in the case of InspiRING. We run each protocol using the parameters provided in the respective paper or implementation to achieve the best performance and compare with InsPIRe that is instantiated with similar parameters. Using the notation of Section 5.3, we use two parameters set for InspiRING which provide 128-bit security based on the lattice estimator:

1. $\log_2 d = 10$, $\log_2 q = 28$, $\log_2 p = 6$, $\ell = 8$, $z = 2^4$
2. $\log_2 d = 11$, $\log_2 q = 56$, $\log_2 p = 15$, $\ell = 3$, $z = 2^{19}$

Table 5.5 summarizes the results.

Table 5.5: Concrete costs of packing 2^{12} LWE ciphertexts into RLWE ciphertexts using the approach in HintlessPIR [110], CDKS [41] and InspiRING.

Packing Type Unpacked Size $\log_2(d, q, p)$	HintlessPIR 16 MB (10, 32, 8)	InspiRING 14 MB (10, 28, 6)	CDKS 56 MB (11, 56, 15)	InspiRING 56 MB (11, 56, 15)
Key Material	360 KB	**60 KB**	462 KB	**84 KB**
Packed Size	180 KB	**32 KB**	56 KB	**56 KB**
Total Size	540 KB	**92 KB**	518 KB	**140 KB**
Offline Runtime	2.0 s	2.4 s	11 s	36 s
Online Runtime	141 ms	**16 ms**	56 ms	**40 ms**

We make the following observations from this table. Firstly, InspiRING requires significantly smaller key material compared to existing work, specifically, 84%, 76%, and over 99% less key material than CDKS and HintlessPIR, respectively. Secondly, the online time of InspiRING is 28% lower than the fastest existing work, CDKS. This, however, comes at the cost of a slower offline phase, which can be attributed to the quadratic dependence on d.

Lastly, we confirm the analytic noise analysis of InspiRING from Theorem 4. In the same experiment, we observe that the bitlength of $||e_{pack}||_\infty$ for ciphertexts packed using CDKS and InsPIRe ($d = 2048$) are equal to 38.5 and 33.4, respectively. This confirms that proposed ring packing construction has less noise growth than CDKS, up to 5 bits in this example.

5.7 Applications of InsPIRe

The server-side preprocessing and low communication of InsPIRe makes it an ideal candidate for the applications of PIR where an a priori setup is not feasible. We explore two such applications and show how InsPIRe is suitable solution with very good performance. The first application is the use of PIR in IPFS [124] for private queries and the second is private device enrollment in Chrome OS [1]. We use the same experimental setup as Section 5.6 of single-threaded execution on an Intel Xeon CPU @ 2.6 GHz.

5.7.1 Private Queries in IPFS

IPFS is a distributed file system which gives clients the ability to access content by querying multiple peers in the network to acquire the necessary routing information and eventually, the content itself. The process of locating and retrieving content can be simplified to

three types of database queries, i.e., peer routing, content discovery, and content retrieval. Mazmudar et al. [124] showed how these queries can be done privately with PIR. However, existing PIR protocols require exchanging large key material or a setup phase, which are not compatible with the query pattern in IPFS. A user iteratively contacts different servers to access different parts of the routing information, so the high cost of setup is not amortized over many queries. Hence, the authors proposed alternative PIR protocols which required no setup and had low overall communication costs.

InsPIRe can be used as a more suitable solution for this application, given the server-side preprocessing and extremely low communication. While the server-side processing requires time to generate the offline material, the server can generate a new preprocessed database while still serving the old database to prevent downtime. This will allow the server to always produce a response, and serve new information with a slight delay. Table 5.6 shows examples of the communication and computation costs of using PIR in IPFS. We provide the communication and computation cost of the best solution for each functionality of Mazmudar et al. [124] and show the corresponding cost of using InsPIRe. In the case of Content Discovery and Content Retrieval, we use InsPIRe with the parameters specified in Section 5.6 but in Peer Routing, due to the extremely small database size, we use smaller parameters ($d = 1024$) to achieve very low communication cost. InsPIRe provides strict improvement in communication and computation for all three functionalities.

Table 5.6: Costs of PIR queries in the three functionalities of IPFS. *Comm.* denotes the total communication required to serve the query and *Comp.* denotes the server runtime.

Step	Metric	Current [124]	InsPIRe	Improvement
Peer Routing (256 × 1.5 KB)	Comm.	> 100 KB	69 KB	32%
	Comp.	> 56 ms	51 ms	8%
Content Discovery (200k Records)	Comm.	> 280 KB	128 KB	46%
	Comp.	> 875 ms	123 ms	86%
Content Retrieval (2^{14}× 256 KB)	Comm.	> 2.1 MB	1.02 MB	51%
	Comp.	> 3.0 s	1.5 s	50%

5.7.2 Privacy-Preserving Device Enrollment

Upon starting a Chrome OS device, the enrollment process requires checking the membership of the device in a server-held database. Since the introduction of Chrome 94, Google has performed this membership check in a privacy-preserving manner using techniques such as PIR [1]. As Chrome devices perform the device enrollment procedure with PIR immediately when it is powered on for the first time or right after a factory reset, there is no opportunity for the device to perform setup with the server. InsPIRe is a solution

for this application due to the server-side preprocessing step. While the precise number of Chromebooks is not disclosed, publicly available data shows over 20 million devices are sold every year from 2019 until 2023.[5] We assume each device requires about 64 bytes of information from the database and provide runtimes for various number of devices. The cost of device enrollment using InsPIRe is shown in Table 5.7.

Table 5.7: Cost of PIR for Chrome Device Enrollment

Devices	DB Size	Offline Time	Communication	Response Time
20 M	1.19 GB	78 s	292 KB	416 ms
40 M	2.38 GB	131 s	292 KB	815 ms
80 M	4.76 GB	241 s	304 KB	1400 ms

Table 5.7 shows that the concrete cost of using PIR for device enrollment is only a few hundred KiloBytes, which is very practical network cost for a personal computer.

5.8 Related Works

PIR was introduced by Chor et al. [50] in the multi-server setting and Kushilevitz and Ostrovsky [103] in the single-server setting. Most early single-server PIR schemes were built upon number-theoretic assumptions [57, 135, 138] using a linearly homomorphic encryption scheme. Multi-server PIR has been built without any cryptographic assumptions [68] and one-way functions [27].

PIR using Server-Stored Client-Specific Keys. In recent years, many works aim to design practically efficient single-server PIR schemes using lattice-based homomorphic encryption starting with XPIR [2] leading to a long line of work [4, 9, 11, 33, 125, 132]. For efficiency purposes, these protocols assume that the server stores a key that is specific to each client that performs PIR queries. With these server-stored client-specific keys, these schemes obtain very low query communication, but require large computation due to relying on RLWE ciphertexts.

PIR using Client-Stored Database Hints. In another line of work, prior works consider single-server PIR where clients download and store database hints that will be used later for queries. The works of SimplePIR [86] and FrodoPIR [59] built schemes using LWE, but require each client to download large database hints before querying. In an orthogonal line, it was shown that client-stored database hints could enable sublinear query

[5] https://www.statista.com/statistics/749890/worldwide-chromebook-unit-shipments/

times [55, 56, 87, 102, 141]. Several very recent works have considered practical single-server implementations of this PIR paradigm [72, 108, 151, 175]. However, these works require either streaming the entire database in the offline phase or requiring the server to perform heavy computation per client.

PIR using the CRS Model. Very recently, several works have considered PIR without offline communication to avoid the obstacles and costs of offline preprocessing. Starting with Tiptoe [85], follow-ups such as HintlessPIR [110] and YPIR [126] consider practical PIR in the CRS model. Prior to InsPIRe, all these works sacrificed larger query communication in order to eliminate offline communication. In a more theoretical line of work, a recent breakthrough work [115] presented a doubly efficient PIR construction with sublinear query time and silent preprocessing from RLWE, but the practical costs remain impractical [134].

PIR with Additional Features. PIR with more advanced query functionalities have also been studied such as batch PIR that efficiently retrieves multiple entries at once [11, 12, 21, 84, 118, 133, 172]. Constructions for keyword PIR has also been presented where queries are performed over sparse databases consisting of keys and values [9, 35, 121, 142]. PIR with additional security features has also been studied such as in the presence of malicious servers including [52, 60] as well as symmetric PIR with database privacy [9, 113].

Ring Packing. The problem of ring packing to transform LWE to RLWE ciphertexts has been studied in three different classes. The row method [41] uses the least amount of cryptographic material while the diagonal [92] and column methods [19, 25, 47, 129] use more cryptographic material to obtain faster packing times.

Labeled Unbalanced PSI. The idea of using polynomial interpolation to represent sets and homomorphic polynomial evaluation for retrieval have been explored in the context of labeled unbalanced PSI [42, 43, 54], but the detailed constructions are quite different from ours. In particular, in these works, the interpolation is done over the plaintext SIMD slots (e.g., over $\mathbb{Z}_p$), while our polynomial interpolation is done over the *entire plaintext ring* R_p. Importantly, the PSI constructions encode the evaluation points as SIMD slots which are then encrypted as RLWE ciphertexts. The encrypted plaintexts generally have high norms, and so each homomorphic multiplication results in a large multiplicative noise growth. To overcome the large depth homomorphic multiplications, these works proposed tricks such as sending multiple powers of the evaluation points to the sender. In contrast, our homomorphic evaluation uses RGSW encrypted unit monomials which only incur additive noise growth for each multiplication, allowing the use of efficient RLWE parameters.

5.9 Conclusions

In this chapter, we present InsPIRe, a family of PIR protocols that obtains higher throughput and smaller query communication than all prior PIR schemes, using only server-side preprocessing. We specifically introduce three protocols, $\mathsf{InsPIRe}_0$, $\mathsf{InsPIRe}^{(2)}$, and InsPIRe which are suited for PIR over databases with various payload sizes. Along the way, we introduce a novel ring packing algorithm, InspiRING, requiring less cryptographic material and faster online packing times as well as a new approach to PIR using homomorphic polynomial evaluation.

InsPIRe has the lowest communication cost compared to all other protocols in the server-preprocessing model, while also having lower runtime. We argue how many existing applications cannot use existing PIR protocols due to the expensive setup costs, and how in these applications, InsPIRe outperforms all existing solutions.

Chapter 6

Conclusion

6.1 Response to Thesis Statement

(Fully) Homomorphic Encryption provides the ability to compute arbitrary functions on encrypted data. However, throughout this thesis we argued that simply implementing a desired functionality in homomorphic encryption rarely yields a practical solution. Instead, we must reshape our desired functionalities so that they are in line with the constraints present in encrypted computation and existing homomorphic encryption schemes. In response to the research question posed in Chapter 1, we presented a collection of protocols that exemplify how to overcome these constraints. We proposed building blocks which can be of independent interest and used in new applications, but also generated insights about how to design algorithms for homomorphic encryption. Our contributions can be summarized as follows:

- We proposed new algorithms for non-interactive Private Decision Tree Evaluation which are optimized for leveled homomorphic encryption by having a bounded multiplicative depth, independent of the structure of the tree and the precision of the input. As part of our protocols, we introduced novel private comparison operators, XXCMP and RCC, optimized for computation over encrypted data. Our private comparison algorithms are able to compare values of arbitrary length, with a multiplicative depth that is independent of the precision of the operands.
- We introduced PEPSI, a private set intersection protocol that improved on the state-of-the-art unbalanced circuit-PSI protocol by four orders of magnitude. We designed

PEPSI using efficient equality operators, in combination with techniques for reducing communication and computation costs, such as Cuckoo hashing and permutation-based hashing. As part of the design of PEPSI, we did a comprehensive analysis to determine optimal parameters for communication and computation, demonstrating the tradeoff between these parameters. Through our work, we demonstrated the complexity of optimizing for these metrics and our approach to find a suitable solution.

- We proposed InsPIRe, a class of novel PIR protocols optimized for server-side preprocessing. InsPIRe and its variants achieve the best performance amongst PIR protocols in the literature in the same model. We observed that InsPIRe is an useful protocol for applications in which an offline setup phase between the client and server is not feasible. As part of InsPIRe, we proposed a novel algorithm for ring packing, optimized for preprocessing, which could be of independent interest. We also use a new approach called homomorphic polynomial evaluation as a second level of PIR, which significantly reduces response sizes in PIR, without the need for additional large key material.

Aside from improvements to individual protocols like PDTE, PSI, and PIR, our work has helped gain insight into the process of designing protocols using homomorphic encryption. Specifically, from the collection of these works, we can draw the following high-level insights:

- Through the work in Chapter 3 and 4, we observe that changing the representation of data can make it much more suitable for implementation in FHE by reducing the multiplicative depth of the computation. This in turn allows the use of smaller parameters in the cryptosystem, which results in faster runtimes.

- Our work in Chapter 4 demonstrated that new data representations can also result in smaller communication costs, due to the use of smaller cryptographic parameters.

- Lastly, in all of our work we observe a communication-computation tradeoff which we can use to balance the cost based on the network and compute costs in each application. Moreover, homomorphic encryption introduces additional cryptographic parameters which affect the performance of the protocol and must be taken into account when optimizing for specific metrics.

It is also important to recognize the relationship between the protocols we have proposed and the possible reductions between these protocols. For example, PSI between two sets of size m and n, can be reduced to m Symmetric keyword PIR queries over a database of

n records [170]. Private decision tree evaluation can also be done using PIR as was shown by Azogagh et al. [18]. Conversely, PIR can be seen as a special case of PDTE, where the bits of the query are the attribute vector [91]. These reductions demonstrate that the improvements that we have proposed for one protocol can be advantageous for improving other protocols as well.

6.2 Future Directions

Our work shows some approaches for making private computation efficient, practical, and accessible for real-world use cases. In this section, we outline potential next steps to continue this effort.

Automated Design. The complexity of designing protocols for private computation makes it inaccessible for an audience without a cryptography background and limits how widely these techniques can be used. To lower the barrier for entry, we can take steps to ease the use of these techniques for developers. One approach is providing tools similar to compilers that translate high-level intent into low-level code and apply known optimizations. We note that FHE compilers for translating high-level code into a program that operates over encrypted data exist [8, 78, 167, 168]. However, existing FHE compilers only perform optimizations after the algorithm has been chosen and do not offer recommendations for better choice of algorithm that is compatible with encrypted computation. Hence, as part of future work, we can augment FHE compilers to also recommend better algorithms for computing over encrypted data.

Attention to hardware acceleration. As private computation becomes practical, hardware acceleration for homomorphic encryption and private computation protocols in general is becoming more appealing [153, 165, 174]. In the theme of this thesis, we see adaptability to hardware acceleration as a new constraint that can be taken into account in the design phase of private computation protocols. For example, many homomorphic encryption schemes or individual protocols work over a prime field, compared to working over a power-of-two modulus. As part of future work, we plan to adapt protocols such as InsPIRe to work over a power of two modulus to be more amenable to hardware acceleration.

Accounting for other deployment costs. As part of this thesis, we were able to address some real world deployment costs and challenges associated with protocols like

PIR. We do, however, see many remaining challenges that need to be addressed. For example, RAM usage is an important cost that is not factored into any existing PIR work. To be able to serve a request efficiently, all PIR protocols store the entire database, or a transformed version of it, in memory. For example, some works store the database encoded as polynomials and in NTT representation [11, 125]. The transformed database could expand its size significantly and incur very high costs for RAM usage. Hence, we foresee the need to design protocols that also take peak RAM usage into account.

References

[1] Google Cloud Vertex AI. https://cloud.google.com/vertex-ai. Accessed Jan 14, 2023.

[2] Carlos Aguilar-Melchor, Joris Barrier, Laurent Fousse, and Marc-Olivier Killijian. XPIR: Private information retrieval for everyone. *Proceedings on Privacy Enhancing Technologies*, 2016(2):155–174, April 2016.

[3] Ehud Aharoni, Allon Adir, Moran Baruch, Nir Drucker, Gilad Ezov, Ariel Farkash, Lev Greenberg, Ramy Masalha, Guy Moshkowich, Dov Murik, Hayim Shaul, and Omri Soceanu. HeLayers: A tile tensors framework for large neural networks on encrypted data. *Proceedings on Privacy Enhancing Technologies*, 2023(1):325–342, January 2023.

[4] Ishtiyaque Ahmad, Yuntian Yang, Divyakant Agrawal, Amr El Abbadi, and Trinabh Gupta. Addra: Metadata-private voice communication over fully untrusted infrastructure. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*, pages 313–329. USENIX Association, July 2021.

[5] Adi Akavia, Max Leibovich, Yehezkel S. Resheff, Roey Ron, Moni Shahar, and Margarita Vald. Privacy-Preserving Decision Trees Training and Prediction. *ACM Trans. Priv. Secur.*, 25(3), may 2022.

[6] Navid Alamati, Pedro Branco, Nico Döttling, Sanjam Garg, Mohammad Hajiabadi, and Sihang Pu. Laconic private set intersection and applications. In Kobbi Nissim and Brent Waters, editors, *TCC 2021: 19th Theory of Cryptography Conference, Part III*, volume 13044 of *Lecture Notes in Computer Science*, pages 94–125, Raleigh, NC, USA, November 8–11, 2021. Springer, Cham, Switzerland.

[7] Martin R Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. *Journal of Mathematical Cryptology*, 9(3):169–203, 2015.

[8] Asra Ali, Jaeho Choi, Bryant Gipson, Shruthi Gorantala, Jeremy Kun, Wouter Legiest, Lawrence Lim, Alexander Viand, Meron Zerihun Demissie, and Hongren Zheng. Heir: A universal compiler for homomorphic encryption, 2025.

[9] Asra Ali, Tancrède Lepoint, Sarvar Patel, Mariana Raykova, Phillipp Schoppmann, Karn Seth, and Kevin Yeo. Communication-computation trade-offs in PIR. In Michael Bailey and Rachel Greenstadt, editors, *USENIX Security 2021: 30th USENIX Security Symposium*, pages 1811–1828. USENIX Association, August 11–13, 2021.

[10] AMD. AMD SEV-SNP: Strengthening VM isolation with integrity protection and more, January, 2020.

[11] Sebastian Angel, Hao Chen, Kim Laine, and Srinath T. V. Setty. PIR with compressed queries and amortized query processing. In *2018 IEEE Symposium on Security and Privacy*, pages 962–979, San Francisco, CA, USA, May 21–23, 2018. IEEE Computer Society Press.

[12] Sebastian Angel and Srinath Setty. Unobservable communication over fully untrusted infrastructure. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, pages 551–569, 2016.

[13] Apple. Getting up-to-date calling and blocking information for your app. https://developer.apple.com/documentation/sms_and_call_reporting/getting_up-to-date_calling_and_blocking_information_for_your_app, 2024.

[14] Diego F. Aranha, Chuanwei Lin, Claudio Orlandi, and Mark Simkin. Laconic private set-intersection from pairings. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022: 29th Conference on Computer and Communications Security*, pages 111–124, Los Angeles, CA, USA, November 7–11, 2022. ACM Press.

[15] Yuriy Arbitman, Moni Naor, and Gil Segev. Backyard cuckoo hashing: Constant worst-case operations with a succinct representation. *2010 IEEE 51st Annual Symposium on Foundations of Computer Science*, pages 787–796, 2010.

[16] Gilad Asharov, Abhishek Jain, Adriana López-Alt, Eran Tromer, Vinod Vaikuntanathan, and Daniel Wichs. Multiparty computation with low communication, computation and interaction via threshold FHE. In David Pointcheval and Thomas Johansson, editors, *Advances in Cryptology – EUROCRYPT 2012*, volume 7237 of

Lecture Notes in Computer Science, pages 483–501, Cambridge, UK, April 15–19, 2012. Springer Berlin Heidelberg, Germany.

[17] Hilal Asi, Fabian Boemer, Nicholas Genise, Muhammad Haris Mughees, Tabitha Ogilvie, Rehan Rishi, Guy N. Rothblum, Kunal Talwar, Karl Tarbe, Ruiyu Zhu, and Marco Zuliani. Scalable Private Search with Wally, 2024.

[18] Sofiane Azogagh, Victor Delfour, Sébastien Gambs, and Marc-Olivier Killijian. PROBONITE: PRivate One-Branch-Only Non-Interactive Decision Tree Evaluation. In *Proceedings of the 10th Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, WAHC'22, page 23–33, New York, NY, USA, 2022. Association for Computing Machinery.

[19] Youngjin Bae, Jung Hee Cheon, Jaehyung Kim, Jai Hyun Park, and Damien Stehlé. HERMES: Efficient ring packing using MLWE ciphertexts and application to transciphering. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part IV*, volume 14084 of *Lecture Notes in Computer Science*, pages 37–69, Santa Barbara, CA, USA, August 20–24, 2023. Springer, Cham, Switzerland.

[20] Jianli Bai, Xiangfu Song, Shujie Cui, Ee-Chien Chang, and Giovanni Russello. Scalable private decision tree evaluation with sublinear communication. In Yuji Suga, Kouichi Sakurai, Xuhua Ding, and Kazue Sako, editors, *ASIACCS 22: 17th ACM Symposium on Information, Computer and Communications Security*, pages 843–857, Nagasaki, Japan, May 30 – June 3, 2022. ACM Press.

[21] Alexander Bienstock, Sarvar Patel, Joon Young Seo, and Kevin Yeo. Batch PIR and labeled PSI with oblivious ciphertext compression. In Davide Balzarotti and Wenyuan Xu, editors, *USENIX Security 2024: 33rd USENIX Security Symposium*, Philadelphia, PA, USA, August 14–16, 2024. USENIX Association.

[22] Nikita Borisov, George Danezis, and Ian Goldberg. DP5: A private presence service. *Proceedings on Privacy Enhancing Technologies*, 2015(2):4–24, April 2015.

[23] Jean-Philippe Bossuat, Juan Ramón Troncoso-Pastoriza, and Jean-Pierre Hubaux. Bootstrapping for approximate homomorphic encryption with negligible failure-probability by using sparse-secret encapsulation. In Giuseppe Ateniese and Daniele Venturi, editors, *ACNS 2022: 20th International Conference on Applied Cryptography and Network Security*, volume 13269 of *Lecture Notes in Computer Science*, pages 521–541, Rome, Italy, June 20–23, 2022. Springer, Cham, Switzerland.

[24] Raphael Bost, Raluca Ada Popa, Stephen Tu, and Shafi Goldwasser. Machine learning classification over encrypted data. In *ISOC Network and Distributed System Security Symposium – NDSS 2015*, San Diego, CA, USA, February 8–11, 2015. The Internet Society.

[25] Christina Boura, Nicolas Gama, Mariya Georgieva, and Dimitar Jetchev. Chimera: Combining ring-lwe-based fully homomorphic encryption schemes. *Journal of Mathematical Cryptology*, 14(1):316–338, 2020.

[26] Florian Bourse, Rafaël del Pino, Michele Minelli, and Hoeteck Wee. FHE circuit privacy almost for free. In Matthew Robshaw and Jonathan Katz, editors, *Advances in Cryptology – CRYPTO 2016, Part II*, volume 9815 of *Lecture Notes in Computer Science*, pages 62–89, Santa Barbara, CA, USA, August 14–18, 2016. Springer Berlin Heidelberg, Germany.

[27] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing: Improvements and extensions. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016: 23rd Conference on Computer and Communications Security*, pages 1292–1303, Vienna, Austria, October 24–28, 2016. ACM Press.

[28] Zvika Brakerski. Fully homomorphic encryption without modulus switching from classical GapSVP. In Reihaneh Safavi-Naini and Ran Canetti, editors, *Advances in Cryptology – CRYPTO 2012*, volume 7417 of *Lecture Notes in Computer Science*, pages 868–886, Santa Barbara, CA, USA, August 19–23, 2012. Springer Berlin Heidelberg, Germany.

[29] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (Leveled) fully homomorphic encryption without bootstrapping. In Shafi Goldwasser, editor, *ITCS 2012: 3rd Innovations in Theoretical Computer Science*, pages 309–325, Cambridge, MA, USA, January 8–10, 2012. Association for Computing Machinery.

[30] Zvika Brakerski and Vinod Vaikuntanathan. Efficient fully homomorphic encryption from (standard) LWE. In Rafail Ostrovsky, editor, *52nd Annual Symposium on Foundations of Computer Science*, pages 97–106, Palm Springs, CA, USA, October 22–25, 2011. IEEE Computer Society Press.

[31] Justin Brickell, Donald E. Porter, Vitaly Shmatikov, and Emmett Witchel. Privacy-preserving remote diagnostics. In Peng Ning, Sabrina De Capitani di Vimercati, and Paul F. Syverson, editors, *ACM CCS 2007: 14th Conference on Computer and*

Communications Security, pages 498–507, Alexandria, Virginia, USA, October 28–31, 2007. ACM Press.

[32] D. J. Britten and F. W. Lemire. A structure theorem for rings supporting a discrete fourier transform. *SIAM Journal on Applied Mathematics*, 41(2):222–226, 1981.

[33] Alexander Burton, Samir Jordan Menon, and David J. Wu. Respire: High-rate pir for databases with small records. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, CCS '24, page 1463–1477, New York, NY, USA, 2024. Association for Computing Machinery.

[34] Jan Camenisch and Gregory M. Zaverucha. Private intersection of certified sets. In Roger Dingledine and Philippe Golle, editors, *FC 2009: 13th International Conference on Financial Cryptography and Data Security*, volume 5628 of *Lecture Notes in Computer Science*, pages 108–127, Accra Beach, Barbados, February 23–26, 2009. Springer Berlin Heidelberg, Germany.

[35] Sofía Celi and Alex Davidson. Call me by my name: Simple, practical private information retrieval for keyword queries. In *ACM CCS*, 2024.

[36] Nishanth Chandran, Divya Gupta, and Akash Shah. Circuit-PSI with linear complexity via relaxed batch OPPRF. *Proceedings on Privacy Enhancing Technologies*, 2022(1):353–372, January 2022.

[37] Melissa Chase, Hao Chen, Jintai Ding, Shafi Goldwasser, Sergey Gorbunov, Jeffrey Hoffstein, Kristin Lauter, Satya Lokam, Dustin Moody, Travis Morrison, Amit Sahai, and Vinod Vaikuntanathan. Security of homomorphic encryption. Technical report, HomomorphicEncryption.org, Redmond WA, USA, July 2017.

[38] Hao Chen, Ilaria Chillotti, and Ling Ren. Onion ring ORAM: Efficient constant bandwidth oblivious RAM from (leveled) TFHE. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019: 26th Conference on Computer and Communications Security*, pages 345–360, London, UK, November 11–15, 2019. ACM Press.

[39] Hao Chen, Ilaria Chillotti, and Yongsoo Song. Improved bootstrapping for approximate homomorphic encryption. In Yuval Ishai and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2019, Part II*, volume 11477 of *Lecture Notes in Computer Science*, pages 34–54, Darmstadt, Germany, May 19–23, 2019. Springer, Cham, Switzerland.

[40] Hao Chen, Ilaria Chillotti, and Yongsoo Song. Multi-key homomorphic encryption from TFHE. In Steven D. Galbraith and Shiho Moriai, editors, *Advances in Cryptology – ASIACRYPT 2019, Part II*, volume 11922 of *Lecture Notes in Computer Science*, pages 446–472, Kobe, Japan, December 8–12, 2019. Springer, Cham, Switzerland.

[41] Hao Chen, Wei Dai, Miran Kim, and Yongsoo Song. Efficient homomorphic conversion between (ring) LWE ciphertexts. In Kazue Sako and Nils Ole Tippenhauer, editors, *ACNS 2021: 19th International Conference on Applied Cryptography and Network Security, Part I*, volume 12726 of *Lecture Notes in Computer Science*, pages 460–479, Kamakura, Japan, June 21–24, 2021. Springer, Cham, Switzerland.

[42] Hao Chen, Zhicong Huang, Kim Laine, and Peter Rindal. Labeled PSI from fully homomorphic encryption with malicious security. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *ACM CCS 2018: 25th Conference on Computer and Communications Security*, pages 1223–1237, Toronto, ON, Canada, October 15–19, 2018. ACM Press.

[43] Hao Chen, Kim Laine, and Peter Rindal. Fast private set intersection from homomorphic encryption. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017: 24th Conference on Computer and Communications Security*, pages 1243–1255, Dallas, TX, USA, October 31 – November 2, 2017. ACM Press.

[44] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. Bootstrapping for approximate homomorphic encryption. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2018, Part I*, volume 10820 of *Lecture Notes in Computer Science*, pages 360–384, Tel Aviv, Israel, April 29 – May 3, 2018. Springer, Cham, Switzerland.

[45] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yong Soo Song. Homomorphic encryption for arithmetic of approximate numbers. In Tsuyoshi Takagi and Thomas Peyrin, editors, *Advances in Cryptology – ASIACRYPT 2017, Part I*, volume 10624 of *Lecture Notes in Computer Science*, pages 409–437, Hong Kong, China, December 3–7, 2017. Springer, Cham, Switzerland.

[46] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *Advances in Cryptology – ASIACRYPT 2016,*

Part I, volume 10031 of *Lecture Notes in Computer Science*, pages 3–33, Hanoi, Vietnam, December 4–8, 2016. Springer Berlin Heidelberg, Germany.

[47] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. Faster packed homomorphic operations and efficient circuit bootstrapping for TFHE. In Tsuyoshi Takagi and Thomas Peyrin, editors, *Advances in Cryptology – ASIACRYPT 2017, Part I*, volume 10624 of *Lecture Notes in Computer Science*, pages 377–408, Hong Kong, China, December 3–7, 2017. Springer, Cham, Switzerland.

[48] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. TFHE: Fast fully homomorphic encryption over the torus. *Journal of Cryptology*, 33(1):34–91, January 2020.

[49] Benny Chor, Niv Gilboa, and Moni Naor. Private information retrieval by keywords. Cryptology ePrint Archive, Report 1998/003, 1998.

[50] Benny Chor, Eyal Kushilevitz, Oded Goldreich, and Madhu Sudan. Private information retrieval. *Journal of the ACM*, 45(6):965–981, November 1998.

[51] Michele Ciampi and Claudio Orlandi. Combining private set-intersection with secure two-party computation. In Dario Catalano and Roberto De Prisco, editors, *SCN 18: 11th International Conference on Security in Communication Networks*, volume 11035 of *Lecture Notes in Computer Science*, pages 464–482, Amalfi, Italy, September 5–7, 2018. Springer, Cham, Switzerland.

[52] Simone Colombo, Kirill Nikitin, Henry Corrigan-Gibbs, David J. Wu, and Bryan Ford. Authenticated private information retrieval. In Joseph A. Calandrino and Carmela Troncoso, editors, *USENIX Security 2023: 32nd USENIX Security Symposium*, pages 3835–3851, Anaheim, CA, USA, August 9–11, 2023. USENIX Association.

[53] Kelong Cong, Debajyoti Das, Jeongeun Park, and Hilder V. L. Pereira. SortingHat: Efficient private decision tree evaluation via homomorphic encryption and transciphering. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022: 29th Conference on Computer and Communications Security*, pages 563–577, Los Angeles, CA, USA, November 7–11, 2022. ACM Press.

[54] Kelong Cong, Radames Cruz Moreno, Mariana Botelho da Gama, Wei Dai, Ilia Iliashenko, Kim Laine, and Michael Rosenberg. Labeled PSI from homomorphic encryption with reduced computation and communication. In Giovanni Vigna and

Elaine Shi, editors, *ACM CCS 2021: 28th Conference on Computer and Communications Security*, pages 1135–1150, Virtual Event, Republic of Korea, November 15–19, 2021. ACM Press.

[55] Henry Corrigan-Gibbs, Alexandra Henzinger, and Dmitry Kogan. Single-server private information retrieval with sublinear amortized time. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology – EUROCRYPT 2022, Part II*, volume 13276 of *Lecture Notes in Computer Science*, pages 3–33, Trondheim, Norway, May 30 – June 3, 2022. Springer, Cham, Switzerland.

[56] Henry Corrigan-Gibbs and Dmitry Kogan. Private information retrieval with sublinear online time. In Anne Canteaut and Yuval Ishai, editors, *Advances in Cryptology – EUROCRYPT 2020, Part I*, volume 12105 of *Lecture Notes in Computer Science*, pages 44–75, Zagreb, Croatia, May 10–14, 2020. Springer, Cham, Switzerland.

[57] Ivan Damgård and Mats Jurik. A generalisation, a simplification and some applications of Paillier's probabilistic public-key system. In Kwangjo Kim, editor, *PKC 2001: 4th International Workshop on Theory and Practice in Public Key Cryptography*, volume 1992 of *Lecture Notes in Computer Science*, pages 119–136, Cheju Island, South Korea, February 13–15, 2001. Springer Berlin Heidelberg, Germany.

[58] Emma Dauterman, Eric Feng, Ellen Luo, Raluca Ada Popa, and Ion Stoica. DORY: An encrypted search system with distributed trust. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 1101–1119. USENIX Association, November 2020.

[59] Alex Davidson, Gonçalo Pestana, and Sofía Celi. FrodoPIR: Simple, scalable, single-server private information retrieval. *Proceedings on Privacy Enhancing Technologies*, 2023(1):365–383, January 2023.

[60] Leo de Castro and Keewoo Lee. VeriSimplePIR: Verifiability in SimplePIR at no online cost for honest servers. In Davide Balzarotti and Wenyuan Xu, editors, *USENIX Security 2024: 33rd USENIX Security Symposium*, Philadelphia, PA, USA, August 14–16, 2024. USENIX Association.

[61] Leo de Castro, Kevin Lewi, and Edward Suh. WhisPIR: Stateless private information retrieval with low communication. Cryptology ePrint Archive, Paper 2024/266, 2024.

[62] Emiliano De Cristofaro, Jihye Kim, and Gene Tsudik. Linear-complexity private set intersection protocols secure in malicious model. In Masayuki Abe, editor, *Advances*

in Cryptology – ASIACRYPT 2010, volume 6477 of *Lecture Notes in Computer Science*, pages 213–231, Singapore, December 5–9, 2010. Springer Berlin Heidelberg, Germany.

[63] Daniel Demmler, Peter Rindal, Mike Rosulek, and Ni Trieu. PIR-PSI: Scaling private contact discovery. *Proceedings on Privacy Enhancing Technologies*, 2018(4):159–178, October 2018.

[64] Dheeru Dua and Casey Graff. UCI machine learning repository, 2017.

[65] Léo Ducas and Daniele Micciancio. FHEW: Bootstrapping homomorphic encryption in less than a second. In Elisabeth Oswald and Marc Fischlin, editors, *Advances in Cryptology – EUROCRYPT 2015, Part I*, volume 9056 of *Lecture Notes in Computer Science*, pages 617–640, Sofia, Bulgaria, April 26–30, 2015. Springer Berlin Heidelberg, Germany.

[66] David Steven Dummit, Richard M Foote, et al. *Abstract algebra*, volume 3. Wiley Hoboken, 2004.

[67] Thai Duong, Duong Hieu Phan, and Ni Trieu. Catalic: Delegated PSI cardinality with applications to contact tracing. In Shiho Moriai and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2020, Part III*, volume 12493 of *Lecture Notes in Computer Science*, pages 870–899, Daejeon, South Korea, December 7–11, 2020. Springer, Cham, Switzerland.

[68] Zeev Dvir and Sivakanth Gopi. 2-server PIR with sub-polynomial communication. In Rocco A. Servedio and Ronitt Rubinfeld, editors, *47th Annual ACM Symposium on Theory of Computing*, pages 577–584, Portland, OR, USA, June 14–17, 2015. ACM Press.

[69] T. Elgamal. A public key cryptosystem and a signature scheme based on discrete logarithms. *IEEE Transactions on Information Theory*, 31(4):469–472, 1985.

[70] David Evans, Vladimir Kolesnikov, and Mike Rosulek. A pragmatic introduction to secure multi-party computation. *Foundations and Trends® in Privacy and Security*, 2(2-3):70–246, 2018.

[71] Junfeng Fan and Frederik Vercauteren. Somewhat Practical Fully Homomorphic Encryption. *Proceedings of the 15th international conference on Practice and Theory in Public Key Cryptography*, 2012:1–16, 2012.

[72] Ben Fisch, Arthur Lazzaretti, Zeyu Liu, and Charalampos Papamanthou. Thorpir: Single server pir via homomorphic thorp shuffles. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, CCS '24, page 1448–1462, New York, NY, USA, 2024. Association for Computing Machinery.

[73] Michael J. Freedman, Carmit Hazay, Kobbi Nissim, and Benny Pinkas. Efficient set intersection with simulation-based security. *Journal of Cryptology*, 29(1):115–155, January 2016.

[74] Michael J. Freedman, Kobbi Nissim, and Benny Pinkas. Efficient private matching and set intersection. In Christian Cachin and Jan Camenisch, editors, *Advances in Cryptology – EUROCRYPT 2004*, volume 3027 of *Lecture Notes in Computer Science*, pages 1–19, Interlaken, Switzerland, May 2–6, 2004. Springer Berlin Heidelberg, Germany.

[75] Craig Gentry. Fully homomorphic encryption using ideal lattices. In Michael Mitzenmacher, editor, *41st Annual ACM Symposium on Theory of Computing*, pages 169–178, Bethesda, MD, USA, May 31 – June 2, 2009. ACM Press.

[76] Craig Gentry. A decade (or so) of fully homomorphic encryption. Keynote talk at EUROCRYPT 2021, May 2021. Invited keynote talk.

[77] Shafi Goldwasser and Silvio Micali. Probabilistic encryption & how to play mental poker keeping secret all partial information. In *Proceedings of the Fourteenth Annual ACM Symposium on Theory of Computing*, STOC '82, page 365–377, New York, NY, USA, 1982. Association for Computing Machinery.

[78] Shruthi Gorantala, Rob Springer, Sean Purser-Haskell, William Lam, Royce Wilson, Asra Ali, Eric P. Astor, Itai Zukerman, Sam Ruth, Christoph Dibak, Phillipp Schoppmann, Sasha Kulankhina, Alain Forget, David Marn, Cameron Tew, Rafael Misoczki, Bernat Guillen, Xinyu Ye, Dennis Kraft, Damien Desfontaines, Aishe Krishnamurthy, Miguel Guevara, Irippuge Milinda Perera, Yurii Sushko, and Bryant Gipson. A general purpose transpiler for fully homomorphic encryption. Cryptology ePrint Archive, Paper 2021/811, 2021. https://eprint.iacr.org/2021/811.

[79] Matthew Green, Watson Ladd, and Ian Miers. A protocol for privately reporting ad impressions at scale. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016: 23rd Conference on Computer and Communications Security*, pages 1591–1601, Vienna, Austria, October 24–28, 2016. ACM Press.

[80] Trinabh Gupta, Natacha Crooks, Whitney Mulhern, Srinath Setty, Lorenzo Alvisi, and Michael Walfish. Scalable and private media consumption with popcorn. In *13th USENIX symposium on networked systems design and implementation (NSDI 16)*, pages 91–107, 2016.

[81] Kyoohyung Han and Dohyeong Ki. Better bootstrapping for approximate homomorphic encryption. In Stanislaw Jarecki, editor, *Topics in Cryptology – CT-RSA 2020*, volume 12006 of *Lecture Notes in Computer Science*, pages 364–390, San Francisco, CA, USA, February 24–28, 2020. Springer, Cham, Switzerland.

[82] Yidi Hao, Baodong Qin, and Yitian Sun. Privacy-preserving decision-tree evaluation with low complexity for communication. *Sensors*, 23(5):2624, 2023.

[83] Trevor Hastie, Robert Tibshirani, Jerome H Friedman, and Jerome H Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, volume 2. Springer, 2009.

[84] Ryan Henry. Polynomial batch codes for efficient IT-PIR. *Proceedings on Privacy Enhancing Technologies*, 2016(4):202–218, October 2016.

[85] Alexandra Henzinger, Emma Dauterman, Henry Corrigan-Gibbs, and Nickolai Zeldovich. Private web search with tiptoe. In *Proceedings of the 29th symposium on operating systems principles*, pages 396–416, 2023.

[86] Alexandra Henzinger, Matthew M. Hong, Henry Corrigan-Gibbs, Sarah Meiklejohn, and Vinod Vaikuntanathan. One server for the price of two: Simple and fast single-server private information retrieval. In Joseph A. Calandrino and Carmela Troncoso, editors, *USENIX Security 2023: 32nd USENIX Security Symposium*, pages 3889–3905, Anaheim, CA, USA, August 9–11, 2023. USENIX Association.

[87] Alexander Hoover, Sarvar Patel, Giuseppe Persiano, and Kevin Yeo. Plinko: Single-server PIR with efficient updates via invertible PRFs. Cryptology ePrint Archive, Report 2024/318, 2024.

[88] Ilia Iliashenko and Vincent Zucca. Faster homomorphic comparison operations for BGV and BFV. *Proceedings on Privacy Enhancing Technologies*, 2021(3):246–264, July 2021.

[89] Intel. Intel software guard extensions programming reference, 2014.

[90] Mihaela Ion, Ben Kreuter, Ahmet Erhan Nergiz, Sarvar Patel, Shobhit Saxena, Karn Seth, Mariana Raykova, David Shanahan, and Moti Yung. On deploying secure computing: Private intersection-sum-with-cardinality. In *2020 IEEE European Symposium on Security and Privacy*, pages 370–389, Genoa, Italy, September 7–11, 2020. IEEE Computer Society Press.

[91] Yuval Ishai and Anat Paskin. Evaluating branching programs on encrypted data. In Salil P. Vadhan, editor, *Theory of Cryptography*, pages 575–594, Berlin, Heidelberg, 2007. Springer Berlin Heidelberg.

[92] Wen jie Lu, Zhicong Huang, Cheng Hong, Yiping Ma, and Hunter Qu. PEGASUS: Bridging polynomial and non-polynomial evaluations in homomorphic encryption. In *2021 IEEE Symposium on Security and Privacy*, pages 1057–1073, San Francisco, CA, USA, May 24–27, 2021. IEEE Computer Society Press.

[93] Mika Juuti, Sebastian Szyller, Samuel Marchal, and N. Asokan. PRADA: Protecting against DNN model stealing attacks. In *2019 IEEE European Symposium on Security and Privacy*, pages 512–527, Stockholm, Sweden, June 17–19, 2019. IEEE Computer Society Press.

[94] Bailey Kacsmar, Basit Khurram, Nils Lukas, Alexander Norton, Masoumeh Shafieinejad, Zhiwei Shang, Yaser Baseri, Maryam Sepehri, Simon Oya, and Florian Kerschbaum. Differentially private two-party set operations. In *2020 IEEE European Symposium on Security and Privacy*, pages 390–404, Genoa, Italy, September 7–11, 2020. IEEE Computer Society Press.

[95] Daniel Kales, Christian Rechberger, Thomas Schneider, Matthias Senker, and Christian Weinert. Mobile private contact discovery at scale. In Nadia Heninger and Patrick Traynor, editors, *USENIX Security 2019: 28th USENIX Security Symposium*, pages 1447–1464, Santa Clara, CA, USA, August 14–16, 2019. USENIX Association.

[96] Manish Kesarwani, Bhaskar Mukhoty, Vijay Arya, and Sameep Mehta. Model Extraction Warning in MLaaS Paradigm. In *Proceedings of the 34th Annual Computer Security Applications Conference*, ACSAC '18, page 371–380, New York, NY, USA, 2018. Association for Computing Machinery.

[97] Aggelos Kiayias, Stavros Papadopoulos, Nikos Triandopoulos, and Thomas Zacharias. Delegatable pseudorandom functions and applications. In Ahmad-Reza Sadeghi, Virgil D. Gligor, and Moti Yung, editors, *ACM CCS 2013: 20th Conference on Computer and Communications Security*, pages 669–684, Berlin, Germany, November 4–8, 2013. ACM Press.

[98] Adam Kirsch, Michael Mitzenmacher, and Udi Wieder. More robust hashing: Cuckoo hashing with a stash. *SIAM Journal on Computing*, 39(4):1543–1561, 2010.

[99] Ágnes Kiss, Masoud Naderpour, Jian Liu, N. Asokan, and Thomas Schneider. SoK: Modular and efficient private decision tree evaluation. *Proceedings on Privacy Enhancing Technologies*, 2019(2):187–208, April 2019.

[100] Lea Kissner and Dawn Xiaodong Song. Privacy-preserving set operations. In Victor Shoup, editor, *Advances in Cryptology – CRYPTO 2005*, volume 3621 of *Lecture Notes in Computer Science*, pages 241–257, Santa Barbara, CA, USA, August 14–18, 2005. Springer Berlin Heidelberg, Germany.

[101] Christian Knabenhans, Alexander Viand, Antonio Merino-Gallardo, and Anwar Hithnawi. vfhe: Verifiable fully homomorphic encryption. In *Proceedings of the 12th Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, WAHC '24, page 11–22, New York, NY, USA, 2024. Association for Computing Machinery.

[102] Dmitry Kogan and Henry Corrigan-Gibbs. Private blocklist lookups with checklist. In Michael Bailey and Rachel Greenstadt, editors, *USENIX Security 2021: 30th USENIX Security Symposium*, pages 875–892. USENIX Association, August 11–13, 2021.

[103] Eyal Kushilevitz and Rafail Ostrovsky. Replication is NOT needed: SINGLE database, computationally-private information retrieval. In *38th Annual Symposium on Foundations of Computer Science*, pages 364–373, Miami Beach, Florida, October 19–22, 1997. IEEE Computer Society Press.

[104] Albert Kwon, David Lazar, Srinivas Devadas, and Bryan Ford. Riffle: An efficient communication system with strong anonymity. *Proceedings on Privacy Enhancing Technologies*, 2016(2):115–134, April 2016.

[105] Mikkel Lambæk. Breaking and fixing private set intersection protocols. Cryptology ePrint Archive, Report 2016/665, 2016.

[106] Adeline Langlois and Damien Stehlé. Worst-case to average-case reductions for module lattices. *Designs, Codes and Cryptography*, 75(3):565–599, 2015.

[107] Kristin Lauter, Sreekanth Kannepalli, Kim Laine, and Radames Cruz Moreno. Password Monitor: Safeguarding passwords in Microsoft Edge. https://www.microsoft.com/en-us/research/blog/password-monitor-safeguarding-passwords-in-microsoft-edge/, 2021.

[108] Arthur Lazzaretti and Charalampos Papamanthou. Single pass client-preprocessing private information retrieval. In Davide Balzarotti and Wenyuan Xu, editors, *USENIX Security 2024: 33rd USENIX Security Symposium*, Philadelphia, PA, USA, August 14–16, 2024. USENIX Association.

[109] Tancrède Lepoint, Sarvar Patel, Mariana Raykova, Karn Seth, and Ni Trieu. Private join and compute from PIR with default. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2021, Part II*, volume 13091 of *Lecture Notes in Computer Science*, pages 605–634, Singapore, December 6–10, 2021. Springer, Cham, Switzerland.

[110] Baiyu Li, Daniele Micciancio, Mariana Raykova, and Mark Schultz. Hintless single-server private information retrieval. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology – CRYPTO 2024, Part IX*, volume 14928 of *Lecture Notes in Computer Science*, pages 183–217, Santa Barbara, CA, USA, August 18–22, 2024. Springer, Cham, Switzerland.

[111] Lucy Li, Bijeeta Pal, Junade Ali, Nick Sullivan, Rahul Chatterjee, and Thomas Ristenpart. Protocols for checking compromised credentials. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, CCS '19, page 1387–1403, New York, NY, USA, 2019. Association for Computing Machinery.

[112] Lucy Li, Bijeeta Pal, Junade Ali, Nick Sullivan, Rahul Chatterjee, and Thomas Ristenpart. Protocols for checking compromised credentials. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019: 26th Conference on Computer and Communications Security*, pages 1387–1403, London, UK, November 11–15, 2019. ACM Press.

[113] Chengyu Lin, Zeyu Liu, and Tal Malkin. XSPIR: Efficient symmetrically private information retrieval from ring-LWE. In Vijayalakshmi Atluri, Roberto Di Pietro, Christian Damsgaard Jensen, and Weizhi Meng, editors, *ESORICS 2022: 27th European Symposium on Research in Computer Security, Part I*, volume 13554 of *Lecture Notes in Computer Science*, pages 217–236, Copenhagen, Denmark, September 26–30, 2022. Springer, Cham, Switzerland.

[114] Hsiao-Ying Lin and Wen-Guey Tzeng. An efficient solution to the millionaires' problem based on homomorphic encryption. In John Ioannidis, Angelos Keromytis, and Moti Yung, editors, *ACNS 2005: 3rd International Conference on Applied Cryptography and Network Security*, volume 3531 of *Lecture Notes in Computer Science*,

pages 456–466, New York, NY, USA, June 7–10, 2005. Springer Berlin Heidelberg, Germany.

[115] Wei-Kai Lin, Ethan Mook, and Daniel Wichs. Doubly efficient private information retrieval and fully homomorphic RAM computation from ring LWE. In Barna Saha and Rocco A. Servedio, editors, *55th Annual ACM Symposium on Theory of Computing*, pages 595–608, Orlando, FL, USA, June 20–23, 2023. ACM Press.

[116] Wen-jie Lu, Zhicong Huang, Qizhi Zhang, Yuchen Wang, and Cheng Hong. Squirrel: A Scalable Secure Two-Party Computation Framework for Training Gradient Boosting Decision Tree. *USENIX Security Symposium*, 2023.

[117] Wenjie Lu, Jun-Jie Zhou, and Jun Sakuma. Non-interactive and output expressive private comparison from homomorphic encryption. In Jong Kim, Gail-Joon Ahn, Seungjoo Kim, Yongdae Kim, Javier López, and Taesoo Kim, editors, *ASIACCS 18: 13th ACM Symposium on Information, Computer and Communications Security*, pages 67–74, Incheon, Republic of Korea, April 2–6, 2018. ACM Press.

[118] Wouter Lueks and Ian Goldberg. Sublinear scaling for multi-client private information retrieval. In Rainer Böhme and Tatsuaki Okamoto, editors, *FC 2015: 19th International Conference on Financial Cryptography and Data Security*, volume 8975 of *Lecture Notes in Computer Science*, pages 168–186, San Juan, Puerto Rico, January 26–30, 2015. Springer Berlin Heidelberg, Germany.

[119] Ming Luo, Feng-Hao Liu, and Han Wang. Faster fhe-based single-server private information retrieval. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 1405–1419, 2024.

[120] Jack P. K. Ma and Sherman S. M. Chow. Secure-computation-friendly private set intersection from oblivious compact graph evaluation. In Yuji Suga, Kouichi Sakurai, Xuhua Ding, and Kazue Sako, editors, *ASIACCS 22: 17th ACM Symposium on Information, Computer and Communications Security*, pages 1086–1097, Nagasaki, Japan, May 30 – June 3, 2022. ACM Press.

[121] Rasoul Akhavan Mahdavi and Florian Kerschbaum. Constant-weight PIR: Single-round keyword PIR via constant-weight equality operators. In Kevin R. B. Butler and Kurt Thomas, editors, *USENIX Security 2022: 31st USENIX Security Symposium*, pages 1723–1740, Boston, MA, USA, August 10–12, 2022. USENIX Association.

[122] Rasoul Akhavan Mahdavi, Nils Lukas, Faezeh Ebrahimianghazani, Thomas Humphries, Bailey Kacsmar, John A. Premkumar, Xinda Li, Simon Oya, Ehsan

Amjadian, and Florian Kerschbaum. PEPSI: Practically efficient private set intersection in the unbalanced setting. In Davide Balzarotti and Wenyuan Xu, editors, *USENIX Security 2024: 33rd USENIX Security Symposium*, Philadelphia, PA, USA, August 14–16, 2024. USENIX Association.

[123] Rasoul Akhavan Mahdavi, Haoyan Ni, Dimitry Linkov, and Florian Kerschbaum. Level up: Private non-interactive decision tree evaluation using levelled homomorphic encryption. In Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda, editors, *ACM CCS 2023: 30th Conference on Computer and Communications Security*, pages 2945–2958, Copenhagen, Denmark, November 26–30, 2023. ACM Press.

[124] Miti Mazmudar, Shannon Veitch, and Rasoul Akhavan Mahdavi. Peer2PIR: Private Queries for IPFS . In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 4438–4456, San Fransisco, CA, USA, May 2025. IEEE Computer Society.

[125] Samir Jordan Menon and David J. Wu. SPIRAL: Fast, high-rate single-server PIR via FHE composition. In *2022 IEEE Symposium on Security and Privacy*, pages 930–947, San Francisco, CA, USA, May 22–26, 2022. IEEE Computer Society Press.

[126] Samir Jordan Menon and David J. Wu. YPIR: High-throughput single-server PIR with silent preprocessing. In Davide Balzarotti and Wenyuan Xu, editors, *USENIX Security 2024: 33rd USENIX Security Symposium*, Philadelphia, PA, USA, August 14–16, 2024. USENIX Association.

[127] Arthur Meyre, Benoit Chevallier-Mames, Jordan Frery, Andrei Stoian, Roman Bredehoft, Luis Montero, and Celia Kherfallah. Concrete-ML: a privacy-preserving machine learning library using fully homomorphic encryption for data scientists, 2022. https://github.com/zama-ai/concrete-ml.

[128] Daniele Micciancio and Chris Peikert. Trapdoors for lattices: Simpler, tighter, faster, smaller. In David Pointcheval and Thomas Johansson, editors, *Advances in Cryptology – EUROCRYPT 2012*, volume 7237 of *Lecture Notes in Computer Science*, pages 700–718, Cambridge, UK, April 15–19, 2012. Springer Berlin Heidelberg, Germany.

[129] Daniele Micciancio and Jessica Sorrell. Ring packing and amortized FHEW bootstrapping. In Ioannis Chatzigiannakis, Christos Kaklamanis, Dániel Marx, and Donald Sannella, editors, *ICALP 2018: 45th International Colloquium on Automata, Languages and Programming*, volume 107 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 100:1–100:14, Prague, Czech Republic, July 9–13, 2018. Schloss Dagstuhl - Leibniz-Zentrum fuer Informatik.

[130] Prateek Mittal, Femi Olumofin, Carmela Troncoso, Nikita Borisov, and Ian Goldberg. PIR-Tor: Scalable anonymous communication using private information retrieval. In *20th USENIX Security Symposium (USENIX Security 11)*, San Francisco, CA, August 2011. USENIX Association.

[131] Prateek Mittal, Femi G. Olumofin, Carmela Troncoso, Nikita Borisov, and Ian Goldberg. PIR-tor: Scalable anonymous communication using private information retrieval. In *USENIX Security 2011: 20th USENIX Security Symposium*, San Francisco, CA, USA, August 8–12, 2011. USENIX Association.

[132] Muhammad Haris Mughees, Hao Chen, and Ling Ren. OnionPIR: Response efficient single-server PIR. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021: 28th Conference on Computer and Communications Security*, pages 2292–2306, Virtual Event, Republic of Korea, November 15–19, 2021. ACM Press.

[133] Muhammad Haris Mughees and Ling Ren. Vectorized batch private information retrieval. In *2023 IEEE Symposium on Security and Privacy*, pages 437–452, San Francisco, CA, USA, May 21–25, 2023. IEEE Computer Society Press.

[134] Hiroki Okada, Rachel Player, Simon Pohmann, and Christian Weinert. Towards practical doubly-efficient private information retrieval. In *Financial Cryptography and Data Security: 28th International Conference, FC 2024, Willemstad, Curaçao, March 4–8, 2024, Revised Selected Papers, Part II*, page 264–282, Berlin, Heidelberg, 2025. Springer-Verlag.

[135] Rafail Ostrovsky and William E. Skeith, III. A survey of single-database private information retrieval: Techniques and applications (invited talk). In Tatsuaki Okamoto and Xiaoyun Wang, editors, *PKC 2007: 10th International Conference on Theory and Practice of Public Key Cryptography*, volume 4450 of *Lecture Notes in Computer Science*, pages 393–411, Beijing, China, April 16–20, 2007. Springer Berlin Heidelberg, Germany.

[136] Rasmus Pagh and Flemming Friche Rodler. Cuckoo hashing. *J. Algorithms*, 51(2):122–144, may 2004.

[137] Pascal Paillier. Public-key cryptosystems based on composite degree residuosity classes. In *Proceedings of the 17th International Conference on Theory and Application of Cryptographic Techniques*, EUROCRYPT'99, page 223–238, Berlin, Heidelberg, 1999. Springer-Verlag.

[138] Pascal Paillier. Public-key cryptosystems based on composite degree residuosity classes. In Jacques Stern, editor, *Advances in Cryptology – EUROCRYPT'99*, volume 1592 of *Lecture Notes in Computer Science*, pages 223–238, Prague, Czech Republic, May 2–6, 1999. Springer Berlin Heidelberg, Germany.

[139] Doni Setio Pambudi, Hudan Studiawan, Baskoro Adi Pratomo, and Diana Purwitasari. *Performance Analysis of Leading Homomorphic Encryption Libraries: A Benchmark Study of SEAL, HElib, OpenFHE, and Lattigo*, page 25–30. Association for Computing Machinery, New York, NY, USA, 2025.

[140] Nicolas Papernot, Patrick D. McDaniel, Ian J. Goodfellow, Somesh Jha, Z. Berkay Celik, and Ananthram Swami. Practical black-box attacks against machine learning. In Ramesh Karri, Ozgur Sinanoglu, Ahmad-Reza Sadeghi, and Xun Yi, editors, *ASIACCS 17: 12th ACM Symposium on Information, Computer and Communications Security*, pages 506–519, Abu Dhabi, United Arab Emirates, April 2–6, 2017. ACM Press.

[141] Sarvar Patel, Giuseppe Persiano, and Kevin Yeo. Private stateful information retrieval. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *ACM CCS 2018: 25th Conference on Computer and Communications Security*, pages 1002–1019, Toronto, ON, Canada, October 15–19, 2018. ACM Press.

[142] Sarvar Patel, Joon Young Seo, and Kevin Yeo. Don't be dense: Efficient keyword PIR for sparse databases. In Joseph A. Calandrino and Carmela Troncoso, editors, *USENIX Security 2023: 32nd USENIX Security Symposium*, pages 3853–3870, Anaheim, CA, USA, August 9–11, 2023. USENIX Association.

[143] Chris Peikert et al. A decade of lattice cryptography. *Foundations and trends® in theoretical computer science*, 10(4):283–424, 2016.

[144] Chris Peikert and Brent Waters. Lossy trapdoor functions and their applications. In Richard E. Ladner and Cynthia Dwork, editors, *40th Annual ACM Symposium on Theory of Computing*, pages 187–196, Victoria, BC, Canada, May 17–20, 2008. ACM Press.

[145] Benny Pinkas, Thomas Schneider, Gil Segev, and Michael Zohner. Phasing: Private set intersection using permutation-based hashing. In Jaeyeon Jung and Thorsten Holz, editors, *USENIX Security 2015: 24th USENIX Security Symposium*, pages 515–530, Washington, DC, USA, August 12–14, 2015. USENIX Association.

[146] Benny Pinkas, Thomas Schneider, Oleksandr Tkachenko, and Avishay Yanai. Efficient circuit-based PSI with linear communication. In Yuval Ishai and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2019, Part III*, volume 11478 of *Lecture Notes in Computer Science*, pages 122–153, Darmstadt, Germany, May 19–23, 2019. Springer, Cham, Switzerland.

[147] Benny Pinkas, Thomas Schneider, Christian Weinert, and Udi Wieder. Efficient circuit-based PSI via cuckoo hashing. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2018, Part III*, volume 10822 of *Lecture Notes in Computer Science*, pages 125–157, Tel Aviv, Israel, April 29 – May 3, 2018. Springer, Cham, Switzerland.

[148] Benny Pinkas, Thomas Schneider, and Michael Zohner. Scalable private set intersection based on ot extension. *ACM Transactions on Privacy and Security (TOPS)*, 21(2):1–35, 2018.

[149] J. Ross Quinlan. Induction of Decision Trees. *Machine learning*, 1:81–106, 1986.

[150] L. Reichert, M. Pazelt, and B. Scheuermann. Circuit-based psi for covid-19 risk scoring. In *2021 IEEE International Performance, Computing, and Communications Conference (IPCCC)*, pages 1–8, Los Alamitos, CA, USA, oct 2021. IEEE Computer Society.

[151] Ling Ren, Muhammad Haris Mughees, and I Sun. Simple and practical amortized sublinear private information retrieval using dummy subsets. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, CCS '24, page 1420–1433, New York, NY, USA, 2024. Association for Computing Machinery.

[152] Ronald L Rivest, Len Adleman, Michael L Dertouzos, et al. On data banks and privacy homomorphisms. *Foundations of secure computation*, 4(11):169–180, 1978.

[153] Nikola Samardzic, Axel Feldmann, Aleksandar Krastev, Srinivas Devadas, Ronald Dreslinski, Christopher Peikert, and Daniel Sanchez. F1: A fast and programmable accelerator for fully homomorphic encryption. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, MICRO '21, page 238–252, New York, NY, USA, 2021. Association for Computing Machinery.

[154] Microsoft SEAL (release 4.0). https://github.com/Microsoft/SEAL, March 2022. Microsoft Research, Redmond, WA.

[155] Elaine Shi, John Bethencourt, Hubert T.-H. Chan, Dawn Xiaodong Song, and Adrian Perrig. Multi-dimensional range query over encrypted data. In *2007 IEEE Symposium on Security and Privacy*, pages 350–364, Oakland, CA, USA, May 20–23, 2007. IEEE Computer Society Press.

[156] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. Membership inference attacks against machine learning models. In *2017 IEEE Symposium on Security and Privacy*, pages 3–18, San Jose, CA, USA, May 22–26, 2017. IEEE Computer Society Press.

[157] Nigel P. Smart and Frederik Vercauteren. Fully homomorphic SIMD operations. *Designs, Codes and Cryptography*, 71(1):57–81, 2014.

[158] Raymond K. H. Tai, Jack P. K. Ma, Yongjun Zhao, and Sherman S. M. Chow. Privacy-preserving decision trees evaluation via linear functions. In Simon N. Foley, Dieter Gollmann, and Einar Snekkenes, editors, *ESORICS 2017: 22nd European Symposium on Research in Computer Security, Part II*, volume 10493 of *Lecture Notes in Computer Science*, pages 494–512, Oslo, Norway, September 11–15, 2017. Springer, Cham, Switzerland.

[159] Jonathan Takeshita, Ryan Karl, Alamin Mohammed, Aaron Striegel, and Taeho Jung. Provably secure contact tracing with conditional private set intersection. In Joaquin Garcia-Alfaro, Shujun Li, Radha Poovendran, Hervé Debar, and Moti Yung, editors, *Security and Privacy in Communication Networks*, Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering, pages 352–373. Springer International Publishing, 2021.

[160] Kurt Thomas, Jennifer Pullman, Kevin Yeo, Ananth Raghunathan, Patrick Gage Kelley, Luca Invernizzi, Borbala Benko, Tadek Pietraszek, Sarvar Patel, Dan Boneh, and Elie Bursztein. Protecting accounts from credential stuffing with password breach alerting. In Nadia Heninger and Patrick Traynor, editors, *USENIX Security 2019: 28th USENIX Security Symposium*, pages 1556–1571, Santa Clara, CA, USA, August 14–16, 2019. USENIX Association.

[161] Florian Tramèr, Fan Zhang, Ari Juels, Michael K. Reiter, and Thomas Ristenpart. Stealing machine learning models via prediction APIs. In Thorsten Holz and Stefan Savage, editors, *USENIX Security 2016: 25th USENIX Security Symposium*, pages 601–618, Austin, TX, USA, August 10–12, 2016. USENIX Association.

[162] Ni Trieu, Kareem Shehata, P. Saxena, R. Shokri, and Dawn Xiaodong Song. Epione: Lightweight contact tracing with strong privacy. *ArXiv*, abs/2004.13293, 2020.

[163] Anselme Tueno, Yordan Boev, and Florian Kerschbaum. Non-Interactive Private Decision Tree Evaluation. In *Database Security*, pages 174–194. Springer International Publishing, 2019.

[164] Anselme Tueno, Florian Kerschbaum, and Stefan Katzenbeisser. Private evaluation of decision trees using sublinear cost. *Proceedings on Privacy Enhancing Technologies*, 2019(1):266–286, January 2019.

[165] Michiel van Beirendonck, Jan-Pieter D'Anvers, Furkan Turan, and Ingrid Verbauwhede. FPT: A fixed-point accelerator for torus fully homomorphic encryption. In Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda, editors, *ACM CCS 2023: 30th Conference on Computer and Communications Security*, pages 741–755, Copenhagen, Denmark, November 26–30, 2023. ACM Press.

[166] Marten van Dijk, Craig Gentry, Shai Halevi, and Vinod Vaikuntanathan. Fully homomorphic encryption over the integers. In *Proceedings of the 29th Annual International Conference on Theory and Applications of Cryptographic Techniques*, EUROCRYPT'10, page 24–43, Berlin, Heidelberg, 2010. Springer-Verlag.

[167] Alexander Viand, Patrick Jattke, Miro Haller, and Anwar Hithnawi. HECO: Fully homomorphic encryption compiler. In Joseph A. Calandrino and Carmela Troncoso, editors, *USENIX Security 2023: 32nd USENIX Security Symposium*, pages 4715–4732, Anaheim, CA, USA, August 9–11, 2023. USENIX Association.

[168] Alexander Viand, Patrick Jattke, and Anwar Hithnawi. SoK: Fully homomorphic encryption compilers. In *2021 IEEE Symposium on Security and Privacy*, pages 1092–1108, San Francisco, CA, USA, May 24–27, 2021. IEEE Computer Society Press.

[169] Frank Wang, Catherine Yun, Shafi Goldwasser, Vinod Vaikuntanathan, and Matei Zaharia. Splinter: Practical private queries on public data. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pages 299–313, 2017.

[170] Zhusheng Wang, Karim Banawan, and Sennur Ulukus. Private set intersection: A multi-message symmetric private information retrieval perspective. *IEEE Transactions on Information Theory*, 68(3):2001–2019, 2021.

[171] Udi Wieder. Hashing, load balancing and multiple choice. *Foundations and Trends® in Theoretical Computer Science*, 12(3–4):275–379, 2017.

[172] Kevin Yeo. Cuckoo hashing in cryptography: Optimal parameters, robustness and applications. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part IV*, volume 14084 of *Lecture Notes in Computer Science*, pages 197–230, Santa Barbara, CA, USA, August 20–24, 2023. Springer, Cham, Switzerland.

[173] Jason H. M. Ying, Shuwei Cao, Geong Sen Poh, Jia Xu, and Hoon Wei Lim. PSI-stats: Private set intersection protocols supporting secure statistical functions. In Giuseppe Ateniese and Daniele Venturi, editors, *ACNS 2022: 20th International Conference on Applied Cryptography and Network Security*, volume 13269 of *Lecture Notes in Computer Science*, pages 585–604, Rome, Italy, June 20–23, 2022. Springer, Cham, Switzerland.

[174] Junxue Zhang, Xiaodian Cheng, Liu Yang, Jinbin Hu, Ximeng Liu, and Kai Chen. Sok: Fully homomorphic encryption accelerators. *ACM Comput. Surv.*, 56(12), October 2024.

[175] Mingxun Zhou, Andrew Park, Wenting Zheng, and Elaine Shi. Piano: Extremely simple, single-server PIR with sublinear server computation. In *2024 IEEE Symposium on Security and Privacy*, pages 4296–4314, San Francisco, CA, USA, May 19–23, 2024. IEEE Computer Society Press.

APPENDICES

Appendix A

PEPSI: Practically Efficient Private Set Intersection in the Unbalanced Setting

A.1 Data Preparation

The algorithm for server and client data preparation is provided as Algorithm 27. The client's secret key is denoted $\mathtt{sk}_c$, and is used for encryption. The encryption procedure is denoted with Enc. CW-ENCODE is the algorithm for mapping elements to constant-weight codewords. Precisely, we use the perfect mapping [121, Algorithm 3] and the lossy mapping [121, Algorithm 8] from the work of Mahdavi and Kerschbaum in the case of small and large elements, respectively.

A.2 PSI-kth-Match

Another function that the server can compute over the intersection is returning only one element in the intersection. This can be used to iteratively return the results of the protocol. For example, a data scientist may wish to see a sample of items in the intersection before deciding whether to recieve the entire intersection.

For simplicity, we only describe the cleartext algorithm for finding the kth match in a vector. The algorithm is designed such that it can be efficiently computed using homomorphic encryption. We include correctness and security proof to show that this algorithm can

Algorithm 27 Client and server data preparation

algorithm ClientDataPrep($\mathbb{X}$)
  Initialize T_c with b bins.
  for each element $x \in \mathbb{X}$ **do**
    Append x to the bins of T_c according to the binning strategy
  Append dummy elements to fill each bin in T_c to the max
  for each bin $k \in [b]$ **do**
    for each batch $i \in [\gamma]$ **do**
      $T'_c[k][i] = \text{CW-Encode}(T_c[k][i], \ell, h)$
  for each batch $i \in [\gamma]$ **do**
    for each bit $j \in [\ell]$ **do**
      $pt[i][j] = [T'_c[1][i][j], T'_c[2][i][j], \cdots, T'_c[b][i][j]]$
      $ct_c[i][j] = \mathsf{Enc}(pt_c[i][j], \mathtt{sk}_c)$
  return ct_c

algorithm ServerDataPrep($\mathbb{Y}$)
  Initialize T_s with b bins
  for each server element $y \in \mathbb{Y}$ **do**
    Append y to bins chosen by the binning strategy.
  Append dummy elements to fill each bin in T_s to μ.
  for each batch $k \in [b]$ **do**
    for each batch $i \in [\mu]$ **do**
      $T'_s[k][i] = \text{CW-Encode}(T_c[k][i], \ell, h)$
  for each batch $i \in [\mu]$ **do**
    for each bit $j \in [\ell]$ **do**
      $pt_s[i][j] = [T'_s[1][i][j], T'_s[2][i][j], \cdots, T'_s[b][i][j]]$
  return pt_s

compute the kth match whilst not revealing any other information about the intersection. The full details on how to implement this algorithm using homomorphic encryption and the FV library are left for future work.

Assume that $I \in \{0,1\}^n$ denotes a vector with ones in some indices. The objective is to compute M such that $M[i_k] = 0$ where i_k is the position of the kth one in I. Algorithm 28 shows the algorithm that can achieve this.

Algorithm 28 Algorithm for kth match

algorithm ComputePSIkthMatch($I \in \{0,1\}^n, k$)
  for each $i \in [n]$ **do**
    $r \xleftarrow{\$} \mathbb{Z}_p$
    $M[i] \leftarrow r \cdot (k \cdot I[i] - 1 - \sum_{i'<i} I[i'])$
  return M

Theorem 15. *Algorithm 28 is correct, i.e., M is zero at the index of the kth one in I, and non-zero in all the indices.*

Proof. Let $i_1 < i_2 < \cdots < i_s$ be the indices such that $I[i_1] = I[i_2] = \cdots = I[i_s] = 1$ and

$I[i] = 0$ for $i \neq i_j$. If $M[i] = k \cdot I[i] - 1 - \sum_{i' \leq i} I[i']$ then

$$M[i_k] = k \cdot I[i_k] - 1 - \sum_{i' < i_k} I[i'] = k - 1 - (k-1) = 0$$

Moreover,

$$j \neq k \Rightarrow M[i_j] = k \cdot I[i_j] - 1 - \sum_{i' < i_j} I[i'] = k - j \neq 0$$

$$i_j < i < i_{j+1} \Rightarrow M[i] = k \cdot I[i] - 1 - \sum_{i' \leq i} I[i'] = -1 - j \neq 0$$

which proves the theorem. □

The security of this algorithm is proven by showing that M reveals nothing about I other than the position of the kth one in the array.

Theorem 16. *If M is defined as in Algorithm 28, then M reveals nothing about I other than the index of the kth one.*

Proof. As shown in the correctness proof,

$$k \cdot I[i] - 1 - \sum_{i' \leq i} I[y'] \neq 0$$

for $i \neq i_k$. Moreover, due to the multiplication of r, which is uniformly random, $M[i]$ is also uniformly random, for $i \neq i_k$. Hence, $M[i']$ for $i' \neq i_k$ does not reveal any information about I. □

Appendix B

InsPIRe: Communication-Efficient PIR with Server-side Preprocessing

B.1 Supplementary Material for Ring Packing

B.1.1 LWE to Intermediate Ciphertexts

We give a detailed mathematical derivation of the first stage of our packing algorithm that transforms the input LWE ciphertexts into intermediate ciphertexts. As discussed, our approach transforms the RLWE ciphertext $(\tilde{a}, \tilde{b})$ into an intermediate ciphertext that resembles an MLWE ciphertext. A key characteristic of this transformed ciphertext is that its non-constant term coefficients are zeroed out. Furthermore, these intermediate ciphertexts retain homomorphic properties, enabling aggregation of multiple ciphertexts into a single one that contains the LWE messages as the coefficients.

To facilitate this transformation, we introduce an alternative interpretation of the $\tilde{b}$ component, leveraging Lemma 2. In essence, Lemma 2 states that for any polynomial $p(X) = \sum_{i=0}^{d-1} c_i X^i$, summing $d/2$ versions of $\tau_g^j(p) + \tau_h \circ \tau_g^j(p)$ (obtained by applying the automorphism $X \mapsto X^{g^j}$ and $X \mapsto X^{g^j \cdot h}$ for $j \in \{0, \ldots, d/2 - 1\}$) yields a polynomial where all coefficients are zero except the constant term, which is scaled by a factor of d. As we will show, this operation is crucial for isolating the LWE message m into a constant term polynomial.

Our initial step involves applying the operator Tr from Lemma 2 to the $\tilde{b}$ component of the RLWE representation $(\tilde{a}, \tilde{b})$. More specifically, from Eq. 5.1, we may re-interpret

everything over $\mathbb{Z}[X]/(X^d+1)$ and obtain $\tilde{b} = -\tilde{a}\tilde{s} + \tilde{m} + q\tilde{u}$ for some $\tilde{u} \in \mathbb{Z}[X]/(X^d+1)$. Applying Tr to both sides, we get:

$$\begin{aligned}\mathrm{Tr}(\tilde{b}) &= \mathrm{Tr}(-\tilde{a}\tilde{s} + \tilde{m} + q\tilde{u}) \\ &= -\mathrm{Tr}(\tilde{a}\tilde{b}) + \mathrm{Tr}(\tilde{m}) + q \cdot \mathrm{Tr}(\tilde{u}).\end{aligned}$$

Given that $\tilde{b}$ is a constant polynomial, $\mathrm{Tr}(\tilde{b}) = \sum_{j=0}^{d/2-1} \tau_g^j(\tilde{b}) + \tau_h \circ \tau_g^j(\tilde{b}) = d \cdot \tilde{b}$. Assuming d is invertible mod q (which is true when q is odd), we can write:

$$\begin{aligned}\tilde{b} &= d^{-1}\left(-\mathrm{Tr}(\tilde{a}\tilde{s}) + \mathrm{Tr}(\tilde{m})\right) \pmod q \\ &= -d^{-1}\mathrm{Tr}(\tilde{a}\tilde{s}) + d^{-1}\mathrm{Tr}(\tilde{m}) \pmod q.\end{aligned}$$

We have $\mathrm{Tr}(\tilde{m}) = d \cdot m$. We define a new message representation as $\hat{m}(X) := d^{-1} \cdot \mathrm{Tr}(\tilde{m}) = m$.

Substituting this expression for $\tilde{b}$:

$$\begin{aligned}\tilde{b} &= -d^{-1} \cdot \mathrm{Tr}(\tilde{a}\tilde{b}) + \hat{m} \pmod q \\ &= -\left(\sum_{j=0}^{d/2-1} \left(d^{-1} \cdot \tau_g^j(\tilde{a})\right) \cdot \tau_g^j(\tilde{s})\right) \\ &\quad -\left(\sum_{j=0}^{d/2-1} \left(d^{-1} \cdot \tau_h \circ \tau_g^j(\tilde{a})\right) \cdot \left(\tau_h \circ \tau_g^j(\tilde{s})\right)\right) + \hat{m} \pmod q \\ &= -\langle \hat{\mathbf{a}}, \hat{\mathbf{s}} \rangle + \hat{m} \pmod q\end{aligned}$$

The second equality follows from the definition of Tr and the property that τ_g and τ_h are ring automorphisms. The final equality is written succinctly as an inner product of two vectors $\hat{\mathbf{a}}, \hat{\mathbf{s}} \in R_q^d$. Here, $\hat{\mathbf{a}}[j] := d^{-1} \cdot \tau_g^j(\tilde{a})$ and $\hat{\mathbf{a}}[j + d/2] := d^{-1} \cdot \tau_h \circ \tau_g^j(\tilde{a})$ for $j < d/2$ e.g. the second half is an automorphic image of the first half by τ_h. Similarly, $\hat{\mathbf{s}}[j] := \tau_g^j(\tilde{s})$ and $\hat{\mathbf{s}}[j + d/2] := \tau_h \circ \tau_g(\tilde{s})$ for $j < d/2$.

This formulation allows $\tilde{b}$ to be interpreted as a pseudorandom component of an MLWE-like ciphertext with the random component $\hat{\mathbf{a}}$ and the secret key $\hat{\mathbf{s}}$.

B.1.2 Conversion to RLWE Ciphertext

This section provides the detailed procedure for converting the aggregated intermediate ciphertext $\mathsf{IRCtx}(\hat{m}) = (\hat{\mathbf{a}}_{agg}, \tilde{b}_{agg}) \in R_q^d \times R_q$ into a standard two-component RLWE ciphertext. The input $\mathsf{IRCtx}(\hat{m}_{agg})$ encrypts the packed message $\hat{m}_{agg}$ (as defined in the main

body) and consists of $d+1$ ring elements. The components $\hat{\mathbf{a}}_{\mathbf{agg}}[j]$ are associated with secret key shares $\hat{\mathbf{s}}[j] = \tau_g^j(\tilde{s})$ and $\hat{\mathbf{s}}[j+d/2] = \tau_h \circ \tau_g^j(\tilde{s})$ for $j < d/2$. The goal is to obtain a ciphertext (a_{fin}, b_{fin}) encrypted under base secret key $\tilde{s}$.

The conversion relies on iteratively reducing the number of components using key-switching.

Key-Switching Preliminaries. Recall the key-switching key generation algorithm $\mathsf{KS.Setup}(s_{in}, s_{out})$ which produces a key $\mathbf{K}$ that can transform a ciphertext component encrypted under s_{in} into one encrypted under s_{out}.

For our specific purpose, we require two elementary key-switching matrices. The first matrix is constructed as $\mathbf{K}_g \leftarrow \mathsf{KS.Setup}(\tau_g(\tilde{s}), \tilde{s})$. By applying Galois automorphisms τ_g^i (resp. $\tau_h \circ \tau_g^i$) to $\mathbf{K}_g$, we obtain a key-switching matrix from $\tau_g^{j+1}(\tilde{s})$ to $\tau_g^j(\tilde{s})$ (resp. $\tau_h \circ \tau_g^{j+1}(\tilde{s})$ to $\tau_h \circ \tau_g^j(\tilde{s})$). The second matrix is constructed as $\mathbf{K}_h \leftarrow \mathsf{KS.Setup}(\tau_h(\tilde{s}), \tilde{s})$ and used in the final step of the algorithm.

The CollapseOne Subroutine. The core building block for this conversion is a subroutine to "collapse" ciphertext components, adapted from standard relinearization techniques. The goal of this subroutine is to use the key-switching to re-express the pseudorandom component masked by one secret key share so that it is now masked by another key share. We refer to these key shares as the source key share and the target key share, respectively.

In line 2, key-switching is performed on the last component $\mathbf{a}[k-1]$ with respect to the current pseudorandom component b. This in turn results in $b' = -\langle \mathbf{a}'', \mathbf{s}' \rangle + m + e_{noise} + e_{ks}$, where $\mathbf{a}''$ and $\mathbf{s}'$ are the "reduced" vectors without the corresponding source key share component. e_{noise} is the initial noise present in the input and e_{ks} is the additive noise from the key-switching. Note that the returned pair $(\mathbf{a}', b')$ is essentially a *partial ciphertext* where $\mathbf{a}'$ masks a portion of the pseudorandom component b'.

The CollapseHalf Subroutine. This subroutine is used to collapse the random components masked by the key shares $\tau_g^k(\tilde{s})$ (resp. $\tau_h \circ \tau_g^k(\tilde{s})$) so that it is masked by a single key share $\tilde{s}$ (resp. $\tau_h(\tilde{s})$).

Using the CollapseOne procedure as the building block, we can iteratively collapse the ciphertext components.

In the k-th iteration of the loop, the component to be eliminated is $\mathbf{a}^{(k)}[k]$, which is associated with the secret $\tau_g^k(\tilde{s})$ (resp. $\tau_h \circ \tau_g^k(\tilde{s})$). The target secret for this component is $\tau_g^{k-1}(\tilde{s})$ (resp. $\tau_h \circ \tau_g^{k-1}(\tilde{s})$). Consequently, the required key-switching matrix is $\mathbf{K}_g^{k-1} = \tau_g^{k-1}(\mathbf{K}_g)$ (resp. $\tau_h \circ \tau_g^{k-1}(\mathbf{K}_g)$ and we can invoke CollapseOne to reduce the number of ciphertext components by one.

After $d/2-1$ such key-switching steps, we obtain $(\mathbf{a}^{(0)}, b^{(0)})$. Note, the masking part of $b^{(0)}$ that was originally masked by secret key shares $\tau_g^j(\tilde{s})$ (resp. $\tau_h \circ \tau_g^j(\tilde{s})$) is now masked by $\tilde{s}$ (resp. $\tau_h(\tilde{s})$) with $\mathbf{a}^{(0)}$ as its random component. This process incurs additive noise corresponding to the $d/2-1$ key-switchings.

The Collapse Procedure. After invoking CollapseHalf on both halves of the random components, the resultant pseudorandom component b_2 can be expressed as $b_2 = -a_1\tilde{s} - a_2\tau_h(\tilde{s}) + \hat{m}_{agg} + e_{tot}$ where e_{tot} is the total accumulated noise from the $d-2$ key-switchings. The final key-switching using $\mathbf{K}_h$ is performed to eliminate the secret key share $\tau_h(\tilde{s})$ in the expression.

Online Computation. We can see that the online execution of each invocation of KS.KeySwitch involves an inner product between $\mathbf{y}^{k-1}$ and the gadget decomposed $\mathbf{a}^{(k)}$, which costs $O(\ell)$ polynomial multiplications and additions. This in turn implies that there are in total $O(\ell d)$ polynomial multiplications and additions. In NTT space, each polynomial multiplication and addition costs $O(d)$, and so the total online running time of packing is $O(\ell \cdot d^2)$.

B.2 Analysis of InsPIRe

Lemma 7 (Polynomials with Subgaussian Coefficients [125], adapted)**.** *Let*

$$f(X) = \sum_{j=0}^{d-1} f_j X^j \in R$$

where each coefficient f_j is independently sampled from a subgaussian distribution with parameter σ. Let $g(X) = \sum_{k=0}^{d-1} g_k X^k \in R$ be a B-bounded polynomial, e.g. $|g_k| \leq B$. Then $h = fg$ has coefficients h_i that are subgaussian with parameter $\sqrt{d}B\sigma$.

Lemma 6. *[Theorem 2.19 [125], adapted] Given $\mathsf{RLWE}(m_0)$, let $\mathsf{RGSW}(m_1)$ be a fresh ciphertext encrypting a unit monomial, e.g. $m_1 = \pm X^k$ for some $0 \leq k < d$. Let the error distribution χ be subgaussian with parameter σ_χ, and let ℓ and z be the gadget parameters of the RGSW ciphertext. Under the independence heuristic, the external product $\mathsf{RLWE}(m_0) \boxdot \mathsf{RGSW}(m_1)$ incurs additive noise $e_{ep} \in R_q$ whose coefficients are subgaussian with parameter σ_{ep} and $\sigma_{ep}^2 \leq \ell d z^2 \sigma_\chi^2/2$.*

Proof. Let $\mathsf{RLWE}(m_0) = (a, b)$ where $b = -as + e_{in} + \Delta m_0$. Let $\mathsf{RGSW}(m_1) = [\mathbf{a}, -s\mathbf{a} + \mathbf{e}] + m_1 \cdot \mathbf{G}_{2,z}$ where $\mathbf{a} \in R_q^{2\ell}$ and $\mathbf{e} \in \chi(R_q^{2\ell})$. Computing the external product expression

directly, we get:

$$\begin{aligned}
&\mathsf{RLWE}(m_0) \boxdot \mathsf{RGSW}(m_1) \\
&\quad = \mathbf{G}_{2,z}^{-1}([a,b]) \cdot ([\mathbf{a}, -s\mathbf{a} + \mathbf{e}] + m_1 \cdot \mathbf{G}_{2,z}) \\
&\quad = [\mathbf{a} \cdot \mathbf{G}_{2,z}^{-1}([a,b]), -s \cdot \mathbf{a} \cdot \mathbf{G}_{2,z}^{-1}([a,b]) + \mathbf{e} \cdot \mathbf{G}_{2,z}^{-1}([a,b])] \\
&\qquad + [m_1 a, m_1 b] \\
&\quad = [a', -a's + e_{ep}] + [m_1 a, m_1(-as + e_{in} + \Delta m_0)] \\
&\quad = [a' + m_1 a, -(a' + m_1 a)s + m_1 e_{in} + e_{ep} + \Delta m_0 m_1] \\
&\quad = [a'', -a''s + m_1 e_{in} + e_{ep} + \Delta m_0 m_1]
\end{aligned}$$

where we have defined $a' := \mathbf{a} \cdot \mathbf{G}_{2,z}^{-1}([a,b])$ and $e_{ep} := \mathbf{e} \cdot \mathbf{G}_{2,z}^{-1}([a,b])$, and $a'' := a' + m_1 a$. Since m_1 is a unit monomial and $\|m_1\|_\infty = 1$, we have $||m_1 e_{in}||_\infty = ||m_1||_\infty ||e_{in}||_\infty = ||e_{in}||_\infty$. Therefore, the only additional noise from the external product is the additive noise e_{ep}.

We now analyze e_{ep}. $\mathbf{e}$ is a fresh noise vector and thus the coefficients of each polynomial are subgaussian with parameter σ_χ. By the definition of $\mathbf{G}_{2,z}^{-1}$, each element $||\mathbf{G}_{2,z}^{-1}([a,b])[k]||_\infty \leq z/2$. By Lemma 7, each product $\mathbf{e}[k] \cdot \mathbf{G}_{2,z}^{-1}([a,b])[k]$ has coefficients that are subgaussian with parameter $\sqrt{d} z \sigma_\chi / 2$. The entire product $e_{ep} = \mathbf{e} \cdot \mathbf{G}_{2,z}^{-1}([a,b])$ involves adding 2ℓ such polynomials. Thus, invoking independence heuristics, e_{ep} has subgaussian coefficients with parameter σ_{ep} satisfying $\sigma_{ep}^2 \leq \ell d z^2 \sigma_\chi^2 / 2$. □

Theorem 11. *Let the error distribution χ be subgaussian with parameter σ_χ. Let ℓ_{ks} and z_{ks} be the gadget parameters of the key-switching matrices. Let ℓ_{gsw} and z_{gsw} be the gadget parameters of the RGSW ciphertext. The error polynomial of the RLWE ciphertext* ct *has the form $e = e_{main} + e_{overflow} \in R_q$. Under the independence heuristics, e_{main} has subgaussian coefficients with parameter $\sigma_{main}^2 \leq Np^2\sigma_\chi^2 + t\ell_{ks} d^2 z_{ks}^2 \sigma_\chi^2/4 + t\ell_{gsw} \cdot d z_{gsw}^2 \cdot \sigma_\chi^2/2$ and $||e_{overflow}||_\infty \leq tp/2$.*

Proof. We first analyze the main noise e_{main} of the final ciphertext ct. The first level of the recursion involves multiplying each LWE ciphertext by an element in $\mathbb{Z}_p$ and summing up N/t of them. Thus, the error term of each of the resulting LWE ciphertexts has subgaussian coefficients with parameter σ_1 and $\sigma_1^2 \leq Np^2\sigma_\chi^2$.

By Theorem 4, InspiRING incurs an additive noise e_{pack} whose coefficients are subgaussian with parameter σ_{pack}: $\sigma_{pack}^2 \leq \ell_{ks} d^2 z_{ks}^2 \sigma_\chi^2/4$. Thus, after packing, each of the n RLWE ciphertexts at this point have noise e_2 whose coefficients are subgaussian with parameter $\sigma_2 : \sigma_2^2 = \sigma_1^2 + \sigma_{pack}^2$.

By Lemma 6, each external product incurs an additive noise e_{ep} with subgaussian coefficients with parameter $\sigma_{ep} : \sigma_{ep}^2 \leq \ell_{gsw} d z_{gsw}^2 \sigma_\chi^2/2$. Since there are $t-1$ external

products and $t-1$ additions of the packed ciphertexts, the final ciphertext has main noise e_{main} with parameter σ_{main}:

$$\begin{aligned}\sigma_{main}^2 &= t\sigma_2^2 + (t-1)\sigma_{ep}^2 \\ &\leq t(\sigma_2^2 + \sigma_{ep}^2) \\ &\leq Np^2\sigma_\chi^2 + t\ell_{ks}d^2 z_{ks}^2 \sigma_\chi^2/4 + t\ell_{gsw} d z_{gsw}^2 \sigma_\chi^2/2\end{aligned}$$

completing the analysis for the e_{main} term.

We now analyze the additional noise introduced from the plaintext overflow from the n homomorphic additions during the homomorphic polynomial evaluation.

Formally, consider the final ciphertext $\mathsf{ct} = (a, -as + \Delta m + e_{main})$ where e_{main} is the primary noise. Decryption yields $[\Delta m + e_{main}]_q$. An overflowed plaintext m can be written as $m = [m]_p + p\cdot\hat{m}$. Here, $[m]_p$ is the intended message (modulo p), and $p\cdot\hat{m}$ is the overflow, with $||\hat{m}||_\infty \leq n$. The scaling factor is $\Delta = \lfloor q/p \rfloor = q/p + \epsilon$ where $\epsilon \in [-1/2, 1/2)$ is the fractional part. Thus,

$$\begin{aligned}\Delta m + e_{main} &= \Delta[m]_p + (q/p + \epsilon)p \cdot \hat{m} + e_{main} \\ &= \Delta[m]_p + q \cdot \hat{m} + \epsilon p \cdot \hat{m} + e_{main}\end{aligned}$$

Modulo q, the $q\cdot\hat{m}$ term vanishes, leaving $[\Delta[m]_p + \epsilon p\cdot\hat{m} + e_{main}]_q$. The overflow-induced error is $e_{overflow} = \epsilon p \cdot \hat{m}$ with $||e_{overflow}||_\infty \leq tp/2$, completing the proof. □

For the $e_{overflow}$ error to significantly affect decryption, its magnitude upper bound $tp/2$ must be comparable to the noise tolerance, e.g. $tp/2 = \Omega(q/p)$. This implies $q = O(tp^2)$. In practice, however, for a typical choice of RLWE parameters and the interpolation degree t, we usually have $q \gg tp^2$. Therefore, the overflow error $e_{overflow} = \epsilon p \cdot \hat{m}$ is negligible compared to the decryption threshold $\approx q/p$.

Theorem 12. *For subgaussian χ with parameter σ_χ, define $\bar{\sigma}_{main}^2 = Np^2\sigma_\chi^2 + t\ell_{ks}d^2 z_{ks}^2\sigma_\chi^2/4 + t\ell_{gsw}\cdot d z_{gsw}^2\cdot\sigma_\chi^2/2$. Then, for $\delta > 2d\exp\left(-\pi(\Delta/2 - tp/2)^2/\bar{\sigma}_{main}^2\right)$, $\mathsf{InsPIRe}$ is $(1-\delta)$-correct.*

Proof. By Theorem 11, we know that the final ciphertext has noise $e = e_{main} + e_{overflow}$ for a subgaussian e_{main} with parameter $\sigma_{main} < \bar{\sigma}_{main}$. Hence, we know that for every i, $\mathbb{P}[e_{main}[i] > \Delta/2 - tp] < \delta/d$, so $\mathbb{P}[||e_{main}||_\infty > \Delta/2 - tp] < \delta$, so $||e||_\infty < \Delta/2$ with probability $1-\delta$. So the decryption is correct with $1-\delta$, which proves the theorem. □

Theorem 13. *Let ℓ_{ks} and ℓ_{gsw} be the gadget parameters of the key-switching matrices and the query RGSW ciphertext respectively. In the CRS model, $\mathsf{InsPIRe}$ runs in offline time $O(Nd^2 + t(d^3 + \ell_{ks}d^2\lg d))$ and online time $O(Nd + t\ell_{ks}d^2 + t\ell_{gsw}d\lg(d))$.*

Proof. The first level of the recursion incurs $O(Nd)$ multiplications and additions over $\mathbb{Z}_q$. The LWEs to RLWE translation involves invoking InspiRING t times, and by Theorem 3, the total time is $O(t\ell_{ks}d^2)$. Each iteration of the loop in the homomorphic polynomial evaluation can be broken down as follows:

1. Inverse NTT ct back to coefficient form.
2. Gadget decompose ct
3. NTT gadget decomposed ct
4. Matrix-matrix product $\mathbf{AB}$ with $\mathbf{A} \in R_q^{1\times 2\ell_{gsw}}$ and $\mathbf{B} \in R_q^{2\ell_{gsw}\times 2}$
5. Add two RLWE ciphertexts

The first three steps can be done in $O(\ell_{gsw} d \lg(d))$ times. The matrix-matrix product costs $O(\ell_{gsw} d)$ time. The addition of two RLWE ciphertext costs d times. Overall, each iteration of the loop takes $O(\ell_{gsw} d \lg(d))$ time. Therefore, the entire homomorphic polynomial evaluation takes $O(t\ell_{gsw} d \lg(d))$ time. Thus, the entire protocol takes $O(Nd + t\ell_{ks}d^2 + t\ell_{gsw} d \lg(d))$ time. □

Theorem 14. *The total communication cost of* InsPIRe *is*

$$d\ell_{ks} \log_2 q + (N/t) \log_2 q + 4\ell_{gsw} d \log_2 q + 2d \log_2 q$$

Proof. The upload cost consists of the packing keys, the indicator vector, and the RGSW ciphertext. The packing key size is $2d\ell_{ks} \log_2 q$. The indicator vector is of length N/t, so the total size is $(N/t) \log_2 q$. Finally, the RGSW ciphertext is of size $4 \times \ell_{gsw}$. The download cost only consists of one ciphertext, which is of size $2d \log_2 q$. □

B.3 Security Analysis of InsPIRe

Key-Dependent RLWE Hardness. The security of InsPIRe relies on the hardness of the following variant of the decision RLWE problem, adapted from [126]:

Definition 1 ([126])**.** *Let λ be a security parameter, $d = d(\lambda)$ a power of two, $m = m(\lambda)$ be the number of samples, $q = q(\lambda)$ a ciphertext modulus, and $\chi(R) = \chi(\lambda, R), \chi(R^m) = \chi(\lambda, R^m)$ be error distributions over the ring $R = \mathbb{Z}[X]/(X^d + 1)$ and vector R^m. Let $\mathcal{F}$*

be an efficiently computable set of functions from R_q to R_q. For a bit $\beta \in \{0,1\}$ and an adversary $\mathcal{A}$, let

$$\mathcal{W}_\beta := \Pr[\mathcal{A}^{\mathcal{O}(\cdot)}(1^\lambda, \mathbf{a}, \mathbf{t}_\beta) \mid {}^{s\leftarrow\chi(R),\mathbf{a}\leftarrow U(R_q^m),\mathbf{e}\leftarrow\chi(R^m)}_{\mathbf{t}_0\leftarrow U(R_q^m),\mathbf{t}_1\leftarrow s\mathbf{a}+\mathbf{e}}]$$

The oracle $\mathcal{O}$ takes as input a function $f \in \mathcal{F}$. If $\beta = 0$, it outputs a uniformly random pair $(x, r) \in R_q \times R_q$. If $\beta = 1$, it outputs $(a, as + e + f(s))$, where $a \leftarrow U(R_q)$ and $e \leftarrow \chi(R)$. We say that the key-dependent RLWE hardness assumption with parameters $(d, m, q, \chi, \mathcal{F})$ holds if for all PPT adversaries $\mathcal{A}$, the advantage $|\mathcal{W}_0 - \mathcal{W}_1|$ is a negligble function in λ.

We require the assumption to hold for a family $\mathcal{F}_{auto}$ of scaled Galois automorphisms:

$$\mathcal{F}_{auto} = \{k \cdot \tau_g \mid k \in \mathbb{Z}_q, g \in \mathbb{N}\}$$

Security Analysis of InsPIRe. We now show that InsPIRe satisfies the query privacy definition assuming the hardness of the key-dependent RLWE problem.

Theorem 17. *Let λ be the security parameter, $z_{ks}, \ell_{ks}, z_{gsw}, \ell_{gsw}$ be the gadget parameters for* InspiRING *and the RGSW encryption respectively. Suppose that the key-dependent RLWE hardness assumption holds for $m \geq N/t + 2\ell_{ks} + 2\ell_{gsw}$ with the function family $\mathcal{F}_{auto}$. Then* InsPIRe *satisfies query privacy.*

Proof. Suppose, for contradiction, that there exists a PPT adversary $\mathcal{A}$ that wins the query privacy game for InsPIRe with a non-negligible advantage $\mathsf{Adv}^{\mathsf{QP}}_{\mathcal{A}}(\lambda) = \epsilon(\lambda)$. We construct a PPT algorithm $\mathcal{B}$ that breaks the key-dependent RLWE assumption with non-negligible advantage.

Algorithm $\mathcal{B}$ is given 1^λ, a vector $\mathbf{a} \in R_q^m$, a vector $\mathbf{t}_\beta \in R_q^m$ (where $\beta \in \{0,1\}$ is the RLWE challenge bit) and access to an oracle $\mathcal{O}(\cdot)$. $\mathcal{B}$ proceeds as follows:

1. **Adversary's DB Choice:** $\mathcal{B}$ runs $\mathcal{A}$ on input 1^λ. $\mathcal{A}$ outputs a database $\mathbf{D}$.
2. **Simulate Offline Query Components:** To generate $\mathsf{qry}_{\mathsf{off}}$ for $\mathcal{A}$, $\mathcal{B}$ uses its RLWE challenge samples and oracle $O(\cdot)$. Recall from InsPIRe.Setup that $\mathsf{qry}_{\mathsf{off}}$ in pp consists of the random matrix $\mathbf{A}$ for the LWE ciphertexts and the random parts of the key-switching matrices $\mathbf{K}_g$ and $\mathbf{K}_h$.
   - $\mathcal{B}$ forms the matrix $\mathbf{A}$ using the first N/t polynomials from its input $\mathbf{a}$.

- For the offline parts of the key-switching matrices, which are $\mathbf{w}_g$ and $\mathbf{w}_h$: For each $k \in [\ell_{ks}]$, $\mathcal{B}$ queries its oracle $\mathcal{O}(\cdot)$ with $f_g^k = z_{ks}^{k-1} \cdot \tau_g$ and $f_h^k = z_{ks}^{k-1} \cdot \tau_h$. Let the outputs be $(w_k^{(g)}, y_k^{(g)})$ and $(w_k^{(h)}, y_k^{(h)})$. $\mathcal{B}$ sets $\mathbf{w}_g = [w_0^{(g)}, \ldots, w_{\ell_{ks}-1}^{(g)}]$ and $\mathbf{w}_h = [w_0^{(h)}, \ldots, w_{\ell_{ks}-1}^{(h)}]$.

$\mathcal{B}$ sets $\mathsf{qry}_{\mathsf{off}} \leftarrow (\mathbf{A}, \mathbf{w}_g, \mathbf{w}_h)$.

3. **PIR Setup:** $\mathcal{B}$ computes $(\mathsf{pp}, \mathbf{D}') \leftarrow \mathsf{InsPIRe.Setup}(1^\lambda, \mathbf{D})$. It replaces pp's offline query parts with the computed $\mathsf{qry}_{\mathsf{off}}$. Let this updated public params be pp'. It gives $(\mathsf{pp}', \mathbf{D}')$ to $\mathcal{A}$.

4. **Adversary's Challenge Indices:** $\mathcal{A}$ outputs a pair of indices $(\mathsf{idx}^{(0)}, \mathsf{idx}^{(1)})$.

5. **$\mathcal{B}$'s Internal Challenge:** $\mathcal{B}$ chooses a random bit $c \in \{0, 1\}$. $\mathcal{B}$ will simulate $\mathsf{qry}^{(c)}$ for $\mathcal{A}$. Let $\mathsf{idx}^{(c)} = (i, j)$.

6. **Construct Challenge Query $\mathsf{qry}^{(c)}$:** $\mathcal{B}$ constructs $\mathsf{qry}^{(c)} = (\mathbf{b}, \mathsf{RGSW}(\omega^j), \mathbf{y}_g, \mathbf{y}_h)$ as follows:

   - For the LWE pseudorandom vector $\mathbf{b}$: For $k = 0 \ldots N/t - 1$, $\mathcal{B}$ computes $b_k = \mathsf{Coeff}(\mathbf{t}_\beta[k] + \Delta\mu_k, 0)$ (e.g. constant term of the polynomial), where $\mu_k = 1$ if $i = k$ and $\mu_k = 0$ otherwise. Set $\mathbf{b} = [b_0, \ldots, b_{N/t-1}]^\top \in \mathbb{Z}_q^{N/t}$.
   - For the pseudorandom parts of the key-switching matrices $\mathbf{y}_g$ and $\mathbf{y}_h$: Set $\mathbf{y}_g = [y_0^{(g)}, \ldots, y_{\ell_{ks}-1}^{(g)}]$ and $\mathbf{y}_h = [y_0^{(h)}, \ldots, y_{\ell_{ks}-1}^{(h)}]$, which are constructed from the second components of the oracle query responses.
   - For $\mathsf{RGSW}(\omega^j)$: $\mathcal{B}$ uses the next $2\ell_{gsw}$ samples from its RLWE challenge $(\mathbf{a}, \mathbf{t}_\beta)$. Let these be $(\mathbf{a}_{gsw}, \mathbf{t}_{gsw})$. $\mathcal{B}$ constructs $\mathsf{RGSW}(\omega^j) = [\mathbf{a}_{gsw}, \mathbf{t}_{gsw}] + \mathbf{G}_{2,z_{gsw}} \cdot \omega^j$ where $\omega = X^{2d/t}$.

   $\mathcal{B}$ gives the fully constructed qry to $\mathcal{A}$.

7. **Adversary's Guess:** $\mathcal{A}$ outputs a bit b'.

8. **$\mathcal{B}$'s Output:** Output 1 if $b' = c$. Otherwise, output 0.

Now, we analyze $\mathcal{B}$'s advantage in the key-dependent RLWE game. Let $\mathcal{W}_\beta = \Pr[\mathcal{B}$ outputs 1 | RLWE challenge bit is $\beta]$.

- **Case 1:** $\beta = 0$ **(RLWE samples are random)**: In this case, we see that the outputs in $\mathsf{qry}_{\mathsf{on}}$ are uniformly random and independent of the query index idx_c. Therefore, the adversary cannot do better than a random guessing, and thus:

$$\mathcal{W}_0 = \Pr[b' = c \mid \beta = 0] = 1/2$$

- **Case 2:** $\beta = 1$ **(RLWE samples are real)**: In this case, $\mathcal{B}$ perfectly simulates the query privacy game for $\mathcal{A}$. The components $\mathbf{t}_\beta[k]$ used for constructing $\mathbf{b}$ generates identically distributed pseudorandom components of the LWE ciphertexts. The outputs from the oracle to generate RLWE encryptions of the automorphic images of the secret keys are identically distributed as the outputs from $\mathsf{KS.Setup}$. The RGSW ciphertext is also identically distributed since $\mathcal{B}$ perfectly simulates the RGSW encryption.

  Thus, $\mathcal{A}$'s view is identical to a real query privacy experiment. $\mathcal{A}$'s advantage is $\mathsf{Adv}^{\mathsf{QP}}_{\mathcal{A}}(\lambda) = |\Pr[b' = c] - 1/2| = \epsilon(\lambda)$. Therefore, $\Pr[b' = c \mid \beta = 1] = 1/2 \pm \epsilon(\lambda)$. $\mathcal{B}$ outputs 1 if $b' = c$, and so

$$\mathcal{W}_1 = \Pr[b' = c \mid \beta = 1] = 1/2 \pm \epsilon(\lambda))$$

Therefore, the advantage of $\mathcal{B}$ in breaking the key-dependent RLWE hardness assumption is:

$$|\mathcal{W}_0 - \mathcal{W}_1| = |1/2 - (1/2 \pm \epsilon(\lambda))| = \epsilon(\lambda)$$

contradicting the hardness of the key-dependent RLWE problem. Therefore, $\mathsf{InsPIRe}$ satisfies query privacy. □

B.3.1 Extension to Multiple Queries

As in prior works, the security of $\mathsf{InsPIRe}$ extend to multiple queries with common random components as long as a new secret key is sampled for each query.

We can modify the single query privacy experiment so that the adversary can adaptively make oracle queries for encryptions of one of the query indices, up to Q oracle queries.

1. **Setup Phase:**
   (a) On input security parameter 1^λ, $\mathcal{A}$ outputs a database $\mathbf{D}$.
   (b) The challenger computes $(\mathsf{pp}, \mathbf{D}') \leftarrow \mathsf{Setup}(1^\lambda, \mathbf{D})$ and gives it to $\mathcal{A}$.

2. **Challenge Phase:**

   (a) The challenger chooses a bit $b \in \{0, 1\}$ uniformly at random.

   (b) For $i = 0 \dots Q - 1$

      i. $\mathcal{A}$ outputs a pair of challenge indices $(\mathsf{idx}_i^{(0)}, \mathsf{idx}_i^{(1)})$.

      ii. The challenger computes the challenge query $(\mathsf{st}_i^{(b)}, \mathsf{qry}_i^{(b)}) \leftarrow \mathsf{Query}(\mathsf{pp}, \mathsf{idx}_i^{(b)})$. The challenger sends $\mathsf{qry}_i^{(b)}$ to $\mathcal{A}$.

3. **Guess Phase:** $\mathcal{A}$ outputs a bit b'.

Proof Sketch. The proof proceeds via a standard hybrid argument. We define a sequence of $Q + 1$ hybrid experiments, $\mathcal{H}_k$ for $k = 0 \dots Q$.

In experiment $\mathcal{H}_k$, the first k challenge queries are "real" (e.g. correctly generated) while the remaining $Q - k$ queries are random. Observe that $\mathcal{H}_Q$ is the real query privacy experiment and $\mathcal{H}_0$ is the experiment where the adversary receives completely random values.

The core of the proof is to show that for any k, the experiments $\mathcal{H}_{k-1}$ and $\mathcal{H}_k$ are computationally indistinguishable. Otherwise, we can construct an algorithm $\mathcal{B}$ that can break the key-dependent RLWE assumption.

The reduction at a high level works as follows. For the k-th query, $\mathcal{B}$ generates the query using the RLWE challenge as in the single query privacy reduction. For the first $k - 1$ queries, $\mathcal{B}$ generates real challenge queries as in $\mathcal{H}_Q$, using the same random components as in the k-th query to generate the encryptions (with a freshly sampled secret key for each query). For the remaining $Q - k$ queries, $\mathcal{B}$ generates random queries as in $\mathcal{H}_0$.

Note that if $\mathcal{B}$ received real RLWE samples, it perfectly simulates $\mathcal{H}_k$, and otherwise, it perfectly simulates $\mathcal{H}_{k-1}$. In particular, any non-negligible advantage $\mathcal{A}$ has in distinguishing $\mathcal{H}_{k-1}$ and $\mathcal{H}_k$ implies $\mathcal{B}$ can break the key-dependent RLWE hardness assumption.

By the triangle inequality, we can thus conclude that any non-negligible advantage $\mathcal{A}$ has in distinguishing between $\mathcal{H}_0$ and $\mathcal{H}_Q$ implies that the key-dependent RLWE hardness assumption is broken. This completes the proof.

□

B.4 Optimizations and Extensions to InsPIRe

B.4.1 Multivariate Homomorphic Polynoimal Evaluation

The main limitation of the original InsPIRe protocol is the constraint $t \leq 2d$, which is necessary to ensure that the evaluation points are unit monomials (of the form $\pm X^k$). We can overcome this limitation by employing a multivariate polynomial interpolation/evaluation instead of univariate, at the cost of small additional query overhead.

Let α be the number of variables. For simplicity, we will assume that $t = \hat{t}^\alpha$ for some power of two $\hat{t} \leq 2d$. Each database entry is indexed by a tuple $(i_0, \ldots, i_{\alpha-1})$ and denoted $y_{i_0,\ldots,i_{\alpha-1}}$, where each $0 \leq i_k < \hat{t}$.

As in the univariate approach, we first define $\hat{t} - th$ primitive root of unity $\omega = X^{2d/\hat{t}}$. We then construct a α-variate interpolation polynomial $h(Z_0, \ldots, Z_{\alpha-1}) \in R_p[Z_0, \ldots, Z_{\alpha-1}]$ such that $h(\omega^{i_0}, \ldots, \omega^{i_{\alpha-1}}) = y_{i_0,\ldots,i_{\alpha-1}}$ for each tuple $(i_0, \ldots, i_{\alpha-1})$. Note that this can be accomplished via the multidimensional FFT-based interpolation. h is constructed in a way that the partial degree of the polynomial for each variable is $\hat{t} - 1$.

The homomorphic evaluation of h leverages the recursive structure of the multivariate polynomial rings. The ring $R_p[Z_0, \ldots, Z_{\alpha-1}]$ can be viewed as a ring of univarate polynomials in the variable $Z_{\alpha-1}$ with coefficients in $R_p[Z_0, \ldots, Z_{\alpha-2}]$.

In particular, h can be expressed as

$$h(Z_0, \ldots, Z_{\alpha-1}) = \sum_{k=0}^{\hat{t}-1} c_k(Z_0, \ldots, Z_{\alpha-2}) \cdot Z_{\alpha-1}^k$$

where each coefficient c_k is a polynomial in $\alpha - 1$ variables. This decomposition can be applied recursively to each c_k. In particular, this structure naturally maps to a complete $\hat{t}$-ary tree of depth α. Each internal node represents a univariate polynomial in some variable Z_j, and its children are the coefficient polynomials.

The homomorphic evaluation of h proceeds via a depth-first traversal of this tree structure. At each node, the corresponding univariate polynomial is evaluated using the Horner's method, with the required coefficients recursively evaluated. This construction requires α RGSW ciphertexts in the query, and costs $O(t)$ RLWE-RGSW external products and RLWE additions, same as in the univariate construction.

B.4.2 Extension to non-power of two t

Cooley-Tukey can only be used when t is a power of two, but the runtime is only $O(t \log t)$ ring operations. For cases where t is not a power of two, we can use Lagrange interpolation, which runs using $O(t^2)$ ring operations. The interpolation happens over R_p with the evaluation points defined as $z_i = X^i$. However, since R_p is not a field, we require that there exist an inverse for $z_i - z_j$ in R_p. For this, we rely on the following lemmas.

Lemma 8. *Consider the plaintext ring $R_p = \mathbb{Z}_p[X]/(X^d + 1)$ where p is odd and d is a power of two. Let $X^k \in R_p$ and $0 \leq k < 2d$. Let $g = \gcd(k, 2d)$ Then*

$$(X^k - 1)^{-1} = (p-2)^{-1} \sum_{i=0}^{d/g-1} X^{ik}$$

Proof. We need to prove that $(X^k - 1)(p-2)^{-1} \sum_{i=0}^{d/g-1} X^{ik} = 1$. We have

$$\begin{aligned}(X^k - 1) \sum_{i=0}^{d/g-1} X^{ik} &= \sum_{i=0}^{d/g-1} X^{(i+1)k} - X^{ik} \\ &= X^{dk/g} - 1 \\ &= (X^d)^{k/g} - 1 \\ &= (-1)^{k/g} - 1 = -2\end{aligned}$$

where the last equality follows because k/g is odd. Since $-2 = p - 2$, multiplying by $(p-2)^{-1}$ (which exists because $\gcd(p, p-2) = 1$) completes the proof. □

Lemma 9. *Suppose that $0 \leq a, b < 2d$ and $a \neq b$. Then*

$$(X^a - X^b)^{-1} = \mathsf{sign}(b-a) \cdot X^{d - \min(a,b)} \cdot (p-2)^{-1} \sum_{i=0}^{d/g-1} X^{i \cdot |a-b|}$$

Proof. We have $(X^a - X^b) = -\mathsf{sign}(b-a) \cdot X^{\min(a,b)}(X^{|a-b|} - 1)$. It is easy to see that $(X^{\min(a,b)})^{-1} = -X^{d-\min(a,b)}$, and applying Lemma 8 to $X^{|a-b|} - 1$ completes the proof. □

Using these two lemmas, we can prove the following theorem about interpolating a polynomial over points in R_p^2.

Theorem 18. *Consider t points $\{(z_i, y_i)\}_{i\in[t]} \in R_p^2$ where $t \leq 2d$ and $z_i = X^i$. Then these points are interpolable over the ring R_p, i.e., there exists $P(Z) \in R_p[Z]$ such that $P(z_i) = y_i$.*

Proof. The i-th Lagrangian basis is defined by

$$L_i(Z) = \prod_{0\leq j<t, j\neq i} \frac{Z - z_j}{z_i - z_j}$$

and by Lemma 9 $(z_i - z_j)^{-1} = (X^i - X^j)^{-1}$ exists. Note, $L_i(z_i) = 1$ and $L_i(z_j) = 0$ for $j \neq i$. So the interpolated polynomial can be computed as

$$P(Z) = \sum_{i=0}^{n-1} y_i \cdot L_i(Z)$$

and it is easy to check that $P(z_i) = y_i$. □

Algorithm 29 shows the pseudocode for Lagrange interpolation. We remark that the same multivariate polynomial evaluation strategy can be used for the multivariate Lagrange interpolated database.

Algorithm 29 Lagrange Interpolation

```
1:  algorithm INTERPOLATE([y_0, ..., y_{t-1}]^⊤ ∈ R_p^t)
2:      for i = 0 ... t − 1 do
3:          z[i] ← y_i
4:      for j = 1, ··· , t − 1 do
5:          for i = t − 1, ··· , j do
6:              z[i] ← (z[i] − z[i − 1]) · (X^i − X^{i−j})^{−1}
7:      for i = 0 ... t − 1 do
8:          c[i] ← 0
9:      c[t − 1] ← z[n − 1]
10:     for k = t − 2, ··· , 0 do
11:         for i = k, ··· , 1 do
12:             c[i] ← X^k (c[i] + c[i − 1])
13:         c[0] ← X^k c[0] + z[k]
14:     return c
```

B.4.3 Approximate Gadget Decomposition

We note that our construction can use the standard approximate gadget decomposition technique to further reduce the query size and the server computation. This was first

introduced in the context of TFHE bootstrapping [46], but the same technique can be applied to the BFV ciphertexts that we use in our work.

We provide a high level overview of this technique in the context of key-switching, but the same principle applies for the RGSW ciphertexts. Consider a RLWE ciphertext $(\tilde{a}, \tilde{b} = -\tilde{a}\tilde{s} + \tilde{e} + \Delta\tilde{m})$. Let $\tilde{a}_{low}$ consist of the least significant digits of the coefficients of $\tilde{a}$ in base z (e.g. $\|a_{low}\|_\infty \leq z$), and $\tilde{a}_{high} = \tilde{a} - \tilde{a}_{low}$. Then, we can express $\tilde{b}$ as:

$$\begin{aligned}\tilde{b} &= -\tilde{a}\tilde{s} + \tilde{e} + \Delta\tilde{m} \pmod q \\ &= -(\tilde{a}_{low} + \tilde{a}_{high})\tilde{s} + \tilde{e} + \Delta\tilde{m} \pmod q \\ &= -\tilde{a}_{high}\tilde{s} + \tilde{e} - \tilde{a}_{low}\tilde{s} + \Delta\tilde{m} \pmod q\end{aligned}$$

In particular, $(\tilde{a}_{high}, \tilde{b})$ is a valid RLWE ciphertext where the term $-\tilde{a}_{low}\tilde{s}$ is now part of the noise. If z and the secret key $\tilde{s}$ have sufficiently low norm (e.g. $\tilde{s}$ from ternary or error distribution), then this additive noise term typically has minimal impact on decryption. Now, suppose that we want to key-switch this ciphertext to be encrypted under a different secret key $\tilde{s}'$. Because the least significant digits (in base z) of $\tilde{a}_{high}$ are zero, the key-switching matrix component corresponding to the least significant digit is not necessary. In particular, instead of the key-switching matrix consisting of ℓ components, it now only requires $\ell - 1$ components. This technique is particularly effective when the number of components ℓ in the standard gadget decomposition is small (as in our case). This key-switching strategy can be applied to Stage 3 of the InspiRING algorithm, and the general idea can also be applied to the homomorphic polynomial evaluation part as well.

We expect this technique to reduce the size of the key-switching matrices and the RGSW encrypted evaluation points by around 33% while reducing the total computation up to $\approx 25\%$.

B.5 Analysis of InsPIRe$^{(2)}$

Lemma 10 (Modulus Switching [125], Theorem 3.4, adapted)**.** . *Let $q \geq \tilde{q} > p$ and let $R = \mathbb{Z}[x]/(x^d+1)$ where d is a power of two. Suppose $c = (c_1, c_2)$ and $c_1 s + c_2 = \lfloor q/p \rfloor \mu + e \pmod q$ for some $s \in R_q$, $c \in R_q^2$, $\|\mu\|_\infty \leq p/2$ and $e \in R$. Suppose the components of s are independent subgaussian random variables with parameter σ_s and e is subgaussian with parameter σ_e. Let $(\tilde{c}_1, \tilde{c}_2) = (\lfloor \tilde{q}c_1/q \rceil, \lfloor \tilde{q}c_2/q \rceil)$. Then $\tilde{c}_1 s + \tilde{c}_2 = \lfloor \tilde{q}/p \rfloor \mu + \tilde{e} \pmod{\tilde{q}}$ where $\tilde{e} = e_1 + e_2$,*

$$\|e_1\|_\infty \leq \frac{1}{2}\left(2 + (\tilde{q} \mod p) + \frac{\tilde{q}}{q}(q \mod p)\right),$$

and the components of e_2 are subgaussian with parameter $\sigma_2^2 = d\sigma_s^2/4 + (\tilde{q}/q)^2\sigma_e^2$.

Lemma 10 also holds in the case where only a subset of the coefficients of e follow a subgaussian distribution. In that case, the claim of the lemma holds for the same subset of coefficients in e_2.

Theorem 9. *Let the error distribution χ be subgaussian with parameter σ_χ. For $i \in \{0,1,2\}$, let e_i denote the error term of an RLWE ciphertext in $\mathsf{resp_i}$, defined as in Algorithm 21. Moreover, let $e[:\gamma]$ denote a polynomial which only consists of the first γ coefficients of e. Then for $i \in \{0,1,2\}$, under the independence heuristic, $\tilde{e}_i[:\gamma_i] = \tilde{e}_i^{(1)} + \tilde{e}_i^{(2)}$ such that*

$$||e_i^{(1)}||_\infty \leq f(q,\tilde{q},p) = \frac{1}{2}(2 + (\tilde{q} \mod p) + \frac{\tilde{q}}{q}(q \mod p))$$

and $\tilde{e}_i^{(2)}$ has subgaussian coefficients with parameter $\tilde{\sigma}_i$ where

$$\tilde{\sigma}_0^2 \leq \Sigma_0 = d\sigma_\chi^2/4 + (\tilde{q}/q)^2((N/t)p^2\sigma_\chi^2 + \ell_{ks}\gamma_0 d z_{ks}^2\sigma_\chi^2/4) \tag{5.3}$$

$$\tilde{\sigma}_1^2 \leq \Sigma_1 = d\sigma_\chi^2/4 + (\tilde{q}/q)^2(tp^2\sigma_\chi^2 + \ell_{ks}\gamma_1 d z_{ks}^2\sigma_\chi^2/4) \tag{5.4}$$

$$\tilde{\sigma}_2^2 \leq \Sigma_2 = d\sigma_\chi^2/4 + (\tilde{q}/q)^2(tp^2\sigma_\chi^2 + \ell_{ks}\gamma_2 d z_{ks}^2\sigma_\chi^2/4) \tag{5.5}$$

Proof. We prove the theorem for $\mathsf{resp_0}$ and proof for the other two follows similarly.

The first level of the recursion involves multiplying each LWE ciphertext by an element in $\mathbb{Z}_p$ and summing up N/t of them. Thus, the error term of each of the resulting LWE ciphertexts is subgaussian with parameter $\sigma_{i.p.}$ and $\sigma_{i.p.}^2 \leq (N/t)p^2\sigma_\chi^2$. By Theorem 6, PartialInspiRING incurs an additive noise e_{pack} such that $e_{pack}[:\gamma_0]$ has subgaussian coefficients with parameter σ_{pack} where $\sigma_{pack}^2 \leq \ell_{ks}\gamma_0 d z_{ks}^2\sigma_\chi^2/4$. Thus, after packing, each of the t RLWE ciphertexts have noise e_0 such that the coefficients of $e_0[:\gamma_0]$ are subgaussian with parameter σ_0 where $\sigma_0^2 = \sigma_{i.p.}^2 + \sigma_{pack}^2 \leq (N/t)p^2\sigma_\chi^2 + \ell_{ks}\gamma_0 d z_{ks}^2\sigma_\chi^2/4$. Finally, after performing modulus switching, based on Lemma 10, we obtain a final error $\tilde{e}_0$ which follows the theorem.

The same rationale holds for $\tilde{e}_1$ and $\tilde{e}_2$, but with inner product of length t and packing parameters γ_1 and γ_2, respectively. □

Theorem 10. *For $f(q,\tilde{q},p)$ and Σ_i defined as in Theorem 9, then if*

$$\delta_0 \geq 2t\gamma_0 \exp\left(-\pi(\tilde{q}/2p - f(q,\tilde{q},p))^2/\Sigma_0^2\right)$$
$$\delta_1 \geq 2\tau d \exp\left(-\pi(\tilde{q}/2p - f(q,\tilde{q},p))^2/\Sigma_1^2\right)$$
$$\delta_2 \geq 2\tau\gamma_0 \exp\left(-\pi(\tilde{q}/2p - f(q,\tilde{q},p))^2/\Sigma_2^2\right)$$

then $\mathsf{InsPIRe}^{(2)}$ is $(1-\delta_0-\delta_1-\delta_2)$-correct.

Proof. Failure can occur if the noise over flows in the three steps outlines below as three events.

- E_0: Failure in the first level of PIR + packing using γ_0
- E_1: Failure in the second level of PIR + packing using γ_1
- E_2: Failure in second level of PIR + packing using γ_2

The total failure can be bounded as $\mathbb{P}[E] = \mathbb{P}[E_0 \vee E_1 \vee E_2] \leq \mathbb{P}[E_0] + \mathbb{P}[E_1] + \mathbb{P}[E_2]$. In each case,

$$\begin{aligned}\mathbb{P}[E_0] &= t\gamma_0 \mathbb{P}[|\tilde{e}_0| > \tilde{q}/2p] \\ &\leq t\gamma_0 \mathbb{P}[|e_0| > \tilde{q}/2p - f(q, \tilde{q}, p)] \\ &\leq 2t\gamma_0 \exp\left(-\pi(\tilde{q}/2p - f(q, \tilde{q}, p))^2/\Sigma_0^2\right) \leq \delta_0\end{aligned}$$

Similarly,

$$\begin{aligned}\mathbb{P}[E_1] &\leq 2\tau d \exp\left(-\pi(\tilde{q}/2p - f(q, \tilde{q}, p))^2/\Sigma_1^2\right) \leq \delta_1 \\ \mathbb{P}[E_2] &\leq 2\tau\gamma_0 \exp\left(-\pi(\tilde{q}/2p - f(q, \tilde{q}, p))^2/\Sigma_2^2\right) \leq \delta_2\end{aligned}$$

Hence, the total failure probability is bounded by $\delta_0 + \delta_1 + \delta_2$. □

Theorem 7. *Let ℓ_{ks} be the gadget parameter of the key-switching matrices. In the CRS model,* $\mathsf{InsPIRe}^{(2)}$ *runs in offline time*

$$O(N\gamma_0 d + t(\gamma_0^2 d + \gamma_0 \ell_{ks} d \lg d) + \tau t d^2 + \tau d(\gamma_1 d + \ell_{ks} d \lg d))$$

and online time

$$O(N\gamma_0 + t\gamma_0 d\ell_{ks} + t\tau\gamma_0 + \tau d^2 \ell_{ks} + \tau\gamma_0 d(\gamma_2 + \ell_{ks} \lg d)).$$

Proof. The runtime consists of five major parts, for which we provide the offline and online runtime.

- The first matrix multiplication runs in offline time $O(Nd\gamma_0)$ and online time $O(N\gamma_0)$.
- The first packing takes advantage of the offline preprocessing and packs t batches of γ_0 ciphertexts, so it runs in offline time $O(t(\gamma_0^2 d + \gamma_0 \ell_{ks} d \lg d))$ and online time $O(t\gamma_0 d\ell_{ks})$.

- The second matrix multiplication runs in offline time $O(\tau t d^2)$ and online time $O(\tau t \gamma_0)$.
- The second packing also takes advantage of offline preprocessing, packing $\frac{\tau d}{\gamma_1}$ batches of γ_1 ciphertexts. Hence, this step runs in offline time $O(\tau d(\gamma_1 d + \ell_{ks} d \lg d))$ and online time $O(\tau d^2 \ell_{ks})$.
- The third packing, which packs $\frac{\tau \gamma_0}{\gamma_2}$ batches of γ_2 ciphertexts, can not use preprocessing since the first component is not known before hand. So all operations occur online and the online time is $O((\tau\gamma_0/\gamma_2)(\gamma_2^2 d + \gamma_2 d \ell_{ks})) = O(\tau \gamma_0 d(\gamma_2 + \ell_{ks} \lg d))$.

□

B.6 Additional Experiments

We provide additional experiments for benchmarking of $\mathsf{InsPIRe}^{(2)}$ and $\mathsf{InsPIRe}$.We provide the breakdown for PIR over a 1 GB, 8 GB, and 32 GB database using $\mathsf{InsPIRe}^{(2)}$ and $\mathsf{InsPIRe}$. These provide better insight into the breakdown of the runtime in our protocols.

Table B.1: Performance metrics for different configurations of $\mathsf{InsPIRe}^{(2)}$

1 GB Configuration												
Entry Size	$\gamma_0 = 16, \gamma_1 = 1024, \gamma_2 = 16$ 32 B				$\gamma_0 = 32, \gamma_1 = 1024, \gamma_2 = 32$ 64 B				$\gamma_0 = 64, \gamma_1 = 1024, \gamma_2 = 64$ 128 B			
N/t	8192	16384	32768	65536	4096	8192	16384	32768	4096	8192	16384	32768
Upload Keys	80 KB	80 KB	80 KB	80 KB	80 KB	80 KB	80 KB	80 KB	80 KB	80 KB	80 KB	80 KB
Upload Query	80 KB	119 KB	219 KB	427 KB	53 KB	66 KB	113 KB	215 KB	40 KB	60 KB	109 KB	214 KB
Download	52 KB	52 KB	52 KB	52 KB	52 KB	52 KB	52 KB	52 KB	52 KB	52 KB	52 KB	52 KB
Total Comm.	211 KB	251 KB	350 KB	559 KB	185 KB	198 KB	244 KB	347 KB	172 KB	191 KB	241 KB	345 KB
First Mat. Mul	104 ms	104 ms	104 ms	102 ms	102 ms	104 ms	101 ms	102 ms	104 ms	103 ms	102 ms	102 ms
First Pack	549 ms	277 ms	140 ms	67 ms	966 ms	487 ms	224 ms	112 ms	852 ms	429 ms	211 ms	107 ms
Second Mat. Mul	3.7 ms	1.8 ms	1.0 ms	0.5 ms	3.3 ms	1.9 ms	1.2 ms	0.8 ms	2.0 ms	1.1 ms	0.6 ms	0.4 ms
Second Pack	28 ms	29 ms	29 ms	28 ms	29 ms	29 ms	28 ms	28 ms	29 ms	29 ms	28 ms	29 ms
Third Pack	10 ms	9 ms	9 ms	9 ms	24 ms	21 ms	21 ms	21 ms	57 ms	58 ms	55 ms	59 ms
Total Time	750 ms	470 ms	330 ms	260 ms	1180 ms	690 ms	440 ms	320 ms	1100 ms	670 ms	470 ms	360 ms
8 GB Configuration												
Entry Size	$\gamma_0 = 16, \gamma_1 = 1024, \gamma_2 = 16$ 32 B				$\gamma_0 = 32, \gamma_1 = 1024, \gamma_2 = 32$ 64 B				$\gamma_0 = 64, \gamma_1 = 1024, \gamma_2 = 64$ 128 B			
N/t	16384	32768	65536	131072	16384	32768	65536	131072	8192	16384	32768	65536
Upload Keys	80 KB	80 KB	80 KB	80 KB	80 KB	80 KB	80 KB	80 KB	80 KB	80 KB	80 KB	80 KB
Upload Query	212 KB	265 KB	450 KB	861 KB	159 KB	238 KB	437 KB	855 KB	106 KB	132 KB	225 KB	431 KB
Download	52 KB	52 KB	52 KB	52 KB	52 KB	52 KB	52 KB	52 KB	52 KB	52 KB	52 KB	52 KB
Total Comm.	344 KB	397 KB	582 KB	993 KB	291 KB	370 KB	569 KB	986 KB	238 KB	264 KB	357 KB	562 KB
First Mat. Mul	844 ms	819 ms	911 ms	836 ms	827 ms	845 ms	827 ms	824 ms	849 ms	840 ms	823 ms	814 ms
First Pack	2221 ms	1038 ms	538 ms	289 ms	1837 ms	994 ms	480 ms	225 ms	3462 ms	1681 ms	875 ms	424 ms
Second Mat. Mul	13.7 ms	7.1 ms	3.9 ms	1.7 ms	7.0 ms	4.1 ms	2.0 ms	1.1 ms	6.4 ms	3.9 ms	1.7 ms	1.1 ms
Second Pack	34 ms	29 ms	30 ms	30 ms	30 ms	28 ms	30 ms	28 ms	34 ms	28 ms	28 ms	28 ms
Third Pack	10 ms	10 ms	11 ms	9 ms	23 ms	22 ms	20 ms	20 ms	63 ms	59 ms	56 ms	56 ms
Total Time	3150 ms	1960 ms	1550 ms	1210 ms	2850 ms	1930 ms	1420 ms	1150 ms	4680 ms	2670 ms	1850 ms	1390 ms
32 GB Configuration												
Entry Size	$\gamma_0 = 16, \gamma_1 = 1024, \gamma_2 = 16$ 32 B				$\gamma_0 = 32, \gamma_1 = 1024, \gamma_2 = 32$ 64 B				$\gamma_0 = 64, \gamma_1 = 1024, \gamma_2 = 64$ 128 B			
N/t	32768	65536	131072	262144	32768	65536	131072	262144	32768	65536	131072	
Upload Keys	80 KB	80 KB	80 KB	80 KB	80 KB	80 KB	80 KB	80 KB	80 KB	80 KB	80 KB	
Upload Query	424 KB	530 KB	901 KB	1722 KB	318 KB	477 KB	874 KB	1709 KB	265 KB	450 KB	861 KB	
Download	52 KB	52 KB	52 KB	52 KB	52 KB	52 KB	52 KB	52 KB	52 KB	52 KB	52 KB	
Total Comm.	556 KB	662 KB	1033 KB	1854 KB	450 KB	609 KB	1006 KB	1841 KB	397 KB	582 KB	993 KB	
First Mat. Mul	4336 ms	3308 ms	3430 ms	3406 ms	3419 ms	3279 ms	3325 ms	3434 ms	3369 ms	3313 ms	3391 ms	
First Pack	8404 ms	2049 ms	1055 ms	555 ms	4131 ms	1855 ms	935 ms	494 ms	3778 ms	1685 ms	870 ms	
Second Mat. Mul	37.9 ms	13.7 ms	7.1 ms	3.3 ms	13.3 ms	7.0 ms	3.9 ms	2.0 ms	6.8 ms	3.6 ms	1.9 ms	
Second Pack	28 ms	103 ms	27 ms	29 ms	36 ms	32 ms	29 ms	30 ms	36 ms	31 ms	30 ms	
Third Pack	11 ms	10 ms	10 ms	10 ms	24 ms	24 ms	23 ms	22 ms	62 ms	61 ms	57 ms	
Total Time	11780 ms	5540 ms	4560 ms	4070 ms	7760 ms	5260 ms	4410 ms	4010 ms	7390 ms	5280 ms	4430 ms	

Table B.2: Performance metrics for InsPIRe using different polynomial interpolation degrees.

Database Size: 1 GB

	1 bit Entry					64 B Entry					32 KB Entry					
N/t	4096	8192	16384	32768	65536	4096	8192	16384	32768	65536	1024	2048	4096	8192	16384	32768
Interpolation Degree	32	32	16	8	4	32	32	16	8	4	32	16	8	4	2	1
Offline Time	113 s	87 s	79 s	99 s	96 s	110 s	86 s	77 s	98 s	99 s	305 s	177 s	106 s	83 s	72 s	86 s
Upload Keys	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB
Upload Query	112 KB	140 KB	196 KB	308 KB	532 KB	112 KB	140 KB	196 KB	308 KB	532 KB	91 KB	98 KB	112 KB	140 KB	196 KB	308 KB
Download	24 KB	12 KB	12 KB	12 KB	12 KB	24 KB	12 KB	12 KB	12 KB	12 KB	96 KB	96 KB	96 KB	96 KB	96 KB	96 KB
Total Comm.	220 KB	236 KB	292 KB	404 KB	628 KB	220 KB	236 KB	292 KB	404 KB	628 KB	271 KB	278 KB	292 KB	320 KB	376 KB	488 KB
Mat Mul.	100 ms	100 ms	100 ms	110 ms	100 ms	100 ms	100 ms	100 ms	100 ms	100 ms	110 ms	110 ms	100 ms	100 ms	100 ms	100 ms
Packing	1070 ms	540 ms	320 ms	180 ms	110 ms	1080 ms	570 ms	310 ms	180 ms	110 ms	4220 ms	2030 ms	1050 ms	540 ms	300 ms	180 ms
Poly Eval	14 ms	7 ms	4 ms	2 ms	1 ms	15 ms	7 ms	4 ms	2 ms	1 ms	59 ms	29 ms	14 ms	7 ms	3 ms	2 ms
Total Time	1180 ms	650 ms	420 ms	280 ms	210 ms	1180 ms	660 ms	400 ms	280 ms	210 ms	4390 ms	2210 ms	1170 ms	640 ms	410 ms	280 ms

Database Size: 8 GB

	1 bit Entry					64 B Entry							32 KB Entry					
N/t	8192	16384	32768	65536	131072	8192	16384	32768	65536	131072	262144	524288	8192	16384	32768	65536	131072	262144
Interpolation Degree	32	32	32	32	16	32	32	32	32	16	8	4	32	16	8	4	2	1
Offline Time	547 s	470 s	596 s	564 s	579 s	544 s	476 s	601 s	581 s	566 s	556 s	560 s	547 s	470 s	598 s	597 s	538 s	467 s
Upload Keys	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB
Upload Query	140 KB	196 KB	308 KB	532 KB	980 KB	140 KB	196 KB	308 KB	532 KB	980 KB	1876 KB	3668 KB	140 KB	196 KB	308 KB	532 KB	980 KB	1876 KB
Download	96 KB	48 KB	24 KB	12 KB	12 KB	96 KB	48 KB	24 KB	12 KB	12 KB	12 KB	12 KB	96 KB	96 KB	96 KB	96 KB	96 KB	96 KB
Total Comm.	320 KB	328 KB	416 KB	628 KB	1076 KB	320 KB	328 KB	416 KB	628 KB	1076 KB	1972 KB	3764 KB	320 KB	376 KB	488 KB	712 KB	1160 KB	2056 KB
Mat Mul.	850 ms	820 ms	800 ms	840 ms	820 ms	810 ms	830 ms	810 ms	810 ms	810 ms	910 ms	830 ms	810 ms	810 ms	810 ms	810 ms	830 ms	810 ms
Packing	4040 ms	2080 ms	1220 ms	560 ms	320 ms	4050 ms	2090 ms	1110 ms	540 ms	310 ms	190 ms	180 ms	3970 ms	2050 ms	1270 ms	540 ms	290 ms	190 ms
Poly Eval	59 ms	30 ms	15 ms	7 ms	4 ms	59 ms	29 ms	15 ms	8 ms	4 ms	2 ms	1 ms	59 ms	30 ms	15 ms	7 ms	4 ms	2 ms
Total Time	5220 ms	3000 ms	1890 ms	1400 ms	1120 ms	5530 ms	3050 ms	1890 ms	1360 ms	1130 ms	1120 ms	970 ms	5630 ms	3010 ms	1920 ms	1340 ms	1120 ms	1020 ms

Database Size: 32 GB

	1 bit Entry					64 B Entry						32 KB Entry						
N/t	32768	65536	131072	262144	1048576	32768	65536	131072	262144	524288	1048576	16384	32768	65536	131072	262144	524288	1048576
Interpolation Degree	32	32	32	32	8	32	32	32	32	16	8	32	32	16	8	4	2	1
Offline Time	2539 s	2393 s	2406 s	2115 s	2111 s	2581 s	2250 s	2420 s	2128 s	2114 s	2178 s	2052 s	2486 s	2569 s	2306 s	2340 s	2053 s	1945 s
Upload Keys	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB	84 KB
Upload Query	308 KB	532 KB	980 KB	1876 KB	7252 KB	308 KB	532 KB	980 KB	1876 KB	3668 KB	7252 KB	196 KB	308 KB	532 KB	980 KB	1876 KB	3668 KB	7252 KB
Download	96 KB	48 KB	24 KB	12 KB	12 KB	96 KB	48 KB	24 KB	12 KB	12 KB	12 KB	192 KB	96 KB	96 KB	96 KB	96 KB	96 KB	96 KB
Total Comm.	488 KB	664 KB	1088 KB	1972 KB	7348 KB	488 KB	664 KB	1088 KB	1972 KB	3764 KB	7348 KB	472 KB	488 KB	712 KB	1160 KB	2056 KB	3848 KB	7432 KB
Mat Mul.	3200 ms	4300 ms	3400 ms	3500 ms	3300 ms	3300 ms	3300 ms	3700 ms	3300 ms	3300 ms	3400 ms	3700 ms	3400 ms	3400 ms	3300 ms	3300 ms	3300 ms	3300 ms
Packing	4210 ms	2440 ms	2150 ms	560 ms	230 ms	4120 ms	2220 ms	2050 ms	560 ms	310 ms	220 ms	15140 ms	4930 ms	2110 ms	1030 ms	540 ms	310 ms	210 ms
Poly Eval	59 ms	30 ms	15 ms	7 ms	2 ms	59 ms	30 ms	15 ms	7 ms	3 ms	2 ms	118 ms	61 ms	30 ms	15 ms	8 ms	4 ms	2 ms
Total Time	8300 ms	6220 ms	4800 ms	4040 ms	3510 ms	7640 ms	5370 ms	5040 ms	3880 ms	3630 ms	3620 ms	15470 ms	8360 ms	5480 ms	4320 ms	3820 ms	3650 ms	3500 ms